



OPERATING SYSTEMS

A Concept-based Approach

Inderjeet Kaur | Sonam Gupta

Distributed by:

dreamtech
PRESS

WILEY

Operating Systems

A Concept-based Approach



Operating Systems

A Concept-based Approach

Inderjeet Kaur

Associate Professor

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
Ghaziabad

Sonam Gupta

Assistant Professor

Department of Computer Science and Engineering
Ajay Kumar Garg Engineering College
Ghaziabad

©Copyright 2020 I.K. International Pvt. Ltd., New Delhi-110002.

This book may not be duplicated in any way without the express written consent of the publisher, except in the form of brief excerpts or quotations for the purposes of review. The information contained herein is for the personal use of the reader and may not be incorporated in any commercial programs, other books, databases, or any kind of software without written consent of the publisher. Making copies of this book or any portion for any purpose other than your own is a violation of copyright laws.

Limits of Liability/disclaimer of Warranty: The author and publisher have used their best efforts in preparing this book. The author make no representation or warranties with respect to the accuracy or completeness of the contents of this book, and specifically disclaim any implied warranties of merchantability or fitness of any particular purpose. There are no warranties which extend beyond the descriptions contained in this paragraph. No warranty may be created or extended by sales representatives or written sales materials. The accuracy and completeness of the information provided herein and the opinions stated herein are not guaranteed or warranted to produce any particulars results, and the advice and strategies contained herein may not be suitable for every individual. Neither Dreamtech Press nor author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

Trademarks: All brand names and product names used in this book are trademarks, registered trademarks, or trade names of their respective holders. Dreamtech Press is not associated with any product or vendor mentioned in this book.

ISBN: 978-93-89795-53-0

EISBN: 978-81-94508-50-2

Edition: 2020



Preface

Operating systems are an essential part of any computer system. The importance of operating system and its concepts are well known in engineering fields.

This book is designed with the aim to provide a step-by-step approach to cover all the concepts of operating systems.

The book covers all the topics starting from basics to all advanced concepts of operating systems. This book uses a simple and lucid language to explain students all the fundamentals.

It assimilates a comprehensive matter on the subject as it is bolstered with the numerical examples.

Summary and representative questions have been added at the end of chapters to help students in picking all the important points from that chapter.

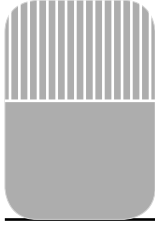
Previous year question papers of various universities have been added for practice and set of case studies of various OS and implemented programs are also added for helping students in better way.

The book covers syllabi of Abdul Kalam Technical University, Lucknow (formerly UPTU), GGSIPU, WBTU, RGTU and many more technological universities for B.E./B. Tech (CSE/IT)/BCA/MCA courses.

We are grateful to Prof. B.M. Kalra, HoD, Computer Science & Engineering, AKGEC, Ghaziabad for his motivational support.

We express our sincere thanks to all those who helped in making this book a reality. Suggestions for the improvement of book will be acknowledged and appreciated.

Inderjeet Kaur
Sonam Gupta



Contents

<i>Preface</i>	<i>v</i>
1. Introduction	1
1.1 Need of Operating System	1
1.2 Objectives of Operating System	1
1.3 Operating System and Its Functions	2
1.4 Viewpoints on Operating System	2
1.5 Functions of Operating System	3
1.6 Classification of Operating Systems	4
1.7 Components of an Operating System	8
1.8 Operating System Structure	11
1.9 Operating System Services	15
1.10 Kernels	16
<i>Summary</i>	18
<i>Multiple-Choice Questions</i>	18
2. Processes	21
2.1 What is a Process?	21
2.2 Process States	22
2.3 Process Control Block	23
2.4 Threads	24
2.5 Schedulers	26
2.6 Operations of Processes	27
2.7 Interprocess Communication (IPC)	29
<i>Summary</i>	32
<i>Multiple-Choice Questions</i>	33
3. CPU Scheduling	36
3.1 Introduction	36
3.2 Basic Concepts of Scheduling	36
3.3 Preemptive vs Non-preemptive Scheduling	38
3.4 Scheduling Criteria	39

3.5 CPU Scheduling Algorithms	39
<i>Numerical Examples</i>	48
<i>Summary</i>	63
<i>Multiple-Choice Questions</i>	64
4. Synchronization	68
4.1 Introduction	68
4.2 Race Condition	68
4.3 The Critical Section Problem	70
4.4 Solution to Critical Section Problem	71
4.5 Algorithm 1	72
4.6 Algorithm 2	72
4.7 Algorithm 3	73
4.8 Bakery Algorithm	74
4.9 Synchronization Hardware	75
4.10 Semaphores	77
4.11 Classic Problems of Synchronization	80
<i>Summary</i>	86
<i>Multiple-Choice Questions</i>	87
5. Deadlock	91
5.1 Introduction	91
5.2 System Model	91
5.3 Cause of Deadlock	92
5.4 Necessary Conditions of Deadlocks	92
5.5 Resource Allocation Graph (RAG)	93
5.6 RAG with a Cycle but No Deadlock	93
5.7 Methods for Handling Deadlock	95
5.8 Deadlock Prevention	95
5.9 Deadlock Avoidance	97
5.10 Deadlock Detection	100
5.11 Recovery from Deadlock	102
5.12 Starvation	103
<i>Numerical Examples</i>	104
<i>Summary</i>	114
<i>Multiple-Choice Questions</i>	115
6. Memory Management	119
6.1 Introduction	119
6.2 Requirements for Memory Management	119
6.3 Basic Hardware for Memory Management	120

6.4	Address Binding	121
6.5	Dynamic Loading	122
6.6	Dynamic Linking	123
6.7	Overlays	123
6.8	Logical vs Physical Address Space	124
6.9	Swapping	125
6.10	Contiguous Memory Allocation	126
6.11	Fragmentation	131
6.12	Non-contiguous Memory Allocation	132
6.13	Main Memory	147
	<i>Numerical Examples</i>	147
	<i>Summary</i>	151
	<i>Multiple-Choice Questions</i>	152
7.	Virtual Memory	156
7.1	Virtual Memory Concept	156
7.2	Demand Paging	157
7.3	Advantages/Disadvantages of Demand Paging	159
7.4	Valid-Invalid Bit	159
7.5	Page Fault	160
7.6	Performance of Demand Paging	160
7.7	Basic Page Replacement	161
7.8	Page Replacement Algorithms	163
7.9	Thrashing	166
7.10	Cache Memory Organization	168
	<i>Numerical Examples</i>	169
	<i>Summary</i>	177
	<i>Multiple-Choice Questions</i>	178
8.	I/O System	181
8.1	Overview of I/O Devices	181
8.2	I/O Subsystem	181
8.3	I/O Software	181
8.4	I/O Buffering	182
	<i>Summary</i>	183
	<i>Multiple-Choice Questions</i>	184
9.	File Management	187
9.1	File and Its Types	187
9.2	File Attributes	187
9.3	File Operations	188

9.4 File Access Methods	188
9.5 File Allocation Methods	190
9.6 Free Space Management	192
9.7 File Directory	193
9.8 Directory Implementation	194
9.9 File Sharing	195
9.10 Implementation of Shared Files and Directory	195
<i>Summary</i>	196
<i>Multiple-Choice Questions</i>	196
10. Disk Storage	199
10.1 Mass Storage Structure	199
10.2 Disk Storage	200
10.3 Disk Performance Parameters	200
10.4 Need of Disk Scheduling	202
10.5 Disk Scheduling Algorithms	202
10.6 RAID (Redundant Array of Independent Disks)	206
<i>Numerical Examples</i>	210
<i>Summary</i>	220
<i>Multiple-Choice Questions</i>	220
11. Protection and Security	223
11.1 Introduction to Protection	223
11.2 Protection Goals	223
11.3 Protection Domain	223
11.4 Access Rights	224
11.5 Access Matrix Implementation	224
11.6 Need of Security	225
11.7 Authentication	225
11.8 Program Threats	226
11.9 System Threats	226
11.10 Threat Monitoring	227
<i>Summary</i>	227
<i>Multiple-Choice Questions</i>	228
Appendices	229
Appendix I : Case Study of Linux & Window XP OS	231
Appendix II : Case Study of Mobile OS	244
Appendix III : Set of Implemented Program in C/C++	249
Appendix IV : Previous University Question Papers	274
Appendix V : Bibliography	315
Index	317



Introduction

1.1 NEED OF OPERATING SYSTEM

Computer is an electronic device and its organization includes two main components, that is, hardware and software. All the hardware components work on the basis of software. The most basic or fundamental software is an operating system. An operating system is a program that manages the computer hardware. It also provides a basis for application program and acts as an intermediary between the computer user and computer hardware. It provides the working environment to execute a program in convenient and efficient manner. Operating system is responsible for overall functioning of machines from booting to resource management. An operating system does the following:

- Manages all the devices
- Provides interface between the user and the system
- Manages the memory allocation and deallocation to process
- Schedules the computational processes
- Manages the files and disks
- Protects the system and data
- Helps resources allocation and management
- Shares of resources in terms of time, space and CPU to achieve optimal utilization of resources.

1.2 OBJECTIVES OF OPERATING SYSTEM

An operating system is a program that controls the execution of application programs and acts as an interface between application and computer hardware. Operating system has three main objectives:

- **Convenience:** An operating system makes a computer more convenient to use.
- **Efficiency:** Allows the computer system resources to be used in efficient manner.
- **Ability to Evolve:** An operating system should be constructed in such a way as to permit effective development, testing and introduction of new system functions without interfering with service.

Apart from the above three objectives, other objectives are:

- Increase productivity of computing resources.
- Facilitate the use of hardware.
- Optimal scheduling of computational activities to ensure good performance of system.
- Creation of user-friendly environment for development and execution of program.

1.3 OPERATING SYSTEM AND ITS FUNCTIONS

An operating system is a program that acts as an intermediary between the user of the computer and the computer hardware. The purpose of an operating system is to provide an environment in which the user can execute programs in a convenient and efficient manner.

The operating system must ensure correct operation of the computer system.

Mainframe operating systems are designed primarily to optimize utilization of hardware. Personal computer (PC) operating systems support complex business applications, games and everything in between.

A computer system can be roughly divided into four components:

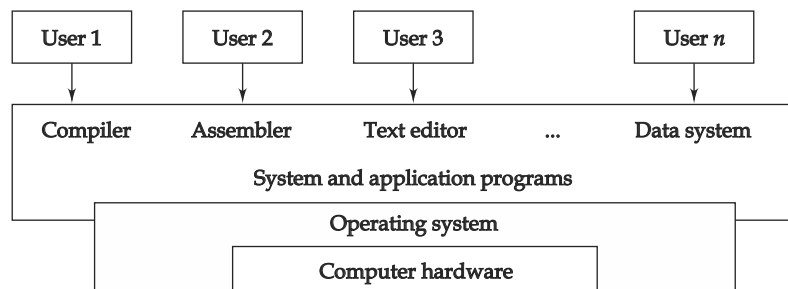


Fig. 1.1 Components of a computer system

1.4 VIEWPOINTS ON OPERATING SYSTEM

Operating system can be explored from two viewpoints:

1.4.1 User View

It varies by the interface being used. Most computer users sit in front of a PC, consisting of a monitor, keyboard, mouse and system unit. Such a system is designed for one user to monopolize its resources, to maximize the work that the user is performing. OS is mostly designed for *ease of use*. Some users sit at the terminal connected to a mainframe or minicomputer. Other users access the same computer through other terminals. These users share resources and may exchange information. OS is designed to maximize *resource-utilization*. Other users sit at workstations, connected to networks of other workstations and servers. These users have dedicated resource at their disposal, but they

also share resources such as networking and servers— file compute and print servers. Thus, OS is designed to *compromise between individual usability and resource utilization*.

1.4.2 System View

From the computer's viewpoint, the OS is a program that is most intimate with the hardware. We can view an OS as a *resource allocator*. A computer system has many sources, hardware and software that may be required to solve a problem.

OS acts as the manager of CPU time, memory space, file storage space, I/O devices and so on. Facing numerous and possibly conflicting requests for resources, the OS must decide how to allocate them to specific programs and users so that it can operate the computer system efficiently and fairly.

An OS is a control program. A control program manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

1.5 FUNCTIONS OF OPERATING SYSTEM

The primary goal of a operating system is to provide convenience and efficeincy.

It keeps a track of who is using which resource, to grant resource request, to account for usage, and to meditate conflicting requests from different programs and users.

The OS is also responsible for security, ensuring that unauthorised users do not access the system. Following are the various functions of an OS:

1. **Device Interfacing:** OS should try and hide the complexity of interfacing to devices from the user program and the user. It should also try and configure the device to start up rather than getting the user to set them up.
2. **File-System:** An OS can create and maintain a file system, where users can create, delete and move files around a structured file system.
Many systems organize files, in directories. In multi-users systems, these folders can have associated user.
3. **User:** This allows one or more users to log into a system. Thus, the OS must contain a user account database. Which contains user name, default home directory user passwords and user rights.
4. **Multiprocessing:** This allows two or more processes to be used at a time. Here the OS must decide if it can run the different processes on individual processors. It must also manage the common memory between processors.
5. **Memory Management:** This involves allocating, and often create a virtual memory for program. Paging which means organizing data so that the program data is loaded into pages of memory. Another method of managing memory is swapping. This involves swapping the content of the memory of disk storage.
6. **Multithreading:** Processes are often split into smaller tasks named threads. This thread allows smoother operations.

1.6 CLASSIFICATION OF OPERATING SYSTEMS

1.6.1 Batch Systems

Earlier the common input devices were card readers and tape drivers. The common output devices were line printers, tape drivers and card punches. The user did not interact directly with the computer systems. Rather, the user prepared a job which consisted of a program, data and some control information about the nature of job and submitted it to the computer operator. The job was usually in the form of punch cards.

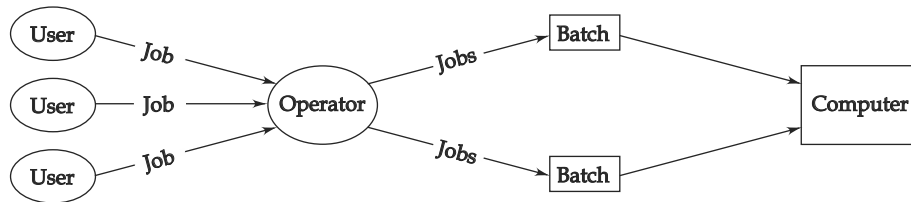


Fig. 1.2 Batch operating systems

1.6.2 Interactive Computer Systems

The interactive computer system is the system which provides direct communication between the user and computer. The user gives instructions to the operating system with input devices and waits for immediate response. When the OS finishes execution of one command, it adds the next control statement, not from the card reader but from the keyboard in the form of command. Most systems have interactive editor for entering the program and then interactive debugger for assigning in debugging the programs.

Example: Programs like word processors and spread sheet applications are interactive.

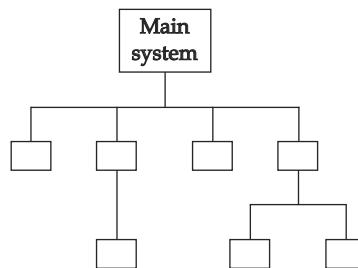


Fig. 1.3 Interactive system

1.6.3 Multiprogramming Operating Systems

When two or more programs are in memory at the same time sharing the processor it is referred to as multiprogramming operating system. Generally, in multiprogramming a single processor is shared. It increases CPU utilization by organising jobs so that CPU

always has one to execute. The operating system keeps several jobs in the memory at a time and picks and begins to execute one of the jobs in the memory.

It provides resource utilisation, but less user interaction.

Advantages

1. High CPU utilisation
2. It appears that many programs are allotted to CPU almost simultaneously.

Disadvantages

1. CPU scheduling is required
2. To accommodate many jobs in memory, memory management is required.

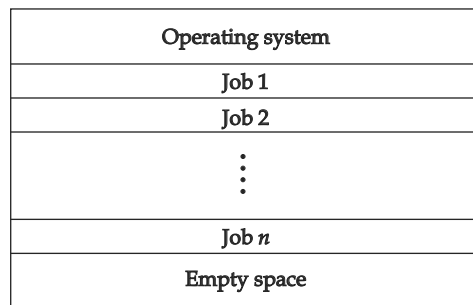


Fig. 1.4 Memory layout of OS

1.6.4 Multitasking Operating Systems

In multitasking operating system multiple programs can be run at a time. The operating system determines which application should run in what order and how much time is allowed for each application before giving another application a turn. Various processes share common processing resources.

Advantages

1. Several applications works simultaneously.
2. To the developer, the advantage is the ability to create applications that use more than one process and to create processes that use more than one thread of executing.

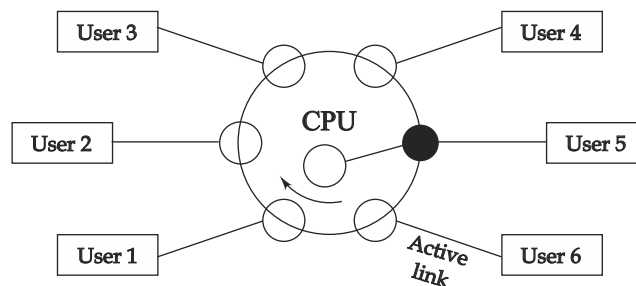


Fig. 1.5 Multitasking OS

1.6.5 Time Sharing Operating Systems

Time sharing is a technique which enables many people, located at various terminals to use a particular computer system simultaneously. Processor's time which is shared among multiple users simultaneously is termed time sharing. The main objective of time sharing system is to minimize response time.

Operating system uses CPU scheduling and multiprogramming to provide each user a small portion of time. Time sharing provides direct access to large number of users where CPU time is divided among all the users on scheduled basis. OS allocates time to each user and when time expires, it passes control to next user on the system. This short period of time is known as *time slice* or *time quantum*. In Fig. 1.4 time sharing system is also shown.

1.6.6 Real-Time Operating Systems

Real-time system is defined as a data processing system in which the time interval required to process and respond to input is so small, that it controls the environment. Real-time processing is always online while vice versa is not true always. So in this system response time is very less as compared to online processing. The time taken by the system to respond to an input and display the required updated information is known as *response time*.

Real-time systems are used where there are rigid time requirements on the operation of a processor. Real time system has well-defined fixed time constraints otherwise the system will fail.

Ward Real-time System: Ward real-time systems guarantee that critical tasks are complete on time. Its application areas are flight control computer (FCC), dynamic positioning system (DPS) found in marine vessels, etc.

Soft Real-time System: Soft real-time are less restrictive. Critical real-time task gets priority over other tasks and the systems retain priority until the jobs are complete. This has limited utility in comparison to ward real-time system, for example, multimedia, virtual reality, advanced scientific projects like sea exploration, etc.

1.6.7 Parallel System

Recently there is a trend towards multiprocessor systems. Such systems have more than one processor in close communication, sharing the computer bus, the clock and sometimes memory and peripheral devices. These are parallel systems. Some systems use both software and hardware duplication to ensure continuous operation. Such systems consist of two identical processors, each with its own local memory. The processors are connected by a bus. One processor is primary and other is backup. Two copies of each processor are kept—one is primary and other is backup. At some fixed checkpoints the state of information of the job is copied from primary to backup. The copy is activated and restarted from the most recent checkpoint. This is expensive and needs considerable hardware duplication.

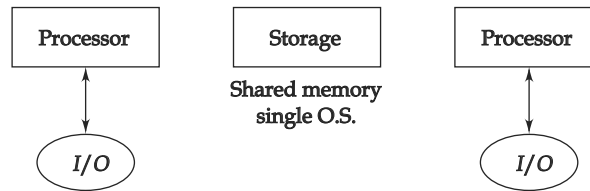


Fig. 1.6 Parallel system memory configuration

Advantages of a multiprocessor are parallel systems are:

1. Increased throughput
2. Save money by sharing peripherals, cabinets and power supplies.
3. Increased reliability
4. Fault-tolerant

1.6.8 Distributed Systems

Distributed systems use multiple central processors to serve multiple real-time applications and multiple users. Data processing jobs are distributed among the processors accordingly to which one can perform each job most efficiently.

The processors communicate with each other through various communication lines. These are loosely coupled systems or distributed systems. Processors may vary in size and function.

Advantages of distributed system:

1. With resource sharing the user at one site may be able to use the resources available at another.
2. Speed up exchange of data with one another.
3. If one site fails in distributed system, the remaining sites can potentially continue operating.
4. Better services to customers.
5. Reduction of load on the main computer.

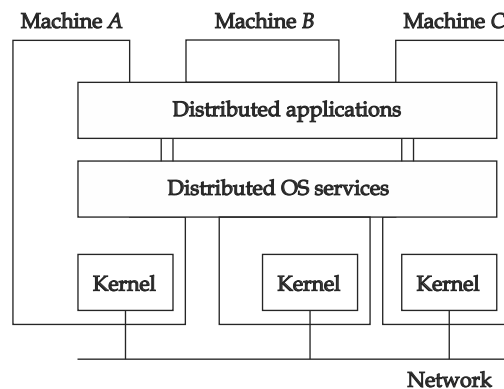


Fig. 1.7 Distributed system

1.6.9 Network Operating Systems

Network operating system runs on a server and provides the server the capability to manage data, users, groups, security, applications and other networking functions. The primary purpose of network operating system is to allow shared file and pointer access among multiple computers in a network, typically a LAN, private network or other network.

Advantages

1. Centralised servers are highly stable
2. Security is server managed
3. Upgrades to new technologies and hardware can be easily integrated into the system
4. Remote access to the server is possible from different locations and types of systems.

Disadvantages

1. High cost of buying and running a server
2. Dependency on a central location for most operations
3. Regular maintenance and updates are required.

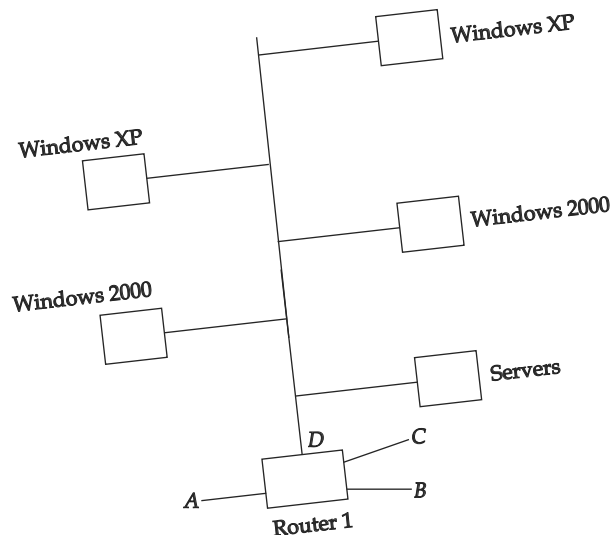


Fig. 1.8 Network operating system

1.7 COMPONENTS OF AN OPERATING SYSTEM

An operating system is a very complex system. To understand it, we will divide it into smaller pieces. An OS is made up of carefully defined inputs, outputs and functions.

It is made up of following system components:

1. Process management
2. Main memory management
3. File management.
4. I/O system management
5. Secondary storage management.
6. Networking
7. Protection system.
8. Command interpreter.

1.7.1 Process Management

- (i) A process can be thought of as a program in execution, e.g., a batch job is a process.
- (ii) A process needs certain resources including CPU time, memory, files and I/O devices to accomplish its task. These resources are either given to the process when it is created or allocated to it while it is running.
- (iii) When the process terminates, the OS will reclaim any reusable resources.
- (iv) A process is the unit of work in a system. Such a system consists of a collection of processes, some of which are OS processes and the rest of which are user processes.

1.7.2 Main Memory Management

- (i) Main memory is a large array of words or bytes, ranging in size from hundreds of thousands to hundreds of millions.
- (ii) Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.
- (iii) The central processor reads instructions from the main memory during the instruction-fetch-cycle and both reads and writes data from the main memory during the data-fetch-cycle.
- (iv) The operating system is responsible for the following activities in connection with memory management.
 - Keeps track of parts of memory which are currently being used and by whom.
 - Allocates and deallocates memory space as needed.
 - Decides which processors are to be loaded into memory when memory space becomes available.

1.7.3 File Management

- (i) Computers can store information on several different types of physical media such as magnetic tape, optical disk and magnetic disk.
- (ii) For convenient use of a complex system, the operating system provides a uniform logical view of information storage.
- (iii) The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file.

- (iv) A file is a collection of related information defined by its creator. Files represent programs and data. A file consists of a sequence of bits, bytes, lines or records whose meanings are defined by their creator.
- (v) Files are normally organised into directories to ease their use. When multiple users have access to files, it may be desirable to control by whom and in what ways files may be accessed.
- (vi) The OS is responsible for the following activities in connection with file management.
 - Creation and deletion of files
 - Creation and deletion of directories
 - The support of primitives for manipulating files and directories
 - The mapping of files onto secondary storage
 - The backup of file on stable storage media

1.7.4 I/O System Management

One of the purposes of an OS is to hide the peculiarities of specific hardware devices from the user.

The I/O subsystem consists of:

- A memory management component including buffering, caching and spooling
- A general device driver interface
- Drivers for specific hardware devices

1.7.5 Secondary Storage Management

- (i) Main memory is too small to accommodate all data and programs and its data and program are lost when power is lost, hence, the computer system must provide secondary storage to back up main memory.
- (ii) Most programs including compilers, assemblers, editors are stored on a disk until loaded into main memory and then use disk as both the source and destination of their processing.

The OS is responsible for the following activities in connection with disk management.

- Free space management
- Storage allocation
- Disk scheduling

1.7.6 Networking

- (i) A distributed system is a collection of processors and does not share memory, peripheral devices or a clock.
- (ii) Each processor has its local memory and clock and the processors communicate with one another through various communication lines like high speed buses or telephone lines.
- (iii) The communication network design must consider routing and connection strategies and the problems of connection and security.

1.7.7 Protection System

- (i) If a computer system has multiple users and allows the concurrent execution of multiple processors, then the various processes must be protected from each other's activity.
- (ii) Protection refers to the mechanism for controlling the access of programs; processes, or users to the resources defined by a computer system.

1.8 OPERATING SYSTEM STRUCTURE

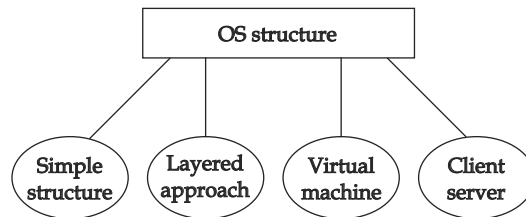


Fig. 1.9 OS structure

1.8.1 Simple Structure

- (i) An OS is written as a collection of procedures, each of which can call any of the other one whenever it needs.
- (ii) Basic structure of the OS can be divided into 3 parts:
 - A main program that invokes the requested service procedure
 - A set of service procedures that carry out system calls
 - A set of utility procedures that help the service procedure
- (iii) For each system call, there is one service procedure, that take care of it. The utility procedures do things that are really needed by several service procedures.

For example: MS DOS structure.

Structure of MS-DOS is shown in Fig. 1.10.

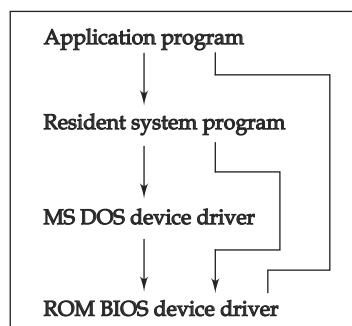


Fig. 1.10 Structure of MS-DOS

1.8.2 Command Interpreter System

- (i) Command interpreter system is the interface between the user and the OS. Some OSs include command interpreter in the kernel.
- (ii) Command line interpreter is often known as shell. OSs are frequently differentiated in the area of the shell.
- (iii) One style of user friendly interface is the mouse based window and menu system is the Macintosh and Mirosoft Windwos.

The problems in this system architecture are:

1. Interfaces and levels of functionality are not well separated.
2. Application programs are able to access the basic I/O routines to write directly to display and disk drive.
3. Leaves MS DOS vulnerable to malicious programs.
4. Limited by hardware.
5. Has 2 separate parts:
 - (a) Kernel
 - (b) system programs

This is for UNIX/LINUX environment.

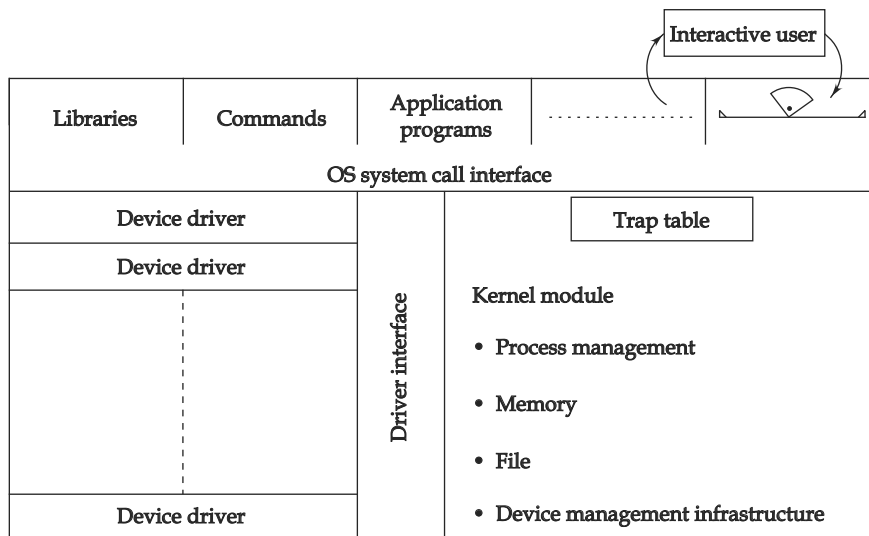


Fig. 1.11 Structure of Unix/Linux

1.8.3 Layered Approach

Modulation of a system can be done in many ways. One method is the layered approach, in which the operating system is broken up into a number of layers (or levels), each built on top of lower layers. The bottom layer (layer 0) is the hardware, (layer N) is the user interface.

The main advantage of the layered approach is modularity. The layers are selected such that each uses functions (or operations) and services of only lower level layers. This approach simplifies debugging and system verification.

The first layer can be debugged without any concern for the rest of the system, because by definition, it uses only the basic hardware (which is assumed correct) to implement its functions. Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged and so on.

If an error is found during debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged. Thus, the design and implementation of the system are simplified when the system is broken down into layers.

Each layer is implemented with only those operations provided by lower-level layers. A layer does not need to know how these operations are implemented, it needs to know only what these operations do. Hence, each layer hides the existence of certain data structures, operations and hardware from higher level layers.

Other requirements may not be so obvious. The backing-store driver would normally be above the CPU scheduler because the driver may need to wait for J/O and the CPU can be rescheduled during this time. However, on a large system, the CPU scheduler may have more information about all the active processes than can fit in memory. Therefore, this information may need to be swapped in and out of memory, requiring the backing store driver routine to be below the CPU scheduler.

Fewer layers with more functionality are designed, providing most of the advantages of modularized code while avoiding the problems of layer definition and interaction.

For example, consider the history of windows NT. The first release had a highly layer oriented organisation, however, it delivered low performance compared to that of Windows 95. Window NT 4.0 partially redressed the performance problem by moving layers from user space to kernel space and more closely integrating them.

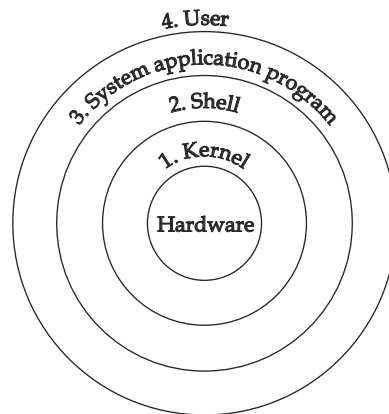


Fig. 1.12 Layered approach

1.8.4 Virtual Machine

Virtual Machine developed by IBM originally called CP/CMS and later renamed as VM/370 was based on two things: multiprogramming and an extended machine with a more convenient interface than the base hardware.

The heart of the system, known as virtual machine monitor, runs on the base hardware and does multiprogramming providing several virtual machines to the next layer.

Virtual machines are exact copies of base hardware including kernel user mode I/O interrupts. Different virtual machines run different operating systems on the single interactive system called conventional monitor system (CMS). When a CMS program executes a system call, the call is trapped to the operating in its own VM. CMS then issues the normal hardware I/O instruction for reading its virtual disk. These instructions are trapped by VM/370.

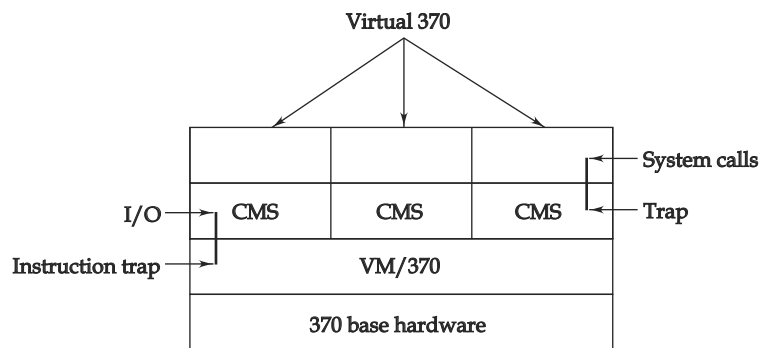


Fig. 1.13 Virtual M/C

Advantages

- Provides security and privacy
- Software development program.

1.8.5 Client-Server Architecture

PC's have become faster, more powerful and cheaper as designer have shifted away centralized system architecture. Terminals connected to centralized are now being supplanted by PCs.

Correspondingly, user interface functionality once handled directly by centralized system is increasingly handled by PCs. As a result, many of today's system act as a server system to satisfy request generated by client-system. This form of specialized distributed system is called a client-server system.

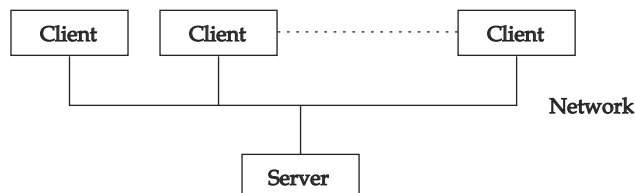


Fig. 1.14 Client server architecture

Server system can be broadly categorised as computer servers and file servers.

- The computer server system provides an interface to which a client can send a request to perform an action, in response the server executes the action/request and sends back the result to the client.
- The file server system provides a file-system interface, where clients can create, update, read and delete files.

Advantages

- Adaptable to distributed system.
- Minimal amount of mechanism into kernel.
- Still leave decision power on server.

Disadvantage

- Some operations are difficult in user space.

1.9 OPERATING SYSTEM SERVICES

Operating system services and facilities can be grouped into the following areas:

1. Program development

- (i) An operating system provides editors and debuggers to assist the programmer in creating programs.
- (ii) Usually, these services are in the form of utility programs and not strictly part of core operating system. They are supplied with operating system and referred as application program development tools.

2. Program execution

- (i) A number of tasks need to be performed to execute a program. Instructions and data must be loaded into the main memory. I/O devices and files must be initialized.
- (ii) The operating system handles these scheduling duties for the user.

3. Access to I/O devices

- (i) Each I/O device requires its own peculiar set of instructions for operations.
- (ii) OS provides a uniform interface that hides these details, so that the programmer can access such devices using simple reads and writes.

4. Controlled access to filter

- (i) In case of file access, OS should provide a detailed understanding of not only the nature of I/O devices, but also the structure of data contained in the files on the storage medium.
- (ii) In case of a system with multiple users, the OS may provide protection mechanism to control access to files.

5. System access

- (i) In case of shared or public systems, the OS controls access to the system as a whole.

- (ii) The access function must provide protection of resources and data from unauthorized users.

6. Error detection and response

- (i) Various types of errors can occur while a computer system is running which includes internal and external hardware errors.
- (ii) In each case, OS must provide a response that clears error condition with least impact on the running applications.

7. Accounting

- (i) A good OS collects a usage statistics for various resources and monitors performance parameters.
- (ii) On any system, this information is useful in anticipating need for future enhancements.

1.10 KERNELS

The kernel is the heart of the operating system. It interacts with hardware and most of the tasks like memory management, task scheduling, file management, I/O management etc. There are two types of kernels: monolithic kernel and microlithic kernel.

1.10.1 Monolithic Kernel

Monolithic kernel OS can be considered or a single large static binary file process running entirely in a single address space. Basic OS services such as process management, memory management, interrupt handling, I/O communication, file systems, device drivers, etc., all run in kernel space. Entire services are loaded on boot up and reside in memory and work is done using system calls. Linux is example of monolithic kernel based OS.

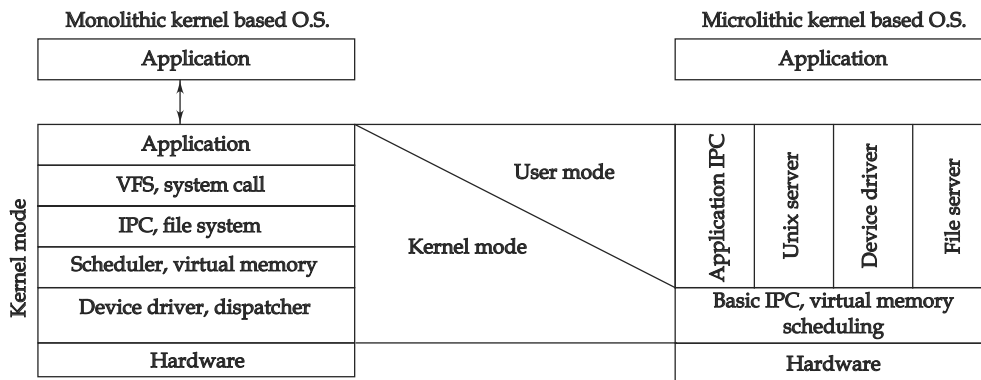


Fig. 1.15 Monolithic vs microkernel kernels

1.10.2 Microkernel

The idea behind micro kernel OS is to reduce the kernel to only basic process communication and I/O control and let other system services use the user space. These

processors are called servers and kept separate in different address spaces. Contrary to monolithic OS where services are directly involved and invoked, communication in microkernel is done via message passing. Mac OS and Nin NT are examples.

Comparison between Monolithic and Microlithic Kernel

Table 1.1 Monolithic vs Microlithic

Monolithic	Microlithic
1. Kernel size is large	1. Kernel size is small
2. OS is complex to design	2. OS is easy to design, and install
3. Different modules can interact less with each other	3. Contains different modules on which work is performed
4. New modules can be dynamically linked, i.e., at run time	4. These can communicate with those modules by linking them
5. All OS services run along main kernel set in same memory architecture	5. They are used in embedded systems
6. Ex. UNIX, MS-DOS, SOLARIS	6. Ex. AIX, OS, Amega OS, Symbian OS

1.10.3 Re-entrant Kernel

A re-entrant kernel enables processes to give away the CPU while in kernel mode, not hindering other processes from also entering kernel mode.

A typical use case is IO wait. The process wants to read a file. It calls a kernel function for this. Inside the kernel function, the disk controller is asked for data. Getting the data will take some time and the function is blocked during that time.

With a re-entrant kernel, the scheduler will assign the CPU to another process unit an interrupt from the disk controller indicates that the data is available and our thread can be resumed.

This process can still access IO, like user i/P. The system stays responsive and CPU time waste due to IO wait is reduced. This is pretty much standard for today's desktop OS.

If the kernel is not re-entrant, a process can only be suspended while it is in user mode because all kernel threads share the same memory and corruption would occur if execution would jump between them arbitrarily.

All UNIX kernels are re-entrants means several processes may be executing in kernel mode at the same time. A one way to provide re-entrance is to write functions so that they modify any local variables and do not alter global data structures. Such functions are called re-entrant functions.

If a hardware interrupt occurs, a re-entrant kernel is able to suspend the current running process even if that process is in kernel mode. This capability is very important because it improves the throughput of the devices controllers that issue interrupts

A kernel control path denotes the sequence of instructions executed by the kernel to handle a system call, an exception or an interrupt.

Summary

- In this chapter we have studied about the operating system, its objectives, functions and the basic structure.
- The operating system acts as a virtual machine, a resource manager, a service provider and as a security agent.
- Similar jobs are batched together and executed in groups. It is called batch processing system. The main function of batch processing system is to automatically keep executing one job to the next job in the batch.
- Multiprogramming operating system is used to increase the system utilization efficiency.
- The operating system allowing execution of multiple tasks at one time, is classified as a multiprocessing operating system.
- Real-time operating system is used when there are rigid time requirements on the operations of a processor or the flow of data. Real time operating system has well defined, fixed time constraints.
- System calls provide an interface to the service mode available by an operating system. These calls are generally available as routines written in C and C++ or these calls are generally available as assembly language instruction.
- Operating system architecture based on layered approach consists of multiple layers each of which is built on top of lower layers.
- The monolithic approach defines a high level virtual interface over the hardware, with a set of primitives or system calls to implement operating system services such as process manager concurrency, memory management, etc.
- Microkernel provides basic functionality that allows the execution of servers.
- A re-entrant kernel enable process to give away the CPU while in kernel mode, not hindering other process from entering kernel mode. All UNIX kernels are re-entrants.

Multiple-Choice Questions

1. Which technique was introduced because a single job could not keep both the CPU and the I/O devices but
 - (a) Time-sharing
 - (b) Spooling
 - (c) Preemptive scheduling
 - (d) Multiprogramming
2. CPU performance is measured through
 - (a) Throughput
 - (b) MHz
 - (c) Flaps
 - (d) None of the above
3. PCB stands for
 - (a) Program Control Block
 - (b) Process Control Block
 - (c) Process Communication Block
 - (d) None of the above

-
4. Software is a program that directs the overall operation of the computer facilitates its use and interacts with the user. What are the different types of this software.
 - (a) Operating system
 - (b) Language compiler
 - (c) Utilities
 - (d) All of the above
 5. Which of the following is contained in process control block (PCB)?
 - (a) Process number
 - (b) List of open files.
 - (c) Memory limits
 - (d) All of the above
 6. Supercomputers typically employ
 - (a) Real-time OS
 - (b) Multiprocessor OS
 - (c) Desktop OS
 - (d) None of the above
 7. Real-time systems are
 - (a) Primarily used on mainframe computers
 - (b) Used for monitoring events as they occur
 - (c) Used for program development
 - (d) Used for real time interactive users
 8. In _____ OS, the response time is very critical.
 - (a) Multitasking
 - (b) Batch
 - (c) Online
 - (d) Real-time
 9. Which of the following is not an advantage of multiprogramming?
 - (a) Increased throughput
 - (b) Shorter response time
 - (c) Decreased operating system overhead
 - (d) Ability to assign priorities to jobs.
 10. The program is known as _____ which interacts with the inner part of called kernel.
 - (a) compiler
 - (b) device driver
 - (c) protocol
 - (d) shell
 11. Information about a process is maintained in a
 - (a) Stack
 - (b) Translation Lookaside Buffer
 - (c) Process Control Block
 - (d) Program Control Block
 12. UNIX operating system is a
 - (a) Time sharing operating system
 - (b) Multi-user operating system
 - (c) Multi-tasking operating system
 - (d) All the above
 13. Distributed OS works on the _____ principle.
 - (a) File foundation
 - (b) Single system image
 - (c) Multisystem image
 - (d) Networking image

14. A _____ is a software that manages the time of a microprocessor to ensure that all time critical events are processed as efficiently as possible this software allows the system activities to be divided into multiple independent elements called tasks.
- (a) Kernel (b) Shell
(c) Processor (d) Device driver
15. The one program running at all times on the computer is called
- (a) Kernel (b) Device Controller
(c) Shell (d) Multiprocessor
16. There are types of multiprocessing
- (a) 3 (b) 4
(c) 2 (d) None of the above

Answers

- | | | | | | |
|---------|---------|---------|---------|---------|---------|
| 1. (d) | 2. (a) | 3. (b) | 4. (d) | 5. (d) | 6. (b) |
| 7. (b) | 8. (d) | 9. (c) | 10. (d) | 11. (c) | 12. (d) |
| 13. (b) | 14. (a) | 15. (a) | 16. (c) | | |

2

Processes

2.1 WHAT IS A PROCESS?

Introduction

Earlier the computers could execute only one program at a time. This program had complete control of the system and had access to all the system. But now, several programs can be loaded into the memory and can be executed concurrently.

It leads to compartmentalization of the various programs and these needs resulted in the notion of a process, which is program in execution. A process is the unit of work in a sharing system.

A system consists of collection of operating system processes, executing system code and user processes executing user code. All these processes can execute concurrently, with the CPU multiplexed among them.

It increases the productivity of the computer.

The Process

In general, we say that a process is the program in execution. A process is more than a program code which is sometimes known as text section. In computing, a process is an instance of a computer program that is being executed. It contains the program code and its current activity. Depending on the operating system, a process may be made up of multiple threads of execution that execute instructing concurrently.

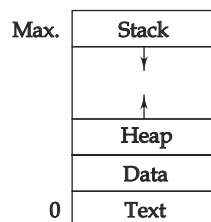


Fig. 2.1 Process in memory

A process generally also includes the process stack, which contains temporary data (such as function parameters, retains addresses and local variables) and a data section

which contains global variables. The heap is also included in the process, which is used for the dynamic allocation of the memory during the runtime.

Program by itself is not a program. It is a process, a passive entity such as file containing a list of instructions stored on disk. On the other hand, a process is an active entity with a program counter specifying the next instruction to be executed and a set of associated resources.

A program becomes a process when an executable file is loaded into memory. Two common techniques for loading executable files are double clicking the icon representing the executable file and entering the name of the executable file on the command line.

2.2 PROCESS STATES

The process as a whole consists of five states. When the process starts executing then the process changes its state. The state of process is defined in part by the current activity of that process.

The different states of process are:

1. **NEW:** The state 'NEW' is the first state in which the process is being created.
2. **Running:** The running state is the state in which instructions are being executed.
3. **Waiting:** In this state the process is waiting for some event to occur. These events may include input/output completion or reception of a signal.
4. **Ready:** In this state the process is waiting for some assigning of a process to a processor.
5. **Terminated:** In this state the process has finished execution.

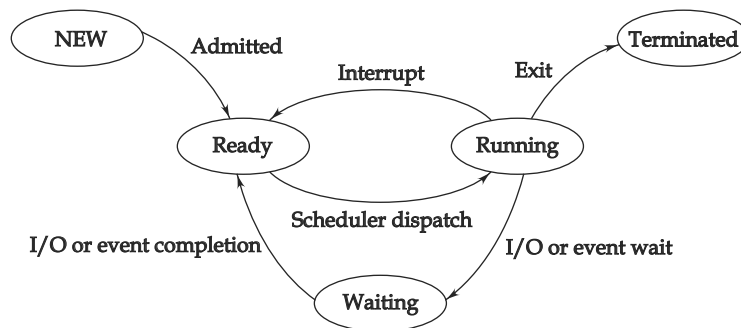


Fig. 2.2 Process states

The names given to various states are arbitrary, and can vary across operating systems. The states that they are present are found on all systems. It is important to realize that only one process can be run on any processor at any instant. Many processes may be ready and waiting.

2.3 PROCESS CONTROL BLOCK

Each process is represented in the operating system by a process control block. The process control block is also known as task control block.

When a process is fetched from job pool and it enters the ready queue, a PCB is assigned to it. Process control block is used to store the information about the processes and it is also called the data structure, which stores the information about the process. The information of the process is used by the CPU at run time.

The information stored in the PCB are the following:

1. **Process State:** The state may be new, ready, running, waiting, halted and so on.
2. **Program Counter:** The counter indicates the address of the next instruction to be executed for that process.
3. **CPU Registers:** The registers vary in number and type, depending on the computer architecture. They include accumulators, index, registers, stack pointers and general-purpose registers plus any condition code information.
4. **CPU Scheduling Information:** This information includes a process priority, pointers to scheduling queues and any other scheduling parameters.
5. **Memory-management Information:** This information may include such information as the value of the base and limit registers the page tables, or the segment tables, depending on the memory system used by the operating system.
6. **Accounting Information:** This information includes the amount of CPU and real-time used, time limits, account numbers, job or process numbers and so on.
7. **I/O Status Information:** This information includes the list of I/O devices allocated to the process, a list to open files and so on.

Process state
Process number
Program counter
Registers
Memory limits
List of open files
• • •

Fig. 2.3 Process control block (PCB)

2.4 THREADS

A thread is a flow of execution through the process code, with its own program counter, system registers and stack. Threads are a popular way to improve application performance through parallelism. A thread is sometimes called a light weight process.

Threads represent a software approach to improve performance of an operating system by reducing the overhead, thread is equivalent to a classical approach. Each thread belongs to exactly one process and no thread can exist outside a process. Each thread represents a separate flow of control.

Threads have been successfully used in implementing network servers. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors.

2.4.1 Implementation of Threads

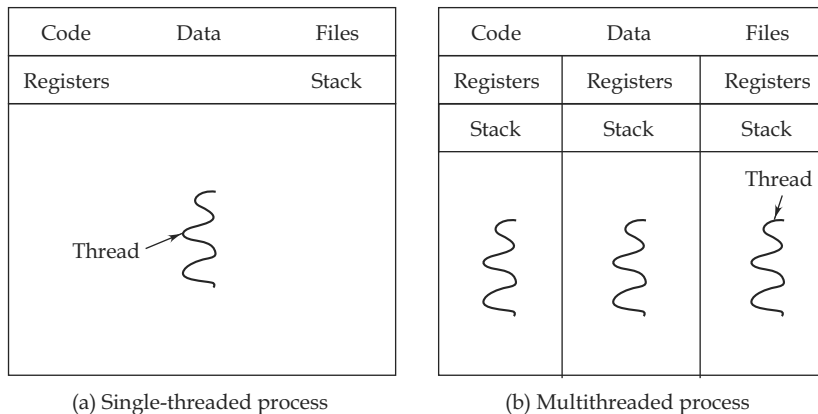


Fig. 2.4

2.4.2 Threads in Solaris

The solaris executes kernel threads for kernel related tasks, such as interrupt handling, memory page management, device drivers, etc. For user process execution, kernel threads have a corresponding light weight process. These kernel threads are scheduled for execution by the kernel on behalf of other processes.

In solaris, the real thread is the kernel thread. The kernel threads can execute kernel related tasks and user-related tasks. For executing user related tasks the kernel threads need a data structure called LWP to provide the environment. So the user thread and LWP are not execution units or thread at all. They are just data structures in the system. The user thread is just the user task needed to be executed which includes the user data and code. The LWP is the kernel data structure providing the kernel information. The real execution unit or flow is kernel thread.

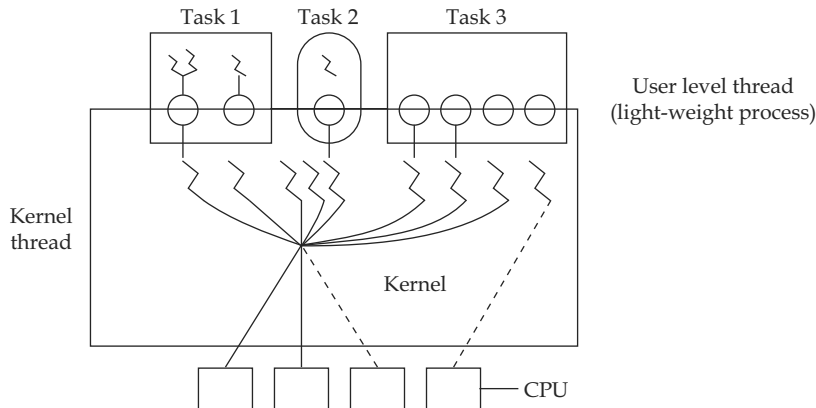


Fig. 2.5 Threads in solaris

Comparison between Process and Thread

Table 2.1 Process vs Thread

No.	Process	Thread
1.	Process is a program in execution. It is heavy weight or resource intensive.	Threads are flow of execution through process code with its own program counter, system registers are stack. Thread is light weight taking lesser resources than a process.
2.	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3.	In multiple processing environment each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4.	If one process is blocked then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, second thread in the same task can run.
5.	In multiprocessing each process operates independently of the others.	One thread can read, write or change another thread's data.

2.4.3 P-Threads

The P-thread or POSIX thread library is a set of functions that enable multiple threads of execution to do multiple tasks simultaneously. It is developed by the IEEE committee in charge of specifying a portable.

- It is fairly low level → you create the thread, you tell it what to do, you wait for it to finish.
- “Communication” is thorough shared memory. A P-threads program cannot span multiple machines.
- No explicit further binding is specified, but you can write a “driven” program in C which makes calls to your further routines.

2.5 SCHEDULERS

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run.

A process comes across various scheduling queues in its life-time. The OS must select the process from these scheduling queues and this selection process is carried out by some appropriate scheduler.

Schedulers are of 3 types:

1. Long-term scheduler
2. Short-term scheduler
3. Medium-term scheduler

2.5.1 Long-term Schedulers

- These are also called job schedulers. Long-term scheduler determines which programs are admitted to the system for processing. It selects processes from pool of processes and loads them into memory for execution.
- The primary distinction between these schedules lies in the frequency of execution. Long-term schedulers execute much less frequently. Minutes may separate creation of one new process and the next.
- They control the degree of multiprogramming (the number of processes in memory). If degree of multiprogramming is stable, then average rate of process creation must be equal to average departure rate of processes leaving the system. Long term schedulers can afford to take more time to decide which process should be selected for execution.
- The primary objective of a job scheduler is to provide a balanced mix of jobs related to I/O bound processes and CPU processes. If all the processes are I/O bound, the ready queue will almost always be empty. If all processes are CPU bound, I/O waiting queue will almost always be empty. Thus, system will give best performance when there is a proper balance between them.
- On some systems (like UNIX and Microsoft Windows), the long-term scheduler may not be available or minimal. Time-sharing OS has no long-term schedulers. When process changes the state from new to ready, then there is use of long-term schedulers.

2.5.2 Short-term Scheduler

- It is also called a CPU scheduler. It selects the processes that are ready to be executed and it allocates the CPU to one of the selected processes.
- Short-term schedulers have to select new processes for CPU more frequently. It is estimated that, short-term schedules executes atleast once every 100 mil/sec. Because of short life of processes, it has to execute many times hence it should be fast.

- If all processes are I/O bound, ready queue will almost always be empty, and short-term scheduler will have little to do.

2.5.3 Medium-term Schedulers

- Medium-term schedulers are part of the swapping. It removes the process from the memory. It reduces the degree of multiprogramming.
- Medium-term scheduler is in-charge of handling the swapped-out processes.

The diagram for medium-term scheduler is shown in Fig. 2.6:

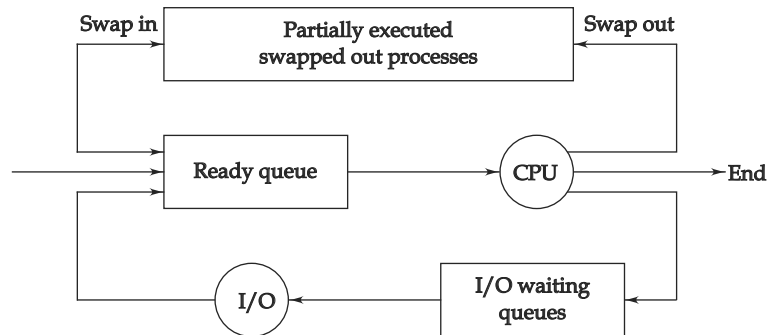


Fig. 2.6 Addition of medium-term scheduler to queuing.

Table 2.2 Types of Schedulers

Long-term scheduler	Short-term scheduler	Medium-term scheduler
It is a job scheduler. It selects processes from a pool and loads them into memory for execution.	It is a CPU scheduler. It selects the process ready to be executed and allocates the CPU to one of them.	It is a process swapping scheduler. The process initially is swapped out, and is later swapped in by this scheduler.
Speed (or frequency of execution) is less than short-term scheduler.	Speed is faster among other two schedulers.	Speed is in between both other two schedulers.
It controls degree of multiprogramming.	It controls/provides less control over degree of multiprogramming.	It reduces degree of multiprogramming.
It is almost absent or minimal in time sharing systems.	It is also minimal in time sharing systems.	It is a part of time sharing systems.

2.6 OPERATIONS OF PROCESSES

The processes in most systems can execute concurrently and they can be created and terminated dynamically.

2.6.1 Process Creation

- One process can create several sub-processes using 'create-process system call' during execution. Creating process is parent process and sub-processes are called child processes.

- OS identifies processes according to a unique process identifier, which is an integer number (or pid).
- It forms a tree-like structure as shown in Fig. 2.7.

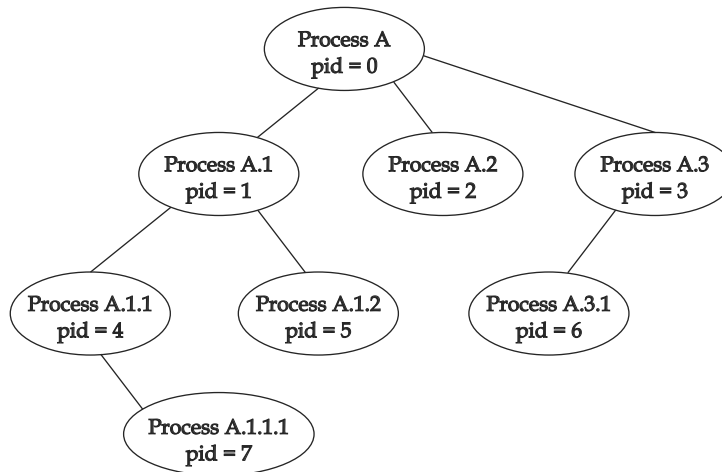


Fig. 2.7 Process creation

- In general, a process may need some resource like CPU time, memory, I/O devices, etc., to accomplish its task. When a process creates a sub-process, it may be able to obtain its resources directly from OS or it may require some other sup-process to get executed and hence provide it the resources.
- As parent process may have to distribute its resources to different child processes, this restricts the process to overload the system by creating many sub-processes.

2.6.2 Process Termination

- A process gets terminated, when it finishes execution of its last statement and asks OS to delete it using 'exit ()' system call.
- When a process finishes its execution it returns a status value (an integer value) to its parent process (by 'wait ()' system call).
- All the resources of the processes like physical and virtual memory, open files, I/O buffers, etc., are deallocated by OS.
- **The termination of process may occur due to the following circumstances.**
 - (i) Process finishes its execution
 - (ii) One process can cause termination of another process by using the required system call. Such a system call can be involved only by parent of the process that needs to be terminated.
 - (iii) Users could arbitrarily kill the jobs.

- **Parent process can terminate the execution of the process due to the following reasons**
 - (i) Task assigned to the child is no longer required.
 - (ii) The parent process is executing. In such a situation OS will not allow child process to remain executing.
 - (iii) Child process has exceeded its usage of some of the resources that it has been allocated.

2.7 INTERPROCESS COMMUNICATION (IPC)

What is IPC?

Interprocess Communication (IPC) is a set of programming interfaces that allow a programmer to coordinate activities among different program processes that can run concurrently in an OS. This allows a program to handle many user requests at the same time. Since even a single user request may result in multiple process running in an OS on user's behalf, the processes need to communicate with each other.

The IPC interface makes this possible. Commonly IPC methods used are shared memory & message passing. Message passing system is more useful and widely used. These mechanisms allow processes to share data and information.

Processes executing in OS can be either independent processes or cooperating processes.

Independent processes: The process that can't affect or be affected by other processes executing in the system. These processes don't share data with other processes.

Cooperating processes: The processes that can affect or be affected by other processes executing in the system. These processes can share data with other processes.

So, any process that shares data (communicate with other processes are cooperating processes. The reasons for providing process cooperation are:

- (i) **Information sharing:** Many users may be interested in similar type of information (like shared files), so it is important to allow, to access such information.
- (ii) **Computation speed-up:** One way to speed-up execution process is breaking a task into sub-tasks. Each of these sub-tasks can be executed in parallel and we can obtain the desired output earlier.
- (iii) **Modularity:** We are able to design modular system, i.e., dividing system functions into separate process or threads.
- (iv) **Convenience:** We are able to provide the user a flexibility to use multiple applications simultaneously like editing, printing, etc.

Cooperating processes require interprocess communication (IPC) mechanism that allows them to share data and information

There are mainly 2 models of IPC:

1. Shared memory
2. Message passing

2.7.1 Shared Memory System

- IPU using shared memory requires communicating process to establish a region of shared memory.
- Shared memory region resides in the address space of the process creating shared memory segment and other processes wish to communicate using that shared-memory must attach it to their address space.
- In this case both the processes have to allow one process to access another process's memory.
- They can then share data by reading and writing data in shared areas.

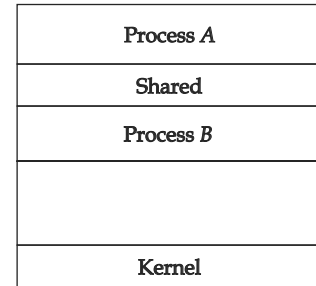


Fig. 2.8 Communication model: Shared memory

2.7.2 Message Passing System

- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing same address space and is very useful in distributed system environment.
- This facility provides at least two operations: send (message) and receive (message)
- A balance has to be made while deciding the message length should be made fixed or not. If fixed message length is kept, this makes task of the programmer more difficult though implementation is straightforward and vice versa.

The diagram of message passing system is shown in Fig. 2.9.

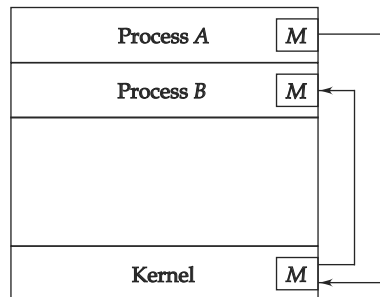


Fig. 2.9 Communication model: Message passing

The message passing system is a much better and widely used system for sharing information and data. So, we discuss this system in following section:

- If processes P and Q want to communicate they must send messages and receive messages; for this to happen a *communication link* must exist between the processes.
- The links can be logically implemented in the following ways:
 - (a) Direct or indirect communication
 - (b) Synchronous or asynchronous communication
 - (c) Automatic or explicit buffering

These features are implemented in the following manner:

(1) Naming

- Process, to communicate must have a way to refer to each other done either by direct or indirect communication.

(i) Direct communication:

- Each process that wants to communicate must explicitly name the recipient or sender of the communication.

For this purpose `send()` and `receive()` functions are defined as:

⇒ `send (P, message)` – send message to process P

⇒ `receive (Q, message)` – receive message from Q

- Communication link established has the following properties:
 - A link is established automatically between every pair of processes that want to communicate.
 - A link is associated with exactly two processes.
 - Between each pair of processes, there is exactly one link.

• Disadvantage

The scheme's (shown above for sending & receiving disadvantage is the limited modularity of resulting process definitions. Changing the identifier of a process may necessitate examining all other process definitions.

All references to the old identifier must be found so that they can be modified to the new identifier.

(ii) Indirect communication

- In this method of communication, messages are sent and received from *mailboxes* or *ports*.
- A mailbox can be viewed as an object into which messages can be placed by processes and also from which messages can be removed. Each mailbox has a unique identification.
- Two processes can communicate only if the processes have a shared mailbox.

The functions used are:

⇒ `send (A, message)` – send message to mailbox A

⇒ `received (A, message)` – receiver message from mailbox

- Communication link established has the following properties:
 - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
 - A link may be associated with more than 2 processes.
 - Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox.

(2) Synchronization

- Communication between processes takes place through calling `send()` and `receive()` functions

- Message passing may be either blocking or non-blocking also known as synchronous and asynchronous
 - **Blocking send:** The sending process is blocked until message is received by the mailbox or receiving process.
 - **Non-blocking send:** The sending process sends message and resumes operation.
 - **Blocking receiver:** The receiver blocks until a message is received.
 - **Non-blocking receive:** The receiver retrieves either a valid message or a null.

(3) Buffering

- When communication is direct or indirect, messages exchanged by communicating processes reside in temporary queues.
These queues can be implemented in 3 ways:

(i) Zero Capacity

- Queue has maximum length of zero
- Thus, link can't have message waiting
- In this, the sender must block until the recipient receives the message.

(ii) Bounded Capacity

- Queue has a finite length of 'n'.
- Thus, at most 'n' messages can reside in it.
- If queue is not full when new message is sent, message is placed in queue and the sender can continue execution without waiting.

(iii) Unbounded Capacity

- Queue length is potentially infinite
- Thus, any number of messages can wait in it
- The sender never blocks.

Summary

- A fundamental function of a separating system is the execution of a program and an executing program is known as process.
- Process is a dynamic entity, that is, a program in execution. A process is a sequence of instruction executions.
- Transition means change of one state of a process to the other state. A process state transition is a dynamic change in the state of process.
- A process has seven states: new, ready, running, blocked, blocked-suspended, ready-suspended and terminated.
- The process changes its state when there is an event causing a state transition.
- The state transitions are admit (new to ready), dispatch (ready to running), I/O wait (running to blocked), suspended (blocked to blocked-suspended), I/O complete (blocked to ready or blocked-suspended to ready-suspended), activate (ready-suspended to ready), and exit (running to terminated).

- When a job arrives in the ready queue, it becomes a process.
- When a process in the ready queue is selected for execution, it is known as process scheduling.
- The process of saving the status of an interrupted process and loading the status of the scheduled process is known as context switching.
- Context switching is a pure overhead from viewpoint of processor time because during context switching no execution is done.
- When a running process is interrupted and the OS assigns another process to the running process and transfers control to it, it is known as process switching.
- A scheduler is a component of the OS that performs the job of scheduling at various levels.

Multiple-Choice Questions

1. A Process Control Block (PCB) does not contain which of the following:
(a) Code (b) Strack
(c) Heap (d) Data
(e) Program Counter (f) Process State
(g) I/O status information (h) bootstrap program
2. The number of processes completed per unit time is known as
(a) Output (b) Throughput
(c) Efficiency (d) Capacity
3. The state of a process is defined by
(a) the final activity of the process
(b) the activity just executed by the process
(c) the activity next be executed by the process
(d) the current activity of the process
4. Which of the following is not the state of a process?
(a) New (b) Old
(c) Waiting (d) Running
(e) Ready (f) Terminated
5. The process control block is
(a) Process type variable (b) Data structure
(c) A secondary storage section (d) A Block in memory
6. The entry of all the PCBs of the current processes is in
(a) Process register (b) Program counter
(c) Process table (d) Process unit

7. The degree of multiprogramming is
 - (a) the number of processes executed per unit time
 - (b) the number of processes in the ready queue
 - (c) the number of processes in the I/O queue
 - (d) the number of processes in memory
8. A single thread of control allows the process to perform
 - (a) only one task at a time
 - (b) multiple tasks at a time
 - (c) All of these
9. The objective of multiprogramming is to: (choose two)
 - (a) Have some process running at all times
 - (b) Have multiple programs waiting in a queue ready to run
 - (c) To minimize CPU utilization
 - (d) To maximize CPU utilization
10. The systems which allows only one process execution at a time, are called
 - (a) uniprogramming systems
 - (b) uniprocessing systems
 - (c) unitasking systems
 - (d) none of the above
11. In an operating system, each process has its own
 - (a) address space and global variables
 - (b) open files
 - (c) pending alarms, signals and signal handlers
 - (d) all of the above
12. In UNIX, which system call creates the new process?
 - (a) fork
 - (b) create
 - (c) new
 - (d) none of the above
13. A process can be terminated due to
 - (a) normal exit
 - (b) fatal error
 - (c) killed by another process
 - (d) all of the above
14. What is the ready state of a process?
 - (a) when process is scheduled to run after some execution
 - (b) when process is unable to run until some task has been completed
 - (c) when process is using the CPU
 - (d) none of the above
15. What is interprocess communication?
 - (a) Communication within the process
 - (b) Communication between two process
 - (c) Communication between two threads of same process
 - (d) None of the above

-
16. A set of processes is deadlock if
 (a) each process is blocked and will remain so forever
 (b) each process is terminated
 (c) all processes are trying to kill each other
 (d) none of the above
17. A process stack does not contain
 (a) function parameters (b) local variables
 (c) return addresses (d) PID of child process
18. Which system call returns the process identifier of a terminated child?
 (a) Wait (b) exit
 (c) fork (d) get
19. The address of the next instruction to be executed by the current process is provided by the
 (a) CPU registers (b) program counter
 (c) process stack (d) pipe
20. Which of the following need not necessarily be saved on a context switch between processes?
 (a) General purpose registers (b) Translation look-aside buffer
 (c) Program counter (d) All of these

Answers

- | | | | | | |
|---------|---------|----------------|---------|---------|---------|
| 1. (h) | 2. (b) | 3. (d) | 4. (b) | 5. (b) | 6. (c) |
| 7. (d) | 8. (a) | 9. (a) and (d) | | 10. (a) | 11. (d) |
| 12. (a) | 13. (d) | 14. (a) | 15. (b) | 16. (a) | 17. (d) |
| 18. (a) | 19. (b) | 20. (b) | | | |



CPU Scheduling

3.1 INTRODUCTION

In computing, scheduling is the method by which threads, processes or data flows are given access to system resources. This is usually done to load balance and share system resources effectively. The concept of multitasking and multiplexing in the modern system requires CPU scheduling. By switching the CPU among various processes, the operating system can make the CPU more productive. In case of the CPU scheduling, the main concern is for the throughput, latency (total time between submission of process and its completion) and the response time. Preference is given to any one of the concerns and also depend on the user's needs and objectives.

3.2 BASIC CONCEPTS OF SCHEDULING

CPU scheduling is the basis of a multiprogrammed operating system. It is done by switching the CPU among processes, that the operating system makes the computer more productive and efficient.

In a single processor system, only one process can run at a time. When the other processes come then the CPU has to wait. When CPU becomes free then the next process will execute. The main aim of multiprogramming is to have some process running at all times to maximize the CPU utilization.

The processes have to wait, typically for the completion of some input-output request. During this time in simple computer system the CPU will just sit idle. All this waiting time is wasted and no useful work is accomplished.

In multiprogramming this time is used for some productive work. One process is over, then it gives CPU to another process.

3.2.1 CPU-I/O Burst Cycle

The success of CPU scheduling depends on an observed property of processes

1. Process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states.

2. Process execution begins with a CPU burst. This is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst and so on.
3. Eventually, the final CPU burst ends with a system request to terminate execution.

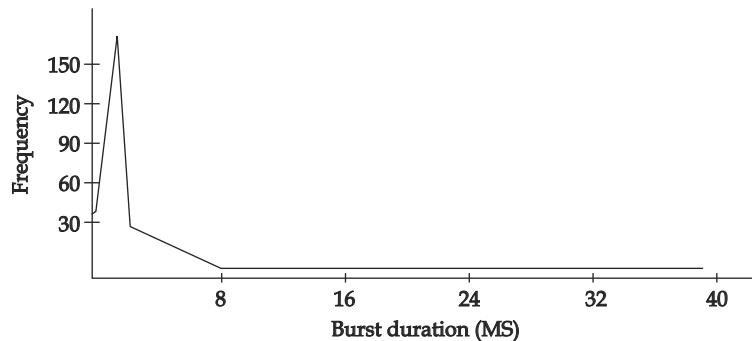


Fig. 3.1 CPU-burst durations

The curve is exponential, having large number of short CPU Bursts and a small number of long CPU bursts.

This distribution is important in the selection of an appropriate CPU scheduling algorithm.

3.2.2 CPU Scheduler

When the situation arises that the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the short-term scheduler. The scheduler selects a process from the processor in memory that are ready to execute and allocates the CPU to that process.

Conceptually, all the processes in the ready queue are lined up waiting for a chance to run on the CPU. The records in the queues are generally process control blocks of the processes.

3.2.3 Dispatcher

Dispatcher is also involved in the process of the CPU scheduling. The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves the following:

- Switching context
- Switching to user mode
- Jumping to the proper location in the user program to restart that program.

The dispatcher should be as fast as possible, since it is involved during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as dispatch latency.

3.2.4 Need for CPU Scheduling

The main need for CPU scheduling is to get the maximum productivity. To maximize the CPU utilization CPU scheduling is required.

Process scheduling is an essential part of a multiprogramming operating system. Such operating systems allow more than one process to be loaded into the executable memory at a time and loaded process shares the CPU using time multiplexing.

The time which is wasted during the waiting by the processes can be taken into use for some productive work. At one time several processes can be kept into memory.

So CPU scheduling is required for the multiprocesses to occur, because in a single processor system only one processor can run at a time.

3.3 PREEMPTIVE VS NON-PREEMPTIVE SCHEDULING

1. Non-Preemptive Scheduling

Non-preemptive algorithms are designed so that once a process enters the running state, it is not removed from the processor until it has completed its service time (or it explicitly yields the processor). "Context-switch ()" is called only when the processor terminates or blocks.

Allowing a process to run has some advantages and disadvantages:

Advantages

- The complexity of the code will get reduced as we serve CPU on first-come-first-basis and neglecting the priority of process.
- We would no longer have to concern ourselves with race conditions as we could be sure that one process could not interrupt another process.

Disadvantages

- A process with high burst time initially will run for a long time and will not free the CPU and other smaller process have to wait for their chance (called *convoy effect*)
- If there are consecutive CPU bound processes, the CPU will remain busy and will not be free. Similarly, if processes are I/O bound then CPU will have to sit idle.

2. Preemptive Scheduling

Preemptive algorithms are driven by the notion of prioritized computation. The process with highest priority should be the one currently using the processor. If one process is using the allocated CPU and a new process with higher priority enters the ready list, the running process is stopped and the CPU is allocated to higher priority process.

Advantages

- Processes with higher priority (like ALU operation) are able to process earlier.
- Processes with low burst time are able to process earlier and become free.

Disadvantages

- The code becomes complex as CPU is allocated on the priority basis. The priority has to be taken in account after every second of execution.
- We also need to cater for race conditions as well as having the responsibility of scheduling the process.

3.4 SCHEDULING CRITERIA

Many criteria have been suggested for comparing CPU-scheduling algorithms. The criteria include the following.

1. **CPU Utilization:** *Our main aim is to utilize the CPU as maximum as possible or to keep the CPU as busy as possible.* Conceptually, CPU utilization can range from 0 to 100 per cent. In the case of the lightly loaded system the CPU utilization is generally around 40 per cent and on the other hand for a heavily used system around 90% of CPU is utilized.
2. **Throughput:** *We can define throughput as the number of processes that are completed per unit time.* Our main aim is to keep the CPU busy by providing the processes again and again. The throughput for the long processes may be one process per hour and for the short processes it may be ten processes per second.
3. **Turnaround Time:** *The turnaround time is the time which a particular process takes for its execution.* The time period in between the time of submission of a process to the time of completion is the turnaround time.
Turnaround time is the sum of periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU and doing I/O.
4. **Waiting Time:** *We can define the waiting time as the sum of the periods spent waiting in the ready queue.* The CPU scheduling algorithm affects the amount of time that a process spends waiting in the ready queue. It does not affect the time for which the process executor or does input/output operations.
5. **Response Time:** In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous result are being output to the user. Then the other criteria is the time from the submission of a request until a first response is produced. This measure is called response time. It is the time it takes to start responding not the time it takes to output the response.

3.5 CPU SCHEDULING ALGORITHMS

CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU. There are many different CPU scheduling algorithms.

3.5.1 First-Come, First-Served

Algorithm Description

It is the simplest CPU scheduling algorithm. The processes are executed in order they arrive. This is simply a non-preemptive CPU scheduling algorithm.

The implementation of FCFS, is easily managed with a FIFO (First-In-First-Out) queue. When a process enters a ready queue, its PCB is linked onto its tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The process that has finished its execution is removed from the queue.

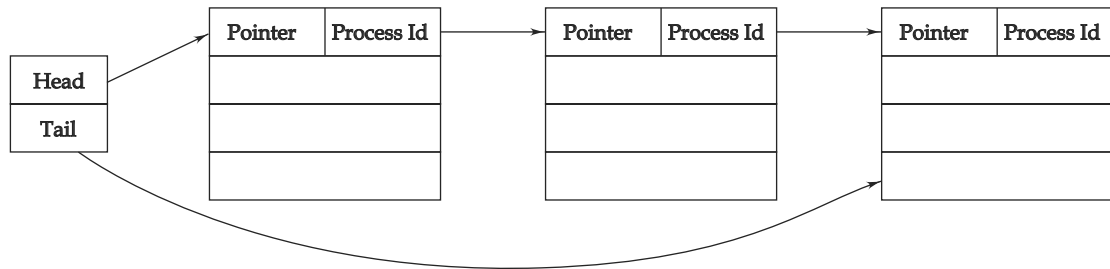


Fig. 3.2 FCFS implementation

Advantages

- FCFS algorithm is easy to implement. We don't need to check the priority of the process, rather we give according to first-come-first-serve basis.
- The process being executed will not be required to stop always due to interely or I/O bound events of other processes.

Disadvantages

- The biggest problem is that the average waiting time can be long.
- CPU bound job may make the I/O bound (once they have finished the I/O) wait for the process. At this point I/O devices are sitting idle.
- When CPU bound job finally does some I/O, the mainly I/O bound processes use the CPU quickly and now the CPU sits idle waiting for CPU bound job to complete its I/O.

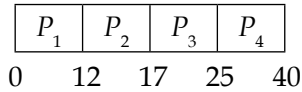
Numerical Example

Consider the following set of processes that arrive at time 0, with length of CPU burst time given in mil-sec.

Case 1: Considering arrival time of all processes = 0

Process	Burst Time	Arrival Time
P_1	12	0
P_2	5	0
P_3	8	0
P_4	15	0

If the processes arrive in order P_1, P_2, P_3, P_4 and are served in FCFS order then Gantt chart will be



As we know: Waiting time = (Service time) - (Arrival time)

So,

For P_1 : WT = 0 - 0 = 0

P_2 : WT = 12 - 0 = 12

P_3 : WT = 17 - 0 = 17

P_4 : WT = 25 - 0 = 25

Similarly, turnaround time (TAT) = (Completion time of process) - (Arrival time)

For P_1 : TAT = 12 - 0 = 12

P_2 : TAT = 17 - 0 = 17

P_3 : TAT = 25 - 0 = 25

P_4 : TAT = 40 - 0 = 40

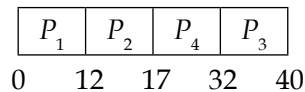
Avg. waiting time = $(0 + 12 + 17 + 25)/4$
= 13.5

Avg. turnaround time = $(12 + 17 + 25 + 40)/4$
= 23.5

Case 2: Considering the arrival time of processes

Process	Burst Time	Arrival Time
P_1	12	0
P_2	5	1
P_3	8	3
P_4	15	2

If processes come at different times then Gantt chart will be:



Now calculating waiting time and turnaround time for each process we get:

Process	Waiting Time	Turnaround Time
P_1	$0 - 0 = 0$	$12 - 0 = 12$
P_2	$5 - 1 = 4$	$17 - 1 = 16$
P_3	$32 - 3 = 29$	$40 - 3 = 37$
P_4	$17 - 2 = 15$	$32 - 2 = 30$

Avg. waiting time = $(0 + 4 + 29 + 15)/4$
= 12

$$\begin{aligned}\text{Avg. turnaround time} &= (12 + 16 + 37 + 30)/4 \\ &= 23.75\end{aligned}$$

3.5.2 Shortest-Job First Scheduling

Algorithm Description

This CPU scheduling algorithm associates with each process the length of the process's next CPU burst. When CPU becomes free from executing one process, it get/finds the process which have next minimum burst time required. It is also known as SRTF (Shortest Remaining Time First), so CPU is given to the process which has minimum left burst time.

Advantages

- It allows the process with minimum burst time to execute first hence saving processes from starvation.
- The biggest advantage is the average waiting time and average turnaround time in SJF process is less as compared to other algorithms.

Disadvantages

- The biggest disadvantage is its complexity of code is much higher as we have to always check the process at the instant with minimum burst times.
- The process if has higher burst time, even on coming early have to wait for a long time

Numerical Example

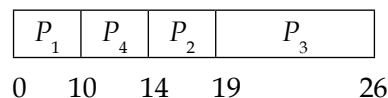
Consider four processes with their burst time and arrival time as follows:

Case 1: (Non-preemptive case)

Process	Burst Time	Arrival Time
P_1	10	0
P_2	5	2
P_3	7	1
P_4	4	3

In the above case, P_1 process has only arrived at 0 sec, so P_1 process starts executing. At 1 sec, the remaining time of P_1 is 9 sec and at same time P_3 process has arrived with burst time of 7 sec.

But, in non-preemptive case once the process is started it will continue it until its execution stops (completed). Following this, after 10 sec all the processes have arrived now it is checked by that time which process has smaller burst time, it is P_4 process. So it will start executing. So, Gantt chart will be:



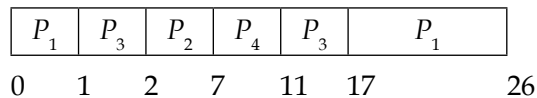
So, waiting time and turnaround time will be:

Process	Burst Time	Waiting Time	Turnaround Time
P_1	10	$0 - 0 = 0$	$10 - 0 = 10$
P_2	5	$14 - 2 = 12$	$19 - 2 = 17$
P_3	7	$19 - 1 = 18$	$26 - 1 = 25$
P_4	4	$10 - 3 = 7$	$14 - 3 = 11$

Case 2: (Preemptive Case)

Process	Burst Time	Arrival Time
P_1	10	0
P_2	5	2
P_3	7	1
P_4	4	3

In this case, P_1 process has arrived at 0 sec, then at 1 sec, P_3 process with burst time 7 sec has arrived. It has less required time for execution so (in preemptive case) the CPU will be allocated to P_3 process which will start execution. Again at 2 sec, process P_2 arrives with less burst time (BT of $P_2 = 5$) so CPU will be allocated to P_2 and execution of P_3 will be stopped. Continuing this way Gantt will be:



So, the waiting time and turnaround time will be:

Process	Waiting Time	Turnaround Time
P_1	$17 - 1 - 0 = 16$	$26 - 0 = 26$
P_2	$2 - 2 = 0$	$7 - 2 = 5$
P_3	$11 - 1 - 1 = 9$	$17 - 1 = 16$
P_4	$7 - 3 = 4$	$11 - 3 = 8$

$$\begin{aligned} \text{Average waiting time} &= (16 + 0 + 9 + 4)/4 \\ &= 7.25 \end{aligned}$$

$$\begin{aligned} \text{Average turnaround time} &= (26 + 5 + 16 + 8)/4 \\ &= 13.75 \end{aligned}$$

3.5.3 Priority Scheduling Algorithm

Algorithm Description

This type of scheduling algorithm is based on priority specified for every process. In this, the process with highest priority (taken as 1) is processed first and then the processes with lower priority are process.

Advantages

- The problem of starvation can be removed by the concept of aging, i.e., OS increases the priority of process time to time.
- It is helpful in first executing the processes which are more important and have higher priority.

Disadvantages

- Low priority process may suffer starvation, i.e., even on coming early the low priority processes have to wait until higher priority processes have completed their execution. Consider five processes with length of CPU burst time given as follows

Numerical Example (Preemptive case)

Process	Burst Time	Arrival Time	Priority
P_1	10	0	3
P_2	1	1	1
P_3	2	2	2
P_4	1	2	4
P_5	5	3	3

In this case, the process P_1 has arrived at 0 sec so it will start its execution. After 1 sec P_2 process has arrived with highest priority (= 1) after 2 sec. P_2 process has finished. Now at time 2 sec P_1 process will start the execution on first-come-first-serve basis as at 2 sec as P_1 and P_3 process have same priority. Following this process Gantt chart will be:

P_1	P_2	P_1	P_5	P_1	P_3	P_4	
0	1	2	3	8	16	18	19

Now, waiting time and turnaround time of each process is

Process	Waiting time	Turnaround time
P_1	$8 - 1 - 1 = 6$	$16 - 0 = 16$
P_2	$1 - 1 = 0$	$2 - 1 = 1$
P_3	$16 - 2 = 14$	$18 - 2 = 16$
P_4	$18 - 2 = 16$	$19 - 2 = 17$
P_5	$3 - 3 = 0$	$8 - 3 = 5$

So, average waiting time = $(6 + 0 + 14 + 16 + 0)/5$
 $= 7.2$

Average turnaround time = $(16 + 1 + 16 + 17 + 5)/5$
 $= 11$

3.5.4 Round Robin Scheduling

Algorithm Description

Round robin is the scheduling algorithm used by the CPU during execution of the process. Round robin algorithm is designed specifically for time sharing systems. It is similar to first come first serve scheduling algorithm but preemption is the added functionality to switch between the processes.

A small unit of time also known as time slice or quantum is defined. The ready queue works like circular queue. All the processes in this queue are kept in circular queue. Each new process is added to the tail of the circular queue.

By using this algorithm, CPU makes sure, time slices are assigned to each process in equal portions and in circular order, dealing with all processes without any priority.

Advantages

The main advantage of round robin over FCFS algorithm is that it is starvation free. So in this way no process is left waiting for its turn to be executed by CPU.

Round robin algorithm is simple and is easy to implement.

The throughput of round robin algorithm is also low as the large process is holding up the central processing unit for execution.

Disadvantages

The round robin algorithm does not give special priority to more important tasks.

Allocating equal amount of times to each task is simple, but it can fast track with those with medium length tasks.

It does not finish the task when it encounters and preemption of tasks is always there.

Numerical Example

Consider the following set of processes that arrive at time 0, with length of CPU burst time given in milliseconds:

Processs	Burst time
P_1	24
P_2	3
P_3	3

Let us consider the time quantum of 4 Ms. So for first 4 seconds the process P_1 occurs and then for the next 4 seconds P_2 to be processed. But it will be over in 3 seconds. Similarly, P_3 will be processed for the next 3 seconds and then P_1 will proceed further.

Gantt Chart

P_1	P_2	P_3	P_1	P_1	P_1	P_1	P_1	
0	4	7	10	14	18	22	26	30

Waiting Time

$$P_1 \rightarrow 6$$

$$P_2 \rightarrow 4$$

$$P_3 \rightarrow 7$$

$$\text{Average waiting time} = \left(\frac{6 + 4 + 7}{3} \right) = \frac{17}{3} = 5.66 \text{ millisecond (ms)}$$

3.5.5 Multilevel Queue Scheduling

Algorithm Description

A multilevel queue scheduling algorithm partitions the ready queue in several separate queue. In a multilevel queue scheduling processes are permanently assigned to one queues.

The processes are permanently assigned to one another, based on some property of the process such as

Memory size

Process priority

Process type

Each queue in the multilevel queue scheduling wave has its own algorithm.

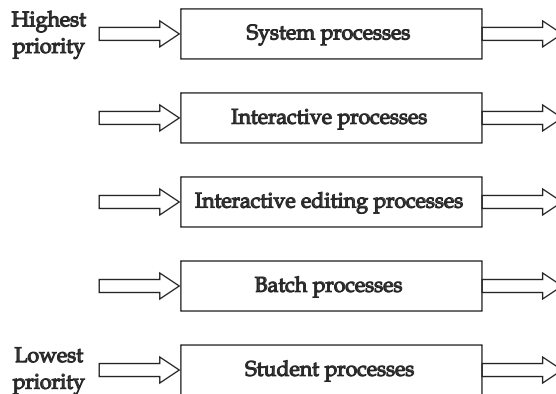


Fig. 3.3 Multilevel queues

Advantages

In case of multilevel queue scheduling we can apply any algorithm on any level of queue. We can apply FIFO, SJF and round robin at any level. By doing this the efficiency of completing the process can be increased and every process will take the CPU Burst.

Disadvantage

The main disadvantage of multilevel queue scheduling is that it is complex and is difficult to understand. It takes a lot of time as the each queue has its own scheduling algorithm.

3.5.6 Multilevel Feedback Queue Scheduling

It allows the process to move between queues. The idea is to separate processes according to the characteristics of their CPU bursts. If a process uses too much time then it will be moved to a lower priority queue. This process leaves I/O bound and interactive processes in the higher priority queues. In addition, a process that waits too long in a lower priority moves to a higher priority queue. This form of aging prevents starvation.

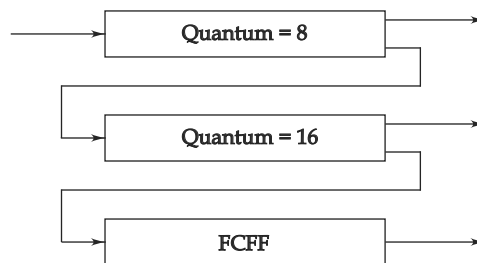


Fig. 3.4 Multilevel feedback queue

Considering the last figure with three queues numbered 0 to 2. The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1. Similarly, processes in queue 2 will only be executed if queues 0 and 1 are empty. A process that arrives in queue 1 will preempt the process in queue 2. A process in queue 1 will, in turn, be preempted by a process arriving by queue 0.

A process entering the ready queue is put in queue 0. A process in queue 0 is given a time quantum of 8 milliseconds. If it does not finish within this time it will move to the tail of queue 1. If queue 0 is empty, the process at head of queue 1 is given a quantum of 16 milliseconds. If it does not complete it is preempted and is put into queue 2. Processes in queue 2 are run on FCFS basis but run only when queues 0 and 1 are empty.

Highest priority → 8 Ms

Then lower priority → Less than 24 Ms

In general, a multilevel feedback queue scheduler is defined by the following parameters:

1. The number of queues
2. The scheduling algorithm for each queue
3. The method used to determine when to upgrade a process to higher priority queue or to demote a process to lower priority queue.
4. The method used to determine which queue a process will enter when that process needs service.

Numerical Examples

3.1 What advantage is there in having different time-quantum sizes at different levels of a multilevel queuing system?

Sol. The processes which need more frequent servicing, for instance interactive processes such as editors, can be in a queue with a small time quantum. Processes with no need for frequent servicing can be in queue with a longer time quantum, requiring fewer context switches to complete the processing, making more efficient use of the computer.

So as a result, it is required to have different time-quantum sizes at different levels of a multilevel queueing system.

3.2 Consider a system running 10 I/O bound tasks and one CPU-bound task. Assume that I/O bound tasks issue an I/O operation once for every millisecond of CPU computing and that each I/O operation takes 10 ms to complete. Also assume context-switching overhead is 0.1 ms and that all processes are long running tasks. Describe CPU utilization for round robin scheduler when:

(a) The time quantum is 1 millisecond.

(b) The time quantum is 10 milliseconds

Sol. (a) The time quantum is 1 millisecond, irrespective of which process is scheduled, the scheduler incurs a 0.1 ms context-switching cost for every context switching.

So the CPU utilization is of:

$$\frac{1}{1.1} \times 100 = 90.9090\%$$

(b) The I/O bound task incurs a context switch after using up only one millisecond of time quantum. The time required to cycle through all the processes is therefore $10 * 1.1 + 10.1$ (as each I/O bound task executes for 1 millisecond and there incurs the context switch task, whereas the CPU bound task executes for 10 milliseconds before incurring context switch).

The CPU utilization is

$$\frac{20}{21.1} \times 100 = 94.78\%$$

3.3 Process	Arrival time	Burst time (Ms)
P_1	0	7
P_2	1	1
P_3	2	2
P_4	3	4

Calculate Avg. wt. time and TAT by FCFS, SJF (Preemptive and non-preemptive).

Sol. FCFS → Gantt chart →

P_1	P_2	P_3	P_4
0	7	8	10
			14

Waiting time

$$\begin{aligned}
 P_1 &= 0 \\
 P_2 &= 7 - 1 = 6 \text{ ms} \\
 P_3 &= 8 - 2 = 6 \text{ ms} \\
 P_4 &= 10 - 3 = 7 \text{ ms}
 \end{aligned}$$

$$\text{Avg. wt. time} = \frac{0+6+6+7}{4} = \frac{19}{4} = 4.75 \text{ ms}$$

Turn around time

$$\begin{aligned}
 P_1 &= 7 - 0 = 7 \text{ ms} \\
 P_2 &= 8 - 1 = 7 \text{ ms} \\
 P_3 &= 10 - 2 = 8 \text{ ms} \\
 P_4 &= 14 - 3 = 11 \text{ ms}
 \end{aligned}$$

$$\text{Avg. TAT} = \frac{7+7+8+11}{4} = \frac{33}{4} = 8.2 \text{ ms.}$$

SJF (non-preemptive)

P_1	P_2	P_3	P_4
0	7	8	10
			14

$$\text{So, Avg. wt time} = \frac{0+6+6+7}{4} = \frac{19}{4} = 4.75$$

$$\text{Avg. TAT} = \frac{7+7+8+11}{4} = \frac{33}{4} = 8.2 \text{ ms}$$

SJF (Preemptive)

P_1	P_2	P_3	P_4	P_1
0	1	2	4	8
				14

Waiting Time

$$P_1 \rightarrow (8 - 1 - 0) = 7 \text{ ms}$$

$$P_2 \rightarrow 0 \text{ ms. Avg. W.T.} = \frac{7+0+0+1}{4}$$

$$P_3 \rightarrow 0 \text{ ms.} = 2 \text{ ms}$$

$$P_4 \rightarrow (4 - 3) = 1 \text{ ms}$$

TAT

$$P_1 \rightarrow (14 - 0) = 14 \text{ ms}$$

$$P_2 \rightarrow (2 - 1) = 1 \text{ ms. Avg. TAT} = \frac{14 + 1 + 2 + 5}{4}$$

$$P_3 \rightarrow (4 - 2) = 2 \text{ ms} = 5.5 \text{ ms}$$

$$P_4 \rightarrow (8 - 3) = 5 \text{ ms}$$

3.4 Process	Arrival time	Execution time
A	0	4
B	2	7
C	3	3
D	3.5	3
E	4	5

1. FCFS algorithm Calculate Avg. waiting time & TAT using

(i) FCFS algorithm

(ii) Round robin algorithm for Quantum size 2

A	B	C	D	E	
0	4	11	14	17	22

Sol: Waiting time

$$A \rightarrow 0 - 0 = 0 \text{ ms}$$

$$B \rightarrow 4 - 2 = 2 \text{ ms}$$

$$C \rightarrow 11 - 3 = 8 \text{ ms}$$

$$D \rightarrow 14 - 3.5 = 10.5 \text{ ms}$$

$$E \rightarrow 17 - 4 = 13 \text{ ms}$$

Avg. waiting time

$$\frac{0 + 2 + 8 + 10.5 + 13}{5} = \frac{33.5}{5} = 6.7 \text{ ms}$$

Turnaround time

$$A \rightarrow (4 - 0) = 4 \text{ ms}$$

$$B \rightarrow (11 - 2) = 9 \text{ ms}$$

$$C \rightarrow (14 - 3) = 11 \text{ ms}$$

$$D \rightarrow (17 - 3.5) = 13.5 \text{ ms}$$

$$E \rightarrow (22 - 4) = 18 \text{ ms}$$

$$\text{Avg. TAT} = \frac{4 + 9 + 11 + 13.5 + 18}{5} = 11.1 \text{ ms.}$$

Form Round robin algorithm

2. Round robin algorithm

A	B	C	D	E	A	B	C	D	E	B	E	B	
0	2	4	6	8	10	12	14	15	16	18	20	21	22

Waiting time

$$A \rightarrow 10 - 2 = 8 \text{ ms}$$

$$B \rightarrow 21 - 6 = 15 \text{ ms}$$

$$C \rightarrow 14 - 2 = 12 \text{ ms}$$

$$D \rightarrow 15 - 2 = 13 \text{ ms}$$

$$E \rightarrow 20 - 4 = 16 \text{ ms}$$

$$\text{Avg. wt time} = \frac{8 + 15 + 12 + 13 + 16}{5} = \frac{64}{5} = 12.8 \text{ sec}$$

Turn around time

$$A \rightarrow 12 \text{ ms}$$

$$B \rightarrow 22 \text{ ms}$$

$$C \rightarrow 15 \text{ ms}$$

$$D \rightarrow 16 \text{ ms}$$

$$E \rightarrow 21 \text{ ms}$$

$$\text{Avg. TAT} = \frac{12 + 22 + 15 + 16 + 21}{5} = \frac{86}{5} = 17.2$$

3.5 Consider the processes

Process	Arrival time	Burst time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

Consider SJF preemptive and non-preemptive cases. Calculate average waiting time and turnaround time.

(i) SJF non-preemptive

P_1	P_3	P_2	P_4	
0	7	8	12	16

(ii) Preemptive SJF

P_1	P_2	P_3	P_2	P_4	P_1	
0	2	4	5	7	11	16

Sol. (i) Waiting time

$$P_1 \rightarrow 0$$

$$P_2 \rightarrow 8 - 2 = 6 \text{ ms}$$

$$P_3 \rightarrow 7 - 4 = 3 \text{ ms}$$

$$P_4 \rightarrow 12 - 5 = 7 \text{ ms}$$

$$\text{So, Avg. waiting time} = \frac{0 + 6 + 3 + 7}{4} = \frac{16}{4} = 4 \text{ ms}$$

Turnaround time

$$P_1 \rightarrow 7 \text{ ms}$$

$$P_2 \rightarrow 12 - 2 = 10 \text{ ms}$$

$$P_3 \rightarrow 8 - 4 = 4 \text{ ms}$$

$$P_4 \rightarrow 16 - 5 = 11 \text{ ms}$$

$$\text{Avg. turnaround time} = \frac{7 + 10 + 4 + 11}{4} = 8 \text{ ms}$$

Sol. (ii) Waiting time

$$P_1 \rightarrow 11 - 2 = 9 \text{ sec}$$

$$P_2 \rightarrow 5 - 2 - 2 = 1 \text{ ms}$$

$$P_3 \rightarrow 4 \text{ ms}$$

$$P_4 \rightarrow 7 \text{ ms}$$

Avg. waiting time

$$\frac{9 + 1 + 4 + 7}{4} = 5.25$$

Turnaround time

$$P_1 \rightarrow 16 \text{ ms}$$

$$P_2 \rightarrow 7 - 2 = 5 \text{ ms}$$

$$P_3 \rightarrow 5 - 4 = 1 \text{ ms}$$

$$P_4 \rightarrow 11 - 5 = 6 \text{ ms}$$

$$\text{Avg. TAT} = \frac{16 + 5 + 1 + 6}{4} = \frac{28}{4} = 7 \text{ ms}$$

3.6 Calculate the turnaround time and waiting time

1. Using SJF (Non-preemptive)

2. Using priority (Non-preemptive)

Process	Burst time	Priority	Arrival time
P_1	7	1	0
P_2	3	2	4
P_3	9	3	7

Sol. (i) Using SJF Gantt chart is

P_1	P_2	P_3	
0	7	10	19

Waiting time

$$P_1 \rightarrow 0$$

$$P_2 \rightarrow 7 - 4 = 3$$

$$P_3 \rightarrow 10 - 7 = 3$$

$$\text{Avg. waiting time} = \frac{0 + 3 + 3}{3} = 2 \text{ ms}$$

TAT

$$P_1 \rightarrow 0$$

$$P_2 \rightarrow 10 \text{ ms} - 4 \text{ ms} = 6 \text{ ms}$$

$$P_3 \rightarrow 19 \text{ ms} - 7 \text{ ms} = 12 \text{ ms}$$

$$\text{Avg. TAT} = \frac{0 + 6 + 12}{3} = 8.33$$

Sol. (ii) Using priority algo. (non-preemptive case) Gantt chart is

	P_1	P_2	P_3
	0	7	10
			19
Process	WT		TAT
P_1	$0 - 0 = 0$		$7 - 0 = 7$
P_2	$7 - 4 = 3$		$10 - 4 = 6$
P_3	$10 - 7 = 3$		$19 - 7 = 12$
Average WT = $(0 + 3 + 3)/3$			
= 2			
Average TAT = $(7 + 6 + 12)/3$			
= 8.3			

3.7 Calculate turnaround time and waiting time for the following processes using (i) SJF and (ii) FCFS

Process	AT	BT
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Sol. (i) Using SJF (Preemptive case)

P_1 0	P_2 1	P_4 5	P_1 10	P_3 17	26
Process	WT			TAT	
P_1	10 − 1 = 9			17 − 0 = 17	
P_2	1 − 1 = 0			5 − 1 = 4	

P_3	$17 - 2 = 15$	$26 - 2 = 24$
P_4	$5 - 3 = 2$	$10 - 3 = 7$

Avg. waiting time = 6.5 ms

Avg. turn around time = 13 ms

(ii) Using FCFS

P_1	P_2	P_3	P_4	
0	8	12	21	26

Process	WT	TAT
P_1	$0 - 0 = 0$	$8 - 0 = 8$
P_2	$8 - 1 = 7$	$12 - 1 = 11$
P_3	$12 - 2 = 10$	$21 - 2 = 19$
P_4	$21 - 3 = 18$	$26 - 3 = 23$

Avg. WT = 8.75

Avg. TAT = 15.25

3.8 Process	Burst time	Arrival time
A	3	0
B	5	1
C	2	3
D	5	9
E	5	12

Using (i) SJF Non-preemptive & SJF-preemptive calculate average waiting time & turnaround time.

Sol. 1. Non-preemptive

Gantt Chart is

A	C	B	D	E	
0	3	5	10	15	20

Waiting time

$A \rightarrow 0$ ms

$B \rightarrow 5 - 1 = 4$ ms

$C \rightarrow 3 - 3 = 0$ ms

$D \rightarrow 10 - 9 = 1$ ms

$E \rightarrow 15 - 12 = 3$ ms

Avg. waiting time

$$\frac{0 + 4 + 0 + 1 + 3}{5} = \frac{8}{5} = 1.6 \text{ ms}$$

Turnaround time

$$A \rightarrow 3 \text{ ms}$$

$$B \rightarrow 10 - 1 = 9 \text{ ms}$$

$$C \rightarrow 5 - 3 = 2 \text{ ms}$$

$$D \rightarrow 15 - 9 = 6 \text{ ms}$$

$$E \rightarrow 20 - 12 = 8 \text{ ms}$$

Avg. TAT

$$\frac{3+9+2+6+8}{5} = \frac{28}{5} = 5.6 \text{ ms}$$

2. Preemptive

A	B	C	D	E	
0	3	5	10	15	20

Waiting time

$$A \rightarrow 0 \text{ ms}$$

$$B \rightarrow 5 - 1 = 4 \text{ ms}$$

$$C \rightarrow 3 - 3 = 0 \text{ ms}$$

$$D \rightarrow 10 - 9 = 1 \text{ ms}$$

$$E \rightarrow 15 - 12 = 3 \text{ ms}$$

Avg. waiting time

$$\frac{0+4+0+1+3}{5} = 1.6 \text{ ms}$$

Turnaround time

$$A \rightarrow 3 \text{ ms}$$

$$B \rightarrow 10 - 1 = 9 \text{ ms}$$

$$C \rightarrow 5 - 3 = 2 \text{ ms}$$

$$D \rightarrow 15 - 9 = 6 \text{ ms}$$

$$E \rightarrow 20 - 12 = 8 \text{ ms}$$

Avg. TAT

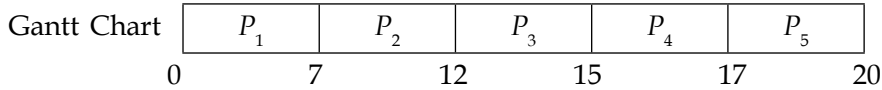
$$\frac{3+9+2+6+8}{5} = \frac{28}{5} = 5.6 \text{ ms}$$

3.9 Consider the snapshot of the system.

Process	Arrival Time	Burst Time (ms)
P_1	0	7
P_2	1	5
P_3	2	3
P_4	6	2
P_5	12	3

Calculate average waiting time and turnaround time for FCSFS & SJF scheduling algorithm.

Sol. FCFS:



(Wt) Waiting time $P_1 = 0$

WT $P_2 = 7 - 1 = 6$

WT $P_3 = 12 - 2 = 10$

WT $P_4 = 15 - 6 = 9$

WT $P_5 = 17 - 12 = 5$

$$\text{Avg. waiting time} = \frac{0 + 6 + 10 + 9 + 5}{5} = \boxed{6 \text{ ms}}$$

(TAT) Turnaround time $P_1 = 7$

TAT $P_2 = 11$

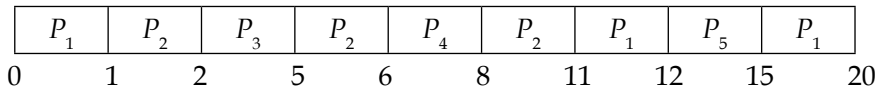
TAT $P_3 = 13$

TAT $P_4 = 11$

TAT $P_5 = 8$

$$\text{Avg. turnaround time} = \frac{7 + 11 + 13 + 11 + 8}{5} = \boxed{10 \text{ ms}}$$

SJF (Preemptive):



WT $P_1 = 15 - 1 - 1 - 0 = 13$

WT $P_2 = 8 - 1 - 1 - 1 = 5$

WT $P_3 = 0$

WT $P_4 = 0$

WT $P_5 = 0$

$$\text{Avg. waiting time} = \frac{13 + 5 + 0 + 0 + 0}{5} = \boxed{3.6 \text{ ms}}$$

TAT $P_1 = 20 - 0 = 20$

TAT $P_2 = 11 - 1 = 10$

TAT $P_3 = 5 - 2 = 3$

TAT $P_4 = 8 - 6 = 2$

TAT $P_5 = 15 - 12 = 3$

$$\text{Avg. turnaround time} = \frac{20 + 10 + 3 + 2 + 3}{5} = \boxed{7.6 \text{ ms}}$$

SJF (Non-Preemptive):

P_1	P_4	P_3	P_2	P_5	
0	7	9	12	17	20

$$WT P_1 = 0$$

$$WT P_2 = 11$$

$$WT P_3 = 7$$

$$WT P_4 = 1$$

$$WT P_5 = 5$$

$$\therefore \text{Avg. waiting time} = \frac{0 + 11 + 7 + 1 + 5}{5} = \boxed{4.8 \text{ ms}}$$

$$TAT P_1 = 0$$

$$TAT P_2 = 16$$

$$TAT P_3 = 10$$

$$TAT P_4 = 3$$

$$TAT P_5 = 8$$

$$\therefore \text{Avg. turnaround time} = \frac{7 + 16 + 10 + 3 + 8}{5} = \boxed{8.8 \text{ ms}}$$

3.10 Consider the system with set of processes

Process	Burst time	Arrival time	Priority
P_1	5	0	2
P_2	15	1	3
P_3	10	2	1

Assuming 1 to be the highest priority, calculate average waiting time and avg. turnaround time using priority (preemptive & non-preemptive) scheduling.

Sol. Priority Preemptive:

P_1	P_3	P_1	P_2	
0	2	12	15	30

$$\text{Waiting time (WT)} P_1 = 12 - 2 - 0 = 10$$

$$WT P_2 = 15 - 1 = 14$$

$$WT P_3 = 0$$

$$\therefore \text{Avg. time} = \frac{10 + 14 + 0}{3} = \boxed{8 \text{ units}}$$

$$\text{Turnaround time (TAT)} P_1 = 15$$

$$\text{TAT } P_2 = 30 - 1 = 29$$

$$\text{TAT } P_3 = 12 - 2 = 10$$

$$\therefore \text{Avg. turn around time} = \frac{15 + 29 + 10}{3} = \boxed{18 \text{ units}}$$

Priority Non-Preemptive:

P_1	P_3	P_2
0	5	15 30

$$\text{WT } P_1 = 0$$

$$\text{WT } P_2 = 15 - 1 = 14$$

$$\text{WT } P_3 = 5 - 2 = 3$$

$$\therefore \text{Avg waiting time} = \frac{0 + 14 + 3}{3} = \boxed{5.66 \text{ units}}$$

$$\text{TAT } P_1 = 5$$

$$\text{TAT } P_2 = 30 - 1 = 29$$

$$\text{TAT } P_3 = 15 - 2 = 13$$

$$\therefore \text{Avg. turn around time} = \frac{5 + 29 + 13}{3} = \boxed{15.66 \text{ units}}$$

3.11 Consider the snapshot of the system:

Process	Arrival Time	Burst Time (ms)
P_1	0	7
P_2	1	1
P_3	2	2
P_4	3	4

Calculated avg. waiting time and avg. trunaround by FCFS, SJF (preemptive & Non-preemptive).

Sol. (i) FCFS:

P_1	P_2	P_3	P_4
0	7	8	10 14

$$\text{Waiting time (WT) } P_1 = 0$$

$$\text{WT } P_2 = 6$$

$$\text{WT } P_3 = 6$$

$$\text{WT } P_4 = 7$$

$$\therefore \text{Avg. waiting time} = \frac{0 + 6 + 6 + 7}{4} = \boxed{4.7 \text{ ms}}$$

Turnaround time (TAT) = $P_1 = 7$

TAT $P_2 = 7$

TAT $P_3 = 8$

TAT $P_4 = 11$

$$\therefore \text{Avg. turnaround time} = \frac{7+7+8+11}{4} = \boxed{8.2 \text{ ms}}$$

(ii) SJF (Non-preemptive):

P_1	P_2	P_3	P_4
0	7	8	10
			14

WT $P_1 = 0$

WT $P_2 = 6$

WT $P_3 = 6$

WT $P_4 = 7$

$$\therefore \text{Avg. waiting time} = \frac{0+6+6+7}{4} = \boxed{4.7 \text{ ms}}$$

TAT $P_1 = 7$

TAT $P_2 = 7$

TAT $P_3 = 8$

TAT $P_4 = 11$

$$\therefore \text{Avg. turnaround time} = \frac{7+7+8+11}{4} = \boxed{8.2 \text{ ms}}$$

(iii) SJF (Preemptive):

P_1	P_2	P_3	P_4	P_1
0	1	2	4	8
				14

WT $P_1 = 8 - 1 - 0 = 7$

WT $P_2 = 0$

WT $P_3 = 0$

WT $P_4 = 4 - 3 = 1$

$$\therefore \text{Avg. waiting time} = \frac{7+0+0+1}{4} = \boxed{2 \text{ ms}}$$

TAT $P_1 = 14 - 0 = 14$

TAT $P_2 = 2 - 1 = 1$

TAT $P_3 = 4 - 2 = 2$

TAT $P_4 = 8 - 3 = 5$

$$\therefore \text{Avg. turnaround time} = \frac{14 + 1 + 2 + 5}{4} = \boxed{5.5 \text{ ms}}$$

3.12 Consider the snapshot of the system:

Process	Arrival Time	Processing Time (ms)
P_1	0	10
P_2	0	2
P_3	1	3
P_4	2	1
P_5	2	4

Calculate waiting time & turnaround time for FCFS & SRJF/SRTF scheduling algorithms for each process.

Sol. FCFS:

P_1	P_2	P_3	P_4	P_5	
0	10	12	15	16	20

Waiting time (WT) $P_1 = 0$ ms

WT $P_2 = 10$ ms

WT $P_3 = 12 - 1 = 11$ ms

WT $P_4 = 15 - 2 = 13$ ms

WT $P_5 = 16 - 2 = 14$ ms

Turnaround time (TAT) $P_1 = 10$ ms

TAT $P_2 = 12$ ms

TAT $P_3 = 15 - 1 = 14$ ms

TAT $P_4 = 16 - 2 = 14$ ms

TAT $P_5 = 20 - 2 = 18$ ms

SRJF/SRTF:

P_2	P_4	P_3	P_5	P_1	
0	2	3	6	10	20

Waiting time (WT) $P_1 = 10 - 0 = 0$ ms

WT $P_2 = 0$ ms

WT $P_3 = 3 - 1 = 2$ ms

WT $P_4 = 2 - 2 = 0$ ms

WT $P_5 = 6 - 2 = 4$ ms

turn around time (TAT) $P_1 = 20 - 0 = 20$ ms

TAT $P_2 = 2$ ms

TAT $P_3 = 6 - 1 = 5$ ms

$$\text{TAT } P_4 = 3 - 2 = 1 \text{ ms}$$

$$\text{TAT } P_5 = 10 - 2 = 8 \text{ ms}$$

3.13 Consider the following set of processes, with length of CPU-burst time given in milliseconds:

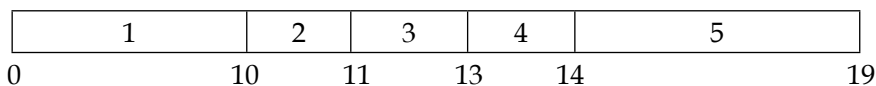
Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	3
P_4	1	4
P_5	5	2

The processes are assumed to have arrived in the order P_1, P_2, P_3, P_4, P_5 all at time 0.

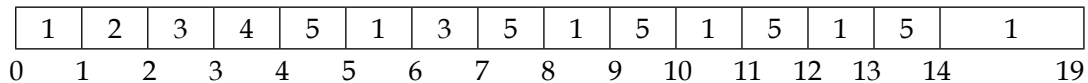
- draw all four Gantt charts illustrating the execution of these processes using FCFS, SJF, a non-preemptive priority (a smaller priority number implies a higher priority) and RR (quantum = 1) scheduling.
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of the scheduling algorithms in part a?
- Which of the schedules in part a results in the minimal average waiting time (over all processes)?

Sol. (a) The four gantt charts are:

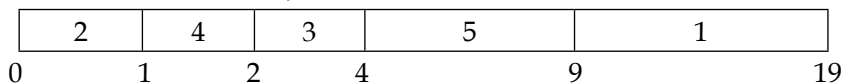
FCFS



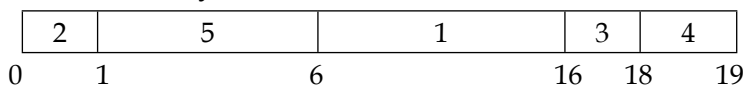
RR (Round robin)



SJF (shortest job first)



Priority



Turnaround time (time to completion - arrival time) here all are arrive at a time)

	FCFS	RR	SJF	Priority
P_1	$10 - 0 = 10$	$19 - 0 = 19$	$19 - 0 = 19$	$16 - 0 = 16$
P_2	$11 - 0 = 11$	$2 - 0 = 2$	$1 - 0 = 2$	$1 - 0 = 1$
P_3	$13 - 0 = 13$	$7 - 0 = 7$	$4 - 0 = 4$	$18 - 0 = 18$
P_4	$14 - 0 = 14$	$4 - 0 = 4$	$2 - 0 = 2$	$19 - 0 = 19$
P_5	$19 - 0 = 19$	$14 - 0 = 14$	$9 - 0 = 9$	$6 - 0 = 6$

(c) Waiting time (turnaround time - burst time)

	FCFS	RR	SJF	Priority
P_1	$10 - 10 = 0$	$19 - 10 = 9$	$19 - 10 = 9$	$16 - 10 = 6$
P_2	$11 - 1 = 10$	$2 - 1 = 1$	$1 - 1 = 0$	$1 - 1 = 0$
P_3	$13 - 2 = 11$	$7 - 2 = 5$	$4 - 2 = 2$	$18 - 2 = 16$
P_4	$14 - 1 = 13$	$4 - 1 = 3$	$2 - 1 = 1$	$19 - 1 = 18$
P_5	$19 - 5 = 14$	$14 - 5 = 9$	$9 - 5 = 4$	$6 - 5 = 1$

(d) Calculate the average of waiting time for all the algorithm as done in part C

$$\text{avg. waiting time for FCFS} = \frac{0+10+11+13+14}{5} = 9.6$$

$$\text{avg. waiting time for RR} = \frac{9+1+5+3+9}{5} = 5.4$$

$$\text{avg. waiting time for SJF} = \frac{9+0+2+1+4}{5} = 3.2$$

$$\text{avg. waiting time for priority} = \frac{6+0+16+18+1}{5} = 8.2$$

Minimum is for SJF (shortest job first = 3.2

3.14 Suppose that following processes arrive for execution at times indicated. Each process will run the listed amount of time. In answering the questions, use non-preemptive scheduling and base all decisions on the information you have at time the decision must be made.

Process	Arrival Time	Burst Time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

- (a) What is the average turnaround time for these processes with FCFS scheduling algorithm?
- (b) What is the average turnaround time for these processes with SJF scheduling algorithms?

Sol. Average turnaround time is calculated as:

- (a) Gantt chart FCFS is

	P_1	P_2	P_3	
	0	8	12	13
	FCFS			
P_1	$8 - 0 = 8$			
P_2	$12 - 0.4 = 11.6$			
P_3	$13 - 1.0 = 12$			

$$\text{avg. TAT} = \frac{8 + 11.6 + 12}{3} = 10.53$$

- (b) Average TAT is calculated as Gantt chart for SJF is called

	P_1	P_3	P_2	P_1	
	0	1	2	6	13
	SJF				
P_1	$13 - 0 = 13$				
P_2	$6 - 0.4 = 5.6$				
P_3	$2 - 1 = 1$				

$$\text{Avg. TAT for SJF} = \frac{13 + 5.6 + 1}{3} = \frac{19.6}{3} = 6.53$$

Summary

- The operating system has scheduling algorithms for execution of programs.
- The scheduling algorithms involve first come first serve, shortest job first, round robin, priority algorithm, multilevel queue scheduling and multilevel-feedback queue scheduling algorithm.
- The selection of scheduling algorithm is based on throughput, waiting time, response time, turnaround time, etc.
- In FCFS, the processes are served on the basis of arrival time. The process which comes first will execute first.
- In SJF processes are served on the basis of shortest execution time. The process which has less time required for execution will be executed first.

- In priority, the processes are served on the basis of process having highest priority. The processes which has the highest priority will be executed first followed by the processes having the lower priority.
- In round robin the processes are served on the basis of fixed time quantum. Each process one by one executed for the specified time quantum.
- A multilevel queue scheduling algorithm partitions the ready queue in several separate queues. Each ready queue has its own scheduling algorithm. In this algorithm the foreground processes have the priority over background processes.
- Multilevel feedback queue-scheduling algorithm allows a process to move between queues. It uses many ready queues and associates a different priority with each queue.
- **CPU utilization:** CPU utilization is the process of keeping the CPU as busy as possible. In real systems it is around 40% to 90%.
- **Throughput:** If the CPU is busy executing processes, then work is being done. Throughput is the number of processes that are completed per unit time.
- **Turnaround time:** The interval from the time of submission of the process to the time of completion is the turnaround time. We want our turnaround time as low as possible.
- **Waiting time:** The waiting time is the sum of the periods spent waiting in the ready queue.
- **Response time:** Response time is the from the submission of a request until the first response is produced.
- **Dispatcher:** The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. The function involves the following
 1. Switching content
 2. Switching to user mode
 3. Jumping to proper location in the time user program to restart that program.

Multiple-Choice Questions

1. Which module gives control of the CPU to the process selected by the short-term scheduler?
 - (a) dispatcher
 - (b) interrupt
 - (c) scheduler
 - (d) none of the above
2. The processes that are residing in main memory and are ready and waiting to execute are kept on a list called
 - (a) list called
 - (b) job queue
 - (c) execution queue
 - (d) process queue
3. The interval from the time of submission of a process to the time of completion is known as

-
- (a) waiting time
 - (b) turnaround time
 - (c) response time
 - (d) throughput
4. Which scheduling algorithm allocates the CPU first to the process that requests the CPU first?
- (a) first-come, first-served scheduling
 - (b) shortest job scheduling
 - (c) priority scheduling
 - (d) none of the above
5. In priority scheduling algorithm
- (a) CPU is allocated to the process with highest priority
 - (b) CPU is allocated to the process with lowest priority
 - (c) equal priority processes cannot be scheduled
 - (d) none of the above
6. In priority scheduling algorithm, when a process arrives at the ready queue, its priority is compared with the priority of
- (a) all process
 - (b) currently running process
 - (c) parent process
 - (d) init process
7. Time quantum is defined in
- (a) shortest job scheduling algorithm
 - (b) round robin scheduling algorithm
 - (c) priority scheduling algorithm
 - (d) multilevel queue scheduling algorithm
8. Processes are classified into different groups in
- (a) shortest job scheduling algorithm
 - (b) round robin scheduling algorithm
 - (c) priority scheduling algorithm
 - (d) multilevel queue scheduling algorithm
9. In multilevel feedback scheduling algorithm
- (a) a process can move to a different classified ready queue
 - (b) classification of ready queue is permanent
 - (c) processes are not classified into groups
 - (d) none of the above
10. Which one of the following cannot be scheduled by the kernel?
- (a) Kernel level thread
 - (b) user level thread
 - (c) process
 - (d) none of the above
11. Under multiprogramming, turnaround time for short jobs is usually _____ and that for long jobs is slightly _____.
- (a) Lengthened; shortened
 - (b) Shortened; lengthened
 - (c) Shortened; shortened
 - (d) Shortened; unchanged
12. Which of the following statements are true?
- (I) Shortest remaining time first scheduling may cause starvation
 - (II) Preemptive scheduling may cause starvation
 - (III) Round robin is better than FCFS in terms of response time
- (a) I only
 - (b) I and III only
 - (c) II and III only
 - (d) I, II and III

-
- (I) Shortest remaining time first scheduling is a preemptive version of shortest job scheduling. It may cause starvation as shorter processes may keep coming and a long CPU burst process never gets CPU.
- (II) Preemption may cause starvation. If priority based scheduling with preemption is used, then a low priority process may never get CPU.
- (III) Round robin scheduling improves response time as all processes get CPU after a specified time.
13. The FCFS algorithm is particularly troublesome for
- (a) time sharing systems
 - (b) multiprogramming systems
 - (c) multiprocessor systems
 - (d) Operating systems
14. Consider the following set of processes, the length of the CPU burst time given in milliseconds:
- P_1 6
- P_2 8
- P_3 7
- P_4 3
- (i) Assuming the above process being scheduled with the SJF scheduling algorithm:
- (a) The waiting time for process P_1 is 3 ms
 - (b) The waiting time for process P_1 is 0 ms
 - (c) The waiting time for process P_1 is 16 ms
 - (d) The waiting time for process P_1 is 9 ms
15. Which module gives control of the CPU to the process selected by the short-term scheduler?
- (a) dispatcher
 - (b) interrupt
 - (c) scheduler
 - (d) none of the above
16. The processes that are residing in the main memory and are ready and waiting to execute are kept on a list called
- (a) job queue
 - (b) ready queue
 - (c) execution queue
 - (d) process queue
17. The interval from the time of submission of a process to the time of completion is known as
- (a) waiting time
 - (b) turnaround time
 - (c) response time
 - (d) throughput
18. Which scheduling algorithm allocates the CPU first to the process that requests the CPU first?
- (a) first-come, first-served scheduling
 - (b) shortest job scheduling
 - (c) priority scheduling
 - (d) none of the above

19. In priority scheduling algorithm
- (a) CPU is allocated to the process with highest priority
 - (b) CPU is allocated to the process with lowest priority
 - (c) equal priority processes cannot be scheduled
 - (d) none of the mentioned
20. In priority scheduling algorithm, when a process arrives at the ready, queue, its priority is compared with the priority of
- (a) all processes
 - (b) currently running process
 - (c) parent process
 - (d) init process

Answers

- | | | | | | |
|---------|---------|---------|---------|---------|---------|
| 1. (a) | 2. (b) | 3. (b) | 4. (a) | 5. (a) | 6. (b) |
| 7. (b) | 8. (d) | 9. (a) | 10. (b) | 11. (b) | 12. (d) |
| 13. (b) | 14. (a) | 15. (a) | 16. (b) | 17. (b) | 18. (a) |
| 19. (a) | 20. (a) | | | | |

4

Synchronization

4.1 INTRODUCTION

A cooperating process is one that can affect or be affected by the process executing in the system. Co-operating process can either directly share a logical address space or be allowed to share data only through files or messages.

Process Synchronization

A mechanism to ensure the orderly execution of a co-operating process that are sharing the logical address space, so that data consistency is maintained.

4.2 RACE CONDITION

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes places is called race condition.

To guard against the race condition, we need to ensure that only one process at a time can be manipulated by the variable counter. To make such a guarantee, we require that the processes be synchronized in some way.

Race Condition Example

Suppose two persons wish to deposit cash to the same bank account simultaneously. One person wants to deposit ₹ 3000 and the other wants to deposit ₹ 5000. If one completes before the other starts. The combined effect would be add ₹ 8000 to the balance.

If they both try to deposit at exactly the same time, initial balance is Rs. 300.

1. Proces P_2 loads into resister, P , add 3000 to its register, giving 3300.
2. P_2 adds 5000 to its register, giving 5300.
3. P_1 stores 3300 is balance.
4. P_2 stores 5300 is balance.
5. The net effect is to add only 5000 to the balance. This situation is known as race condition.

Producer-Consumer Problem

A producer who produces the item in the buffer and consumer who consumes the item from the buffer.

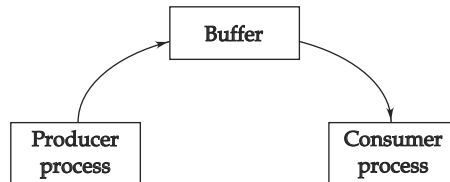


Fig. 4.1 Producer-consumer problem overview

For this type of scenario, synchronization is required, i.e., the following two conditions must be satisfied:

1. The consumer must wait if buffer is empty.
2. The producer must wait if buffer is full.

The flow chart of the above the problem is described as follows:

The pseudocode for producer-consumer problem is shown in Fig. 4.2:

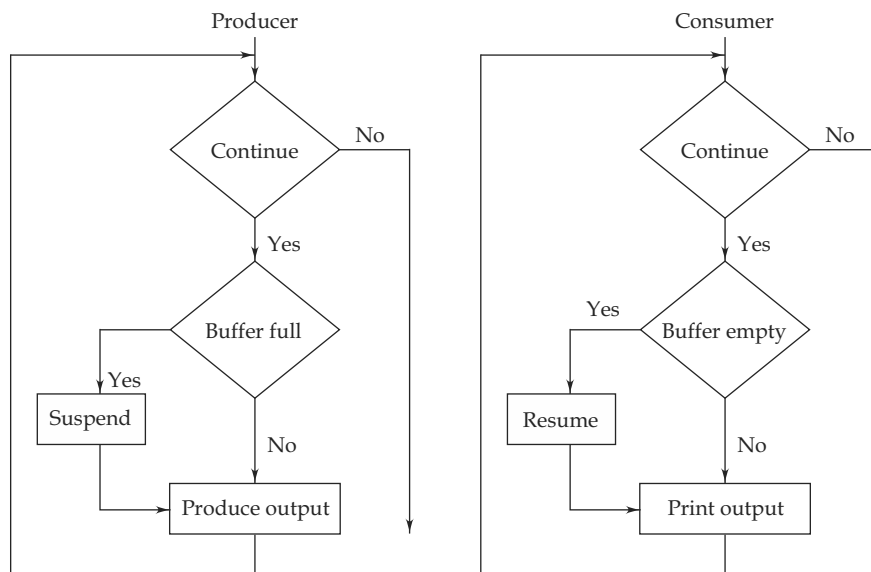


Fig. 4.2 Producer and consumer process

The pseudocode for producer-consumer problem is as shown below:

Repeat	Repeat
Produce an item in nextp While counter = n do no-op; buffer [in]: = nextp; in = in + 1 mod n; counter: = counter + 1; until false	While counter = 0 do no-op; nextc = buffer [out]; out = out + 1 mod n; counter: = counter - 1 — consume item in next C until false
Producer process	Consumer process

Race Condition in Producer-Consumer Problem

The producer and consumer process are correct separately but they may not function correctly when executed concurrently.

Counter = Counter + 1 can be implemented as:

register 1 = counter;

register 1 = register 1 + 1

counter = register 1

Where register 1 is local CPU register

Counter = Counter - 1 can be implemented as:

register 2 = counter;

register 2 = register 2 - 1

counter = register 2

Initially take "counter = 5"

S0: Producer register 1 = Counter [register = 5]

S1: Producer register 1 = register 1 + 1 [register 1 = 6]

S2: Consumer register 2 = counter [register 2 = 5]

S3: Consumer register 2 = register 2 - 1 [register 2 = 4]

S4: Producer counter = register 1 [counter = 6]

S5: Consumer counter = register 2 [counter = 4]

This is called *race condition*. It is arrived because we allowed both processes to manipulate the counter variable concurrently. To guard against this race condition we should follow the *mutual exclusion* concept strictly.

4.3 THE CRITICAL SECTION PROBLEM

1. Consider a system of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code, called a *critical section*, in which a process may be changing common variables, updating a table, writing a file and so on.

2. The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to be executed in its critical section.
3. The critical-section problem is to design a protocol that the processes can use to co-operate.
4. Each process must request permission to enter its critical section. The section of code implementing this request is the *entry section*.
5. The critical section may be followed by an *exit section*. The remaining code is the *remainder section*.

```

do
{
    entry section
    Critical section
    exit section
    remainder section
}
while (true);

```

Fig. 4.3 General structure of a process

The most important point is that when one process is executing shared modifiable data in its critical section, no other process is to be allowed to execute in its critical section. Thus, the execution of critical sections by the processes is mutually exclusive in time.

The critical section problem is to design a protocol that the processes can use to co-operate. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by exit section and remaining code is the remainder section.

4.4 SOLUTION TO CRITICAL SECTION PROBLEM

Solution to the critical section problem must satisfy the following three requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section then no other process can be executing in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only these processes that are not executing in their remainder section can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before this request is granted.

Two general approaches are used to handle critical sections in operating systems:

1. Pre-emptive kernels allow a process to be pre-empted while it is running in kernel mode.

2. Non-preemptive kernels do not allow a process running in kernel mode to be preempted. These are free from race conditions on kernel data structures, as only one process is active in the kernel at a time.

But a preemptive kernel is more suitable for real-time programming, as it will allow a real-time process to preempt a process currently running in the kernel. Further more, a pre-emptive kernel may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the watching processes.

4.5 ALGORITHM 1

This algorithm is a solution to critical section problem in which system has only two processes represented as P_0 and P_1

1. Both the processes P_0 and P_1 share the common integer variable. We assign the name to this variable as 'turn' and is initialized to 0 or 1.
2. If turn = i

Then process P_0 is allowed to execute in critical section.

Algorithm 1 allows only one process to enter into critical section.

Structure of P_0 in Algorithm 1

```
do
{
    while (turn!=i)
        critical section
    turn=i;
    remain section
}
while (1);
```

Fig. 4.4 Structure of process P_0 using Algorithm 1

3. For example, if turn = 0 and P_1 is ready to enter its critical section, P_1 cannot do so, even though P_0 may be in its remainder section.

4.6 ALGORITHM 2

1. Algorithm 1 cannot give sufficient information about the state of each process. It keeps record of the process which is entered into the critical section. To solve this problem variable 'turn' is replaced with new variable 'Flag'. It is initialized as

boolean Flag [2];

First the elements of array are initialised to false.

If flag[i] is true.

Then P_i is ready to enter into critical section.

Structure for Algorithm 2

```

do
{
    Flag[i]=true;
    While (Flag[i])
        Critical section
    Flag[i]=False
    remainder section
}

while (1);

```

Fig. 4.5 Structure of process P_i using Algorithm 2

In this algorithm process P_i first sets $\text{flag}[i]$ to be true, then process P_i is ready to enter its critical section. Process P_i also checks for process P_j . If process P_j were ready then P_i would wait until $\text{flag}[j]$ was false. So process P_i would enter into critical section.

4.7 ALGORITHM 3

Peterson's Solution

It is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_0 and P_1 . For convenience, we are using P_i , and P_j to denote

Peterson's solution requires the two processes to share two data items:

int turn;		Variables are initialized as
boolean flag[2];		$\text{flag}[0] = \text{flag}[1] = \text{false};$

The structure of process P_i in the Peterson's solution.

```

do
{


flag[i]=TRUE;
        turn=j;
        while(flag[j] && turn==j);


    critical section


flag[i]=FALSE


    remainder section
}

while (TRUE);

```

Fig. 4.6 Structure of P_i using Peterson's solution (Algorithm 3)

To prove the solution is correct, we need to show the following properties preserved:

- (1) **Mutual exclusion is preserved:** P_i enters its critical section only if either $\text{flag}[j] == \text{false}$ on $\text{turn} == i$. If both processes can be executing in their critical sections at the same time, then $\text{flag}[0] == \text{flag}[1] == \text{true}$. These two observations imply that P_0 and P_1 could not have successfully executed their while statements at about the same time, since value of turn can be either 0 or 1 but cannot be both. Hence, one of the processes, P_j - must have successfully executed the while statement, whereas P_i had to execute atleast one additional statement (" $\text{turn} == j$ "). At that time, $\text{flag}[j] == \text{true} \ \& \ \text{turn} == j$, & this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.
- (2) The progress requirement is satisfied.
- (3) The bounded-waiting requirement is met.

Process P_i can be prevented from entering the critical section only if it is stuck in the while loop with the condition $\text{flag}[j] == \text{true}$ and $\text{turn} == j$, and this condition will persist as this loop is the only one possible. If P_j is not ready to enter the critical section, then $\text{flag}[j] == \text{false}$, and P_i can enter critical section. If P_j has set $\text{flag}[j] == \text{true}$ and is also executing in its while statement, then either $\text{turn} == i$ or $\text{turn} == j$. If $\text{turn} == i$, then P_i will enter the critical section. If $\text{turn} == j$, then P_j will enter the critical section. However, once P_j exits its critical section, it will reset $\text{flag}[j]$ P_i to enter its critical section. If it will reset $\text{flag}[j]$ to false, allowing true, it must also set turn to i . Thus, since P_i does not change the value of the variable turn while executing the while statement, P_i will enter the critical section (progress) after at most entry by P_j (bounded waiting).

Solution to critical-section problem using locks.

```

do
{
    acquire lock
    critical section
    release lock
    remainder section
}
while(TRUE);

```

Fig. 4.7 Structure of process P_i using locks

4.8 BAKERY ALGORITHM

- (1) It is also an algorithm used for critical section problem.
- (2) This is a multiple-process solution.

Key Features are:

- Each process wanting to enter critical section (CS), gets a token number
- Process with lowest token number enters CS
- If two processes have same token number, process with lower PID (the process identified enters CS).

Pseudocode for Bakery's Algorithm

```

shared char choosing [N] = {0, ..., 0};
shared int number [N] = {0, ..., 0};
do {
    choosing[i] = 1;
    number [i] = MAX (number[0], number [1], ...,
                      number [n - 1] + 1);

    choosing[i] = 0;
    for(j=0; j < n, j + t)
    {
        while (choosing[j]);
        while ((number[j]!=0)&&(number[j] < number[i]))
        {
        }
    }
    critical-section();
    number[i]=0;
    remainder-section
}
while(1);

```

Fig. 4.8 Structure of process P_i using Bakery's algorithm**4.9 SYNCHRONIZATION HARDWARE**

Feature of the hardware can make the programming task easier and improve system efficiency. Many machines therefore provide special hardware instructions that allow us either to test and modify the content of a word, or to swap the contents of two words, automatically. We can use these special instructions to solve the critical section problem in a relatively simple manner. The Test-and-Set instruction can be defined as shown in Fig. 4.9. The important characteristic is that this instruction is executed atomically, that is, as one uninterrupted unit. Thus, if two Test-and-Set instructions are executed simultaneously (each on a different (PU), they will be executed sequentially in some arbitrary order.

If the machine supports the Test-and-Set instruction, then we can implement mutual exclusion by declaring a Boolean variable lock, initialized to false. The structure of process P_1 is shown in Fig. 4.9.

(Test & Set) Definition

1. The definition of the TestAndSet() instruction is as follows:

```
boolean TestAndSet (boolean & target)
{
    boolean rv = * target;
    * target = TRUE;
    return rv;
}
```

Fig. 4.9 Definition of test and set instruction

One important characteristic of this instruction is that it is executed automatically. Thus, if two TestAndSet() instructions are executed simultaneously, they will be executed sequentially in some arbitrary order.

```
mutual exclusion with TestAndSet()
do
{
    while(TestAndSet(& lock))
        ;// do nothing
    //critical section
    lock=FALSE;
    // remainder section
}
while (TRUE)
```

Fig. 4.10 Test and set instruction with mutual exclusion

2. Bounded-waiting mutual exclusion with TestAndSet().

```
do
{
    waiting[i]=TRUE;
    key = TRUE;
    while(waiting[i] && key)
        key = TestAndSet (& lock);
    waiting[i] = FALSE;
    // critical section
    j=(i+1)%n;
    while ((j!=i)&&! waiting[j])
        j=(j+1)%n;
    if(j==i)
        lock=FALSE
    else
        waiting[j]=FALSE; //remainder section
}
```

Fig. 4.11 Test and set with bounded-waiting mutual exclusion

3. The Swap() instruction in contrast to the TestAndSet() instruction, operates on the contents of two words. It is executed automatically.

The definition of Swap() instruction.

```
void swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a=*b;
    *b=temp;
}
```

(a)

(1) Mutual exclusion with the Swap() instruction.

```
do
{
    key=TRUE;
    while (key==TRUE)
        Swap(&lock, *key);
    //critical section
    lock=FALSE;
    //remainder section
}
while(TRUE);
```

(b)

Fig. 4.12 (a) Definition of swap function (b) Swap function with mutual exclusion

4.10 SEMAPHORES

Introduction

1. It is a process synchronization tool.
2. A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations:
Wait() & Signal().
3. The wait () operation was originally termed P(from the Dutch proberm, "to test");

```
wait (S)
{
    while S<=0
        ; // no-op
    S--;
```

4. The signal () was originally called V (from verhog, "to increment").

```
Signal(S)
{
    S++;
}
```

5. All modifications to the integer value of the semaphore in the wait() and signal () operation must be executed indivisibly.

4.10.1 Usage

OS often distinguishes between counting and binary semaphores. The value of counting semaphore can range over an unrestricted domain.

The value of binary semaphore can range only between 0 and 1. On some systems, these are known as mutex locks, as they are locks that provide mutual exclusion.

1. Binary semaphores are used to deal with the critical section problem for multiple processes. The n processes share a semaphore, mutex, initialized to 1.

```
do
{
    wait(mutex);
    //critical section
    signal(mutex);
    //remainder section
}
while(true);
```

2. Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to number of resources available.

(a) A process that wishes to use a resource, performs wait() operation on semaphore.

(b) A process when releases a resource, it performs a signal() operation.

(c) When the count for the semaphore goes to 0, all the resources are being used.

Semaphores can be used to solve various synchronization problems, e.g., consider two concurrently running processes P_1 with a statement S_1 & P_2 with a statement S_2 . Suppose we require that S_2 be executed only after S_1 has completed.

```
        S1;
        signal(synch);
in process P1 & the statements
        wait(synch);
        S2;
in process P2
```

4.10.2 Implementation

1. The main disadvantage of the semaphores definition given here is that it requires *busy waiting*.
2. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
3. This continual looping is clearly a problem in real multiprogramming system, where a single CPU is shared among many processes. Busy waiting wastes CPU cycles that some other process might be able to use productively.
4. This type of semaphore is also called spinlock because the process “spins” while waiting for the lock.

To overcome the need of busy waiting, we can modify the definition of wait () and signal () semaphore operations.

(a) The semaphore is defined as:

```
typedef struct
{
    int value;
    struct process *list;
}
semaphore;
```

(b) Wait() semaphore operation can be defined as:

```
wait(semaphore *S)
{
    S->value - -;
    if(S->value<0)
    {
        add this process to S->list;
    }
}
```

(c) The signal () operation can be defined as:

```
Signal (semaphore *S)
{
    S->value ++;
    if(S->value <=0)
    {
        remove a process P from S->list,
        wakeup(P);
    }
}
```

The block operation suspends the process that invokes it. The wakeup(P) operation resumes the execution of blocked process P.

6. The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is execution of signal() operation. When such a state is reached, then processes are said to be deadlocked.

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
Signal(S);	Signal(Q);
Signal(Q);	Signal(S);

7. Another problem related to deadlock is indefinite blocking or starvation. A situation in which processes wait indefinitely within the semaphore.

4.11 CLASSIC PROBLEMS OF SYNCHRONIZATION

4.11.1 The Bounded-Buffer Problem

It is commonly used to illustrate the power of synchronization primitives.

(a) Algorithm Description

Assuming that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to value 1. The empty and full semaphore count the number of empty and full buffers. The semaphore empty is initialized to value n ; the semaphore full is initialized to value 0.

(b) The structure of *Producer Process*:

```

do
{
...
// produce an item in next
...
wait(empty);
wait(mutex);
...
// add next p to buffer.
...
signal (mutex);
signal (full);

```



```

}
while (TRUE);

```

(c) The structure of consumer process

```

do
{
wait(full)
wait(mutex);

...
// remove an item from buffer to next C.

...
signal (mutex);
signal (empty);

...
// consume the item in next C.

...
} while (TRUE);

```

4.11.2 The Readers-Writers Problem

(a) Algorithm description

1. Suppose that a database is to be shared among several concurrent processes.
 - (a) Some of these processes may want only to read database, known as readers.
 - (b) Others may want to update the database, known as writers.
2. If two readers access the shared data simultaneously, no adverse effects will result. However, if a writer and some other (reader/writer) access the database simultaneously, chaos may be ensured.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database while writing to the database.

This synchronization problem is referred to as the readers-writers problem.

Solution:

No reader will be kept waiting until a writer has already obtained permission to use that object i.e., no reader should wait for other readers to finish because the writer is writing.

- (a) Once a writer is ready, writer performs its write as soon as possible that means if a writer is waiting to access object, no new reader can come and start reading. The variables used are as follows:

```

semaphore mutex, wrt;
int readcount;
mutex = wrt = 1;
readcount = 0;

```

(b) The structure of *Writer Process*

```
do
{
wait(wrt);
...
// writing is performed
...
signal (wrt);
}
while(TRUE);
```

(c) The structure of *Reader Process*

```
do
{
wait(mutex);
    readcount++;
    if(readcount==1)
        wait(wrt);
    signal(mutex);
    //reading is performed
    ...
wait(mutex)
readcount --;
if(readcount==0)
    signal(wrt);
    signal(mutex);
}
while(TRUE);
```

4.11.3 Dining Philosophers Problem

Consider 5 philosophers who spend their lives in thinking and eating. The philosophers share a circular table surrounded by 5 chairs, each belonging to one philosopher.

In the centre of the table is a bowl of rice, and the table is laid with 5 single chopsticks.

The situation of Dining-Philosophers is shown as:

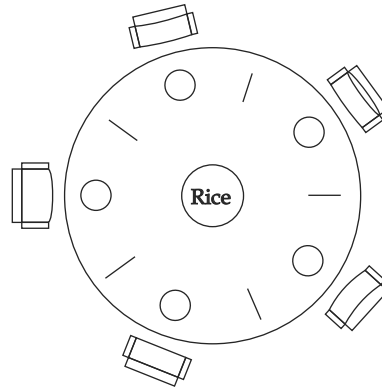


Fig. 4.13 Dining philosophers

- (1) When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her.
- (2) A philosopher may pick up only one chopstick at a time when a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

Problem Analysis

1. *Shared Resource*: Here each fork is shared by two philosophers.
2. *Race condition*: We do not want a philosopher to pick up a fork that has already being picked up by his neighbour. This is called race condition.

Philosopher states are described as:

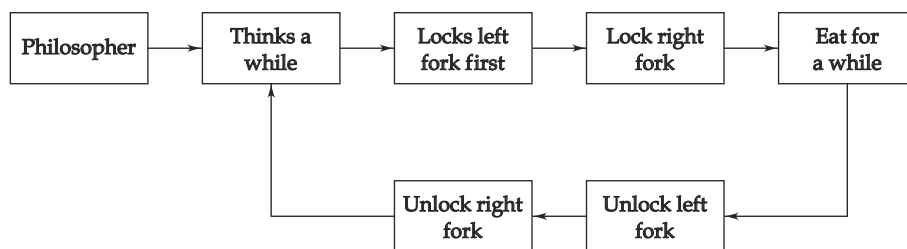


Fig. 4.14 Philosopher states

The structure of philosopher is:

```

do
{
wait(chopstick[i]);
wait(chopstick[i+1]%5);
...

```

```
        // eat
        ...
        signal(chopstick[i]);
        signal(chopstick[i+1]%5);
        ...
        //think
    }
    while(true);
```

Solution:

- (1) Allow at most 4 philosophers to be sitting simultaneously at the table.
- (2) Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- (3) Use an asymmetric solution; i.e., an odd philosopher picks up first her left chopstick and then her right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick.

```
# define N = 5;
# define left  (i + N - 1) % N;
# define right (i + 1) % N;
# define thinking 0;
# define eating 2;
    typedef int semaphore;
    int state N;
    semaphore mutex = 1;
    semaphore S[N]; //one semaphore for each purpose
    void philosopher(int i)
    .
    .
    .
    while(TRUE)
    {
        think()
        takeforks(i);
        eat();
        putfork(i);
    }
}
void takeforks(int i)
{
    down(&, mutex);
    state[i] = hungry
    test(i);
```

```

    up(&, mutex);
    down(&, S[i]);
}
void putfork(i)
{
    down(s, mutex);
    state[i]=thinking;
    test(left);
    test(right);
    up(&, mutex);
}
void test(i)
{
    if (state[i] == hungry && state[left] != eating && state [right] != eating)
    {
        state[i] = eating;
        up(&, mutex);
    }
do
{
    wait(chopstick[i]);
    wait(chopstick{[i+1]%5})
    eat;
    signal(chopstick[i]);
    signal(chopstick[(i+1)%5])
    think;
    while(1);
}

```

4.11.4 The Sleeping Barber Problem

(a) Algorithm description

1. Consider a barber's shop where there is only one barber, one barber chair and a number of waiting chairs for the customers.
2. When there are no customers, the barber sits on the barber chair and sleeps.
3. When a customer arrives he awakes the barber or waits in one of the vacant chairs if the barber is cutting someone else's hair.
4. When all the chairs are full, the newly arrived customer simply leaves.

(b) Problem

1. There might be a scenario in which the customer ends up waiting on a barber and a barber waiting on the customer which would result a deadlock.

2. There might also be a case of starvation when the customer doesn't follow an order to cut hair, as some people won't get a hair cut even though they had been waiting long.

(c) Barber code

```
# define chairs 5
typedef int semaphore;
    semaphore customers = 0;
    semaphore barber = 0;
    semaphore mutex = 1;
int waiting = 0;
void barber(void)
{
    while(TRUE)
    {
        down(& customers);
        down(& mutex);
        waiting = waiting - 1;
        up(& barber);
        up(& mutex);
        cut-hair();
    }
}
```

(d) Customer code

```
void customer(void)
down(& mutex)
if(waiting ≤ chairs)
    {waiting = waiting +1;
    up(& customers);
    up(& mutex);
    down(& barber);
    get haircut();
    }
else up(& mutex);
```

Summary

- Given a collection of co-operating sequential processes that share data, mutual exclusion must be provided to ensure that a critical section of code is used by only one process or thread at a time.

- Semaphores can be used to solve various synchronization problems and can be implemented efficiently, especially if hardware support for atomic operations is available.
- Various synchronization problems are important mainly, because they are example of a large class of concurrency-control problems. These problems are used to test nearly every newly proposed synchronization scheme.
- A mechanism that allows cooperating processes to exchange data and information is known as inter-process communication (IPC).
- The two fundamental models of inter-process communication are shared memory model and message passing model.
- Message passing may be either blocking or non-blocking.
- The situations, where several processes are allowed to access and manipulate the same data concurrently and the outcome of the execution depends on the order in which access takes place is called a race condition.
- Critical-section is a segment of code where the process may be exchanging common variables, updating a table, writing a file, etc.
- To solve the critical section problem, the three main requirements are: mutual exclusion, progress and bounded wait.
- In OS, to handle critical-section, the two approaches are: preemptive and non-preemptive kernels.
- Semaphore is a synchronization tool.
- Two types of semaphores are available, i.e., counting and binary semaphore.

Multiple-Choice Questions

1. The bounded buffer problem is also known as:
 - (a) Readers-Writers problem
 - (b) Dining-Philosophers problem
 - (c) Producer-Consumer problem
 - (d) None of these
2. In the bounded buffer problem, there are the empty and full semaphores that
 - (a) count the number of empty and full buffers
 - (b) count the number of empty and full memory spaces
 - (c) count the number of empty and full queues
 - (d) None of these
3. In the bounded buffer problem
 - (a) there is only one buffer
 - (b) there are n buffers (n being greater than one but finite)
 - (c) there are infinite buffers
 - (d) the buffer size is bounded
4. To ensure difficulties do not arise in the readers-writers problem, _____ are given exclusive access to the shared object.

- (a) readers
 - (b) writers
 - (c) none of these
5. The dining-philosophers problem will occur in case of
- (a) 5 philosophers and 5 chopsticks
 - (b) 4 philosophers and 5 chopsticks
 - (c) 3 philosophers and 5 chopsticks
 - (d) 6 philosophers and 5 chopsticks
6. A deadlock free solution to the dining philosophers problem
- (a) necessarily eliminates the possibility of starvation
 - (b) does not necessarily eliminate the possibility of starvation
 - (c) eliminates any possibility of any kind of problem further
 - (d) None of these
7. All processes share a semaphore variable **mutex**, initialized to 1. Each process must execute wait (mutex) before entering the critical section and signal (mutex) **afterward**.
- (i) Suppose a process executes in the following manner:
signal(mutex);
....
critical section
....
wait(mutex);
In this situation:
 - (a) a deadlock will occur
 - (b) processes will starve to enter critical section
 - (c) several processes may be executing in their critical section
 - (d) All of these
 - (ii) Suppose a process executes in the following manner (replacing signal with wait):
 - (a) a deadlock will occur
 - (b) processes will starve to enter critical section
 - (c) several processes may be executing in their critical section
 - (d) All of these
8. Consider the methods used by processes P1 and P2 for accessing their critical sections whenever needed, as given below. The initial values of shared Boolean variables S1 and S2 are randomly assigned.
- Method used by P₁:
- ```
while(S1==S2);
Critical section
S1 = S2;
```
- Method used by P<sub>2</sub>:
- ```
while(S1!=S2);
```


Critical section

S1 = not (S1);

Which of the following statements describes properties achieved?

- (a) Mutual exclusion but not progress
 - (b) Progress but not mutual exclusion
 - (c) Neither mutual exclusion nor progress
 - (d) Both mutual exclusion and progress
9. Bounded waiting implies that there exists a bound on the number of times a process is allowed to enter its critical section:
- (a) after a process has made a request to enter its critical section and before the request is granted
 - (b) when another process is in its critical section
 - (c) before a process has made a request to enter its critical section
 - (d) None of these
10. A minimum of _____ variable(s) is/are required to be shared between processes to solve the critical section problem.
- (a) one
 - (b) two
 - (c) three
 - (d) four
11. In the bakery algorithm to solve the critical section problem:
- (a) each process is put into a queue and picked up in an ordered manner
 - (b) each process receives a number (may or may not be unique) and the one with the lowest number is served next
 - (c) each process gets a unique number and the one with the highest number is served next
 - (d) each process gets a unique number and the one with the lowest number is served next
12. A semaphore is a shared integer variable
- (a) that cannot drop below zero
 - (b) that cannot be more than zero
 - (c) that cannot drop below one
 - (d) that cannot be more than one
13. Mutual exclusion can be provided by the
- (a) mutex locks
 - (b) binary semaphores
 - (c) both (a) and (b)
 - (d) none of the above
14. When high priority task is indirectly preempted by medium priority task effectively inverting the relative priority of the two tasks, the scenario is called
- (a) priority inversion
 - (b) priority removal
 - (c) priority exchange
 - (d) priority modification
15. Process synchronization can be done on
- (a) hardware level
 - (b) software level
 - (c) both (a) and (b)
 - (d) none of the above

16. A monitor is a module that encapsulates
(a) shared data structures
(b) procedures that operate on shared data structure
(c) synchronization between concurrent procedure invocation
(d) All of the above
17. Spinklocks are:
(a) CPU cycles wasting locks over critical sections of programs
(b) locks that avoid time wastage in context switches
(c) locks that work better on multiprocessor systems
(d) All of these
18. The main disadvantage of spinlocks is that
(a) they are not sufficient for many processes
(b) they require busy waiting
(c) they are unreliable sometimes
(d) they are too complex for programmers
19. The wait operation of the semaphore basically works on the basic _____ system call.
(a) stop() (b) block()
(c) hold() (d) wait()
20. The signal operation of the semaphore basically works on the basic _____ system call.
(a) continue() (b) wakeup()
(c) getup() (d) start()

Answers

- | | | | | | |
|------------|-------------|---------|---------|---------|---------|
| 1. (c) | 2. (a) | 3. (b) | 4. (b) | 5. (a) | 6. (b) |
| 7. (i) (c) | 7. (ii) (a) | 8. (d) | 9. (a) | 10. (b) | 11. (b) |
| 12. (a) | 13. (c) | 14. (a) | 15. (c) | 16. (d) | 17. (d) |
| 18. (b) | 19. (b) | 20. (b) | | | |

5.1 INTRODUCTION

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources, if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is not able to change its state, because the resources are held by other waiting process. This situation is called deadlock.

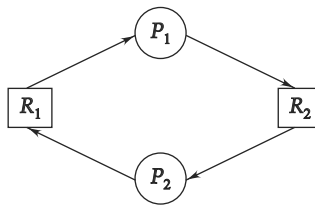


Fig. 5.1 Deadlock situation

Figure 5.1 illustrates the concept of deadlock. Here P_1 and P_2 are processes, R_1 and R_2 are the resources. Process P_1 acquires resource R_1 and needs resource R_2 to complete its execution, but process P_2 is having the resource R_2 , which needs resource R_1 to complete its execution, so both the processes are in waiting state, no one is able to complete its execution. This gives rise to a situation called deadlock.

5.2 SYSTEM MODEL

A system consists of a finite number of resources distributed among a number of competing processes.

Sources can be partitioned into resource types, each consisting of identical instances, e.g., many CPU cycles, files and I/O devices. If a process requests an instance of a source type, the allocation of any instance will satisfy the request.

A process must request a resource before using it and then release it after using it.

A process may request as many resources as required, but must not exceed the total number of resources available.

Sequence of Utilising a Resource

- Request
- Process must wait till it acquires the resource
- Use
- Release

A system table records the information about resources and processes.

If a process requests a resource that is currently allocated to another process, it can be added to a queue of processes waiting for this resources.

A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.

Multithreaded programs are good candidates for deadlock because multiple threads can compete for shared resources.

5.3 CAUSE OF DEADLOCK

Each process needs what other process has. For example:

1. Traffic only in one direction.
2. Each section of a bridge can be viewed as a resource.
3. If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback).
4. Several cars may have to be backed up if a deadlock occurs.
5. Starvation is possible.

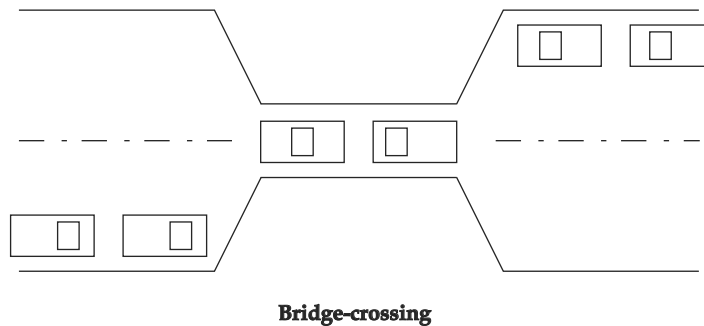


Fig. 5.2 Example of deadlock

5.4 NECESSARY CONDITIONS OF DEADLOCKS

Four conditions if occur simultaneously cause deadlock.

1. Mutual Exclusion: At least one resource must be in non-sharable mode. (Only one process at a time can use the resource.) If another process requests that resource, the requesting process must be delayed till the resource is released.

2. Hold and wait: A process must be holding at least one resource and waiting to acquire additional sources currently being held by other processes.

3. Preemption: Resources cannot be preempted (resource can be released voluntarily by the process).

4. Circular wait: A set $\{P_0 P_1 \dots P_n\}$ exists, such that P_0 is waiting for a resource held by $P_1 \dots P_{n-1}$ is waiting for P_n and P_n is waiting for P_0 .

Circular wait implies hold and wait.

5.5 RESOURCE ALLOCATION GRAPH (RAG)

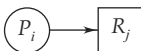
Deadlock can be described more precisely in terms of a directed graph called system resource allocation graph.

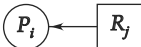
$r = (V_1 F_2) \rightarrow$ graph contains nodes and edges.

V Nodes consist of processes $\{P_1, P_2, P_3\}$ and resource types $\{R_1 R_2, R_3\}$

E Edges are (P_i, R_j) or (R_i, P_j)

An arrow from process to resource indicates the process is requesting the resource

(request edge) 

An arrow from resource to process shows an instance of the resource has been allocated to the process (assignment edge) 

Process is a circle, resource type is a square. Dots represent number of instances of resource in type.

Requests point to square, assignment comes from dot.

If a graph contains no cycle $\Rightarrow$ no deadlock

If a graph contains a cycle

- if only one instance per resource type, then deadlock
- if several instances per resource type, possibility of deadlock.

5.6 RAG WITH A CYCLE BUT NO DEADLOCK

When process P_i requests an instance of resource type R_j , a request edge is inserted and when fulfilled, it is instantaneously transformed to as assignment edge.

Consider a system with following sets of information

$P = \{P_1, P_2, P_3\}$.

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

Resource instances:

One instance of resource R_1

Two of R_2 .

One of R_3 , and 3 of R_4

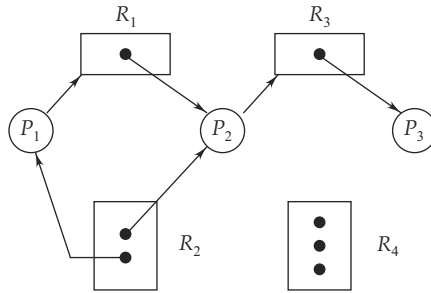


Fig. 5.3 Resource allocation graph

Process states

P_1 is holding an instance of R_2 and is waiting for R_1 ;

Process P_2 is holding an instance of R_1 and an instance of R_2 and is waiting for an instance of R_3 .

Process P_3 is holding an instance of R_3 . The RAG is drawn as Fig. 5.3.

This shows that a cycle in a graph is necessary but not a sufficient condition for a deadlock.

If P_3 requests an instance of resource R_2 $P_3 \rightarrow R_2$ is added.

Process P_1 , P_2 and P_3 are deadlocked (Fig. 5.4).

Two minimal cycles exist in the system.

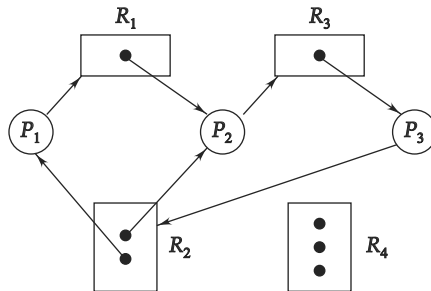


Fig. 5.4 Resource allocation graph with a deadlock

Figure 5.4 depicts the graph with a cycle and a deadlock.

In Fig. 5.5 there is a cycle but no deadlock. Process P_4 may release its instance, that can then be allocated to P_3 breaking the cycle.

The system states are written as follows:

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

$$P = \{P_1, P_2, P_3, P_4\}$$

$$R = \{R_1, R_2\}.$$

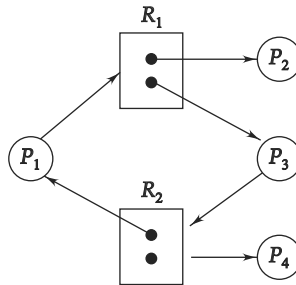


Fig. 5.5 Resource allocation graph with a cycle but no deadlock

5.7 METHODS FOR HANDLING DEADLOCK

- Ensure that the system never enters a deadlock state (avoidance/prevention)
- Allow the system to enter the system and then the resource.
- Ignore the problem and pretend that deadlocks never occur in the systems.

Prevent any one of the four conditions from happening.

Allow all dedlock conditions, but detect cycles about to happen and handle dangerous operations.

5.8 DEADLOCK PREVENTION

Deadlock prevention is preventing occurrence of deadlock by ensuring that at least one of the necessary conditions cannot hold.

Mutual Exclusion

It may hold for non-sharable resources as shared entities don't need mutual exclusion. Prevention is not possible (read only files) as some resources are intrinsically non-sharable.

Hold and Wait

Ensure whenever a process requests a resource, it does not hold any other resource.

Disadvantage: Low utilization, starvation is possible.

Two approaches of hold and wait

Approach I

Process requests all of its resources at one time and blocks it until all requests are granted simultaneously.

Disadvantages

- Process may be held up for a long waiting for all of its resource requests to be accepted.
- Resources allocated may remain unused.
- Process may not know in advance, e.g., process must initially request the tape drive, disk file and the printer. The printer is held for the entire execution.
- Low utilization.

Approach II

Process requests resources only when the process has none.

A process may request some resources and use them. Before it can request any additional resource, it must release all its resources currently allocated.

Disadvantage

It may lead to starvation, e.g., if the process requests the tape drive. It will release both the resources (tape drive and disk file) after copying from tape drive to disk file.

Now it will again request disk file and printer and then release both.

A process may have to wait indefinitely because at least one of the resource it needs is always allocated to some other process.

No Preemption

- (a) Release resources already being held if can't get an additional resource.
- (b) If a needed resource is held by another process, then process should wait or steal it.
- (c) Possible for CPU registers and memory space but not possible for printers and tape drivers.

Circular Wait

- Imposes a total ordering of all resources and each process must request resources in an increasing order of enumeration.

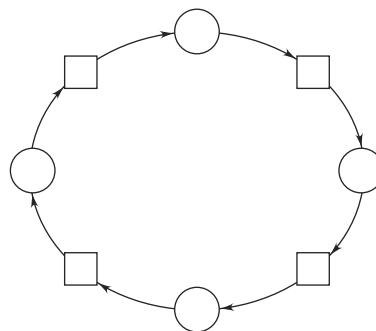


Fig. 5.6 Circular wait

- Whenever a process requests an instance of resource type R_j , it has released any other resources R_i such that $F(R_i) \geq F(R_j)$

5.9 DEADLOCK AVOIDANCE

Ensure that there can never be a circular wait condition.

Possible states:

1. **Deadlock:** No forward progress can be made.
2. **Unsafe state:** A state that may allow deadlock.
3. **Safe state:** A state is safe if a sequence of processes exist such that there are enough resources for the first to finish and as each finishes and releases its resources there are enough for the next.

5.9.1 Safe State

To illustrate safe state, we consider a system with 12 magnetic tape drives and processes: P_0 , P_1 and P_2 .

	Maximum needs	Current needs
P_0	10	5
P_1	4	2
P_2	9	2

The sequence $\langle P_1 P_0 P_2 \rangle$ satisfies the safety condition.

Note: It is possible to go from a safe state to an unsafe state. If P_2 is granted one more tape drive, user would reach in unsafe state.

Thus, to ensure that the system never reaches deadlock, we can define avoidance algorithms.

Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or if the process must wait. The resource is granted only if the allocation leaves the system in safe state.

Thus, resource utilization may be lower than it would be without a deadlock avoidance algorithm.

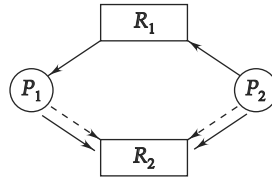
5.9.2 Resource Allocation Graph Algorithm

In addition to the request and assignment edges, we introduce a new type of edge, called a claim edge.

A claim edge $(P_i) \cdots \rightarrow (R_j)$ indicates that process may request resource R_j , and is represented by dashed line.

Resources must be claimed prior from the system. Claim edge can be converted to a request edge and assignment to claim edge. A claim edge can be added to the system only if all the edges associated with the process are claim edges, otherwise they need to be mentioned before the execution. The request can be granted only if it does not form a cycle.

We check cycle by using cycle detection algorithm. An algorithm for detecting a cycle requires an order of n^2 operations, where n is the number of processes.



Although R_2 is free, but cannot be allocated to P_2 , as it will create a cycle.

5.9.3 Banker's Algorithm

The resource allocation graph algorithm does not apply to a resource allocation system with multiple instances of each resource type, for such a situation Banker's algorithm is used. Banker's algorithm could be used in a banking system to ensure that the bank never allocates its available cash such that it can no longer satisfy the needs of all its customers.

When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system. When a user request a set of resources, the system must determine, whether the allocation of these resources will leave the system in a safe state. If so, then the resources are allocated else the process must wait until some other process releases enough resources. Banker's algorithm is less efficient than the resource allocation graph algorithm.

Let n be the number of processes in the system and m be the number of resource types. We need the following data structure to implement Banker's algorithm.

1. **Available:** A vector of length m indicates the number of available resource of each type, if available $[j] = k$, there are k instances of resource type R_j in the system.
2. **Max:** An $(n * m)$ matrix defines the maximum demand of each process. If $\max [i, j] = k$, then process P_i may request at most k instances of resource type, R_j .
3. **Allocation:** An $(n * m)$ matrix defines the number of resources of each type, currently allocated to each process. If allocation $[i, j] = k$, then process P_i is currently allocated k instance of resource type, R_j .
4. **Need:** An $(n * m)$ matrix indicates the remaining resource need of each process. If need $[i, j] = k$, then process P_i may need k more instances of resource type R_j to complete its task.

$$\text{Need } [i, j] = \text{Max } [i, j] - \text{Allocation } [i, j]$$

These data structures may vary over time in both size and value. For each row in the matrices, allocation and need can be treated as a vector. So we can refer to them as allocation i and Need i respectively. The vector allocation, specifies the resources

currently allocated to process P_i , the vector need_i , specifies the additional resources that process P_i may still request to complete its task.

(a) Safety algorithm

1. Let work and finish be vectors of length m and n .
Initialize work = Available and finish $[i] = \text{false}$
 2. Find an i such that both
Finish $[i] = \text{false}$.
 $\text{Need}_i \leq \text{work}$
If no such i exists, go to step 4.
 3. Work = work + allocation $_i$
Finish $[i] = \text{true}$
go to step 2.
- If finish $[i] = \text{true}$, then the system is in a safe state.
The algorithm may require $m \times n^2$ operations to decide about the safe state.

(b) Resource-request algorithm

Let request i be the request vector for process P_i .

1. If request $_i \leq \text{Need}_i$ go to step 2.
2. If request $_i \leq \text{Available}$ go to step 3 otherwise P_i must wait.
3. Have the system pretend to have allocated the requested resource, modify as follows.

$$\begin{aligned}\text{Available} &= \text{Available} - \text{Request}_i \\ \text{Allocation}_i &= \text{Allocation}_i + \text{Request}_i \\ \text{Need}_i &= \text{Need}_i - \text{Request}_i\end{aligned}$$

5.9.4 Limitations of Banker's Algorithm

1. It is very conservative in nature.
2. New processes arriving work well only:
 - (a) Process requested claim is less than the total number of units.
 - (b) When new process can be placed at the end (provided system was earlier safe).
 - (c) Insuring fairness (starvation freedom) needs a little more work.
3. A resource can become unavailable, that can lead to unsafe state.
4. Only works with fixed number of resources and processes.
5. Unnecessary delay in avoiding unsafe states.
6. Not suitable for multi-access systems.

5.9.5 Numerical Examples

Consider the system with five processes, P_0 through P_4 and three resource type ABC. Resource A has 10 instances, B has 5 and C has 7 instances. At time T_0 the following snapshot is given: Determine the need matrix and safe state.

	Allocation	Max	Available
	A B C	ABC	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Solution: The content of the matrix need are as follows:

$$\text{Need}_i = [\text{Max}_i - \text{allocated}_i]$$

	A B C	Instances of each resource	Available A B C
P_0	7 4 3	$A = 10$	2 3 0
P_1	1 2 2	$B = 5$	
P_2	6 0 0	$C = 7$	
P_3	0 1 1		
P_4	4 3 1		

Currently the system is in safe state. $\langle P_1 P_3 P_4 P_2 P_0 \rangle$ sequence satisfy the safety criteria

New Request

If it would have been for process P_1 , $\text{request}_1 = (102)$, since $\text{request}_1 < \text{available}$

$$(1, 0, 2) \leq (3, 3, 2)$$

The new state now becomes:

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Applying safety algorithm, the sequence becomes $\langle P_1 P_3 P_4 P_0 P_2 \rangle$ and is safe.

Hence, request can be immediately granted to process P_1 .

However, when the system is in this state, a request for $(3, 3, 0)$ by P_4 cannot be granted, (request > available) because resources are not available.

A request for $(0, 2, 0)$ by P_0 can also be not granted even though it satisfies $\text{request} \leq \text{available}$. This allocation would lead to an unsafe state.

5.10 DEADLOCK DETECTION

If the system does not employ deadlock prevention or avoidance, then it must provide:

- (i) an algorithm that examines the state of system to whether the deadlock has occurred.

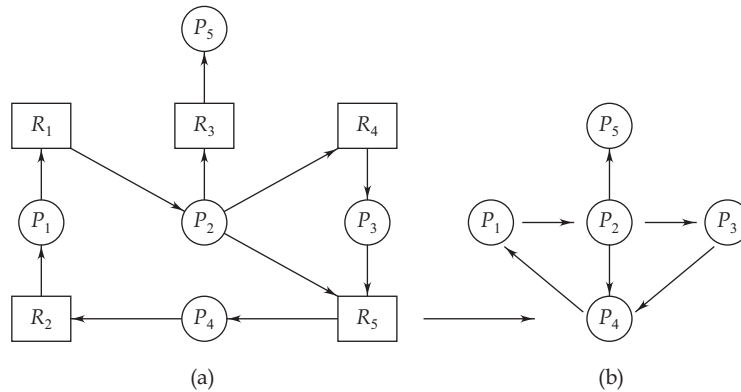


Fig. 5.7 (a) RAG (b) Wait for graph

(ii) An algorithm to recover from deadlock.

(iii) Wait for graph is used.

Figure 5.7 shows a RAG and its corresponding wait for graph.

5.10.1 Resource Allocation with Single Instances

If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of resource allocation graph called a wait for graph. It is obtained by remaining the resource nodes and collapsing the appropriate edges. An edge from $p(j)$ to $p(i)$ implies that $p(j)$ is waiting for $p(i)$ to release.

A deadlock exists in the system if and only if it contains a cycle.

The algorithm requires an order of n^2 operations.

5.10.2 Several Instances of Resources

Complexity: $m \times n^2$

We need to keep track of

- (i) Available – A vector of length m indicating the number of available resources of each type
- (ii) Allocation – An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- (iii) Request – An $n \times m$ matrix indicates the current request of each process. If Request $[i][j] = k$ then P_i is requesting k more instances of resources.

Let work and finish be vectors of lengths m and n respectively.

5.10.3 Detection of Algorithm

1. Initialize work = available
for $i = 0, 1, 2 \dots n - 1$
if allocation _{i} != 0 then finish $[i] = \text{false}$ otherwise finish $[i] = \text{true}$
2. Find an i such that finish $[i] == \text{false}$ and request _{i} ≤ work
If no such i exists go to 4

3. Work = work + allocation_i
 finish [i] = true
 goto 2.
4. if finish [i] == false, then deadlock.

5.10.4 Numerical Example

Consider a system in which we have 3 resources A, B and C. A has 7 instances, B has 2 and C has 6. Allocation: at time to is given as follows:

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

This system is not in a deadlock state and the sequence $\langle P_0 P_2 P_3 P_1 P_4 \rangle$ results in finish [i] == true for all i. Suppose P_2 requests an additional instance.

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 1	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

The above matrix will lead the system into deadlock.

5.11 RECOVERY FROM DEADLOCK

There are two options:

1. Simply abort one or more processes to break the circular wait.
2. Preempt some resources from one or more of the deadlocked processes.

(a) Process Termination

1. Abort all deadlocked processes
 - this is expensive
2. Abort one at a time until the deadlock is broken. This is time consuming, since after each process is aborted the algorithm must be invoked.
3. Selecting the process is based upon process priority, time executed, time to completion, needs for completion, or depth of rollback.
4. Its easier to preempt the resource, than to terminate the process.

(b) Resource Preemption

1. Select a victim which resource to preempt and which process to minimise cost.
2. Rollback to previously defined safe state.
3. Prevent one process from always being preempted. (i.e. starvation)

It is difficult to determine a safe state, the simplest solution is total rollback. It must be ensured that the process can be picked as a victim only a finite number of times.

5.11.1 Deadlock State

1. A deadlock state occurs when 2 or more processes are waiting indefinitely for an event.
2. A deadlock can occur only if four necessary conditions hold simultaneously: mutual exclusion, hold and wait, no preemption and circular wait.
3. To prevent deadlock, we must ensure that at least one of the necessary conditions does not hold.
4. Combined approach to deadlock handling. Combine the three basic approaches (prevention, avoidance and detection) allowing the use of the optimal approach for each class of resources in the system.
5. A deadlock detection algorithm must be invoked to determine whether the deadlock has occurred.
6. None of the basic approach alone is appropriate for the entire spectrum of resource allocation problem.
7. In a system that selects victims for rollback, primarily on the basis of cost factor, starvation may occur, and the selected process can never complete its designated task.
8. A method for avoiding deadlock requires the prior information about processes and resources.
9. Deadlocks can be handled in three major ways: prevent them, detect them or ignore them.
10. In deadlock avoidance, processes can be blocked for long.
11. Deadlock detection can lead to inherent preemption losses.
12. Deadlock prevention is inefficient and restrictive.
13. Avoidance schemes require specification of future needs.
14. Avoidance schemes work with global information, therefore, are less conservative.
15. RAG algorithm considers resources with only one instance.
16. Use most appropriate technique for handling deadlocks within each class.

5.12 STARVATION

Starvation is a situation in which some clients cannot access a service, others can. It occurs when a scheduler process refuses to give a particular thread any quantity of a particular resource. If there are too many high priority threads, a lower priority thread may be starved.

This can have negative impacts, though, particularly when the lower priority thread has a lock on some resource.

It means two threads want the same resource, and one always gets it in favour of the other.

It arises due to deadlock.

If same process is selected as a victim for rollback again and again, then it is known as starvation.

Table 5.1 Summary of Deadlock Handling Strategic

Principle	Resource allocation policy	Different schemes	Major advantages	Major disadvantages
(1) Prevention		Requesting all resources at once	• Work well for processes that perform a single burst of activity. No preemption, necessary	• Inefficient • Delays process initiation • Future resource requirements must be known
	Conservative, undercommits resources	Preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary • Subject to cyclic restart
		Resource ordering	• Fearable to enforce via compile time checks • Needs no run-time computation since problem is solved in system design	• Preempts without much time • Disallows increment resource requests
(2) Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known. • Processes can be blocked for long periods.
(3) Detection	Very liberal; requested resources are granted where possible	Involve periodically to test for deadlock	• Never delays process initiation • Facilitates online handling	• Inherent preemption losses

Numerical Examples

Deadlock Avoidance: (Banker's Algorithm)

5.1. Consider the following snapshot of a system. Which 1 condition out of 4 conditions Banker's algorithm avoid?

	Allocation	Max	Available
	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

Answer the following questions using the Banker's algorithm:

- What is the content of the matrix Need?
- Is the system in a safe state?
- If a request from process P_1 arrives for (0, 4, 2, 0), can it be granted immediately?

Sol. (a) Need $[i, j] = \text{Max } [i, j] - \text{Allocated } [i, j]$

	Need			
	A	B	C	D
P_0				
P_1	0	0	0	0
P_2	1	0	0	2
P_3	0	0	2	0
P_4	0	6	4	2

(b) Applying safety algorithm

Work = Available, Finish $[i] = \text{false}$ for $i = 0, 1, 2, 3, 4$.

Finish $[0] = \text{false}$, Need $<$ available (or work)

$\Rightarrow$ work = [1520 + 0012] fi [1532], finish $[0] = \text{true}$

Finish $[1] = \text{false}$ need $_1 <$ available, [1532] ABCD

Finish $[2] = \text{false}$ need $_2 <$ available, work = 1354 fi available = [2 8 8 6] finish $[2] = \text{true}$

Finish $[3] = \text{false}$ need $_3 <$ work work = 0632 fi available = [2 14 11 8] finish $[3] = \text{true}$
2 14 11 12

Finish $[4] = \text{false}$ need $_4 <$ work, work = 0014 $\Rightarrow$ available [2 14 12 16] finish

$\therefore$ Finish $[i] = \text{True}$ for all $i = 0, 1, 2, 3, 4$

Hence System is in safe state $< P_0, P_2, P_3, P_4, P_1 >$

(c) If P_1 requests for (0, 4, 2, 0) then,

(i) request $_1 = [0420] \leq \text{need}_i$

(ii) request $_1 <$ available $\therefore$ System is safe state, request can be granted immediately $< P_0, P_2, P_3, P_1, P_4 >$

(iii) Available = 1 5 2 0 $\Rightarrow$ 1 1 0 0

$$\begin{array}{r} - 0 4 2 0 \\ \hline 1 1 0 0 \end{array}$$

$$\text{Allocation}_i = 1 \ 0 \ 0 \ 0 \Rightarrow \underline{1 \ 4 \ 2 \ 0}$$

$$+ 0 \ 4 \ 2 \ 0$$

$$\text{Need}_i = 0 \ 7 \ 5 \ 0 \Rightarrow \underline{0 \ 3 \ 3 \ 0}$$

$$- 0 \ 4 \ 2 \ 0$$

Apply safety algorithm work = available = 1100

1100

Finish[0] = false, need₀ < work ⇒ work = 0012 ⇒ available = [1, 1, 1, 2]; finish[0] = true

Finish[1] = false, need₁ < work 1112

Finish[2] = false, need₂ < work ⇒ work = 1354 ⇒ available = [2, 4, 6, 6]; finish[2] = true

Finish[3] = false, need₃ < work ⇒ work = 0020 ⇒ available = [2, 4, 8, 6], finish[3] = true

Finish[4] = false, need₄ < work available = [2, 4, 8, 6]

Finish[1] = false, need₁ < work ⇒ work = 1420 ⇒ available = [3, 8, 10, 6] finish = true

Finish[4] = false, need₄ < work ⇒ work = 0014 ⇒ available = [3, 8, 11, 10]

Deadlock avoidance (Banker's algorithm)

5.2. Consider 5 processes P_0 through P_4 : 3 resources types A (10 instances), B (5 instances) and C (7 instances).

Snapshot at time T_0 .

	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

(a) Define matrix Need.

Matrix Need Max-Allocation.

	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

(b) Is system in safe state ⇒ Yes

Sol. Safety algorithm.

finish[i] = false, for all i = 0, 1, 2, 3, 4; work = Available

finish[0] = false

finish[1] = true, work = [6 5 4]

finish[2] = true, work = [12 5 4]

finish[3] = true, work = [12 6 5]

finish[4] = true, work = [16 9 6]

finish[0] = true, work = [23 13 9] Safe sequence is $\langle P_1 P_2 P_3 P_4 P_0 \rangle$

(c) P_1 requests (1, 0, 2), can it be granted immediately?

Sol. Request₁ < Need₁ Yes

$$[1 \ 0 \ 2] < [1 \ 2 \ 2]$$

Request₁

is $[1 \ 0 \ 2] < \text{available } [3 \ 3 \ 2]$ yes

Now Allocation = 2 0 0

$$+ 1 \ 0 \ 2 \Rightarrow 3 \ 0 \ 2$$

Available = 3 3 2

$$- 1 \ 0 \ 2 \Rightarrow 2 \ 3 \ 0$$

Need = 1 2 2

$$- 1 \ 0 \ 2 \Rightarrow 0 \ 2 \ 0$$

Execute Safety Algorithm.

finish[0] = false

finish[1] = true $\therefore$ need1 < available, work = [5 3 2]

finish[2] = false $\therefore$ need2 > available

finish[3] = true $\therefore$ need3 < available work = [7 4 3]

finish[4] = true $\therefore$ need4 < available work = [7 4 5]

finish[0] = true $\therefore$ need0 < work work = [7 5 5]

finish[2] = true work = [7 5 5]

$$+ [3 \ 0 \ 1] \Rightarrow [10 \ 5 \ 5]$$

$\therefore$ finish[i] = true $\forall i = 0, 1, 2, 3, 4$. Thus system is in safe state with safe sequence $\langle P_1 P_3, P_4, P_0, P_2 \rangle$.

Deadlock Detection Algorithm

5.3. Consider five processes P_0 through P_4 ; three resource types A(7 instances), B(2 instances), and C(6 instances)

Snapshot: at time T_0

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Determine whether the system is deadlocked or not

initialize, work = available = 0 0 0

Sol. Finish = false $\forall i = 0, 1, 2, 3, 4$.

1. Finish[0] = false, request₀ ≤ work = [0, 1, 0], finish[0] = true
2. Finish[1] = false, request₁ ≤ work
3. Finish[2] = false, request₂ ≤ work, work = [3 1 3], finish[2] = true
4. Finish[3] = false, request₃ ≤ work, work = [5 2 4], finish[3] = true
5. Finish[4] = false, request₄ ≤ work, work = [5 2 6], finish[4] = true
6. Finish[1] = false, request₁ ≤ work $\Rightarrow$ work = 7 2 8, finish[1] = true

Thus, finish[1] = true $\forall i = 0, 1, 2, 3, 4$.

Thus, system is in safe state with a safe sequence of $\langle P_0 P_2 P_3 P_4, P_1 \rangle$.

No, system is not deadlocked.

5.4 Consider the following snapshot of system at time T_0 .

	Allocation	Max	Available
	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	2 1 0 0
P_1	2 0 0 0	2 7 5 0	
P_2	0 0 3 4	6 6 5 6	
P_3	2 3 5 4	4 3 5 6	
P_4	0 3 3 2	0 6 5 2	

(i) Calculate the need matrix.

(ii) State whether the system is safe? If yes, give safe sequence.

(iii) For the system snapshot at time T_0 . If a request from P_2 arrives for (0, 1, 0, 0). Can be request be safely granted?

(iv) For the system snapshot at time T_0 . If a request from P_3 arrives for (1, 0, 0, 0). Can be request be safely granted?

Sol. (i) Need = Max - Allocation

	A	B	C	D
P_0	0	0	0	0
P_1	0	7	5	0
P_2	6	6	2	2
P_3	2	0	0	2
P_4	0	3	2	0

(ii) Finish[0] = true, work = [0, 0, 1, 2] + [2, 1, 0, 0] $\Rightarrow$ [2, 1, 1, 2]

Finish[1] = false

Finish[2] = false

Finish[3] = true, work = [2, 1, 1, 2] + [2, 3, 5, 4] $\Rightarrow$ [4, 4, 6, 6]

Finish[4] = true, work = [4, 4, 6, 6] + [0, 3, 3, 2] $\Rightarrow$ [4, 7, 9, 8]

Finish[1] = true, work = [4, 7, 9, 8] + [2, 0, 0, 0] $\Rightarrow$ [6, 7, 9, 8]

Finish[2] = true, work = [6, 7, 9, 8] + [0, 0, 3, 4] $\Rightarrow$ [6, 7, 12, 12]

Finish[i] = true; i = 0, 1, 2, 3, 4

System is in safe state $\langle P_0 P_3 P_4 P_1 P_2 \rangle$ safe sequence.

- (iii) P_2 request (0, 1, 0, 0): request₂ < need₂ (6, 6, 2, 2) [Max - allocated]
request₂ < available (2, 1, 0, 0)

So it can be allocated.

Now Allocation = 0 0 3 4 Need: 6 6 2 2 Available: 2 1 0 0

+ 0 1 0 0	- 0 1 0 0	0 1 0 0
0 1 3 4	6 5 2 2	2 0 0 0

for new allocated, need & available matrix, we execute safety matrix

Finish[0] = true, work =

Finish[1] = false = [2, 0, 0, 0] + [0, 0, 1, 2] $\Rightarrow$ [2, 0, 1, 2]

Finish[2] = false

Finish[3] = true, work = [2, 0, 1, 2] + [2, 3, 5, 4] $\Rightarrow$ [4, 3, 6, 6]

Finish[4] = true, work = [4, 3, 6, 6] + [0, 3, 3, 2] $\Rightarrow$ [4, 6, 9, 8]

Finish[1] = false, finish[2] = false

Finish[i] $\neq$ true for i = 1, 2 so, system is *not in safe state*.

- (iv) P_3 (1, 0, 0, 0): request < need (2002)
request < available (2, 1, 0, 0)

Allocation = 2 3 5 4 Need: 2 0 0 2 Available: 2 1 0 0

+ 1 0 0 0	- 1 0 0 0	1 0 0 0
3 3 5 4	3 0 0 2	3 1 0 0

Finish[0] = true, work = [3, 1, 0, 0] + [0, 0, 1, 2] = [3, 1, 1, 2]

Finish[1] = false

Finish[2] = false

Finish[3] = true, work = [3, 1, 1, 2] + [2, 3, 5, 4] = [5, 4, 6, 6]

Finish[4] = true, work = [5, 4, 6, 6] + [0, 3, 3, 2] = [5, 7, 9, 8]

Finish[1] = true, work = [5, 7, 9, 8] + [2, 0, 0, 0] = [7, 7, 9, 8]

Finish[2] = true, work = [7, 7, 9, 8] + [0, 0, 3, 4] = [7, 7, 12, 12]

Finish[i] = true, i = 1, 2, 3, 4.

System is in safe state. $\langle P_0 P_3 P_4 P_1 P_2 \rangle$ safe sequence.

5.5. Consider the following current resource allocate state:

Process	Allocation	Max	Available
	$R_1 R_2 R_3$	$R_1 R_2 R_3$	$R_1 R_2 R_3$
P_1	2 2 3	3 6 8	7 7 10
P_2	2 0 3	4 3 3	
P_3	1 2 4	3 4 4	

- (a) Is the current allocation state safe?

(b) Would the following requests be granted in the current state if

- Process P_1 requests (1, 1, 0)
- Process P_2 requests (0, 1, 0)
- Process P_3 requests (0, 1, 0)

Sol. (a) $\text{Need}_i = \text{Max}_i - \text{Allocated}_i$

	Need		
	A	B	C
P_1	1	4	5
P_2	2	3	0
P_3	2	2	0

$\text{finish}[1] = \text{true}$, $\text{work} = [7, 7, 10] + [2, 2, 3] \Rightarrow [9, 9, 13]$

$\text{finish}[2] = \text{true}$, $\text{work} = [9, 9, 13] + [2, 0, 3] \Rightarrow [11, 9, 16]$

$\text{finish}[3] = \text{true}$, $\text{work} = [11, 9, 16] + [1, 2, 4] \Rightarrow [12, 11, 20]$

(b) P_1 request (1, 1, 0) < need (1, 4, 5)

(1, 1, 0) < available (7, 7, 10)

So, it can be allocated.

Now, Allocation = 2 3 3 Need: 1 4 5 Available: 7 7 10

$$\begin{array}{r} + 1 \ 1 \ 0 \\ \hline 3 \ 3 \ 3 \end{array}$$

$$\begin{array}{r} - 1 \ 1 \ 0 \\ \hline 0 \ 3 \ 5 \end{array}$$

$$\begin{array}{r} - 1 \ 1 \ 0 \\ \hline 6 \ 6 \ 10 \end{array}$$

	Allocation	Max	Available	New Need
	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$
P_1	3 3 3	3 6 8	6 6 10	0 3 5
P_2	2 0 3	4 3 3		2 3 0
P_3	1 2 4	3 4 4		2 2 0

$\text{Finish}[1] = \text{true}$, $\text{work} = [6, 6, 10] + [3, 3, 3] \Rightarrow [9, 9, 13]$

$\text{Finish}[2] = \text{true}$, $\text{work} = [9, 9, 13] + [2, 0, 3] \Rightarrow [11, 9, 16]$

$\text{Finish}[3] = \text{true}$, $\text{work} = [11, 9, 16] + [1, 2, 4] \Rightarrow [12, 11, 20]$

Yes safe with $\langle P_1, P_2, P_3 \rangle$ sequence

(b) (ii) P_2 requests (0, 1, 0) < need < available (7, 7, 10)

	Allocation	Max	Available	Need
	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$
P_1	2 2 3	3 6 8	7 6 10	1 5 5
P_2	2 1 3	4 3 3		2 2 0
P_3	1 2 4	3 4 4		2 2 0

$\text{Finish}[1] = \text{true}$, $\text{work} = [2, 2, 3] + [7, 6, 10] \Rightarrow [9, 9, 13]$

$\text{Finish}[2] = \text{true}$, $\text{work} = [9, 9, 13] + [2, 1, 3] \Rightarrow [11, 10, 16]$

Finish[3] = true, work = [11, 10, 16] + [1, 2, 4] $\Rightarrow$ [12, 12, 20]

Yes safe with $\langle P_1, P_2, P_3 \rangle$ safe sequence.

(iii) P_3 request (0, 1, 0) $<$ need (2, 2, 0) $<$ available (7, 7, 10)

	Allocation	Max	Available	Need
	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$
P_1	2 2 3	3 6 8	7 6 10	1 4 5
P_2	2 0 3	4 3 3		2 3 0
P_3	1 3 4	3 4 4		2 1 0
		7 6 10		

Finish[1] = true, work = [2, 2, 3] + [7, 6, 10] $\Rightarrow$ [9, 8, 13]

Finish[2] = true, work = [9, 8, 13] + [2, 0, 3] $\Rightarrow$ [11, 8, 16]

Finish[3] = true, work = [11, 8, 16] + [1, 3, 4] $\Rightarrow$ [12, 11, 20]

Yes safe with $\langle P_1, P_2, P_3 \rangle$ safe sequence

5.6 Describe the Banker's algorithm for safe allocation. Consider a system with five processes and three resource types and at time t_0 the following snapshot of the system has been taken:

	Allocated	Maximum	Available
Process ID	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$	$R_1 \ R_2 \ R_3$
P_1	1 1 2	4 3 3	3 1 0
P_2	2 1 2	3 2 2	
P_3	0 2 0	9 0 2	
P_4	0 2 0	7 5 3	
P_5	1 1 2	11 2 3	

(i) Determine the total amount of resources of each type.

(ii) Compute the need matrix.

(iii) Determine if the state is safe or not using Banker's algorithm.

(iv) Would the following request be granted in the current state?

(a) $P_1 < 3, 3, 1 >$ (b) $P_2 < 2, 1, 0 >$

Sol. (i) $R_1 = (8 + 3) = 11$

$R_2 = (5 + 1) = 6$

$R_3 = (7 + 0) = 7$

(ii) Need = Maximum - Allocation

	R_1	R_2	R_3
P_1	3	2	1
P_2	1	1	0
P_3	5	0	1
P_4	7	3	3
P_5	10	1	1

- (iii) Finish[1] = false, finish[2] = true, work = [2, 1, 2] + [3, 1, 0] $\Rightarrow$ [5, 2, 2]
 finish[3] = true, work = [5, 2, 2] + [0, 2, 0] = [5, 4, 2]
 finish[4] = true, work = [5, 4, 2] + [0, 2, 0] = [5, 6, 2]
 finish[5] = finish[1] = true, work = 1 1 2 $\Rightarrow$ 10 5 5 [5, 6, 2] + [1, 1, 2] = [6, 7, 4]
 finish[5] = true, work = [6, 7, 4] + [1, 1, 2] $\Rightarrow$ [7, 8, 6]
 $\therefore$ finish[i] = true $\forall$ i = 1, 2, 3, 4, 5. System is in safe state with a safe sequence of $\langle P_1, P_3, P_4, P_1, P_5 \rangle$
- (iv) (a) P_1 request $\langle 3, 3, 1 \rangle < \text{need}(3, 2, 1)$
 P_1 request can't be granted as per Banker's algorithm.
- (b) $P_2 \langle 2, 1, 0 \rangle < \text{need}(1, 1, 0)$
 request P_2 can't be granted
- Resource allocation algorithm in Banker's algorithm do not allocate the requested resources to a process whose request is greater than the need of the process. Otherwise, it will lead to unsafe state.

5.7 Consider a system with five process – P0 to P4 & three resources A, B, C. Given that:

Resource type A has 10 instances

Resources type B has 5 instances

Resource type C has 7 instances

Suppose that at that time, t_0 the following snapshot of the system has been taken:

Process	Allocation			Max		
	R_1	R_2	R_3	R_1	R_2	R_3
P_0	0	1	0	7	5	3
P_1	2	0	0	3	2	2
P_2	3	0	2	9	0	2
P_3	2	1	1	2	2	2
P_4	0	0	2	4	3	3

Calculate the available matrix of resources. Find the need of the matrix and the safe sequence.

Sol. Available = Total - Allocation

$$\therefore \begin{aligned} R_1 &= 10 - (2 + 3 + 2) = 3 \\ R_2 &= 5 - (1 + 1) = 3 \\ R_3 &= 7 - (2 + 1 + 2) = 3 \end{aligned}$$

$\therefore$ Available matrix is (3, 3, 2)

Need = Max - Allocation

∴	Process	R_1	R_2	R_3
	P_0	7 - 0	5 - 1	3 - 0
	P_1	3 - 2	2 - 0	2 - 0
	P_2	9 - 3	0 - 0	2 - 2
	P_3	2 - 2	2 - 1	2 - 1
	P_4	4 - 0	3 - 0	3 - 2

Applying Banker's algorithm, we now search the need-matrix from P_0 to P_4 which is less than the available matrix (3, 3, 2). We find that $(1, 2, 2) < (3, 3, 2)$. So, allocate the requesting resource to P_1 . Now the process P_1 's allocated is:

$$(2, 0, 0) + (1, 2, 2) = (3, 2, 2)$$

After the completion of job P_1 , it releases the total resources. Then the available resource is:

$$(3, 3, 2) + (2, 0, 0) = (5, 3, 2)$$

We search the need matrix again, which is less than the available. Now, we find that:

$$(0, 1, 1) < (5, 3, 2)$$

So, allocate resources to P_3 . After P_3 completes, it releases all its resources. Then,

$$\text{Available} = \text{Available} + \text{Allocation}$$

$$\text{i.e., } (5, 3, 2) + (2, 1, 1) = (7, 4, 3)$$

This process is continued till we find the safe state and as shown below:

$$\text{Work} = \text{Available} = (3, 3, 2)$$

P_0 : Finish [i] = false ($i = 1$)

$$\begin{array}{ll} \text{Need}_1 & \text{work} \\ (7, 4, 3) & (3, 3, 2) \end{array}$$

Step 4: Finish [1] = false X ∴ usage = P_0

P_1 : Finish [i] = false [$i = 2$]

$$\begin{array}{ll} \text{Need}_2 & \text{Work} \\ (1, 2, 2) & (3, 3, 2) \end{array}$$

Step 3: Work = work + allocation

$$\therefore \text{work } (3, 3, 2) + (2, 0, 0) = (5, 3, 2)$$

$$\text{Finish } [2] = \text{true} \quad \therefore \boxed{\text{safe } P_1}$$

P_2 Finish [i] = false [$i = 3$]

$$\begin{array}{ll} \text{Need}_3 & \text{work} \\ (6, 0, 0) & (5, 3, 2) \end{array}$$

Step 4: Finish [3] = true = usage P_2

P_3 : Finish [i] = false [$i = 4$]

Need₄ Work
 (0, 1, 1) (5, 3, 2)

Step 3: Work = work + allocation
 = (5, 3, 2) + (2, 1, 1)
 = (7, 4, 3)

Finish [4] = true ∴ safe P₃

P₄: Finish [i] = false (i = 5)
 Need₅ = work
 (4, 3, 1) ≤ (7, 4, 5)
 Work = work + allocation
 = (7, 4, 3) + (0, 0, 2)
 = (7, 4, 5) ∴ safe P₄

P₀: Finish[i] = false [i = 1]
 Need₁ ≤ (7, 4, 5)
 work = work + allocation
 = (7, 4, 5) + (0, 1, 0)
 = (7, 5, 5)
 work = (7, 5, 5)

P₂: Finish [2] = false ∴ safe P₀
 Need₃ ≤ work [i = 3]
 (6, 0, 0) ≤ (7, 5, 5)
 Work = work + allocation
 = (7, 5, 5) + (3, 0, 2)
 = (10, 5, 7) ∴ safe P₂

∴ safe sequence = (P₁, P₃, P₄, P₀, P₂)

Summary

- A process requests resources, if resources are not available at that time, the process enters in a waiting state. Sometimes a waiting process is never again able to change state, because the resources, it has requested are held by other waiting process. This situation is called deadlock.
- A preemptable resource is a resource allocated to a process is preempted and allocated to other process after its use.
- A non preemptable resource is a resource once allocated to a process is not preempted to any other process.

- Only one process may use a resource at a time is called mutual exclusion.
- A safe state is a state when there is at least one order in which all processes can be run to completion without resulting deadlock.
- Safety algorithm is used for finding out whether or not a system is in safe state.
- Resource-request algorithm is used to determine whether request can be safely granted or not.

Multiple-Choice Questions

1. What is a reusable resource?
 - (a) That can be used by one process at a time and is not depleted by that use.
 - (b) That can be used by more than one process at a time.
 - (c) That can be shared between various threads.
 - (d) None of the above.
2. Which of the following condition is required for deadlock to be possible?
 - (a) Mutual exclusion.
 - (b) A process may hold allocated resources while awaiting assignment of other resources.
 - (c) No resource can be forcibly removed from a process holding it.
 - (d) All of the above.
3. A system is in the safe state if
 - (a) the system can allocate resources to each process in some order and still avoid a deadlock.
 - (b) there exists a safe sequence.
 - (c) both (a) and (b)
 - (d) none of the above
4. The circular wait condition can be prevented by
 - (a) defining a linear ordering of resource types.
 - (b) using thread.
 - (c) using pipes.
 - (d) all of the above.
5. Which one of the following is the deadlock avoidance algorithm?
 - (a) Banker's algorithm
 - (b) Round Robin algorithm
 - (c) Elevator algorithm
 - (d) Karn's algorithm
6. Which of the following is the drawback of Banker's algorithm?
 - (a) In advance process rarely know that how much resource they will need.
 - (b) The number of processes changes as time progresses.
 - (c) Resource once available can disappear.
 - (d) All of the above.

7. For effective operating system, when to check for deadlock?
- (a) Every time a resource request is made.
 - (b) At fixed time intervals.
 - (c) Both (a) and (b).
 - (d) None of the above.
8. A problem encountered in multitasking when a process is perpetually denied necessary resources is called
- (a) deadlock
 - (b) starvation
 - (c) inversion
 - (d) aging
9. Which one of the following is a visual (mathematical) way to determine the deadlock occurrence?
- (a) Resource allocation graph
 - (b) Starvation graph
 - (c) Inversion graph
 - (d) None of the above
10. To avoid deadlock
- (a) there must be a fixed number of resources to allocate.
 - (b) resource allocation must be done only once.
 - (c) all deadlocked processes must be aborted.
 - (d) inversion technique can be used.
11. A system has 12 magnetic tape drives and 3 processes: P_0 , P_1 , and P_2 . Process P_0 requires 10 tape drives, P_1 requires 4 and P_2 requires 9 tape drives.
- Process
- P_0
- P_1
- P_2
- Maximum needs (process-wise: P_0 through P_2 top to bottom)
- 10
- 4
- 9
- Currently allocated (process-wise)
- 5
- 2
- 2
- Which of the following sequence is a safe sequence?
- (a) P_0, P_1, P_2
 - (b) P_1, P_2, P_0
 - (c) P_2, P_0, P_1
 - (d) P_1, P_0, P_2
12. If no cycle exists in the resource allocation graph
- (a) then the system will not be in a safe state.
 - (b) then the system will be in a safe state.

-
- (c) Either (a) or (b).
 - (d) None of these.
13. For mutual exclusion to prevail in the system
- (a) at least one resource must be held in a non-shareable mode.
 - (b) the processor must be a uniprocessor rather than a multiprocessor.
 - (c) there must be at least one resource in a shareable mode.
 - (d) All of these.
14. For a hold and wait condition to prevail
- (a) a process must be holding a resource, but waiting for one to be freed, and then request to acquire it.
 - (b) a process must be holding at least one resource and waiting to acquire additional resources that are being held by other processes.
 - (c) a process must hold at least one resource and not be waiting to acquire additional resources.
 - (d) None of these.
15. Deadlock prevention is a set of methods
- (a) to ensure that at least one of the necessary conditions cannot hold.
 - (b) to ensure that all of the necessary conditions do not hold.
 - (c) to decide if the requested resources for a process have to be given or not.
 - (d) to recover from a deadlock.
16. A deadlock can be broken by (choose all that apply)
- (a) abort one or more processes to break the circular wait.
 - (b) abort all the process in the system.
 - (c) preempt all resources from all processes.
 - (d) to preempt some resources from one or more of the deadlocked processes.
17. The two ways of aborting processes and eliminating deadlocks are: (choose all that apply)
- (a) Abort all deadlocked processes
 - (b) Abort all processes
 - (c) Abort one process at a time until the deadlock cycle is eliminated
 - (d) All of these
18. Those processes should be aborted on occurrence of a deadlock, the termination of which
- (a) is more time consuming
 - (b) incurs minimum cost
 - (c) safety is not hampered
 - (d) All of these.
19. The process to be aborted is chosen on the basis of the following factors: (choose all that apply)
- (a) priority of the process
 - (b) process is interactive or batch

- (c) how long the process has computed
 - (d) how much more long before its completion
 - (e) how many more resources the process needs before its completion
 - (f) how many and what type of resources the process has used
 - (g) how many resources are available in the system
 - (h) All of these
20. Cost factors of process termination include: (choose all that apply)
- (a) number of resources the deadlock process is holding
 - (b) CPU utilization at the time of deadlock
 - (c) amount of time a deadlocked process has thus far consumed during its execution
 - (d) All of the above

Answers

- | | | | | | |
|---------|----------------------------------|---------|-----------------|-----------------|---------|
| 1. (a) | 2. (d) | 3. (c) | 4. (a) | 5. (a) | 6. (d) |
| 7. (c) | 8. (a) | 9. (a) | 10. (a) | 11. (d) | 12. (b) |
| 13. (a) | 14. (b) | 15. (a) | 16. (a) and (b) | 17. (a) and (c) | |
| 18. (b) | 19. (a), (b), (c), (d), (e), (f) | | 20. (a) and (c) | | |



Memory Management

6.1 INTRODUCTION

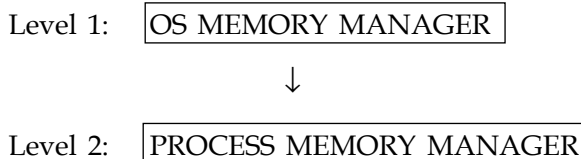
Memory is central to the operation of a modern computer system. Memory consists of a large array of words or bytes, each with its own address. The CPU fetches instruction from memory according to the value of the program counter. These instructions may cause additional loading from and storing to specific memory address. A typical instruction-execution cycle, for example, first fetches an instruction from memory. The instruction is then decoded and may cause operands to be fetched from memory. After the instruction has been executed on the operands, results may be stored back in the memory. The memory unit sees only a stream of memory addresses; it does not know how they are generated (by the instruction counter, index, etc.) or what they are for (instruction or data). Accordingly, we can ignore how a program generates a memory address. We only require the sequence of memory address generated by the running program.

The part of OS that manages the memory hierarchy is known as *memory management*.

Job:

- Keep track of available and used memory space.
- Allocation/deallocation of memory to processes.
- Manage swapping between main memory and disk.

Levels of memory management:



6.2 REQUIREMENTS FOR MEMORY MANAGEMENT

Linking: Modification of addresses where one object module refers to a location in another object module

Protection: Each process should be protected against unwanted interference by other processes. This should be satisfied by the processes rather than OS and this is achieved with the capability of hardware processor.

Sharing: Allow cooperating processes (on the same disk) controlled access to shared areas of memory.

Logical Address: Main memory in computer system is organised as linear 1D address space which consists of sequence of bytes and such type of organisation is called logical organisation and set of logical addresses is called logical address space.

Physical Address: Computer library is organisation in 2 levels, one is main memory and other is secondary memory. This type of organisation is called physical organisation of memory address and set is called physical address space.

$$\text{Physical Address} = \text{Logical address} + \text{Contents of base register.}$$

Relocation

The mechanism of relocation of logical addresses to be mapped into actual physical address is called relocation.

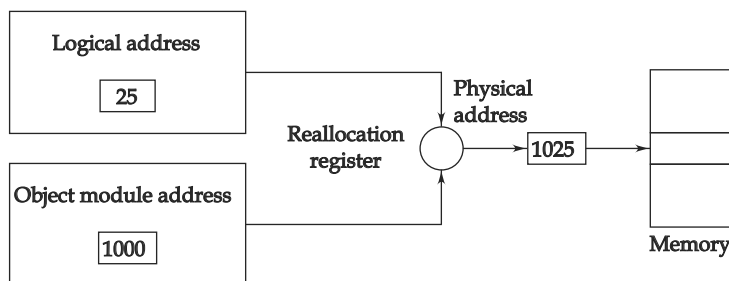


Fig. 6.1 Dynamic relocation using relocation register

When a program is linked with the main program, user written procedures and library routines are combined into a single address space is known as relocation.

6.3 BASIC HARDWARE FOR MEMORY MANAGEMENT

Main memory and the registers built into the processor itself are the only storage that the CPU can access directly. There are machine instructions that take memory addresses as arguments, but none takes disk addresses. Therefore, any instruction in execution, and any data being used by the instructions, must be in one of these direct access storage devices. If the data are not in memory, then they must be moved there before the CPU can operate on them.

Some Important Points

Memory unit only sees stream of addresses. A program must be provided in memory for the execution input queue — collection of processes that are waiting to be brought in memory for execution. User program needs to go through several steps before execution.

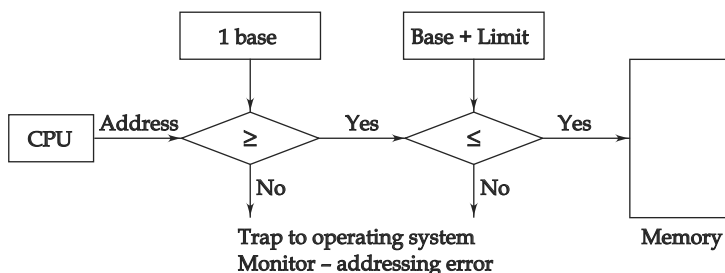


Fig. 6.2 Hardware address protection with base and limit registers

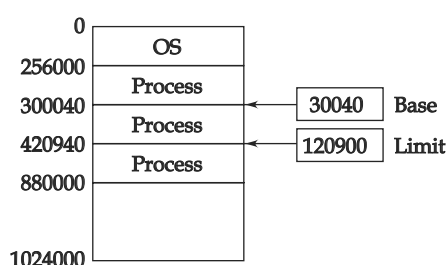


Fig. 6.3 Base and limit register defining logical address space

6.4 ADDRESS BINDING

Addresses in the source program are generally symbolic. A compiler 'binds' these symbolic addresses to allocatable addresses. Then the linkage editor or loader binds these allocatable addresses to absolute address.

Each binding represents mapping from one address space to another.

Multi-step Processing of a User Program

The 'binding' of instructions and data to memory addresses can be done at any stage:

Compile Time: If address is known at this time, then the absolute code can be generated. (Address of the process where it resides in the memory.) If after some time, the location or address of process changes to new location then it is necessary to recompile this code.

For example, MS-DOS. COM-format programs are absolute code bound at compile time.

Load time: If address is not known at compile time then the compiler generates a relocatable code hence, final binding is delayed till load time (in this case).

If the starting address changes we need only to reload the user code to incorporate the changed value.

Execution time: If the process can be moved during its execution from one memory segment to another then binding must be delayed until run time.

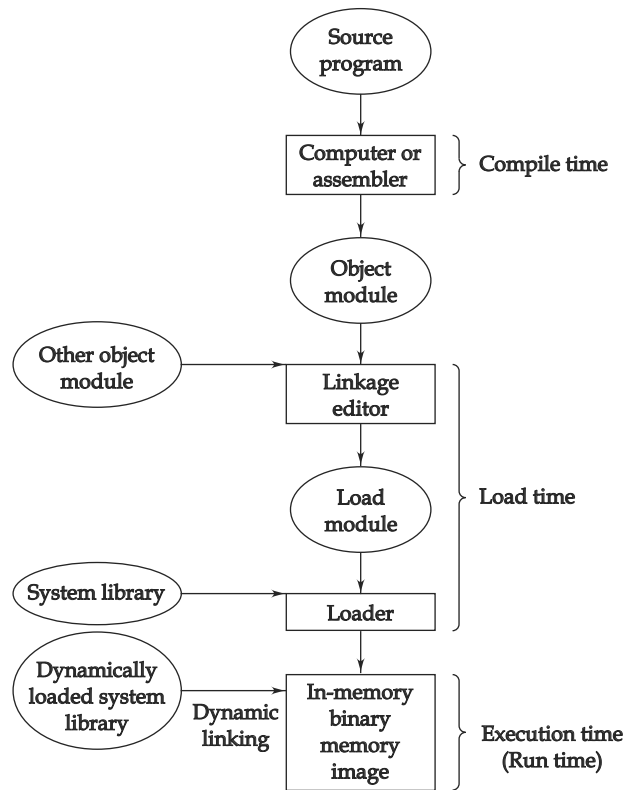


Fig. 6.4 Processing of a user program

- Need hardware support for address maps.
- Most OSs use this stage for binding (complete).

6.5 DYNAMIC LOADING

- This is done so that there is better memory space utilization.
- In this a routine is not loaded until it is called. All these routines lie on disk in a relocatable load format.
- The process goes on like this: First, only the main program is loaded into the memory and is executed. When a routine needs to call another routine, then calling routine checks if the called routine is loaded or not. If not then the relocatable linking loader is called to load the called routine into memory and updates the programs address tables to reflect the change.

Then control switches to the called routine.

Advantages

- It does not require any special support from OS. It is the user's responsibility to design programs in such a way that they can utilize such a scheme.

- Unused routine is never loaded.
- Helpful when large amounts of codes are be handled, e.g., error routines.
- The memory-space utilization is efficient.

6.6 DYNAMIC LINKING

- In Static linking, language library is treated like any other object module and is combined by the loader into the binary program image.
- In this, the linking is postponed until execution.
- If dynamic linking is not there then we need to have the copy of the files that the program requires for execution in the executable image. This requirement wastes both disk space and main memory.
- With dynamic linking, a 'stub' is included in the image for each library reference.
- Stub is a small code that indicates how to load the library if routine is not present.
- When a stub is executed, it checks whether the needed routine is present in memory or not. If not then the program loads it into the memory.

The stub replaces itself with the address of routine and executes its routine. Thus, the next time the code is reached; the library routine is executed directly, incurring no code for dynamic linking.

In this the process that need these library files execute only one copy of the library code.

Help of OS is required when processes in memory are protected from one another and they share a common routine that is present in another memory space (process). So OS allows multiple processes to access same memory addresses.

6.7 OVERLAYS

As the size of process is limited to the size of physical memory, the concept of overlays came into scene. It allows the process to be larger than the amount of memory allocated to it.

The basic idea is to keep those instructions and data in memory that are required at a given point of time. When other instructions are needed, then they are loaded in place of the previously loaded instructions that are no longer required.

For example consider a 2-pass assembler. During pass 1, it constructs a symbol table; then; during pass 2, it generates a machine language code. Only 150 K is available.

Pass 1	70 K
Pass2	80 K
Symbol table	20 K
Common routines	30 K

To load everything required space = 200 K

But, available = 150 K

And, we know Pass 1 and Pass 2 do not require memory at the same time.

Thus, we define 2 overlays:

Overlay A: symbol table, common routines pass 1.

Overlay B: symbol table, common routines pass 2.

We also add an overlay driver (10 K).

This driver is required for switching from A to B.

Firstly, overlay A is loaded into memory.

So it requires = 120 K (70 + 20 + 30)

After that, the overlay B is loaded, overwriting overlay A

So it requires = 130 K (80 + 20 + 30)

This reloading is a bit faster as only few data is to be transformed.

This way in limited area, i.e., 150 K the program of 200 K can run.

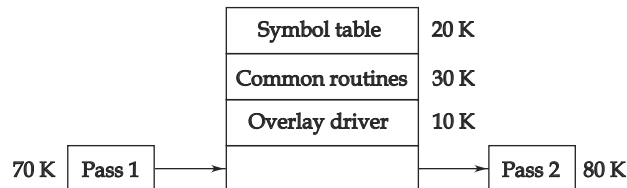


Fig. 6.5 Overlays

- Overlays do not require any special support from the OS.
- Can be implemented completely by the user with simple file structures.
- But there is more I/O than usual.
- The use of overlay is restricted to microcomputers and systems that have limited physical memory and that lack hardware support for more advanced techniques.

6.8 LOGICAL VS PHYSICAL ADDRESS SPACE

When there is compile-time or load-time address binding schemes, then logical and physical addresses are the same.

If binding is done at the time of execution, physical and logical addresses behave differently.

Logical/Virtual	Physical
Address generated by CPU.	Address seen by memory unit (i.e., the one loaded into memory address register of the memory)
Set of all logical addresses generated by a program is referred as a logical address space.	The set of all physical addresses corresponding to logical addresses is referred as physical address space
User program deals with logical addresses	User program never deals with logical physical addresses

The run-time mapping from virtual to physical address is done by memory-management unit (MMU). It is a hardware device.

The most simplest MMU scheme is a generalization of the base-register scheme.

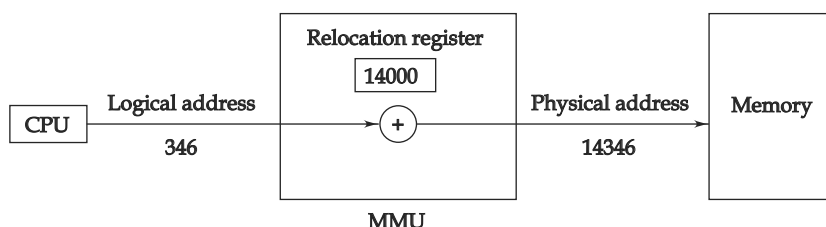


Fig. 6.6 Dynamic relocation

This scheme requires hardware support.

The value in the relocation register is added to every address generated by a user process at the time it is sent to memory.

6.9 SWAPPING

In this (fast disk) process is swapped out from the main memory to the backing store or swapped in from backing store to the main memory. The swapping is done by memory manager.

This policy is used in round robin CPU-scheduling algorithm and priority-based scheduling algorithms.

This swap in and out is also called roll in and out.

The process that is swapped out will be swapped into the same memory space, i.e., accepted previously. This is possible through the method of addressing binding. If binding is done at assembly or load time, then the process cannot be moved to different locations. If execution-time binding is used then it is possible to swap a process into a different memory space, as physical addresses are computed during execution time.

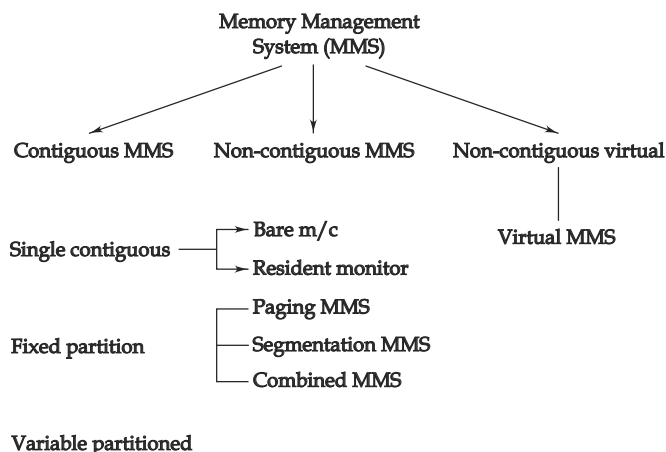


Fig. 6.7 MMS

The backing store should be large enough to accommodate copies of all memory images for all users, and should provide process of swapping:

Ready queue: Queue of memory images of processes that are in backing store or memory and are ready to run.

Whenever CPU scheduler decides to execute a process, it calls the dispatcher. Then this dispatcher checks whether the next process in ready queue is there in memory or not, and there is no free memory region, the dispatcher swaps out the process currently in memory and swaps in the desired process. It then reloads registers as normal and transfer control to the selected process.

For efficient CPU utilization, execution time must be comparatively long as compared to swap time.

The major part of swap time is transfer time and is directly proportional to the amount of memory swapped.

For this scheme to be effective, the user must keep the system informed of any changes in memory requirements. Thus, a process with dynamic memory requirement will need to issue system calls (request/release memory) to inform OS of its changing memory needs.

Constraints on swapping.

Before swapping, process must be idle.

If a process is waiting for I/O and we need to swap it out, if I/O is asynchronously accessing the user memory for I/O buffers, then processes cannot be swapped. To handle such problems: (i) Never swap a process with pending I/O; (ii) execute I/O operations only into OS buffers. Transfers only occur when processes are swapped in. (The part of OS manages the bakery hierarchy).

6.10 CONTIGUOUS MEMORY ALLOCATION

The main memory must accommodate both the operating system and the various user processes. We therefore need to allocate the main memory in the most efficient way.

The memory is usually divided into two parts; one for the resident monitor and the other for the user processes. We can place the operating system in either low or high memory. The major factor affecting this decision is the location of the interrupt vector. Since the interrupt vector is often in low memory, programmers usually place the operating system in low memory as well.

We usually want several user processes to reside in memory at the same time. We therefore need to consider how to allocate available memory to the processes that are in the input queue waiting to be brought into memory. In contiguous memory allocation, each process is contained in a single contiguous section of memory.

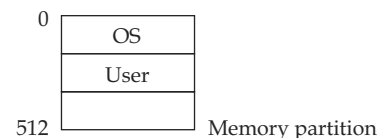


Fig. 6.8 Memory layout

6.10.1 Single Contiguous

The memory is divided into 2 parts, one for resident OS and one for the user processes.

OS lies in low memory and user processes in high.

Single partition allocation, e.g., as shown in Fig. 6.8. If OS lies in low memory and user in high, we need to protect the OS from changes by user processes and processes from one another. The relocation register contains the value of the smallest physical address; the limit register contains the range of logical addresses.

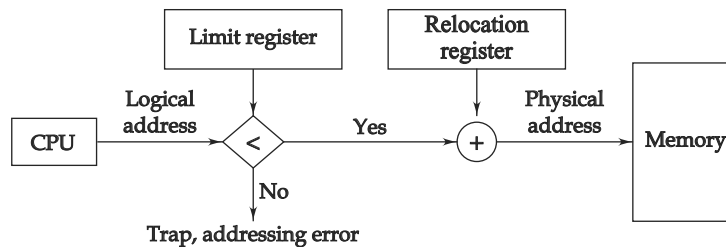


Fig. 6.9 Hardware support for relocation and limit register

When the CPU scheduler selects a process for, execution dispatcher loads the relocation and limit registers.

Relocation-register scheme provides an effective way to allow the OS size to change dynamically.

Transient OS code: These are codes that make up OS, when these codes are not used (commonly) then they can go out and come back when needed. This, in turn, changes OS size dynamically.

6.10.1.1 Bare M/C

It is the simplest and basic MMS.

- User has complete control over the memory
- Single user approach

Advantages: Flexible

Does not require hardware, software or OS support

Disadvantages: No service from OS so no control over memory. No virtual memory benefits.

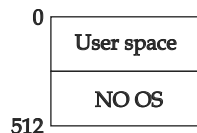
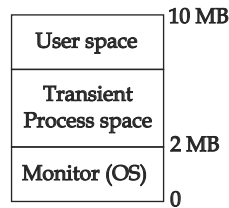


Fig. 6.10 Bare machine

6.10.1.2 Resident Monitor

- Also a single MMS
- Memory consists of two contiguous areas in which one is permanently allocated to resident portion of OS, hence the name resident monitor.

**Fig. 6.11** Resident monitor

The remaining memory is allocated to the resident processes which are executed by one user command.

Advantages

- Simple to design
- Little of hardware support.

Disadvantages

Less utilization of processor and memory.

Multiple Partition Allocation (Fixed Partition) Simplest scheme for memory allocation is to divide memory into fixed partitions.

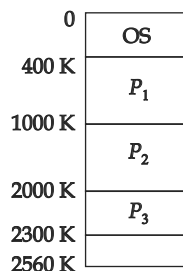
Each partition contains one process. Thus, degree of multiprogramming is bound by the number of partitions. The OS keeps a table indicating available and uses parts of memory.

Initially, all memory is available as a large chunk called 'hole'. When process arrives, a hole that can be used is searched and then allocated to the process. The remaining is available for future requests.

		Job queue			
		Process	Memory	Time	
Example	0				
	400 K	P_1	600 K	10	
		P_2	1000 K	5	
		P_3	300 K	20	
	2560 K	P_4	700 K	8	
		P_5	500 K	15	

FCFS
job
scheduling

Allocate P_1 , P_2 , P_3 memory spaces:

**Fig. 6.12** (a) Memory partitions

Now we have hole of 260 K that cannot be used for any other process.

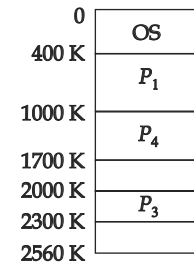
Now applying round-robin CPU scheduling with $tq = 1$ unit. We have process P_2 getting terminate at 14 unit. So it releases the memory.

Now we allocate.

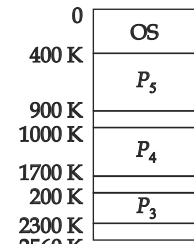
Now P_1 terminates at 28 unit and we allocate P_5 :

6.10.2 Structure of Allocation

- Processes enter the system, they are put into input queue.
- The OS takes into account the required memory space and available space.
- When space is available it brings the process into memory so that it can now compete for CPU cycle.
- After finishing, the execution process leaves and OS allocates the freed memory to another successive process.
- The allocation is in such a way that if a process comes and has a hole/chunk of memory where this process can reside. So OS allocates the required space and leaves the rest as a hole.
- If the new hole is adjacent to the previous one then we merge them to create a bigger one.



(b)



(c)

Fig. 6.12

6.10.3 Variable Partition

- Allows variable length and number of partitions.
- Process is allocated as much memory as it requires and no more.

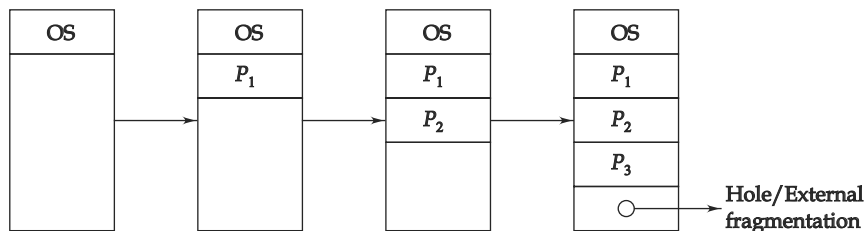


Fig. 6.13 Variable partition

Placement strategies for variable partition.

Best fit: Chooses the free block closest to the requested site rather than breaking big hole.

First fit: Chooses the first site which is greater or equal to the requested space.

Worst fit: Chooses the biggest chunk available.

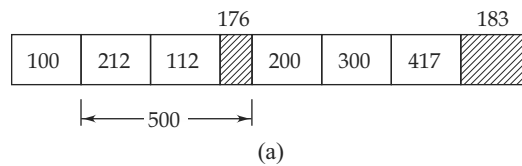
Example: Given memory partition of 100 kB, 500 kB, 200 kB, 300 kB, 600 kB, in order. How would each of the first fit, best fit, worst fit algorithm place these processes of 212 kB, 417 kB, 112 kB, 426 kB? Which is the best algorithm?

Ans.

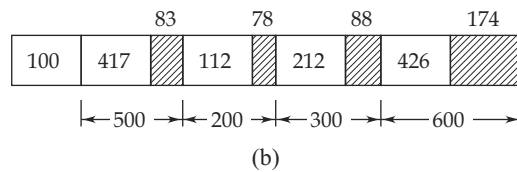
100	500	200	300	600
-----	-----	-----	-----	-----

First fit:

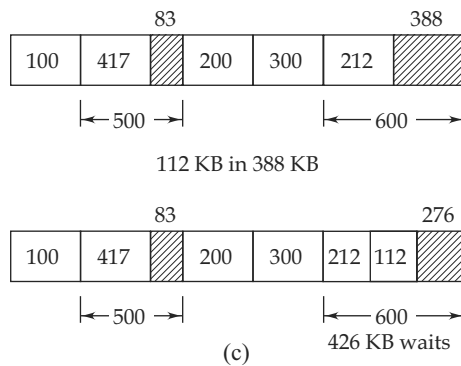
212 KB in 500 KB
 417 KB in 600 KB
 112 KB in 288 KB

**Best fit:**

212 KB in 300 KB
 417 KB in 500 KB
 112 KB in 200 KB
 426 KB in 600 KB

**Worst fit:**

212 KB $\xrightarrow{\text{in}}$ 600 KB
 417 KB in 500 KB

**Fig. 6.14** Variable partition

The best in all the three is best fit algorithm.

6.11 FRAGMENTATION

Fragmentation is the situation in which the memory gets divided into so many small blocks that none of them are useful because they are too small to satisfy a pending request.

6.11.1 External and Internal Fragmentation

External fragmentation exists when there is enough total memory. Space is available to satisfy a request but is not contiguous, i.e., it is fragmented into small holes.

Example: In the previous example after allocating P_4 , we had enough space to allocate to P_5 ($360 + 260 = 560$ K) but it was not configured. This is called external fragmentation.

Note: The selection of first fit or best fit can affect the amount of fragmentation.

So this is the problem.

50 per cent: Rule for first $\rightarrow N$ allocated blocks, another OS blocks are unusable (lost due to fragmentation) this is called 50% rule.

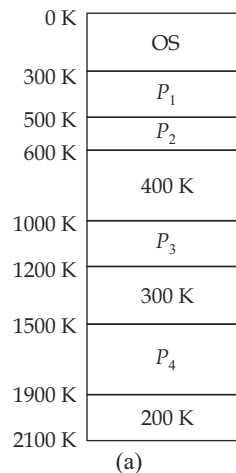
Internal fragmentation: Arises in partitioning where the process uses requested space, and rest is unused. This is called internal fragmentation.

Solution to External Fragmentation

Compaction: In this all the free memory are combined to form contiguous free memory area.

The processes are assigned new locations. Compaction is not always possible, as stated before the address of process change i.e., this internal addresses are relocated. If relocation is static and is done at assembly or load time, then compaction cannot be done. It is possible only if relocation is dynamic and is done at execution time. (requires only moving of data and changing base register)

When compaction is possible, we must determine cost. Example: considerate situation.



In this case there are 3 possible ways of compaction:

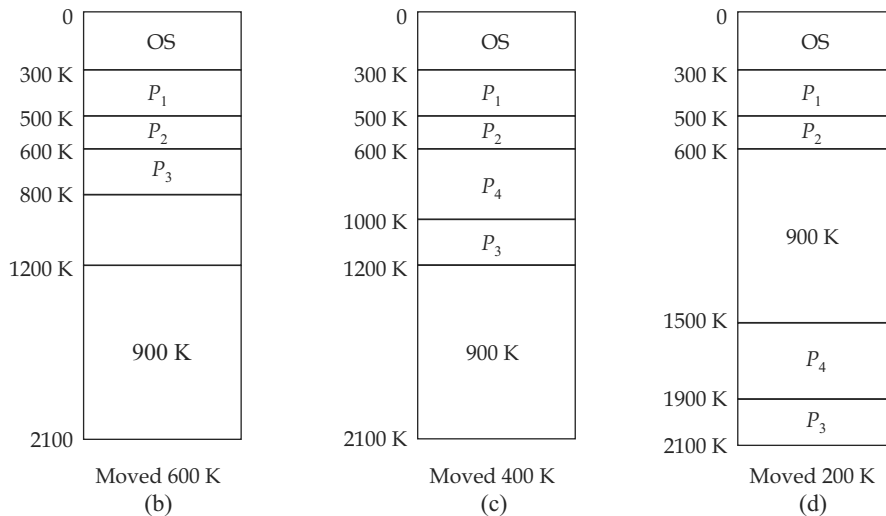


Fig. 6.15 Example of compaction

This shows that there are a number of ways to perform compaction but which is most optimal is difficult to find.

Swapping can be combined with compaction.

When process is rolled out then several problems may arise.

In static relocation the process should return to the same memory location. To obey or follow it other processes need to be rolled out.

So it is again a problem.

If dynamic allocation is there, then we can roll in the process to any other location.

6.12 NON-CONTIGUOUS MEMORY ALLOCATION

To permit sharing of code and data among processes, to resolve the problem of external fragmentation of physical memory, to enhance degree of multiprogramming and to support virtual memory concept, it was decided to have non-contiguous physical address space of a process. So now a process could be allocated wherever it was available. However, it involves complex implementation and an additional cost in terms of memory and processing.

There are three main methods of non-contiguous storage allocations:

- Paging
- Segmentation
- Segmentation with paging

6.12.1 Paging

In this, the logical address space is permitted to be non-contiguous, thus allowing a process to be allocated physical memory wherever the latter is available.

A better technique as compared to compaction, so used a number of OSs.

Physical memory and logical memory is broken into fixed-sized blocks called frames and pages respectively. Then the process is to be executed,

Pages lie in (backing store) loaded → Frames (true)

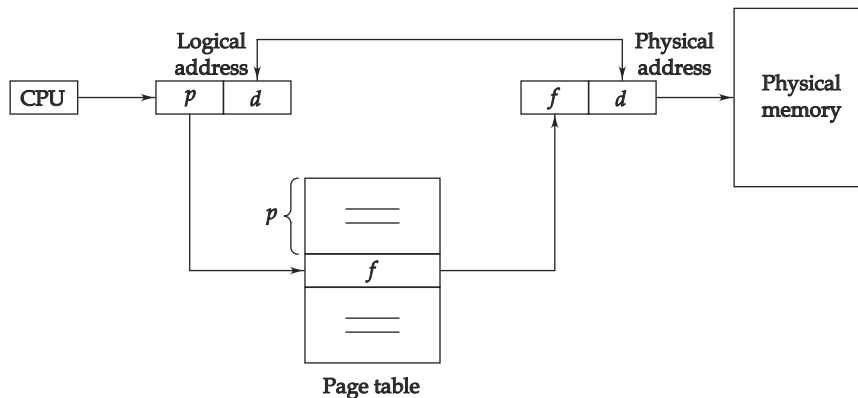


Fig. 6.16 Paging hardware

Backing store is divided into fixed-size blocks as that of memory frames.

Hardware support of paging:

Page number is used as an index	d : page offset	page table: contains base address of each page in physical memory
------------------------------------	-------------------	--

$m - n$ bits → page no.

n bits $\xrightarrow{\text{give}}$ offset number

The page size is defined by hardware (in power of 2 varying between 512 bytes and 8192 bytes).

The selection of a power of 2 as a page size makes the translation of logical address space into page number and offset becomes easy.

If size of logical address space is 2^m and page size is 2^n then higher order $m-n$ designate page number and lower n represents offset.

Logical Address:

P	d
$m - n$	n

 d : displacement
within page

Example: page size = 4 bytes; physical memory = 32 bytes.

Note: Paging is dynamic relocation.

No external fragmentation however internal exists.

If process size is independent of page size, then internal fragmentation becomes one-half page per process. So small page size is desirable.

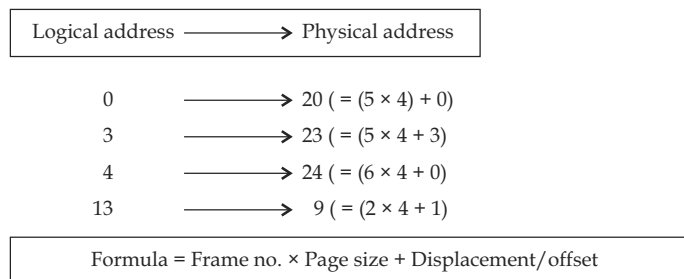
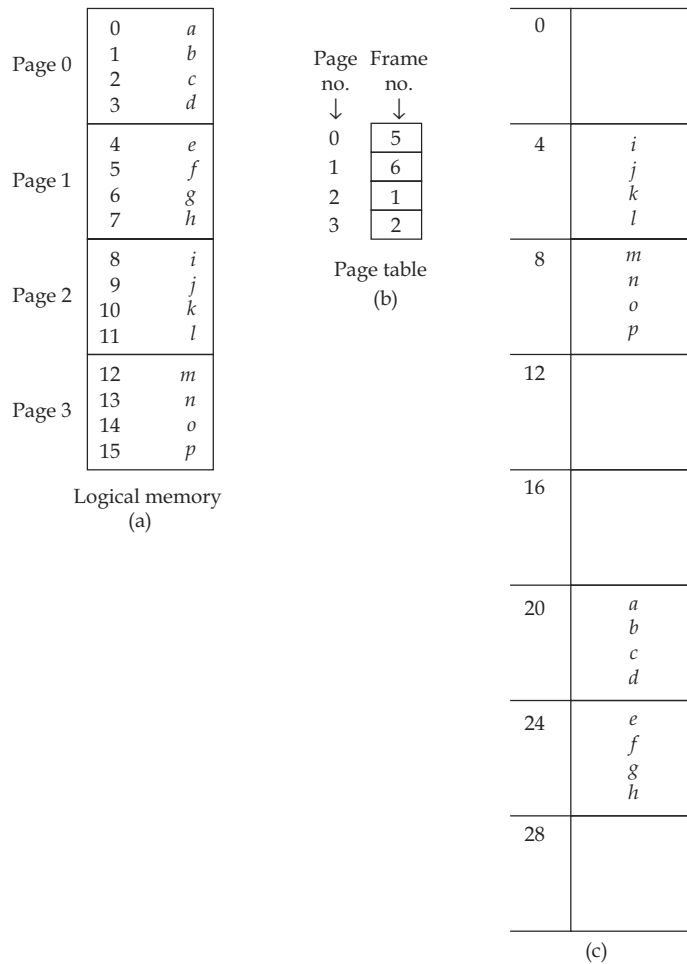


Fig. 6.17 Paging example for 32 byte memory with 4 byte pages

- Each page of the process needs one frame. Thus, for n pages there must be at least n frames available.
- Paging clearly separates user's view of memory and actual physical memory.
- User believes that his program resides in a contiguous space which is not true.

- To remove the difference of view's address translation hardware is used (L.A. $\rightarrow$ P.A.)

Frame table: It is a data structure that keeps the information of each physical page frame which is occupied or not. If occupied then by which process?

- The OS keeps a copy of page table with each process, so as to translate the logical address to physical address manually. It is also used by CPU dispatcher to define hardware page table when process gets allocated.

6.12.1.1 Structure of Page Table

In simplest case page table is implemented as a set of dedicated registers.

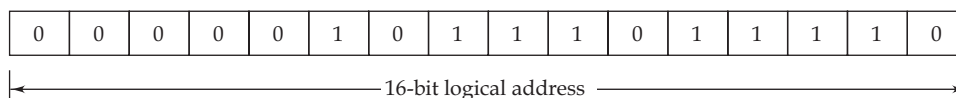
These registers should be built with very high speed logic to make paging address translation efficient. The use of registers for page table is satisfactory if page table is reasonably small.

The problem lies in the time required to access a user memory location. In this 2 memory accesses are needed to access a byte. Thus, memory access is slowed by factor of 2.

The solution to this is to have small, special, fast-lookup hardware cache called associative registers or translation look aside buffers (TLBS).

This consists of 2 parts: a key and value.

For example, consider a logical address.



number of offset bits = 10

number of bits for page no. = 6

$\therefore$ Our page number = 1 and offset = 478

Page size = $2^{\text{no. of bits in offset}} = 2^{10} = 1024$ bits

Address translation (main 3 steps):

- Extract page number from logical address
- Use page no. as index into process page table to find frame number ' f '.
- The starting physical address of the frame is:
 $f \times 2^{\text{no. of bits in offset}}$ (Fig. 6.18)

Why page size is power of 2?

- The logical addressing scheme is transparent to the program assembler and the linker.

Each logical address is identical to relative address. Note that, relative address is defined with reference to origin of program and logical address.

- It becomes easy to implement a function in hardware to perform dynamic address translate at runtime.

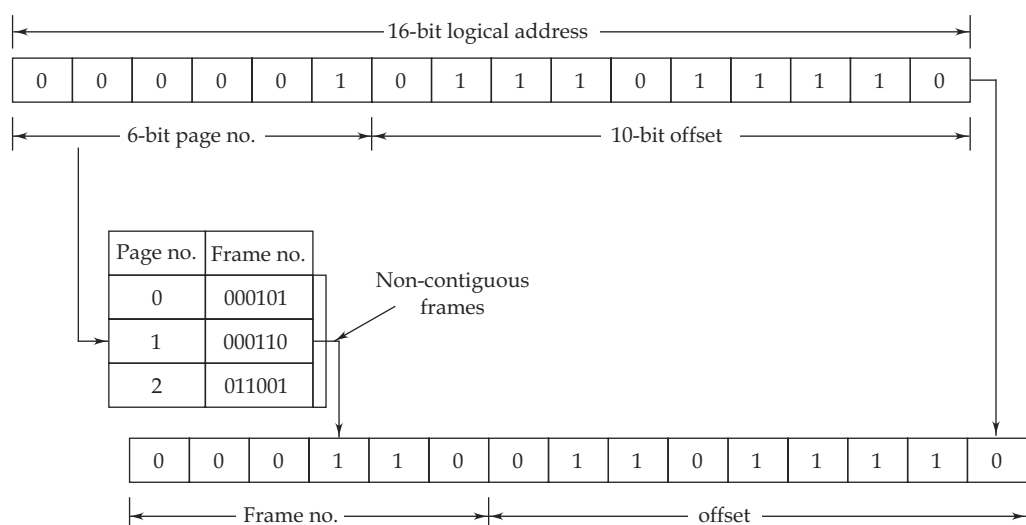


Fig. 6.18 Logical to physical address translation in paging

6.12.1.2 Paging with TLB or associative mapping

The associative registers contain only a few of the page-table entries. When CPU generates logical address, its page number is searched in these registers, if found then the whole task takes 10% less time. If not found then a memory reference to the page table is made when frame number is obtained, it is used to access memory. It is also added to associative registers, so that next time it would be found quickly. If TLB is full then OS uses one of the replacement strategies.

Note: Every time a new page table is selected, the TLB must be flushed so that invalid data/address is not used.

Hit ratio: % of times a page no. is found in TLB is called hit ratio.

6.12.1.3 Protection

In paged environment protection is achieved by protection bits that are associated with each frame. One bit can define a page to read and write or read-only. When physical address space is being computed at the same time protection bits can be checked to verify that no writes are made to read-only page such an attempt causes a hardware trap to OS (memory-protection violation).

Valid-invalid bit.

Valid: indicates that the associated page is in the process logical address space.

Invalid: indicates that the page is not in the processes logical address space.

Also used to allow or disallow access to pages.

Note: Paging results in internal fragmentation, so to overcome it some (PTLF) systems provide hardware, in the form of a page-table length register, to indicate length/size

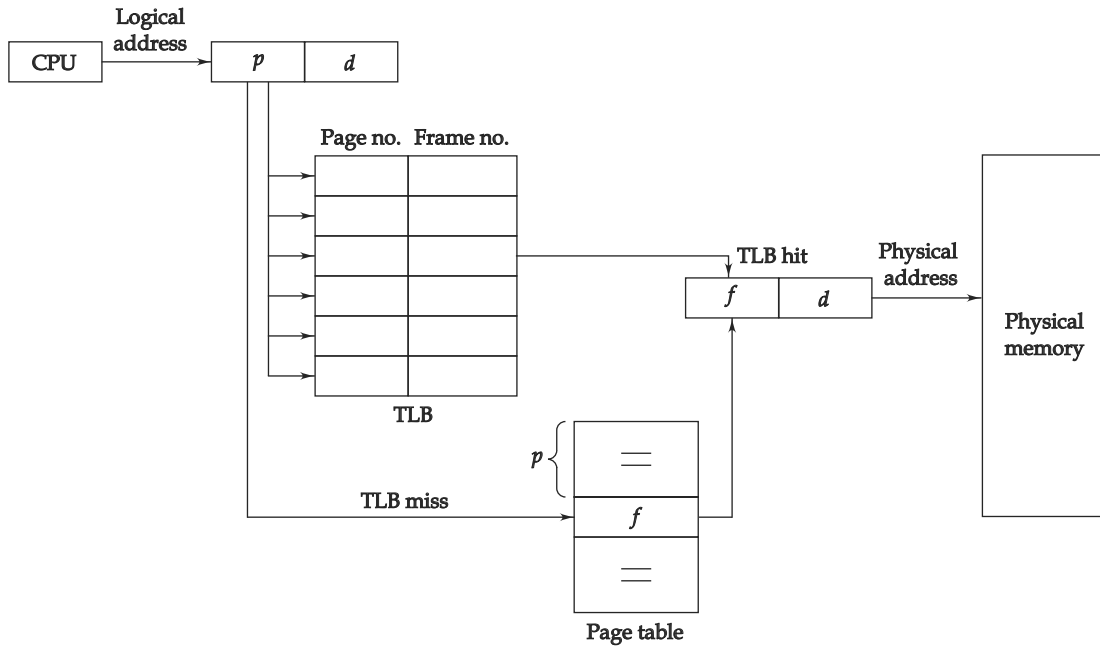


Fig. 6.19 Paging with TLB

of page table. This value is checked against every logical address to verify that the address is in the valid range of the process. Failure causes an error trap to the OS.

6.12.1.4 Multilevel paging

In env. where there is a large logical address space, it becomes impossible to allocate such large page table. The solution to this is to divide the page table into smaller pieces.

2-Level paging:

Page table is itself paged.

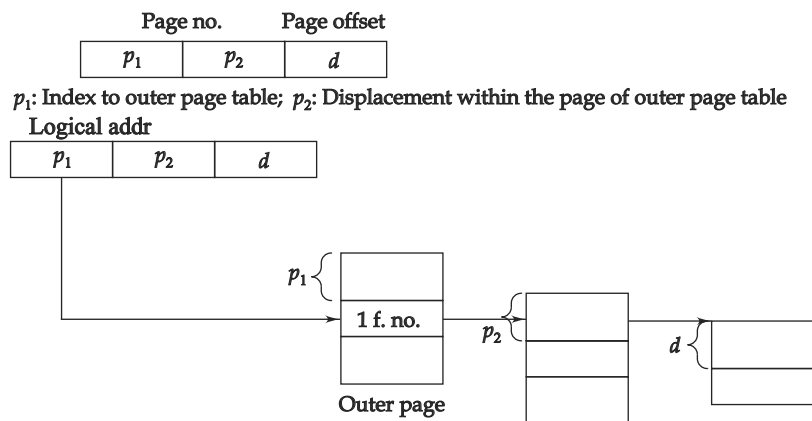


Fig. 6.20 Address translation for a 2-level 32-bit paging architecture

p_i -index to outer page table; p_i : displacement within the page of outer page table.

Flow chart for paging with a TLB.

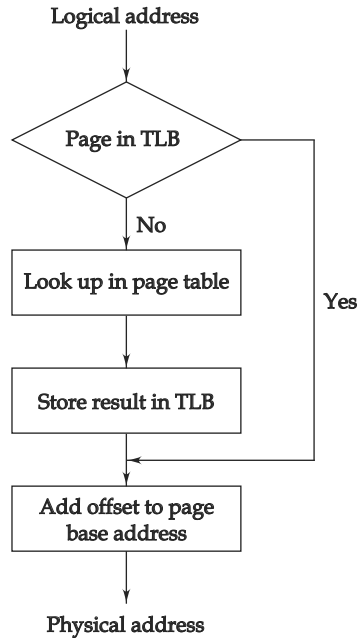


Fig. 6.21 Paging flow chart

6.12.1.5 Advantages of paging

- Physical memory can be easily allocated from the free list of frames.
- External fragmentation is eliminated.
- Easy to “page out” chunks of programs. All chunks are of the same size (page size).
- Processor and memory utilized efficiently.

6.12.1.6 Disadvantages of paging

- Fragmentation cannot be removed completely as internal fragmentation is possible. Memory reference is an overhead.

VAX architecture:

Section	Page	Offset
s	p	d

For a 64-bit logical addr space, 2-level paging scheme is no longer appropriate. So divide the outer page also that leads to 3-level paging scheme.

2nd outer page	Outer page	Inner page	Offset
p_1	p_2	p_3	d

SPARC supports 3-level paging scheme

Motorola supports 4-level paging scheme

Benefits of 2-level paging:

1. The pages do not have to be in contiguous physical memory, i.e., the master page table and secondary page table can be anywhere.
2. We can allow page faults when fetching the secondary page tables, i.e., secondary page table need not be in the main memory.
3. The main benefit of this scheme is visible in larger programs. The large program only needs page table for active processes to be kept in the main memory. This way we are again exploiting the principle of locality of page references.
4. Another advantage is that we can have large holes in logical address space; because master table and secondary table can be non-contiguous memory.

6.12.1.7 Hashed page tables

A common space for handling address space larger than 32 bits is to use a hashed page table, with the hash value being the virtual page number. Each entry in the hash table contains a linked list of elements that hash to the same location (to handle collisions). Each element consists of three fields: (1) the virtual page number, (2) the value of the mapped page frame, (3) a pointer to the next element in the linked list.

The algorithm works as follows: The virtual page numbers in the virtual address are hashed into the hash table.

The virtual page number is compared with field 1 in the first element of the linked list. If there is a match the corresponding page frame (field 2) is used to form the desired physical address. If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.

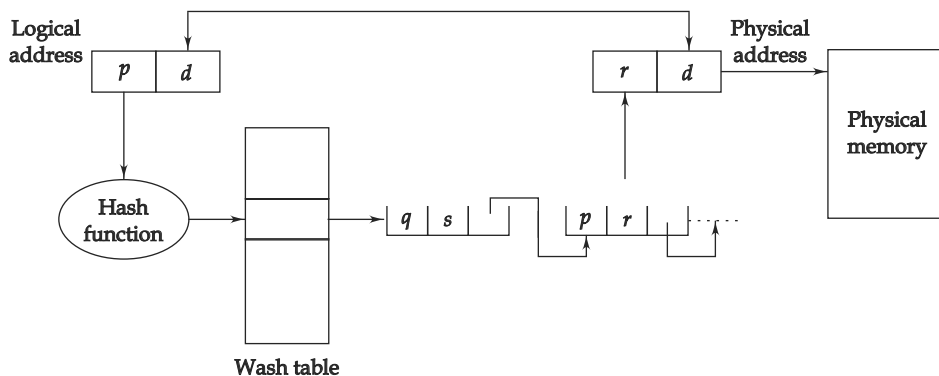


Fig. 6.22 Hashed page table

The previous page table technique has a drawback, i.e., each page table consists of million entries. So these consume large amount of physical memory. To solve this, inverted page table technique came into picture.

In this the table has one entry for each real page (frame) of memory. Each entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.

Thus, there is only one page table in the system, and it has only one entry for each page of physical memory.

Each virtual address consists of:

<process id, page-number, offset>

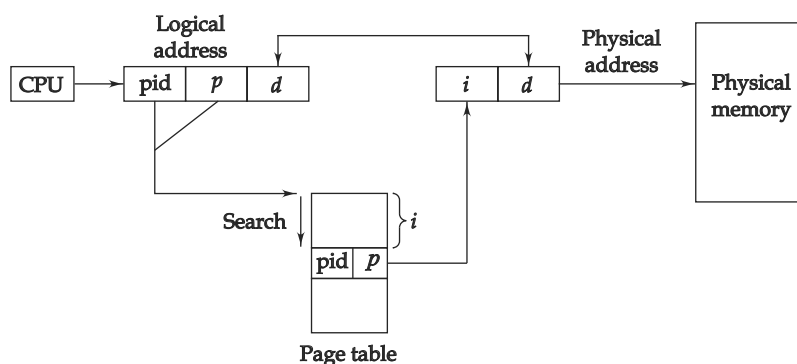


Fig. 6.23 Index or the entry where the < pid, p> is found in page table

If no match is found then an illegal address access is attempted.

Advantages

- It decreases the amount of memory needed but increases the amount of searching.
- To improve performance, associate memory registers are used to hold recently located entries. So when these registers are used first this improves the speed.

Shared Pages

- Advantage of paging is the possibility of sharing command code.

Example: Consider a system that supports 40 users, each executes a text editor (text editor consists of 150 K of code and 50 K of data space).

If each user has its own test editor so each user would require 200 K so 40 users = 8000 K.

But if code is re-entrant; this text editor can be shared. Re-entrant code is non-self-modifying code. If the code is re-entrant, then it never changes during execution. Thus, 2 or more processes can execute the same code at the same time.

The text editor maps onto same physical copy of editor but data pages in different frames.

6.12.1.8 Shared pages

An advantage of paging is the possibility of sharing common code. This consideration is particularly important in a time sharing environment. Consider a system that supports 40 users, each of whom executes a text editor. If the text editor consists of 150 kB of code and 50 kB of data space, we need 8000 kB to support the 40 users. If the code is re-entrant code (or pure code), however it can be shared.

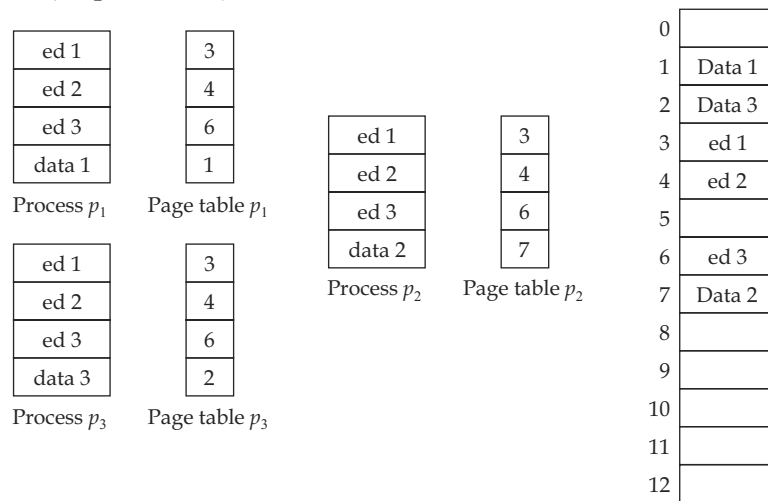


Fig. 6.24 Sharing of code in a paging environment

Here we see three page editors, each page 50 kB in size (the large page size is used to simplify the figure) being shared among three processes. Each process has its own data page.

Re-entrant code is non-self modifying code. It never changes during execution. Thus, two or more processes can execute the same code at the same time. Each process has its own copy of registers and data storage to hold the data for the process's execution.

Note: To be sharable, code must be re-entrant.

Systems that use inverted page tables have difficulty in implementing shared memory.

6.12.2 Segmentation

User view of memory: Memory is a collection of variable-size segments, with no necessary ordering among segments.

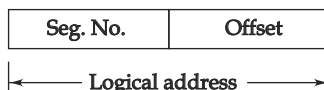
Segmentation is a memory management scheme that supports the user view of memory.

A logical address space is a collection of segments. Each segment has a name and a length.

The addresses specify both the segment name and the offset within the segment.

User specifies both segment name and offset.

Logical address space: $\langle \text{Segment-no, offset} \rangle$



6.12.2.1 Hardware

- Now mapping is of the user provided 2-dimensional into 1D physical address. This is effected by segment table. Each entry of the table has segment base and limit.
- Segment base: Starting physical address where the segment resides in memory.
- Segment limit: Specifies the length of the segment.

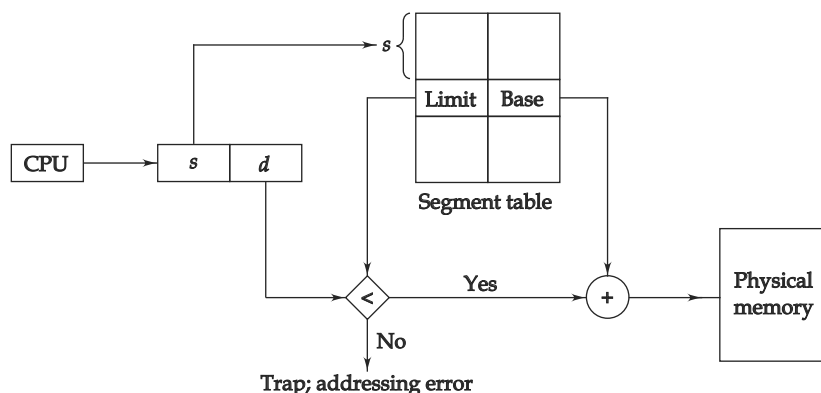


Fig. 6.25 Segmentation hardware

Example: We have segments numbered from 0 to 4.

Reference to byte 53 of segment 2 mapped an = $4300 + 53 = 4353$

byte 852 of segment 3 = 3200 (base of segment + 3) + $852 = 4052$.

reference to 1222 byte of segment 0

= would result in trap as this segment only has 1000 bytes long.

6.12.2.2 Implementation

Segmentation is closely related to partition models of memory management, the main difference is that the program number consists of several segments.

Segment table can also be put into fast register or in memory when a program consists of large number of segments then it is not feasible to keep segment table in memory

STBR: Segment Table Base Register points to segment table

STBL: Segment Table Base Length.

For logical address (s, d) we first check $S < \text{STLR}$. Then, we add STBR and S.

Here also, mapping requires 2 memory references per logical address, effectively slowing the computer system by a factor of 2.

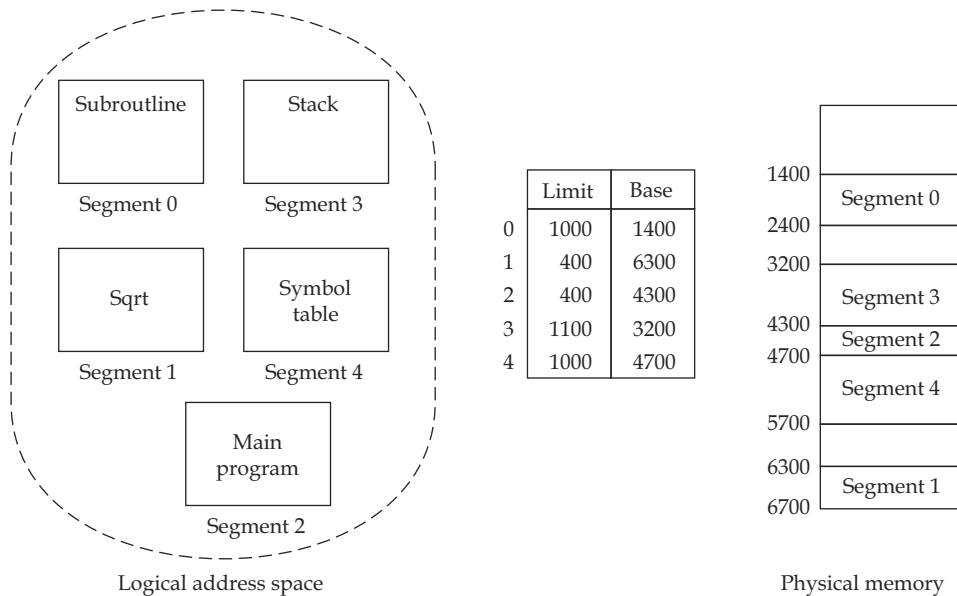


Fig. 6.26 Example of segmentation

To improve this associative registers can be used to reduce the time required of memory access.

6.12.2.3 Protection and sharing

Segments represent semantically defined portions of the program, so it is likely that all entries are used in same way in the segment.

As instructions are non-self modifying they can be defined as read-only or execute only.

The memory hardware checks the protection bits associated with each segment-table entry to prevent illegal access to memory. By placing an array in its own segment, the memory management hardware will automatically check that array induces are legal and do not stray outside the array boundaries. Thus, many program errors will be detected by the hardware before a damage sharing:

Segmentation involves sharing of code or data. Segments are shared when entries in the segment table of 2 different process point to same physical locations. Sharing happens at segment level.

It is also possible to share only parts of program.

Note: Code segments contain references to themselves.

So they need to refer itself in the same way for all the users that share them, i.e., it must have unique segment number. As number of users sharing the segment increases so does the difficulty of finding an acceptable segment number.

Example:

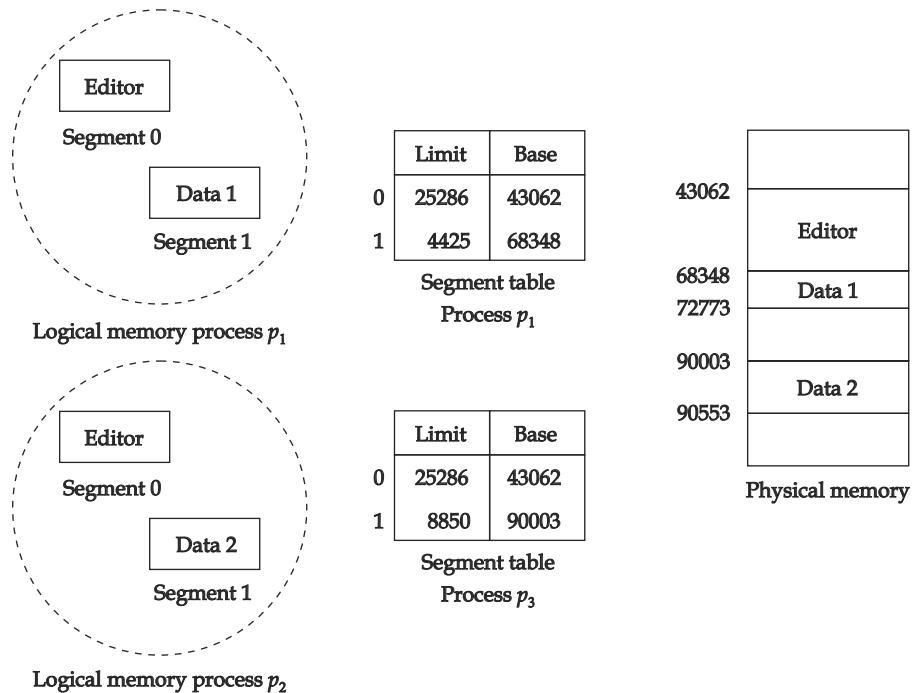


Fig. 6.27 Example of segmentation

Rather than having n copies of editor, only one can be utilised.

Fragmentation

The long-term scheduler needs to find memory for segments (variable length).

Segmentation may cause external fragmentation, when free blocks are small to accommodate a segment.

In this case, the process needs to wait, or compaction can be done as segmentation is by nature a dynamic relocation algorithm, so compaction is possible any time.

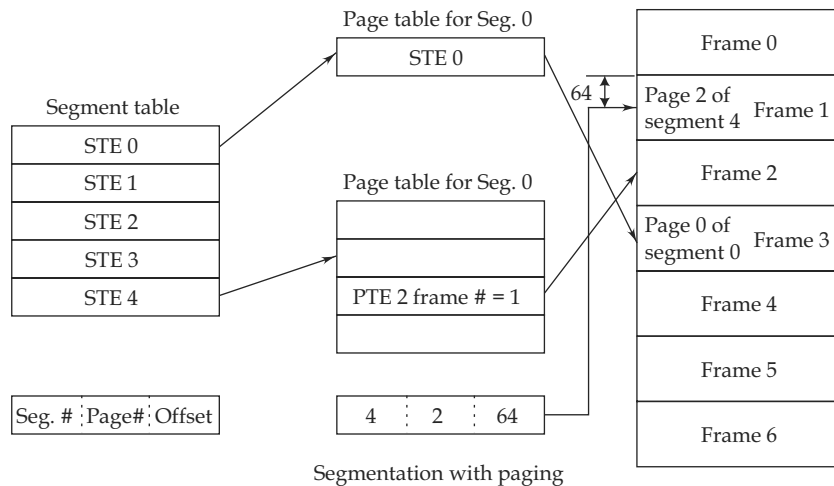
If average segment size is small, external fragmentation will also be small.

6.12.3 Segmentation with Paging

(a) In this the virtual address becomes a triplet:

<Seg#, page#, offset>

- Seg#: Segment table entry (STED) points to the page table for that segment.
- The size of each segment is a multiple of the page size. Perhaps not. Can keep the exact size in the STE (limit value) and shoot the process if it referenced beyond the limit. In this case the last page of each segment is partially valid.
- page#: Gives the entry in the chosen page table.

Example**Fig. 6.28** Segmentation with paging**Significance**

Intel combined paging and segmentation in the same MMU.

This way advantages of both are cashed on one single chip.

Unfortunately, 32-bit OS fails to take advantage of segmentation.

Table 6.1 Paging v/s Segmentation

Paging		Segmentation	
1. Main memory is partitioned into frames.		1. Main memory is not partitioned	
2. Program is broken into pages by the compiler or MMU		2. Program is divided into segments and programmer makes decision	
3. It causes internal fragmentation		3. It may cause external fragmentation because of segments not utilising the physical address space due to varying lengths	
4. A page table is used for each process showing which frame is occupied by each page		4. Segment table for each process shows base address and length of each segment	
5. OS must maintain free frame list		5. OS must maintain 4 holes in main memory	
6. Logical address consists of:		6. Logical address consists of:	
Page no.	Offset	Segment no.	Offset within segment
7. Presents memory as a linear array		7. Memory can be referenced in 2D.	
8. All pages need be in the main memory for a process to run		8. All segments need not be in the main memory for process to run	

Table 6.2 External Fragmentation v/s Internal Fragmentation

External fragmentation	Internal fragmentation
1. It happens when dynamic memory allocation algorithm allocates some memory and a small piece is left over that cannot be effectively used.	1. It is the space wasted inside allocated memory blocks because of restriction on the allowed size of allocated blocks
2. Allocated memory may be slightly larger than requested memory; this size difference in memory is internal to a partition, but not being used.	2. Enough memory space may exist to satisfy a request, but it is not contiguous. It exists as a larger number small unusable holes
3. Paging causes internal fragmentation because a frame can be allocated to a process that doesn't need the entire frame	3. Segmentation suffers from external fragmentation.

Table 6.3 Memory Management Techniques

S. No.	Technique	Description
1.	Fixed Partitioning	Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size
2.	Variable or dynamic partitioning	Partitions are created dynamically, so that each process is loaded into a partition of exactly the same size as that of process
3.	Paging	Main memory is divided into equal size frames. Each process is divided into equal size pages. Note that size of page is equal to size of frame. A process is loaded by loading all its pages into available frames which might not be contiguous.
4.	Segmentation	Each process is divided into segments. A process is loaded by loading all its segments in dynamic partitions

Table 6.4 Dynamic Memory Allocation Policies

	Description	Performance
First fit	The memory manager scans along the list of blocks until it finds a hole that is big enough. The hole is then broken up into 2 pieces, one for process and one for unused memory	It is a fast algorithm because it searches as little as possible. It is simplest and the best. However, it is at the front end with small free partitions that need to be searched over on each first fit pass. First fit generates larger holes on an average
Next fit	It keeps track of where it is whenever it finds a suitable hole. It starts searching from the place where it is left out last time; instead of beginning. It begins to scan memory from the location of last placement & chooses the next available block.	It tends to produce slightly worse results than the first fit. The next fit algorithm will lead to an allocation of free block from the end of memory. This results in increasing fragmentation at the end of memory space.
Best fit	It chooses the free block closest to the requested size. Rather than breaking up a big hole, it tries to find a hole close to actual size needed.	Best fit is slower than first fit because it must search the entire list every time it is called. This results in more wastage of memory than the first-fit/next-fit because it tends to fill up memory with tiny, useless holes.

6.13 MAIN MEMORY

- The part of the operating system that manages the memory hierarchy is known as memory management.
- Relocation is combining of address space of all object modules into single address space of load module.
- Linking is modification of address, where one object module refers to a location in another object module.
- Internal fragmentation is the space wasted inside allocated memory blocks because of restrictions on the allowed size of allocated blocks.
- External fragmentation happens when a dynamic memory allocation algorithm allocates some memory and a small piece is left over that cannot be effectively used.
- Paging is a non-contiguous allocation memory management scheme.
- Segmentation is a memory management scheme supporting user's view of memory.
- A special hardware memory management unit (MMU) is used to translate the virtual address to physical address in virtual memory.
- Thrashing is a phenomenon of moving pages from primary to secondary storage or vice versa. It consumes a lot of computer's energy but accomplishes very little useful result.

Numerical Examples

6.1 There are 125 pages in a logical address space, with page size of 1024 bytes. How many bits will be there in the logical address?

Sol. The logical address space contains 128 pages, i.e., 2^7 pages.

That means, the number of bits required for the page no., $P = 7$

Page size $2^b = 1024 \text{ bytes} = 2^{10}$

That means the number of bits required for the offset,

$$b = 10.$$

$$P = a - b$$

$$7 = a - 10$$

$$a = 17$$

$\therefore$ 17 bits are required in the logical address.

6.2 There is a system with 64 pages of 512 bytes page size and a physical memory of 32 frames. How many bits are required in the logical and physical address?

Sol. Given page size = 512 bytes = 2^9 i.e., $b = 9$

No. of pages = 64 = 2^6 i.e., $p = 6$

Since $P = a - b$; $a = 15$

- ∴ 15 bits are required in logical address.
Since the number of bits required for offset $b = 9$ and the number of bits required to address $32(2^5)$ frames = 5
∴ after conversion, 14 bits are required in the physical address.

6.3 Given memory partitions of 100 K, 500 K, 200 K, 300 K and 600 K (in order), how would each of the first fit, best fit and worst fit algorithms place processes of 212 K, 417 K, 112 K and 426 K (in order)? Which algorithm makes the most efficient use of memory?

Sol. (i) First fit

- (a) 212 K is put in 500 K partition.
- (b) 417 K is put in 600 K partition.
- (c) 112 K is put in 288 K partition (New portion 288 K = 500 K - 112 K)
- (d) 426 K must wait

(ii) Best fit

- (a) 212 K is put in 300 K partition.
- (b) 417 K is put in 500 K partition
- (c) 112 K is put in 200 K partition
- (d) 426 K is put in 600 K partition

(iii) Worst fit

- (a) 212 K is put in 600 K partition
- (b) 417 K is put in 500 K partition
- (c) 112 K is put in 388 K partition
- (d) 426 must wait

In this example, best fit turns out to be the best.

6.4. Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 frames.

- (a) How many bits are there in the logical address?**
- (b) How many bits are there in the physical address?**

Sol.

- (a) Logical address 13 bits ($8 \times 1024 = 2^3 \times 2^{10} = 2^{13}$)
- (b) Physical address 15 bits ($32 \times 1024 = 2^5 \times 2^{10} = 2^{15}$)

6.5. Consider a paging system with the page table stored in memory.

- (a) If a memory reference takes 200 nanoseconds, how long does a paged memory reference takes?**
- (b) If we add associative registers, and 75 per cent of all pages-table references are found in the associative registers what is the effective memory reference time? (Assume that finding a page table entry in the associative registers takes zero time, if the entry is there.)**

Sol.

- (a) 400 nanoseconds; 200 nanoseconds to access the page table and 200 nanoseconds to access the word in memory.
 (b) Effective access time = $0.75 \times (200 \text{ nanoseconds}) + 0.25 \times (400 \text{ nanoseconds})$
 = 250 nanoseconds.

6.6 Consider the following segment table

Segment No.	Base	Bound
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are the physical addresses for the following logical addresses?

(a) 0,430 (b) 1,10 (c) 2,500 (d) 3,400 (e) 4,112

Where (0, 430) implies 430th instruction/data is 0th segment.

Sol. (a) logical address = (0, 430)

i.e., $(s, d) = (0, 430)$

Now, d lies between 0 & 600 and that $d < \text{bound}$ or $430 < 600$ is valid; so,

Physical address = $d + \text{base address}$

$$= 219 + 430 = 649$$

(b) Now, $m, LA = (1, 10) = (s, d)$

$$s = 1, d = 0$$

Now, $d = 10$ lies between 0 & segment limit = 14

Physical address = $d + \text{base address}$

$$= 10 + 2300 = 2300$$

(c) Now, $LA = (2, 500) = (s, d)$

Now, $d = 500$ doesn't lie between 0 & segment limit of 100. This will result in illegal reference and a trap to OS will be generated.

(d) Given now, $LA = (3, 400) = (s, d)$

$$d = 400 < 580 \text{ (i.e. } d < \text{bound)}$$

$\therefore$ physical memory address = $1327 + 400 = 1727$

(e) Now, $LA = (4, 112) = (s, d)$

$\therefore d = 112$ does not lie between 0 & 96.

$\therefore$ Invalid reference

$\Rightarrow$ Trap to O.S.

6.7 On a simple paging system with 2^{27} bytes of physical memory, 256 pages of logical address space, and a page size of 2^{10} bytes.

(a) How many bits are in a logical address

Sol. The logical address space contains $256 = 2^8$ pages of 2^{10} bytes, making the total logical address space $2^{10} \times 2^8 = 2^{18}$ bytes. An 18-bit address is required to cover a 2^{18} byte address space.

(b) How many bits does the physical address specify in the page frame?

Sol. A 24-bit address is required to cover the 2^{24} bytes physical address space. Since pages, and therefore page frame, are 2^{10} bytes, the last 10 bits are the physical address specify the page offset. The remaining 14 bits specify the page frame number.

(c) How many entries are there in a page table and how long is the page table?

Sol. The page table must contain an entry for each page. Since there are 256 pages in the logical address space, the page table must be 256 entries long.

(d) How many bits are needed to store an entry in the page table (how wide is the page table)? Assume each page table entry contains a valid/invalid bit in addition to the page frame number.

Sol. Each entry contains 1 bit to indicate if the page is valid & 14 bits to specify the frame number.

6.8 On a simple paging system with a page table contains 64 entries of 11 bits (including valid/invalid bits) each and a page size of 512 bytes:

(a) how many bits does the logical address specify for the page number?

Sol. Since there are $64 = 2^6$ pages, 6 bits of logical address is required.

(b) how many bits does the logical address specify for the offset within the page?

Sol. Since a page is $512 = 2^9$ bytes long.

(c) How many bits are there in a logical address?

Sol. 6 most significant bits of the logical address would specify the page number, the 9 least significant bits would specify the page offset, $\therefore$ total bits = 15.

(d) What is the size of logical address space?

Sol. $2^{15} = 32,768$.

(e) How many bits does the physical address specify for the page frame number?

Sol. Since, total number of bits is 11,

$\therefore$ number of bits in page frame number are 10.

(f) How many bits does the physical address specify for the offset within the page frame?

Sol. The page frame offset is same as the page offset, $\therefore$ 9-bit value.

6.9 On a simple page system, associative registers hold the most active page entries and the full page table is 1 stored in the main memory. If references are satisfied by the associative registers take 100 ns, & 180 ns, what must the hit ratio be to active in effective access time of 125 ns?

Sol. If h is the hit ratio, then
 $100h + 180(1 - h) = 125$

$\therefore \boxed{h = 68.75\%}$

6.10 On a system using simple segmentation, computer the physical address for each of the logical address given below segment table. If the address generates a segment fault, indicate so.

Segment	Base	Length
0	330	124
1	876	211
2	111	99
3	498	302

(a) 0, 99

Sol. 429

offset 99 < segment length 124. Segment 0 begin at location 330, so offset 99 is at physical address $99 + 330 = \boxed{429}$.

(b) 2,78

Sol. offset 78 < segment length 99. Segment 2 begins at location 111, so offset 78 is at physical address $78 + 111 = \boxed{189}$

(c) 1,265

Sol. Fault.

offset 265 > segment lengths 211.

(d) 3,222

Sol. offset 222 < segment length 302. Segment 3 begins at location 498, so offset 222 is at physical address $222 + 498 = \boxed{720}$

(e) 0, 111

Sol. Offset 111 < segment length 124. Segment 0 begins at location 330, so offset 111 is at physical address $111 + 330 = \boxed{441}$

Summary

- Address binding is done in 3 ways.
 - (a) Compile time
 - (b) Load time
 - (c) Run time
- A compiler will typically bind the symbolic addresses to relocatable addresses.
- Compile time address binding generates absolute code.

- Logical address is generated by CPU.
- Logical address is also called virtual address.
- Logical address and physical address are different for execution time address binding.
- Memory management unit is used to map from virtual to physical address.
- Base register is also called relocation register.
- User program never sees the real physical address.
- User program deals with logical address.
- Base register holds the smallest legal physical memory address.
- Limit register contains the size of the range of process.
- Advantage of dynamic loading is that an unused writing is never loaded.
- Swapping is a technique of temporarily removing inactive program from the memory of a system.
- If binding is done at assembly or load time, then the process cannot be moved to different locations.
- Memory allocation methods are:
 - (a) First fit allocates the first hole, i.e., big enough
 - (b) Best fit allocates the smallest hole, i.e., big enough
 - (c) Worst fit allocates the largest hole.
- Physical memory is broken into fixed size blocks called frames.
- Logical memory is broken into blocks of same size called pages.
- Page size is defined by hardware.
- TLB is associative, high speed memory.
- Paging eliminates fragmentation.
- In segmentation, program is divided into various size segments.

Multiple-Choice Questions

1. The main memory accommodates
 - (a) operating system
 - (b) CPU
 - (c) user processes
 - (d) All of these
2. The operating system is
 - (a) in the low memory
 - (b) in the high memory
 - (c) either (a) or (b) (depending on the location of the interrupt vector)
 - (d) None of these
3. In contiguous memory allocation
 - (a) each process is contained in a single contiguous section of memory
 - (b) all processes are contained in a single contiguous section of memory

-
- (c) the memory space is contiguous
 - (d) None of these
4. The relocation register helps in
- (a) providing more address space to processes
 - (b) a different address space to processes
 - (c) to protect the address spaces of processes
 - (d) None of these
5. With relocation and limit register, each logical must be _____ the limit register.
- (a) less than
 - (b) equal to
 - (c) greater than
 - (d) None of these
6. The operating system and the other processes are protected from being modified by an already running process because
- (a) they are in different memory spaces
 - (b) they are in different logical addresses
 - (c) they have a protection algorithm
 - (d) every address generated by the CPU is being checked against the relocation and limit registers
7. Transient operating system code is code that
- (a) is not easily accessible
 - (b) comes and goes as needed
 - (c) stays in the memory always
 - (d) never enters the memory space
8. Using transient code, _____ the size of the operating system during program execution.
- (a) increases
 - (b) decreases
 - (c) changes
 - (d) maintains
9. When memory is divided into several fixed sized partitions, each partition may contain _____ .
- (a) exactly one process
 - (b) at least one process
 - (c) multiple processes at once
 - (d) None of these
10. In fixed sized partition, the degree of multiprogramming is bounded by _____
- (a) the number of partitions
 - (b) the CPU utilization
 - (c) the memory size
 - (d) All of these
11. The first fit, best fit and worst fit are strategies to select a _____ .
- (a) process from a queue to put in memory
 - (b) processor to run the next process
 - (c) free hole from a set of available holes
 - (d) All of these

12. The _____ is used as index into the page table.
 - (a) frame bit
 - (b) page number
 - (c) page offset
 - (d) frame offset
13. The _____ table contains the base address of each physical memory.
 - (a) process
 - (b) memory
 - (c) page
 - (d) frame
14. The size of a page is typically
 - (a) varied
 - (b) power of 2
 - (c) power of 4
 - (d) None of these
15. Each entry in a segment table has a
 - (a) segment base
 - (b) segment peak
 - (c) segment limit
 - (d) segment value
16. The segment base contains the
 - (a) starting logical address of the process
 - (b) starting physical address of the segment in memory
 - (c) segment length
 - (d) None of these
17. Address binding is
 - (a) going to an address in memory
 - (b) locating an address with the help of another address
 - (c) binding two addresses together to form a new address in a different memory space
 - (d) mapping from one address space to another
18. Binding of instructions and data to memory addresses can be done at
 - (a) compile time
 - (b) load time
 - (c) execution time
 - (d) All of these
19. If the process can be moved during its execution from one memory segment to another, then binding must be:
 - (a) delayed until run time
 - (b) preponed to compile time
 - (c) preponed to load time
 - (d) None of these
20. Dynamic loading is:
 - (a) loading multiple routine dynamically
 - (b) loading a routine only when it is called
 - (c) loading multiple routine randomly
 - (d) None of these

Answers

- | | | | | | |
|--------------|---------|---------------|---------|---------|---------|
| 1. (a) & (c) | 2. (c) | 3. (a) | 4. (c) | 5. (c) | 6. (d) |
| 7. (b) | 8. (c) | 9. (a) | 10. (c) | 11. (c) | 12. (b) |
| 13. (c) | 14. (b) | 15. (a) & (c) | 16. (c) | 17. (d) | 18. (d) |
| 19. (a) | 20. (b) | | | | |



Virtual Memory

7.1 VIRTUAL MEMORY CONCEPT

Virtual memory is a feature of an operating system that enables a process to use a memory address space that is independent of other processes running in the same system and use a space that is larger than the actual amount of RAM present, temporarily relegating some contents from RAM to a disk, with little or no overhead. Virtual memory is simply the operating system using some amount of disk space as if it were real memory.

- First requirement for execution is that instruction must be in physical memory.
 - (a) One approach is to place the entire logical address in main memory.
 - (b) Overlays and dynamic loading may relax this criteria.
 - (c) But size of the program is limited to the size of the main memory.
- Normally, the entire program may not be needed in main memory because:
 - (a) Programs have error conditions.
 - (b) Arrays, lists and tables may be declared by 100 by elements, but seldom larger than 10 by 10 elements.
 - (c) Assembler program may have room for 3000 symbols although average program may contain less than 200 symbols. Benefits of the ability to execute program that partially in memory:
 - (i) The user can write programs and software for entirely large virtual address space.
 - (ii) More programs can run at the same time.
 - (iii) Less I/O would be needed to load or swap each user program into memory.
- Virtual memory is a technique that allows the execution of processes that may not be completed in memory.
- Programs are often larger than main memory.
- Virtual memory abstract main memory into an external uniform array of storage.
- Advantages of virtual memory:
 - (i) Separation of user logical memory from physical memory as perceived by users.
 - (ii) Only part of the program needs to be in memory for execution.

- (iii) Logical address space can therefore be much larger than physical address space.
- (iv) Allows address space to be shared by several processes.
- (v) Allows for more efficient process creation.
- (vi) Frees the programmer from memory constraints.
- Virtual memory can be implemented as shown in Fig. 7.1.

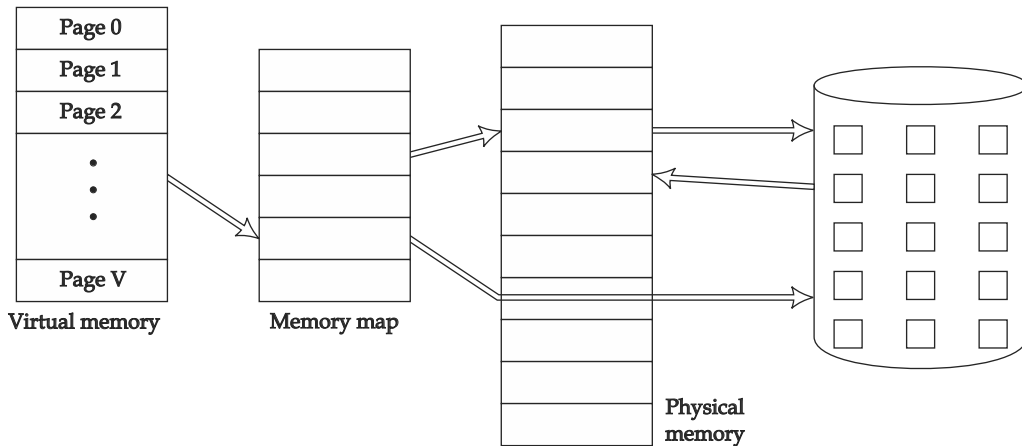


Fig. 7.1 Virtual memory

- The virtual address space of a process refers to the logical (or virtual) view of how a process is stored in memory. Typically, this view is that a process begins at a certain logical address, say, address 0-and exists in contiguous memory, as shown in Fig. 7.2.

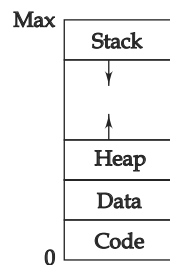


Fig. 7.2 Virtual address space

7.2 DEMAND PAGING

- A demand-paging system is similar to a paging system with swapping where processes reside in secondary memory. When we want to execute a process, we swap it into memory.

- For demand paging, we use '*lazy pager*'. A lazy pager never swaps a page into memory unless required. It brings a page into memory only when it is needed.

Advantages of demand paging:

- Less I/O needed
- Less memory needed
- Faster response
- More users.

When a page is needed, reference to it. If invalid reference $\Rightarrow$ abort

If not in memory $\Rightarrow$ bring the needed page to memory.

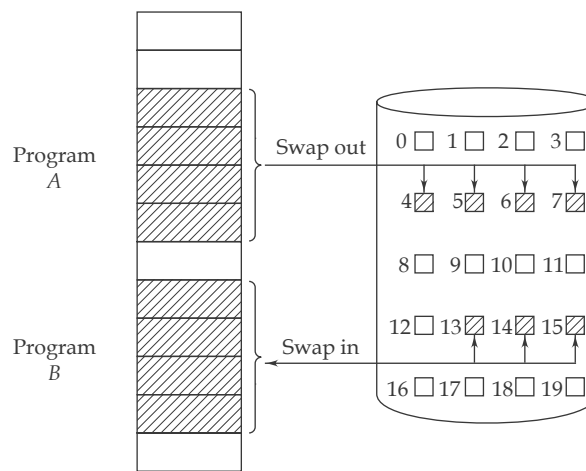


Fig. 7.3 Demand paging

Why is it called demand paging and not simply paging?

An OS page when it moves pages in and out of memory.

If space is needed and a particular page has not been used recently, it is *paged out* (stored to disk) and the space is freed up for more active pages. Pages that have been migrated to disk are *paged in* (returned to memory) when they are needed.

If an item can be paged, it implies that the item resides in virtual space. It is accessed using virtual address and space is allocated for its mapping information. The CPU allocates physical memory for the item itself only when the item is paged in. A virtual page is considered mapped when the OS has information on its location (in memory or in disk).

Thus, the demand paging is more like "*lazy swapping*", it swaps pages only if they are needed. Thus, it is known as demand paging rather than simply paging.

In demand paging, we use "*lazy swapper*" which brings the needed page into the memory.

Note: Since in demand paging, lazy swapper brings a page into memory only when it is needed, it is also referred as "*lazy pager*".

7.3 ADVANTAGES/DISADVANTAGES OF DEMAND PAGING

Advantages

- Demand paging, only loads pages that are demanded by the executing process.
- As there is more space in the main memory, more processes can be loaded reducing context switching time which utilizes large amounts of resources.
- Less loading latency occurs at program startup, as less information is accessed from secondary storage and less information is brought into main memory.
- Does not need extra hardware support since protection fault can be used to get page fault.

Disadvantages

- Demand paging may have lower performance as individual program face extra latency.
- Programs running on low-cost, low-power embedded systems may not have a memory management unit that supports page replacement.
- Memory management with page replacement algorithm becomes slightly more complex.

7.4 VALID-INVALID BIT

With each page valid-invalid bit is associated. When this bit is set to “valid”, the associated page is both legal and in memory.

If the bit is set to “invalid”, the page either is not valid (that is, not in the logical address space of the process) or is valid but is currently on the disk.

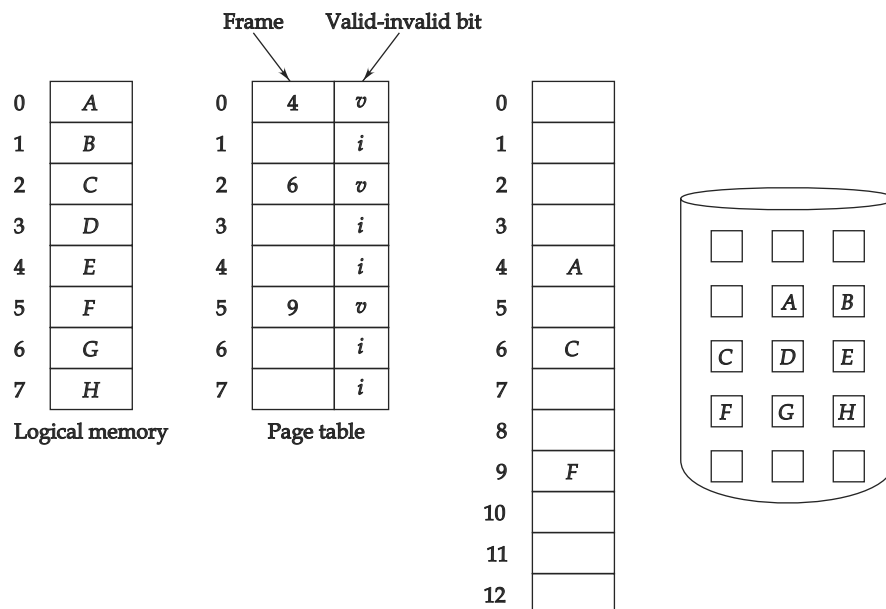


Fig. 7.4 Page table when some pages are not in main memory

The page-table entry for a page that is brought into memory is set as usual, but the page-table entry for a page that is not currently in memory is either simply marked invalid or contains the address of the page on disk. This situation is depicted in Fig. 7.4.

7.5 PAGE FAULT

If there is ever a reference to a page marked invalid, first reference will trap to OS $\Rightarrow$ page fault trap.

OS handles the page fault as follows:

1. Check the internal table, to determine whether the reference is valid or invalid.
2. If the reference is invalid, then terminate.
3. Find free frame.
4. Schedule disk operation.
5. Modify the internal table to indicate that page is in main memory.
6. Restart the instruction.

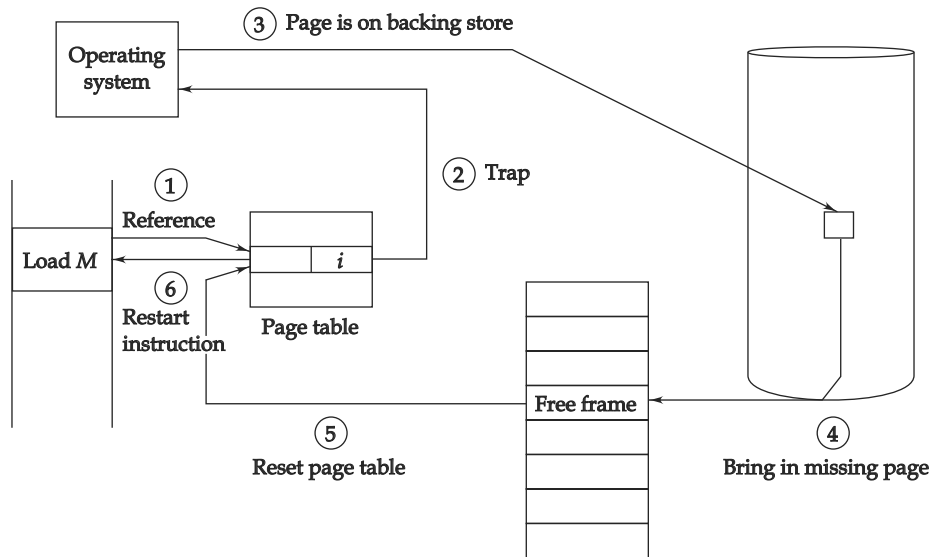


Fig. 7.5 Page fault handling steps

7.6 PERFORMANCE OF DEMAND PAGING

Let p be the probability of page fault ($0 \leq p \leq 1$). We would expect p to be close to zero, that is, we would expect to have only a few page faults.

if $p = 0$, no page faults.

* Memory access time = effective access time

if $p = 1$, every reference is a fault.

Effective access time (EAT)

$$\text{EAT} = (1 - p) * \text{memory access} + p (\text{page fault overhead}).$$

Major operations during page fault:

- Trap to OS
- Save user registers and process state
- Issue a disk read
- Wait for interrupt from disk
- Wait for the CPU
- Restore the process status

Demand paging example:

With an average page-fault service time of 8 milliseconds and a memory access time of 200 nanoseconds.

$$\begin{aligned} \text{The EAT (in nanoseconds)} &= (1 - p) \times 200 + p (8 \text{ ms}) \\ &= (1 - p) \times 200 + 8,000,000 p \\ &= 200 + 7,999,800 p. \end{aligned}$$

We see that EAT is directly proportional to the page fault rate

It is important to keep the *page fault rate low* otherwise EAT increases and slowing the process execution dramatically.

7.7 BASIC PAGE REPLACEMENT

Page replacement is basic to demand paging.

If the memory is full then the inactive pages which are not needed currently for execution are removed and are replaced by those pages from the secondary device which are to be executed. This is called page replacement.

Basic Page Replacement

1. Find the location of the desired page on the disk
2. Find a free frame:
 - If there is a free frame, use it.
 - If there is no free frame, use a page replacement algorithm to select a victim frame.
3. Read the desired page into the (new) free frame.
4. Restart the process.

There are many different page replacement algorithms. Every OS probably has its own replacement scheme.

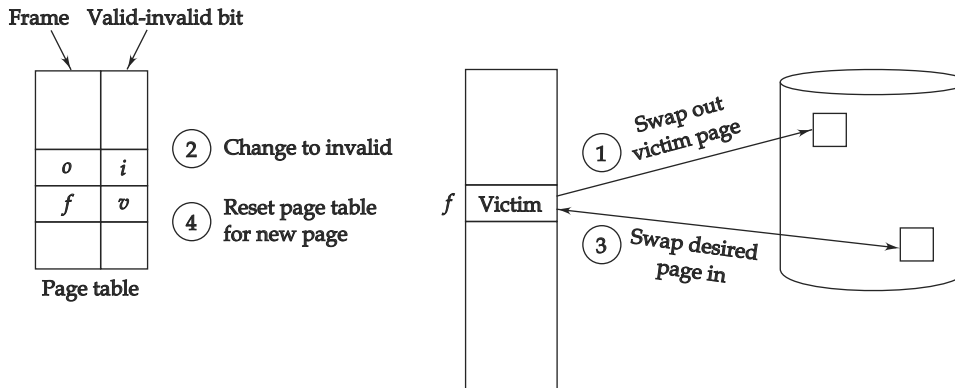


Fig. 7.6 Page replacement

How do we select a particular replacement algorithm?

In general, we want the one with the lowest page-fault rate.

We evaluate an algorithm by running it on a particular *reference string* (string of memory references) and computing the number of page faults.

We can generate *reference strings* artificially (by a random number generator, for example) or we can trace a given system and record the address of each memory reference. But the later choice produces a large number of data. To reduce the number of data, we use two facts:

- (i) For a given page size, we need to consider only the page number rather than the entire address.
- (ii) If we have a reference to a page p , then any immediately following reference to page p will never cause a page fault. Page p will be in the memory after the first reference; the immediately following references will not fault.

For example, if we trace a particular process, we might record the following address sequence: 0102, 0103, 0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105, which at 100 bytes per page, is reduced to the following reference string:

1, 4, 1, 6, 16, 16, 1, 6, 1

- To determine the number of page faults for a particular reference string and page-replacement algorithm, we also need to know the *number of page frames* available. Obviously, as the number of frames available increases, the number of page faults decreases [Except in a few cases, like FIFO replacement algorithm].

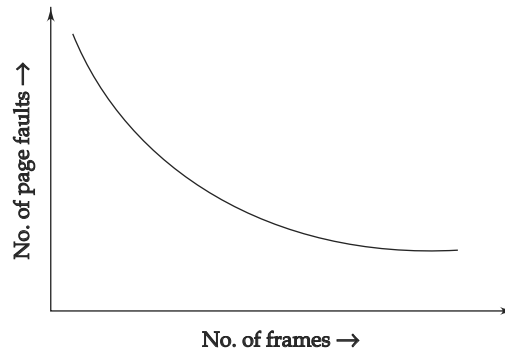


Fig. 7.7 Page faults vs number of frames

Reducing overhead: Modify bit

- Use modify (dirty) bit to reduce overhead of page transfers → only modified pages are written to disk.
- Modify bit can be used to reduce the overhead with the help of hardware.
- It is set, indicating the page has been modified.
- While replacing a page,
If the modify bit is set, we must write that page to disk.
If it is not set we can avoid overwriting it, if it is not overwritten.

7.8 PAGE REPLACEMENT ALGORITHMS

Page replacement algorithms decide which memory pages to page out or swap out when a page of memory needs to be allocated. Paging happens when a page fault occurs and a free page cannot be used to satisfy the allocation, either because there are none, or because the number of free pages is less than some threshold.

When the page that was selected for replacement and paged out is referenced again, it has to be paged in, and this involves waiting for I/O completion. This determines the quality of the page replacement algorithm: the less time waiting for page-in is, the better the algorithm.

A page replacement algorithm looks at the limited information about accesses to the pages provided by reference string while balancing this with the costs (primary storage and processor time) of the algorithm itself.

7.8.1 First-In-First-Out (FIFO) Algorithm

- A FIFO replacement algorithm associates with each page when that page was brought into memory. When a page must be replaced, the oldest page is chosen.
- It is not necessary to record the time when a page is brought in. We can create a FIFO queue to hold all the pages in the memory. We replace the page at the head of the queue. When a page brought into memory, we insert it at the tail of the queue.

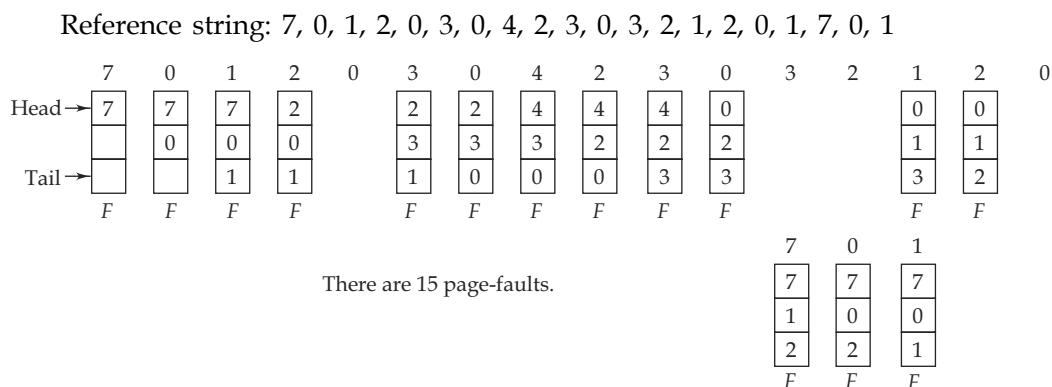


Fig. 7.8 FIFO page-replacement algorithm

7.8.2 Belady's Anomaly

For some page replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

More frame $\Rightarrow$ More page faults

Example:

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

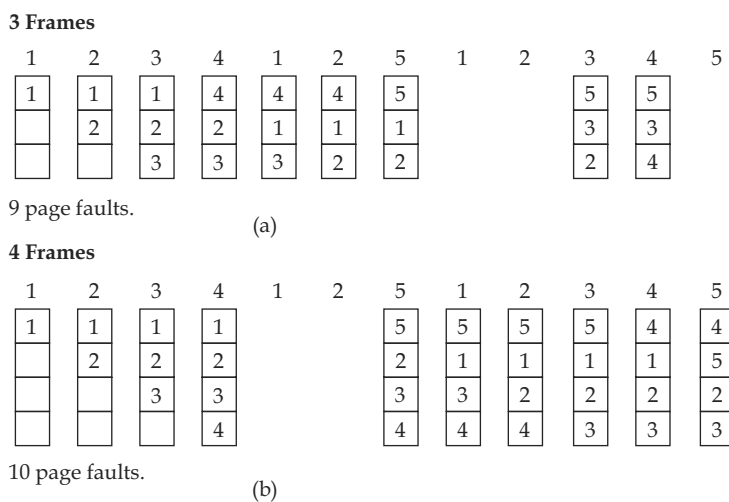


Fig. 7.9 FIFO page-replacement algorithm example using (a) 3 frames (b) 4 frames

As number of page faults increases with increase in number of frames.

7.8.3 Optimal Algorithm

Optimal algorithm replaces the page that will not be used for the longest period of time.

Use of this page-replacement algorithm guarantees the lowest possible page-fault rate for a fixed number of frames.

Choose from the pages a page that will not be used for longest.

7.8.4 Least Recently Used (LRU) Algorithm

LRU associates with each page the time of that page's last use. The page that has not been used for longest period of time is replaced, when there is the option to replace a page.

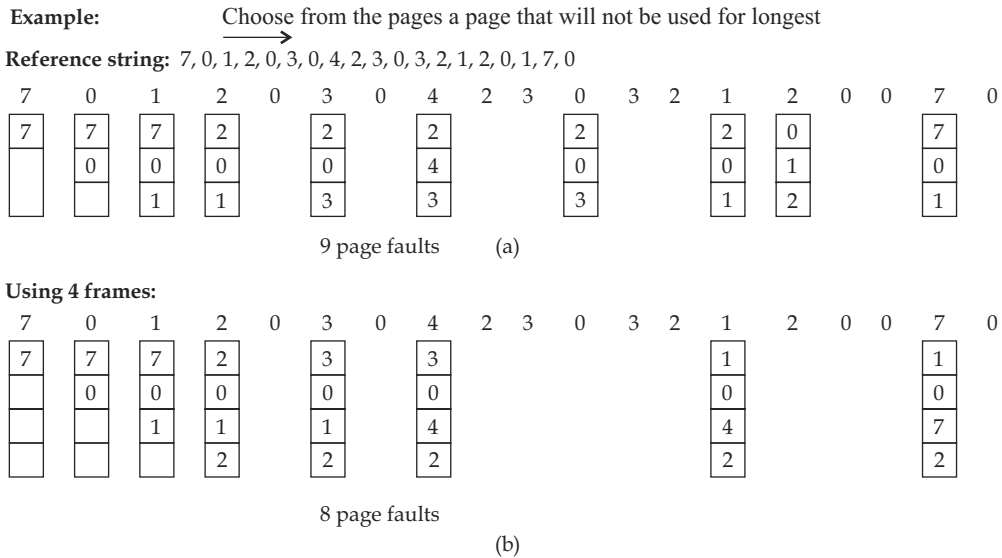


Fig. 7.10 Optimal page replacement algorithm (a) using 3 frames (b) using 4 frames

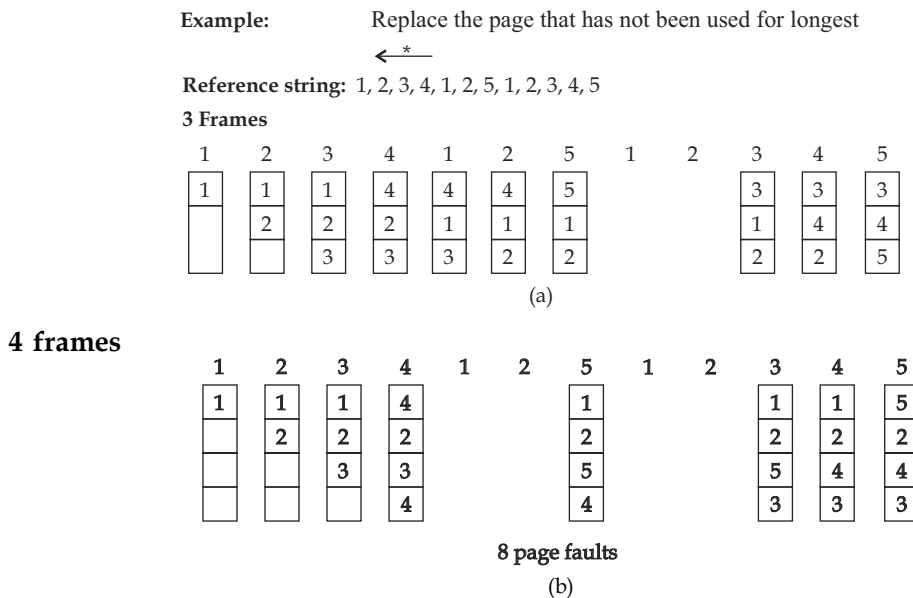


Fig. 7.11 LRU page replacement algorithm (a) using 3 frames (b) using 4 frames

Implementation of LRU Page Replacement: Counter implementation

An LRU page-replacement algorithm may require substantial hardware assistance. The problem is to determine an order for the frames defined by the time of last use. *Counter implementation* is feasible:

1. We associate with each page-table entry a time-of-use field, and add to the CPU a logical clock or counter.
2. The clock is incremented for every memory reference.
3. Whenever a reference to a page is made, the contents of the clock register are copied to the time-of-use field in the page-table entry for that page. In this way, we always have the "time" of the last reference to each page.
4. We replace the page with the smallest time value.

This scheme requires a search of the page table to find the LRU page, and a write to memory (to the time-of-use field in the page table) for each memory access. The time also must be maintained when page tables are changed due to CPU scheduling.

Overflow of the clock must be considered.

7.8.5 Comparison of Algorithms

Page replacement algorithm	Advantages	Disadvantages
Optimal	• Lowest page fault rate • Never suffers from Belady's anomaly • Twice as good as FIFO	• Difficult to implement • It needs future knowledge
FIFO	• Easy to understand • Easy to execute	• Not very effective • Track of every frame is maintained • Suffers from Belady's anomaly • Bad replacement choices increase the page faults rate
LRU	• Overcomes the drawback of optimal algorithm of keeping future knowledge • It keeps track of the last	• It is sometimes difficult to determine an order for the frames defined by the time of last use

7.9 THRASHING

If a process does not have "enough" pages, the page fault rate is very high. This leads to:

- low CPU utilization
- OS thinks that need to increase the degree of multiprogramming
- If the process does not have number of frames equivalent to the number of active pages, it will very quickly page fault.
- Since all the pages are in active use, it will page fault again.

Thrashing is a high paging activity.

Thrashing: A process is thrashing if it is spending more time in swapping pages in and out.

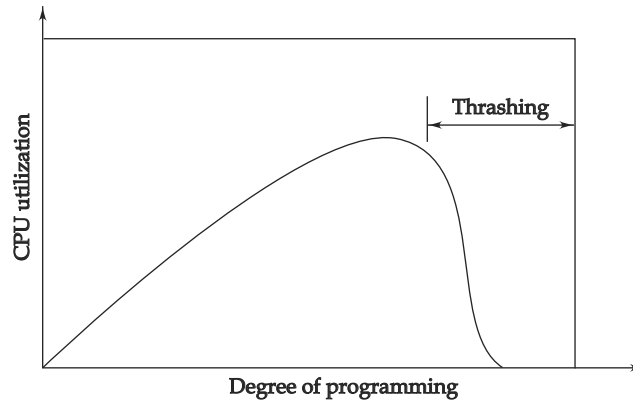


Fig. 7.12 Thrashing

- Paging work with locality model:
 - (i) Process migrates from one locality to another
 - (ii) A locality is set of pages that are actively used together.
 - (iii) Localities may overlap.
- Thrashing is a phenomenon of moving pages back and forth between memory and secondary storages.

Causes of Thrashing

- OS monitors CPU utilization.
- If it is low, increases the degree of multiprogramming.
- Consider that a process enters new execution phase and starts faulting.
 - It takes pages from other processes.
 - If processes need those pages, and they also fault, then process takes frames from other processes.
- The queue increases for paging device and ready-queue empties.
- CPU utilization decreases.

The *solution of this problem* is to provide the process as many frames as it needs. Working set model based on assumption of locality helps us in knowing how many frames a process needs.

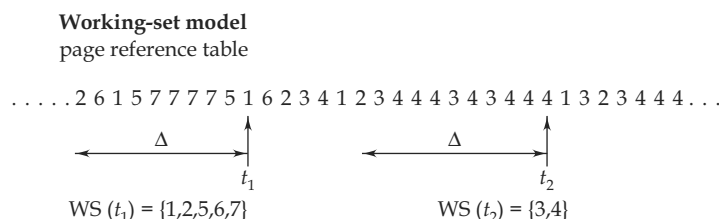


Fig. 7.13 Working set model

- Based on locality.
- Define a parameter Δ
 $\Delta \equiv$ working set window $\equiv$ a fixed number of page references, e.g., - 10,000 instructions.
- Most recent references are examined.
- WSS_i (working set process P_i) = total number of pages referenced in the most recent Δ (varies in time).
 - (i) if Δ too small $\Rightarrow$ will not encompass entire locality
 - (ii) if Δ too large $\Rightarrow$ will encompass several localities
 - (iii) if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \sum WSS_i \equiv$ total demand frames
- if $D > m \Rightarrow$ thrashing

Advantages of Thrashing

1. More utilization of main memory
2. Flexible, adjusts to divergent process needs.
3. Virtual memory gives the opportunity for the system to sustain an indefinite number of process simultaneously

Disadvantages of Thrashing

1. The system spends most of its time in swapping rather than executing instructions.
2. Uses more hardware space as page faults are handled therefore more overhead with tables.
3. System's throughput decreases.

7.10 CACHE MEMORY ORGANIZATION

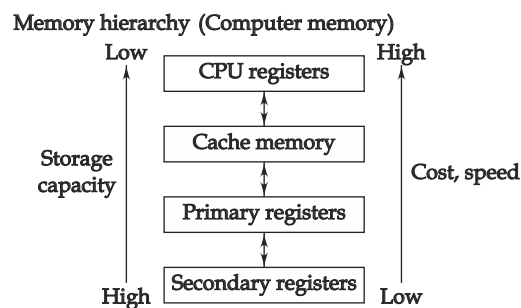


Fig. 7.14 Memory hierarchy

Computer memory is organized into a hierarchy

At the highest level are the processor registers. Next comes one or more levels of cache. Next comes main memory followed by secondary memory.

As one goes down, the memory hierarchy finds another memory.

The overall goal of using memory hierarchy is to obtain the highest possible average access speed while minimizing the total cost of the entire memory system.

Cache Memory

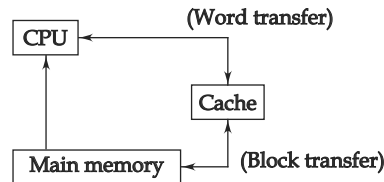


Fig. 7.15 Use of cache memory in system

Cache memory is a specially designed faster but smaller memory area, generally used to keep recently referenced data and data near recently referenced data, which can lead to potential performance increase.

The cache automatically retains a copy of some of the recently used words from the main memory. If the cache is designed properly, then most of the time the processor will request memory words that are already in the cache memory.

Average access time of cache memory $\bar{T} = ht_c + (1 - h)(t_c + t_m)$

$\swarrow$ $\downarrow$ $\downarrow$
 hit rate access access
 of cache time of time
 memory cache of main
 memory memory

$$\text{Efficiency} = \frac{t_c}{\bar{T}}$$

- Cache memory is a specially designed faster but smaller memory area.
- Cache memory is used to keep the recently referenced data.
- Cache memory can lead to potential performance increase.
- Cache memory automatically retains a copy of some of the recently used words from the main memory.
- Cache memory sits between normal main memory and CPU.
- Cache memory is typically used for both, i.e., programs and data.
- It works because of locality

Numerical Examples

- 7.1 Consider a demand-paged computer system where the degree of multiprogramming is currently fixed a four. The system was recently measured to determine utilization of CPU and the paging disk. The degree of mutliprogramming should be increased to increase the CPU utilization? Does paging helping?

(a) CPU utilization 13% and disk utilization 97%**Sol.** System is thrashing

- The degree of multiprogramming should be decreased.
- Paging is not helping.

(b) CPU utilization 87% and disk utilization 3% system**Sol.** System is well utilized, CPU is being kept busy most of the time.

- The degree of multiprogramming probably should stay the same, increasing it may lead to thrashing.
- Paging is helping.

(c) CPU utilization 13%; and disk utilization 3%;**Sol.** System is under utilization; the CPU is not getting enough work.

- The degree of multiprogramming should be increased.
- Paging is not really helping or hurting.

7.2 A FIFO page replacement is used with four page-frames. How many page-faults will occur with the reference string 0 1 7 2 3 2 7 1 0 3 if the four frames are initially empty. Repeat the problem for LRU.**Sol.****FIFO**

0	1	7	2	3	2	7	1	0	3
0	0	0	0	3				3	
	1	1	1	1				0	
		7	7	7				7	
			2	2				2	
F	F	F	F	F				F	

6 page faults

LRU

0	1	7	2	3	2	7	1	0	3
0	0	0	0	3				0	0
	1	1	1	1				1	1
		7	7	7				7	7
			2	2				2	3
F	F	F	F	F				F	F

7 page faults

7.3 Consider the following pages of a reference string:

1, 2, 0, 3, 5, 1, 5, 7, 2, 4, 1, 2, 5, 3, 7

Implement FCFS, LRU and optimal page replacement algorithm and calculate the number of page faults and hit ratio by considering three frames in a block.

Sol.

FCFS

1	2	0	3	5	1	5	7	2	0	3	5	4	1	2	5	3	7
1	1	1	3	3	3		7	7	7	3	3	3	1	1	1	3	3
	2	2	2	5	5		5	2	2	2	5	5	5	2	2	2	7
		0	0	0	1		1	1	0	0	0	4	4	4	5	5	5

17 page faults

$$\text{Hit ratio} = \frac{1}{18} = 0.056$$

LRU

1	2	0	3	5	1	5	7	2	0	3	5	4	1	2	5	3	7
1	1	1	3	3	3		7	7	7	3	3	3	1	1	1	3	3
	2	2	2	5	5		5	5	0	0	0	4	4	4	5	5	5
		0	0	0	1		1	2	2	2	5	5	5	2	2	2	7

17 page faults

Optimal replacement

1	2	0	3	5	1	5	7	2	0	3	5	4	1	2	5	3	7
1	1	1	1	1			7		0	3		4	1			3	3
	2	2	2	2			2		2	2		2	2			2	7
		0	3	5			5		5	5		5	5			5	5

12 page faults

$$\text{Hit ratio} = \frac{6}{12} = 0.5$$

7.4 Consider a demand paged system. Page tables are held in registers. It takes 8 milliseconds to service a page fault if an empty page is available or the replaced page is not modified, and 20 milliseconds if the replaced page is modified. Memory access time is 100 nanoseconds. Assume that the page to be replaced is modified 70% of the time. What is the maximum acceptable page fault rate for an effective access time of not more than 200 nanoseconds?

Sol. $0.2 \text{ us} = ((1 - p) * 0.1 \text{ us}) + (0.2 p * 8000 \text{ us}) + (0.7 p * 20,000 \text{ us})$

7.5 Consider a page replacement algorithm should minimize the number of page faults. We can do this minimization by distributing heavily used pages evenly all over the memory rather than having them complete for small number of page frames. We can associate with each frame a counter of number of pages that are associated with that frame and then to replace a page we search for the page frame with the smallest counter.

(a) Define the page replacement algorithm with the above idea specifically addressing the problem of:

- (i) What are the initial values of counters?
- (ii) When counters are increased or decreased.
- (iii) How the page is replaced.

Sol.**(a) Define a page replacement algorithm addressing the problems of**

- (i) Initial value of the counter 0
- (ii) Counters are increased—whenever a new page is associated with that frames
- (iii) Counters are decreased—whenever one of the pages associated with that frame.
- (iv) How the page to be replaced is selected—find a frame with the smallest counter. Use FIFO for breaking ties.

(b) How many page faults occur for your algorithms for the following reference string, for four page frames?**1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 5, 4, 5, 4, 5, 2****Sol. (b)**

1	2	3	4	5	3	1	4	6	7	8	7	8
Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count		
1 0	1 1	1 1	1 1	5 2	5 2	5 2	5 2	5 2	5 2	5 2		
 0	2 1	2 1	2 1	2 1	1 2	1 2	1 2	1 2	1 2	1 2		
 0	 0	3 1	3 1	3 1	3 1	6 2	6 2	6 2	6 2	6 2		
 0	 0	 0	4 1	4 1	4 1	4 1	4 1	7 2	7 2	4 1		
1	2	3	4	5	6	7	8	9				
9	7	8	9	5	4	5	4	2				
Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count	Frame Count				
8 3				8 3	5 2			2 4				
9 3				9 3	2 1			9 3				
6 2				5 3	3 1			5 3				
7 2				7 2	4 1			4 3				
10				11	12			13				

= 13 page faults

(c) What is the minimum number of page faults for an optimal page replacement strategy for reference string in part (b) with four frames.

- (i) Initial values of counters is 0
- (ii) Counters are increased whenever a new page is associated with the frame no longer required.
- (iii) Find the frame with the smallest counter and use FIFO for breaking the tie.

Sol.

1	2	3	4	5	3	4	1	6	7	8	7	8	9	7	8	9	5	4	5	4	2
1	1	1	1	1				6	7	7			7					4			2
	2	2	2	5				5	5	5			5				5	5		5	
								3	3	8			8				8	8		8	
																				9	
1	2	3	4	5				6	7	8			9				10			11	

= 11 page faults

7.6 An operating system supports a paged virtual memory, using a central processor with a cycle time of 1 microsecond. It costs an additional 1 microsecond to access a page other than the current one. Page has 100 words, and the paging device is a drum that rotates at 300 revolution per minutes and transfers 1 million words per second. The following statistical measurements were obtained from the system:

- 1% of all instructions executed accessed a page other than the current page.
- of the instruction that accessed another page, 80% accessed page already in memory.
- When a new page was required, the replaced page was modified 50% of the time.

Calculate the effective instruction time on this system, assuming that the system is running one process only, and that the processor is idle during drum transfers.

Sol.

$$\begin{aligned}
 \text{Effective access time} &= 0.99 \times (1 \mu\text{sec} + 0.008 \times (2 \mu\text{sec}) + 0.002 \times \\
 &\quad (10,000 \mu\text{sec} + 1,000 \mu\text{sec}) \\
 &\quad + (+ 0.001 \times 10,000 \mu\text{sec} + 1,000 \mu\text{sec}) \\
 &= (0.99 + 0.016 + 22.0 + 11.0) \mu\text{sec} \\
 &= 34.0 \mu\text{sec}
 \end{aligned}$$

7.7 Consider the following page reference string

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, or seven frames? Remember all frames are initially empty. So your first unique pages will all cost one fault each.

- LRU replacement
- FIFO replacement
- Optimal replacement

Sol.

Number of frames	LRU	FIFO	Optimal
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

7.8 On a system using demand paged memory, it takes 120 ns to satisfy a memory request if the page is in memory. If the page is not in memory, the request takes on an average it is 5 ms. What would be the page fault to achieve an effective access time of microsecond? Assume the system is running only a single process and the CPU is idle during page swaps.

Sol. $120 \text{ ms} \times (1 - P) + P \times 5 \text{ ms} = 1 \text{ msec}$

But, $5 \text{ ms} = 5 \times 10^{-3} \text{ sec}$

$120 \text{ ms} = 120 \times 10^{-9} \text{ sec}$

$1 \text{ msec} = 1 \times 10^{-6} \text{ sec}$

$\therefore 120 \times 10^{-9} \times (1 - P) + P(5 \times 10^{-3}) = 1 \times 10^{-6}$

$\therefore \boxed{P = 176.004 \text{ msec}}$

7.9 A paging system consists of physical memory 2^{24} bytes, pages of logical address spaces is 256. Page size of 2^{10} bytes, how many bits are in logical address.

Sol. Logical address space = $256 = 2^8$

Page size = 2^{10} bytes

So the total logical address space = $2^8 \times 2^{10}$

= 2^{18} bytes

for 2^{18} byte address space $\Rightarrow$ 18 bit address is required.

7.10 System using a paging and segmentation, the virtual address space consists of up to 8 segments where each segment can be 2^{29} byte long. The hardware pages each segment into 256 bytes pages. How many bits in the virtual address specify the

(i) Segment number

(ii) Page number

(iii) Offset within page

(iv) Entire virtual address

Sol.

(i) Segment number

Virtual address space consists of up to 8 segments.

So $8 = 2^3$

∴ 3 bits are needed to specify segment number.

(ii) Page number

Hardware pages each segment into 256 byte pages. So

$256 = 2^8$ bytes pages

Size of segment is = 229 bytes

∴ $\frac{2^{29}}{2^8} = 2^{29-8} = 2^{21} \Rightarrow 21$ pages

So 21 bits are required to specify the page number.

(iii) offset with in page: for 2^8 byte page

8 pages are required

(iv) Entire virtual address:

= Segment numbers + Page number + offset

= $3 + 21 + 8$

= 32 **Ans.**

7.11 Consider a demand paging system with a paging disk that has an average access and transfer time of 20 ms. Addresses are translated through a page table in the main memory, with an access time of 1 microsecond per memory access. Thus, each memory reference through the page table takes two accesses. To improve this time, we have added an associative memory that reduces access time to one memory reference, if the page table entry is in associative memory. Assume that 80% of accesses are in associative memory and that of the remaining 10% (or 2% of total) cause page faults. What is the effective memory access time?

Sol. Effective access time = $(0.8) \times 1 \mu\text{sec} + (0.1) \times (2 \mu\text{sec}) + (0.1) \times (5002 \mu\text{sec})$
 = 501.2 μsec
 = 0.5 millsec.

7.12 Consider the following page reference string:

1, 2, 3, 1, 4, 5, 6, 2, 1, 3, 2, 7, 6, 4, 1, 2, 6 Find the number of page faults using LRU using 6 frames.

Sol. LRU

	1	2	3	1	4	5	6	2	1	3	2	7	6	3	4	1	2	6
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4				4	4	4	4	4	4	4	7	7	7	7	7	7	7	7
5					5	5	5	5	5	5	5	5	5	4	4	4	4	4
6						6	6	6	6	6	6	6	6	6	6	6	6	6

∴ Total number of page faults = 8.

7.13 What is the cause of thrashing? How does the system detect thrashing? Once it detects thrashing what can the system do to eliminate this problem?

Sol. Thrashing is caused by under allocation of the minimum number of pages required by a process, forcing it to continuously page fault. The system can detect thrashing by evaluating the level of CPU utilization as compared to the level of multiprogramming. It can be eliminated by reducing the level of multiprogramming.

7.14 Given a system with four page frames, the following table indicates page, load time, last reference time, dirty bit and reference bit.

Page	Load time	Last reference	Dirty bit	Reference bit
0	167	374	1	1
1	321	321	0	0
2	254	306	1	1
3	154	331	0	1

(a) Which page will FIFO replace?

Sol. 3, FIFO selects the page loaded first, page 3 at time 154.

(b) Which page will LRU replace?

Sol. 2, LRU selects the page referenced last, page 2 at time 306.

(c) Which page will NRU replace?

Sol. 1, NRU first looks for a clean page that was recently referenced. Page 1 is only such page.

(d) Which page will second chance replace?

Sol. Second chance first checks the oldest page, i.e., page 3. Since its reference bit is 1, it reacts that bit to 0 & entry's load time to the current time. The same thing happens to page 0, the next oldest page. The reference bit for page 2 is then checked, and since it is 0 it is the selected page.

7.15 Consider the following page reference. Indicate page faults and calculate the total number of page faults for optimal and LRU. The total number of available frames is 4.

1 2 3 2 5 6 3 4 6 3 7 3 1 5 3 6
 3 4 2 4 3 4 5 1

Sol. (1) optimal (frames = 4)

	1	2	3	2	5	6	3	4	6	3	7	3	1	5	3	6	3	4	2	4	3	4	5	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	2	2	2	2	1
1	-	2	2	2	2	6	6	6	6	6	6	6	6	6	6	6	6	4	4	4	4	4	4	4
2	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
3	-	-	-	-	5	5	5	4	4	4	7	7	7	5	5	5	5	5	5	5	5	5	5	5
Page fault	F	F	F		F	F		F			F			F				F	F					F

No. of page fault = 11.

(ii) LRU

	1	2	3	2	5	6	3	4	6	3	7	3	1	5	3	6	3	4	2	4	3	4	5	1
0	1	1	1	1	1	6	6	6	6	6	6	6	6	5	5	5	5	5	2	2	2	2	2	1
1	-	2	2	2	2	2	2	4	4	4	4	4	1	1	1	1	1	4	4	4	4	4	4	4
2	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
3	-	-	-	-	5	5	5	5	5	5	7	7	7	7	7	6	6	6	6	6	6	6	5	5
Page fault	F	F	F		F	F		F			F		F	F		F		F	F				F	F

No. of page fault = 14.

Summary

- Virtual memory allows execution of partially loaded process.
- Virtual memory is the separation of user logical memory from physical memory.
- Virtual memory is commonly implemented by demand paging.
- Demand segmentation can also be used to provide virtual memory. Lazy swapper never swaps a page into memory unless that page will be needed.
- String of memory references is called a reference string.
- Page replacement algorithms are
 - (i) First In First Out (FIFO)
 - (ii) Optimal
 - (iii) Least Recently Used (LRU)
 - (iv) Least Frequently Used (LFU)
- FIFO: Replace the page at the head of the queue.
- Optimal: Replace the page that has not been used for the longest period of time.

- LFU: Page with the smallest count be replaced.
- Page replacement algorithm deals with the selection of the page in memory to be replaced when a new page must be brought in.
- Belady's anomaly: Page fault rate may increase as the number of allocated frames increases.
- FIFO replacement algorithm suffers from Belady's anomaly
- Thrashing: Phenomenon of moving pages back and forth between memory and secondary storage.
- A process is thrashing if it is spending more time in paging than executing.
- Locality of reference are:
 - (i) Spatial locality
 - (ii) Temporal locality.
- Prepaging is an attempt to prevent high level of initial paging.
- Real-time processing system almost never have virtual memory.

Multiple-Choice Questions

1. Which of the following page replacement algorithms suffers from Belady's anomaly?
 - (a) Optimal replacement
 - (b) LRU
 - (c) FIFO
 - (d) both optimal replacement and FIFO
2. A process refers to 5 pages, A, B, C, D, E in the order: A, B, C, D, A, B, E, A, B, C, D, E. If the page replacement algorithm is FIFO, the number of page transfers with an empty internal store of 3 frames is
 - (a) 8
 - (b) 10
 - (c) 9
 - (d) 7
3. In question 2, if the number of page frames is increased to 4, then the number of page transfers
 - (a) decreases
 - (b) increases
 - (c) remains the same
 - (d) None of these
4. A memory page containing a heavily used variable that was initialized very early and is in constant use is removed, then the page replacement algorithm used is
 - (a) LRU
 - (b) LFU
 - (c) FIFO
 - (d) None of these
5. A virtual memory system uses First-In-First-Out (FIFO) page replacement policy and allocates a fixed number of frames to a process. Consider the following statements:
 - P: Increasing the number of page frames allocated to a process sometimes increases the page fault.
 - Q: Some programs do not exhibit locality of reference.

- Which of the following is true?
- (a) Both P and Q are true, and Q is the reason for P
 - (b) Both P and Q are true, but Q is not the reason for P
 - (c) P is false but Q is true
 - (d) Both P and Q are false
6. Users _____ that their processes are running on a paged system.
 - (a) are aware
 - (b) are unaware
 - (c) None of these
 7. If no frames are free, _____ page transfer(s) is/are required.
 - (a) one
 - (b) two
 - (c) three
 - (d) four
 8. When a page is selected for replacement, and its modify bit is set
 - (a) the page is clean
 - (b) the page has been modified since it was read in from the disk
 - (c) the page is dirty
 - (d) (a) and (b)
 9. The aim of creating page replacement algorithms is to
 - (a) replace page faster
 - (b) increase the page fault rate
 - (c) decrease the page fault rate
 - (d) to allocate multiple pages to processes
 10. A FIFO replacement algorithm associates with each page the _____.
 - (a) time it was brought into memory
 - (b) size of the page in memory
 - (c) page after and before it
 - (d) All of these
 11. Optimal page replacement algorithm is
 - (a) Replace the page that has not been used for a long time
 - (b) Replace the page that has been used for a long time
 - (c) Replace the page that will not be used for a long time
 - (d) None of these
 12. Optimal page-replacement algorithm is difficult to implement, because
 - (a) it requires a lot of information
 - (b) it requires future knowledge of the reference string
 - (c) it is too complex
 - (d) it is extremely expensive
 13. LRU page-replacement algorithm associates with each page the _____.
 - (a) time it was brought into memory
 - (b) the time of that page's last use
 - (c) page after and before it
 - (d) All of these
 14. For 3 page frames, the following is the reference string
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

- (i) How many page faults does the LRU page replacement algorithm produce?
 (a) 10 (b) 15
 (c) 11 (d) 12
- (ii) How many page faults does the FIFO page replacement algorithm produce?
 (a) 10 (b) 15
 (c) 11 (d) 12
15. The two methods how LRU page replacement policy can be implemented in hardware are:
 (a) Counters (b) RAM
 (c) Stack (d) Registers
16. When the page fault rate is low
 (a) the turnaround time increases (b) the effective access time increases
 (c) the effective access time decreases (d) (a) and (b)
17. Locality of reference implies that the page reference being made by a process:
 (a) will always be to the page used in the previous page reference
 (b) is likely to be one of the pages used in the last few page references
 (c) will always be one of the pages existing in memory
 (d) will always lead to page faults
18. A process is thrashing if
 (a) it spends a lot of time executing, rather than paging
 (b) it spends a lot of time paging, than executing
 (c) it has no memory allocated to it
 (d) None of these
19. Thrashing _____ the CPU utilization.
 (a) increases (b) keeps constant
 (c) decreases (d) None of these
20. If there are 32 segments, each of size 1 kB, then the logical address should have:
 (a) 13 bits (b) 14 bits
 (c) 15 bits (d) 16 bits

Answers

- | | | | | | |
|---------|----------------|--------------|-----------------|---------|---------|
| 1. (c) | 2. (c) | 3. (b) | 4. (c) | 5. (c) | 6. (b) |
| 7. (b) | 8. (b) and (c) | 9. (c) | 10. (a) | 11. (c) | 12. (b) |
| 13. (b) | 14. (i) (d) | 14. (ii) (b) | 15. (a) and (c) | 16. (c) | |
| 17. (b) | 18. (b) | 19. (c) | 20. (a) | | |



I/O System

8.1 OVERVIEW OF I/O DEVICES

I/O devices are of two types: block device and character device. *Block device* stores information in fixed size blocks. The operating system assigns address to each block. A block device is one with which the driver communicates by sending entire blocks of data. A block device is a device that can host the file system. Every block device driver must provide an interface to the buffer cache as well as the normal file operation interface. Block devices are characterised by the capability to read and write blocks of data to and from random location on an addressable medium. Character device is one with which the driver communicates by sending and receiving single characters.

The only difference between character device and regular file is that user can always move back and forth in the regular file, where characters devices are just data channels, where user can only access sequentially. Block and character devices are represented for user space applications as files that can be manipulated using the traditional file API.

8.2 I/O SUBSYSTEM

The control of device connected to a computer is a major concern of operating system designers. Because I/O devices vary so widely in their functions and speed, various methods need to control them. These methods form the I/O subsystem of the kernel. Kernels provide many services related to I/O. Several services scheduling, buffering, caching, spooling, device reservation and error handling are provided by kernel's I/O subsystem and build on the hardware and device driver infrastructure. The device drivers present a uniform device access interface to the I/O subsystem, much as system calls provide a standard interface between the application and the operating system. The I/O subsystem is also responsible for protecting itself from errant process and malicious users.

8.3 I/O SOFTWARE

I/O software are of two types: device independence and device dependence. Device independence is the key concept in the design of I/O software. Most of the operating systems are unaware of the characteristic of specific devices attached to the system.

Device independence has been accomplished well for the files stored on various devices. Most operating systems include the file system code and most applications can perform read and write operation on the file. Device impedance also applies for user program reading or writing streams. Device impedance permits the program to be implemented as if they will read and write generic or abstract devices, with the actual device specified at run time. The device-independent code contains most of the I/O functionality, but not most of the codes since there are many drivers. All drivers of same class perform the same operations but in different way due to different controllers. Device independent I/O software can perform various functions like:

- Provides buffering
- Support for error reporting
- Provide uniform interfacing for device drive

8.4 I/O BUFFERING

Buffering is the technique by which the device manager can keep slower I/O devices busy during times, when a process is not requiring I/O operations. I/O buffering is a mechanism that improves the throughout of input and output operations. I/O buffering can be of two types: input buffering and output buffering.

Input buffering is the technique of having the input device read information into the primary memory before the process requests it.

Output buffering is the technique of saving information in memory and then writing it to the device while the process continues execution.

8.4.1 Types of I/O Buffering Schemes

- (a) Single buffering
- (b) Double buffering
- (c) Circular buffering
- (d) No buffering

(a) Single buffering: Operating system assigns a buffer in the system, portion of main memory.

Input transfers are made to the system buffer. After transferring, the process moves the block

- into user and request for another block
- user processes can process one block of data
- OS is able to swap the process.

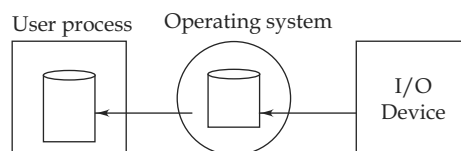


Fig. 8.1 Single buffering

(b) Double buffering: Double buffering is also called buffer swapping.

- There are two buffers used in double buffering.
- One buffer is for the driver or controller to store data while waiting for it to be retrieved by higher level of hierarchy.
- Other buffer is used to store data from lower level.

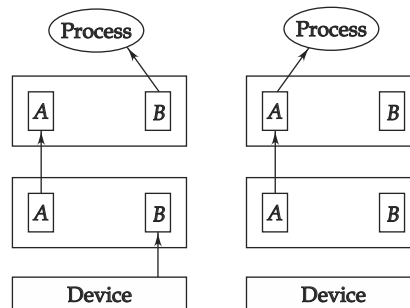


Fig. 8.2 Double buffering

(c) Circular buffering: When more than two buffers are used, the collection of buffer is referred to as circular buffer. In this the producer cannot pass to the consumer because it would overwrite buffers before they had consumed.

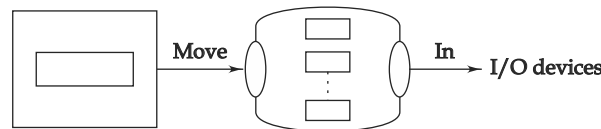


Fig. 8.3 Circular buffering

(d) No buffering: There is a buffer between user process, operating system and I/O devices.

Summary

- The basic hardware elements involved in I/O are buses, device controller and the device themselves.
- The kernel module that controls a device is device driver.
- The kernel I/O subsystem provides numerous services, like I/O scheduling, buffering, caching, spooling, etc.
- Buffering is a technique by which the device manager can keep slower I/O devices busy during times, when a process is not requiring I/O operations.
- I/O system calls are costly in terms of CPU consumption because of the many layers of software between a physical device and an application.

Multiple-Choice Questions

1. If one or more devices use a common set of wires to communicate with the computer system, the connection is called.
(a) CPU (b) Monitor
(c) wirefull (d) bus
2. A _____ a set of wires and a rigidly defined protocol that specifies a set of messages that can be sent on the wires.
(a) port (b) node
(c) bus (d) None of these
3. When device A has a cable that plugs into device B, and device B has a cable that plugs into device C and device C plugs into a port on the computer, this arrangement is called a
(a) port (b) daisy chain
(c) bus (d) cable
4. The _____ present a uniform device-access interface to the I/O subsystem, much as system calls provide a standard interface between the application and the operating system.
(a) devices (b) buses
(c) device drivers (d) I/O system
5. A _____ is a collection of electronics that can operate a port, a bus, or a device.
(a) controller (b) driver
(c) host (d) bus
6. An I/O port typically consists of four register status, control, _____ and _____ registers.
(a) system in, system out (b) data in, data out
(c) flow in, flow out (d) input, output
7. The _____ register is read by the host to get input.
(a) flow in (b) flow out
(c) data in (d) data out
8. The _____ register is written by the host to send output.
(a) status (b) control
(c) data in (d) data out
9. The hardware mechanism that allows a device to notify the CPU is called.
(a) polling (b) interrupt
(c) driver (d) controlling

-
10. The CPU hardware has a wire called _____ that the CPU senses after executing every instruction.
 - (a) interrupt request line
 - (b) interrupt bus
 - (c) interrupt receive line
 - (d) interrupt sense line
 11. The _____ determines the cause of the interrupt, performs the necessary processing and executes a return from the interrupt instruction to return the CPU to the CPU to the execution state prior to the interrupt.
 - (a) interrupt request line
 - (b) device driver
 - (c) interrupt handler
 - (d) All of these
 12. In general, the two interrupt request lines are:
 - (a) non-maskable interrupt
 - (b) blocked interrupt
 - (c) maskable interrupt
 - (d) None of these
 13. The _____ are reserved for events such as unrecoverable memory errors.
 - (a) non-maskable interrupt
 - (b) blocked interrupt
 - (c) maskable interrupt
 - (d) None of these
 14. Caching: (choose all that apply)
 - (a) holds a copy of the data
 - (b) is fast memory
 - (c) holds the only copy of the data
 - (d) holds output for a device
 15. Spooling: (choose all that apply)
 - (a) holds a copy of the data
 - (b) is fast memory
 - (c) holds the only copy of the data
 - (d) holds output for a device
 16. The _____ keeps state information about the use of I/O components.
 - (a) CPU
 - (b) OS
 - (c) kernel
 - (d) shell
 17. The kernel data structures include:
 - (a) process table
 - (b) open file table
 - (c) close file table
 - (d) All of these
 18. Window NT uses a _____ implementation of I/O
 - (a) message-passing
 - (b) draft-passing
 - (c) secondary memory
 - (d) cache
 19. A _____ is a full duplex connection between a device driver and a user level process.
 - (a) bus
 - (b) I/O operation
 - (c) stream
 - (d) flow
 20. I/O is a _____ in system performance.
 - (a) major factor
 - (b) minor factor
 - (c) useless matter
 - (d) None of these

Answers

- | | | | | | |
|---------|-----------------|---------|-----------------|---------|-----------------|
| 1. (d) | 2. (c) | 3. (b) | 4. (c) | 5. (a) | 6. (b) |
| 7. (c) | 8. (d) | 9. (b) | 10. (a) | 11. (c) | 12. (a) and (c) |
| 13. (a) | 14. (a) and (b) | | 15. (c) and (d) | | 16. (c) |
| 17. (b) | 18. (a) | 19. (c) | 20. (a) | | |



File Management

9.1 FILE AND ITS TYPES

File type represents the internal structure of the file. According to the structure, file is divided into certain types: text, source, executable and object file.

1. Text File: It is the sequence of characters which organised into lines. Text file is a regular file.
2. Source File: It contains sequence of subroutines, functions and procedures.
3. Executable File: It contains a series of code sections. This file is input to the loader and loader loads this file into memory and then executes.
4. Object File: Object file is input to the linker. An object file is a binary file that the compiler creates, which converts the source code to object code.

9.2 FILE ATTRIBUTES

One of the characteristics of a file is a set of file attributes that give the operating system more information about the file and how it is intended to be used. File attributes vary from system to system. File attributes are as follows:

- Name
- Identifier
- Type
- Location/Place
- Size
- Protection
- Date and Time

File name: It is in human readable. User can perform any operation on file using its name.

Identifier: When file is created by the user, the operating system assigns unique identification number to each file. OS uses this identifier for its operation.

Type: Information is required for system that support different types of files.

Location: Information is pointer to a device and actual location of files on the device. The information needed to locate the file on disk is kept in memory.

Size: Size is measured in bits or bytes. It gives idea about current size of the file.

Protection: This information is stored on the peer-proces table so that the operating system can allow or deny subsequent request. Attributes like password, creator and owner provide protection to a file.

Date and Time: This information is related to creation and modification of files, protection, security and monitoring purpose, this data is used by the operating system.

9.3 FILE OPERATIONS

File operations are simply those things that the user can perform on a file. For example, user can create a file, save a file, read a file, etc. There are many different types of file operation supported by the operating system. Operating system can provide the system call for performing various operations on the file. Six basic file operations are: create, read, write, delete, repositioning and delete.

File creation: Any time user can create a file. File is also created without data. The steps for creating a file. File is also create without data. The steps for creating a file:

- (a) File system must provide a space for the file
- (b) New file entry is made in the directory.

Write: To perform write operation on the file, file name and data are required. It updates a file by adding new data and changes some content of file.

Close: Process is not used a file after closing. Process cannot perform any operation on file. Operating system removes it entry from open file table.

Read: A process reads all or some portions of the data in file. Read system call is used for reading a file. For reading a file, name and position of the memory is specified.

Delete: Operating system removes the file from the file structure. For deleting a file, directory is searched with particular name and file is deleted. Deleting means making space free from memory and disk.

Repositioning: The directory is searched for the proper entry, and the current file position pointer is repositioned to a given value. Repositioning within a file need not involve any actual I/O. This file operation is also known as a file seeking.

Open: Before performing any operation on file, it is necessary to open file in the required mode.

Append: Append is related to write operation of a file. Append only add data to the end of file.

9.4 FILE ACCESS METHODS

Information and data are kept in files. Files are stored on the secondary storage device. When this information is to be used, it has to be accessed and brought into the main memory. Information in files could be accessed in many ways. It is usually dependent on an application. File organisation checks how efficiently the input-output storage

medium used. File access methods define the way processes read and write files. Different access methods are available.

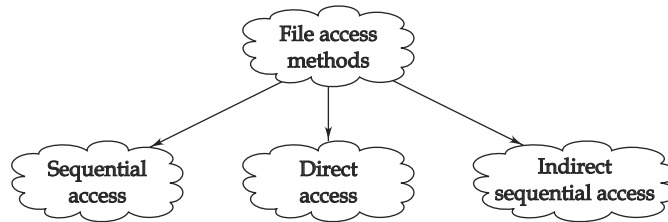


Fig. 9.1 File access methods

9.4.1 Sequential Access Method

Sequential access method is a simple method. The information in a file is accessed sequentially one record after another. In this method, a process could read all the records in a file in order, starting at the beginning. It cannot skip any records and cannot read out of order.

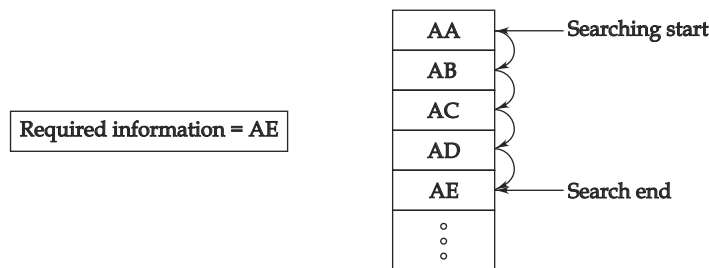


Fig. 9.2 Sequential file access

Batch application uses sequential files. A byte stream file uses sequential access method.

9.4.2 Direct Access Method

It is also called random access method. This access allows a user to position to read/write. Mark before reading or writing. Direct access file method provides accessing

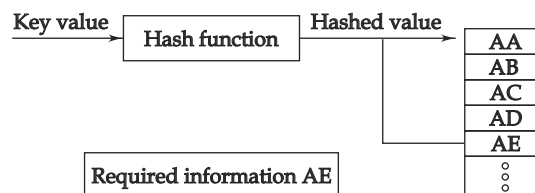


Fig. 9.3 Direct access method

the records directly. Directly access method is based on the hard disk that is a direct access device. It allows random access of any file block.

Each record has its own address on the file with the help of which it can be directly accessed for reading or writing. This feature is used by editors. At the time of file creation, access method is defined. According to defined access method, file is accessed. Sequential access of a direct access file is possible but direct access of sequential file is not possible.

9.4.3 Indexed Sequential Access

It is a modified method of direct access method. This method is a combination of direct and sequential access methods. In simple indexed sequential method, simple single level of indexing is used. Sequential file is used as index.

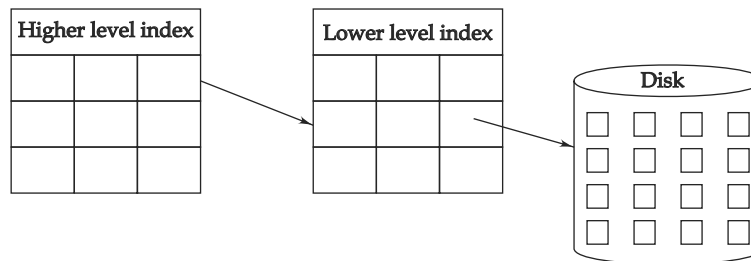


Fig. 9.4 Index sequential access

Indexed sequential files store records in the order that they are written to the disk. Records may be retrieved in sequential order or in random using an index to represent the record number in the file. File system maintains an overflow file. The overflow file is merged with the main file during a batch update. The multiple indexes for the same key field can be set up to increase efficiency lowest level of index file is treated as a sequential file and a higher index file is created for that file. Higher index file would contain pointers to lower index files.

9.5 FILE ALLOCATION METHODS

The direct access nature of disks allows flexibility in the implementation of files. In almost every case many files are stored on the same disk. The problem is to allocate space to these files so that disk space is utilized. There are three methods of file allocation:

- contiguous
- linked
- indexed

9.5.1 Contiguous Allocation

Contiguous allocation requires that each file occupies a set of contiguous blocks on the disk. Disk addresses define a linear ordering on the disk. With this ordering assume

that only one task is accessing the disk, accessing block bit after blocked $b + 1$ normally requires no head movement. When head movement is needed, the head only moves from one track to the next. Thus, the number of disk seeks required for accessing contiguously allocated file is minimal, contiguous allocation of a file is defined by the disk address and length of the first block. If the file is n blocks long and starts at location b , then it occupies blocks $b, b + 1, b + 2, \dots, b + n - 1$. The directory entry for each file indicates the address of starting block and the length of the area allocated for this file.

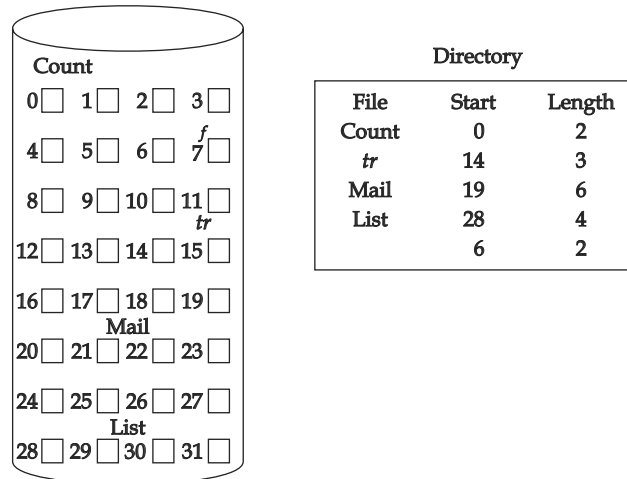


Fig. 9.5 Contiguous allocation of disk space

9.5.2 Linked Allocation

Each file is a linked list of disk blocks. The disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. For example, a file of five blocks might start at block 9 and continue at block 16, then block 1, then block 10 and finally block 25. Each block contains a pointer to the next

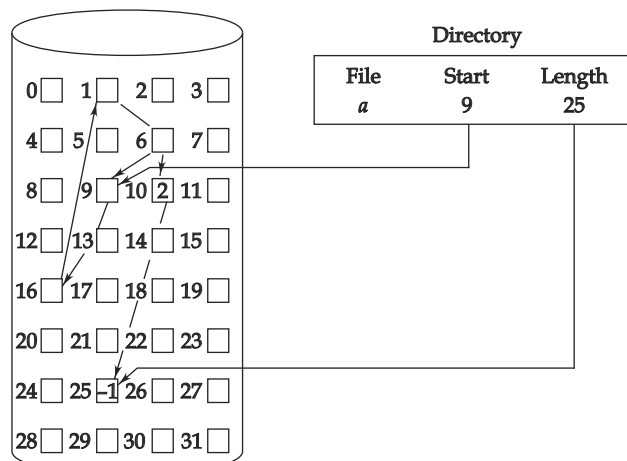


Fig. 9.6 Linked allocation of disk space

block. These pointers are not made available to the user. Linked allocation solves the problem of external fragmentation.

9.5.3 Indexed Allocation

Each file has its own index block, which is an array of disk-block addresses. The i th entry in the index block points to the i th block of the file. The directory contains the address of the index block. To find and read the i th block, we use the pointer in the i th index-block entry. When the file is created all pointers in the index block are set to null. When the i th block is first written, a block is obtained from the free space manager and its address is put in the i th index-block entry. Index allocation supports direct access, without suffering from external fragmentation.

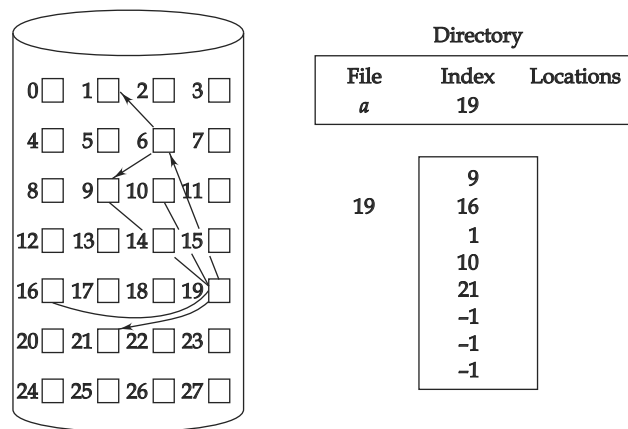


Fig. 9.7 Index allocation

9.6 FREE SPACE MANAGEMENT

- Space that is allocated to files must be managed and space which is not allocated to any file must be managed.
- To keep track of free disk space, the system maintains a free space list.
- File allocation and disk allocation both are required to manage free space properly.
- When a file is deleted, its disk space is added to the free-space list.
- Following are the techniques used for free space management:
 1. Bit vector
 2. Linked list
 3. Indexing or grouping
 4. Free block list or counting.

9.6.1 Bit Vector

- Each block on the disk is represented by a bit. It uses vector chain for blocks. Free block is represented by '0', and used block is represented by '1'.

- The memory required for a block bit map = (Disk size/8) × (block size of file system)
- Bit map requires extra space.

For example:

Block size = 2^{30} bytes

Disk size = 2^{12} bytes

$$\text{Block number } (n) = \frac{\text{Block size}}{\text{Disk size}} = \frac{2^{30}}{2^{12}} = 2^{18}$$

Advantages

- (a) Easy to find a free block.
- (b) It is as small as possible

Disadvantage

It may not be feasible to keep the bitmap in memory for large disks.

9.6.2 Link List

- In this method all free space disk blocks are linked, keeping a pointer to the first free block.
- Disk will become quite fragmented after some use. Many portions will be a single block long.

9.6.3 Grouping

- First free block contains the address of n free blocks. The first $n - 1$ of these blocks are actually free. The last block also contains the address of another n free block.
- When all the blocks in the group have been allocated then use the block that held the pointer.
- Because of grouping method, address of a large number of free blocks can be found quickly.

9.6.4 Counting

- It keeps the address of the first free block and the number n of free contiguous blocks that follows the first block. Each entry in the free space list then consists of a disk address and count.

9.7 FILE DIRECTORY

- User stores file in file system using directory. File system stores files of many users. Files are stored on the secondary storage device like hard disk, optical disk and memory disk. These device support random access method.
- Directory structure organizes and provides information about all files in the system. Every partition has a file system which consists of files and directory.

- Hard disks split into one or more partitions. It is known as mini disk or volumes.
- Each disk contains minimum one partition. Operating system supports 24 partitions. Partition can be larger than a disks, i.e., combining two hard disks. These partitions are called virtual disks. Partition maintains information about files in device directory.
- Files are stored in the directory. Directory is created by the user. User is free to create as many directories as he wishes and assign suitable names.
- In some operating systems, directory itself is considered a file. A set of logically associate file is called directory. Directory structure of a file system connects a directory to other directories in the system.
- Directory is a data structure that lists files and subdirectories in a collection. Directory structure contains a list entries one for each file.

Operation on directory

- Create a file – user creates new file and added to the directory.
- Delete a file – user deletes the file after completing the work on files.
- Rename a file – user changes the file name if the file content is changed.
- File searching – directory structure is searched for particular file entry.

9.8 DIRECTORY IMPLEMENTATION

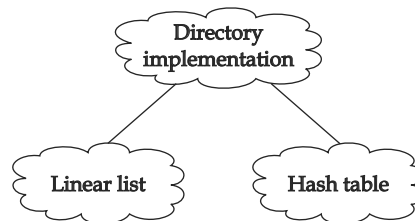


Fig. 9.8 Methods for directory implementation

Linear List

- Linear list is a simple method of directory implementation. It uses the file name with pointer to the data block. It is time consuming because of searching overheads.
- To create new file, file name is searched in the directory to avoid the file name duplication. Then it adds new entry at the end of the directory. To delete a file, again file name is searched in the directory. Space is released after deleting a file. Directory entry is also removed. Unused space is marked as free entry. Last entry of the directory is copied into the free space list.

Hash Table

Hash table is one more data structure used for directory implementation. Hash table takes a value computed from the file name and returns a pointer to the file name in the linear list.

- Hash table reduces the searching time.
- Hash table uses fixed size blocks.

9.9 FILE SHARING

- Multiprogramming and multitasking operating system supports multiple users. Users perform various operations on the files. Files are shared between a number of users. File protection, naming and sharing is issued for file sharing.
- Directory structure may allow files to be shared by users. Sharing must be done through a protection scheme.
- In a single-user system, the operating system maintains attributes for all files and directory. According to the attributes, the operating system allows the user to operate files for users. But on multiuser system, more attributes are needed. An attribute like owner, group and other users are replied. These are the access permissions for file and directory.
- Access permissions are of three types: Read, write and execute.
 1. A 'r' indicates read permission for a file or directory.
 2. A 'w' indicates write permission.
 3. A 'x' indicates execute permission if it is a file and search permission if it is a directory.
 4. Owner is the file owner who created the file and has all rights (read, write and execute).
 5. The group of other users have some limited access permission.

9.10 IMPLEMENTATION OF SHARED FILES AND DIRECTORY

Shared files and directory can be implemented in either of two ways.

- Using link
- Duplicating Information

Using Link

- Link is used for creating shared files and directory. A link is a pointer to another file or subdirectory. If reference to a file is made, the directory is first searched. A link is a logical relationship between a node and file that relates the name of a file to its physical location.
- UNIX operating system provides two types of links: hard link and symbolic link.
- A hard links contains multiple directory entries that both refer to the same file. Hard links are only used with ordinary files in the same file system.
- A symbolic link is also called soft link. It contains a special file which maintain the information about the file, where to find the linked file symbolic links may be used to link directories and files in the other file system.

Duplicating Information

- Duplicate all information about shared files in both sharing directories. Original file will remain as it is without modifying any content by the shared user.
- Consistency is not supported by copied file and shared file. Shared files may have more than one absolute path.

Summary

- A file is an abstract data type. It is a sequence of logical records. A logical record may be a byte, a line, or a more complex data item.
- In a file system each device keeps a device directory that lists the location of the files on the device.
- The file system reside permanently on the secondary storage which has the main requirement that it must be able to hold a large amount of data.
- The various files can be allocated space on the disk in three ways; through contiguous, linked or indexed allocation.
- Free space allocation methods also influence the efficiency of use of disk space, the performance of the file system; and the reliability of secondary storage.
- The directory management routines must consider efficiency, performance and reliability.
- A hash table is the most frequently used method; it is fast and efficient.
- Files are the main information, therefore they need to be protected. For each type of access such as read, write, execute, append, delete, list and so on, access to files can be controlled individually.
- Files can be made protected by using pass words, by access lists, or by other technique.

Multiple-Choice Questions

1. Management of metadata information is done by
 - (a) file-organisation module
 - (b) logical file system
 - (c) basic file system
 - (d) application programs
2. A file control block contains the information about
 - (a) file ownership
 - (b) file permissions
 - (c) location of file contents
 - (d) All of the above
3. Which table contains the information about each mounted volume?
 - (a) Mount table
 - (b) System-wide open-file table
 - (c) Per-process open-file table
 - (d) All of the above
4. To create a new file application program calls
 - (a) basic file system
 - (b) logical file system
 - (c) file-organisation module
 - (d) none of the mentioned

-
5. When a process closes the file
 - (a) per-process table entry is removed
 - (b) system wide entry's open count is decremented
 - (c) both (a) and (b)
 - (d) None of the above
 6. What is raw disk?
 - (a) Disk without file system
 - (b) Empty disk
 - (c) Disk lacking logical file system
 - (d) Disk having file system
 7. The data structure used for file directory is called
 - (a) mount table
 - (b) hash table
 - (c) file table
 - (d) process table
 8. In which type of allocation method each file occupy a set of contiguous block on the disk?
 - (a) Contiguous allocation
 - (b) Dynamic-storage allocation
 - (c) Linked allocation
 - (d) Indexed allocation
 9. If the block of free space list is free then bit will
 - (a) 1
 - (b) 0
 - (c) both 0 and a can be possible
 - (d) None of the above
 10. Which protocol establishes the initial logical connection between a server and a client?
 - (a) transmission control protocol
 - (b) user datagram protocol
 - (c) mount protocol
 - (d) datagram congestion control protocol
 11. The NFS servers
 - (a) are stateless
 - (b) save the current state of the request
 - (c) maybe stateless
 - (d) None of these
 12. Every NFS request has a _____ allowing the server to determine if a request is duplicated or if any are missing.
 - (a) name
 - (b) transaction
 - (c) sequence number
 - (d) All of these
 13. A sever crash and recovery will _____ to a client.
 - (a) be visible
 - (b) affect
 - (c) be invisible
 - (d) harm
 14. _____ tend to represent a major bottleneck in system performance.
 - (a) CPUs
 - (b) Disks
 - (c) Programs
 - (d) I/O
 15. In UNIX, even an 'empty' disk has a percentage of its space lost to
 - (a) programs
 - (b) inodes
 - (c) virtual memory
 - (d) stacks

-
16. By preallocating the nodes and spreading them across the volume, we _____ the system performance.
(a) improve (b) decrease
(c) maintain (d) do not affect
17. To organise file systems on disk
(a) they are split into one or more partitions
(b) information about files is added to each partition
(c) they are made on different storage spaces
(d) All of these
18. The directory can be viewed as a _____, that translates file names into their directory entries.
(a) symbol table (b) partition
(c) swap space (d) cache
19. To control access the three bits used in UNIX are represented by
(a) *r* (b) *w*
(c) *ex* (d) *x*
20. If each access to a file is controlled by a password, then the disadvantage is that
(a) user will need to remember a lot of passwords
(b) it is not reliable
(c) it is not efficient
(d) All of these

Answers

- | | | | | | |
|---------|-------------------|---------|---------|-----------------|---------|
| 1. (b) | 2. (d) | 3. (d) | 4. (b) | 5. (c) | 6. (a) |
| 7. (b) | 8. (a) | 9. (a) | 10. (c) | 11. (a) | 12. (c) |
| 13. (c) | 14. (b) | 15. (b) | 16. (a) | 17. (a) and (b) | |
| 18. (a) | 19. (a), (b), (d) | 20. (a) | | | |

File system, logically can be viewed as consisting of three parts

1. User and programmer interface to the file system.
2. Internal data structures and algorithms used by OS to implement this interface.
3. The secondary and tertiary storage structures (lowest level)

10.1 MASS STORAGE STRUCTURE

Overview of physical structure of secondary and tertiary storage devices.

(i) Magnetic Disks

Provide the bulk of secondary storage for modern computer systems. Conceptually, disks are simple.

Structure: Each disk platter has a flat circular shape, like a CD. Diameter range from 1.8 to 5.25 inches.

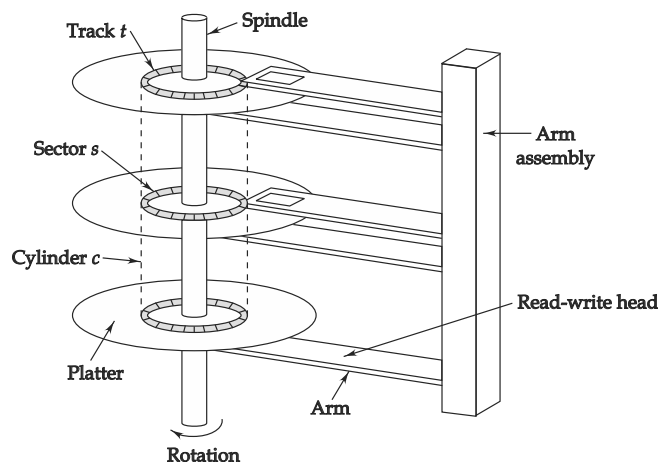


Fig. 10.1 Moving-head disk mechanism

Structure of disk

1. Made up of one or more platters.
2. Each platter is a metal disk (material is magnetic).

3. Each platter has a number of concentric tracks.
 4. Each track is a circle around the platter and can be used to record data.
 5. Track is divided into sectors, each of which is a disk block there are about 50 kB on a track.
 6. Tracks that are at the same position on each platter form a group called cylinder.
 7. Each disk block has an address that tells where it is on the disk.
- The address has three parts: cylinder, track and sector.

Disk drive consists of a motor and all disks together at a very high speed along cylinder.

10.2 DISK STORAGE

Information is stored on the surface of a disk along concentric set of rings called tracks. These tracks are divided into sections called *sectors*.

Let S bytes be stored per sector; there are P sectors per track, t tracks per surface and m surfaces.

$$\text{Capacity} = m \times t \times P \times s \text{ bytes}$$

if d is diameter,

$$\text{Density} = \frac{P \times s}{\pi \times d} \text{ bytes/inch}$$

Disk Bandwidth: Total number bytes transferred and, divided by the total time between the first request of services and the completion of the last transfer.

The address of disk has three parts:

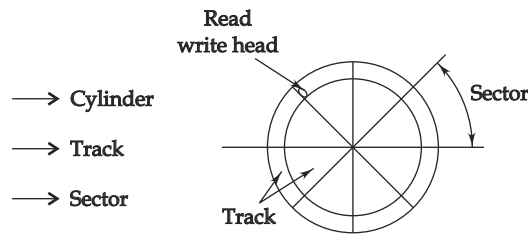


Fig. 10.2 One-surface of disk

10.3 DISK PERFORMANCE PARAMETERS

The actual details of disk input-output operation depend on the *computer system*, the *operating system* and *nature of disk controller* and *device driver*.

When disk drive is operating, the disk rotates at a constant speed.

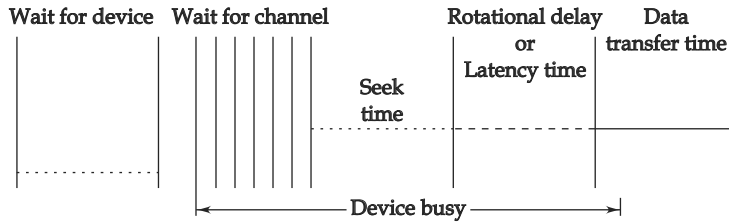


Fig. 10.3 Timing diagram of a disk input/output transfer

Seek time: The time for the disk arm to move the heads to the cylinder containing the desired sector.

Rotational latency: It is the additional time for the disk to rotate the desired sector to the disk head.

Bandwidth: The disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.

Access time: The sum of seek time and latency time

$$\text{Data transfer time (T) to (or) from the disk} = \frac{b}{rN}$$

b = number of bytes to be transferred

N = number of bytes on a track

r = rotation speed in revolution per second.

$$\text{Total average access time (T}_a\text{)} = T_s + \frac{1}{2r} + \frac{6}{rN}$$

T_a = total average access time

T_s = estimated seek time

Estimated seek time or average seek time is defined as

$$T_s = m \times n + s$$

n = number of tracks traversed

m = a constant that depends on the disk drive

s = start uptime.

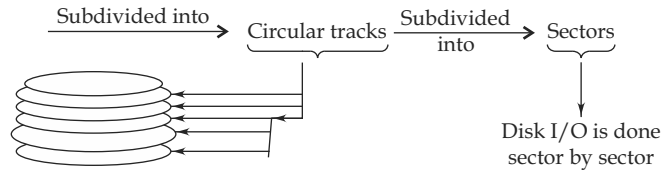
Since requests from different processes may be scattered all over the disk, awful lot of jumping around on disk results in *rotational latency*.

To avoid this, disk scheduling algorithm are division.

Throughput: It is the number of processes that complete its execution per unit time.

Responsible time: Amount of time a process takes to get the first response or the request submitted.

How the algo works: A hard disk is a collection of plates called platters.

**Fig. 10.4** Hard disk

There is a head connected to an arm for each surface, which handles all I/O operations, for each I/O request head is selected first and it is then moved over the destination track. The disk is then rotated to position the desired sector under the head and finally the read, write operation is performed.

10.4 NEED OF DISK SCHEDULING

1. File system must be accessed in an efficient manner, especially with hard drives that are the slowest part of computer.
2. As computer deals with multiple processes over a period of time, a list of requests to access the disk builds up, for efficiency purpose all requests are combined together.
3. The technique that OS uses to determine which requests to satisfy first is called disk scheduling.
4. Disk scheduling specifies that at which time which process will be executed by CPU, so that scheduling means to execute all the processes those are given to CPU at a time.

10.5 DISK SCHEDULING ALGORITHMS

Two main objectives of DS algorithm:

1. Minimize throughput
2. Minimize response time

Following are the disk scheduling algorithms:

1. FCFS
2. SSTF (shortest seek time first)
3. SCAN
4. LOOK
5. SCAN
6. LOOK

10.5.1 FCFS (First Come First Serve)

This algorithm performs operations in the order requested. It has poor performance and generally does not provide the fastest service. It does lead to starvation. Larger head movement is the main drawback of this algorithm resulting in poor performance.

Disk Queue: 30, 85, 130, + 5, 175

Consider total number of cylinders = 200 (0 - 199)
head = 100

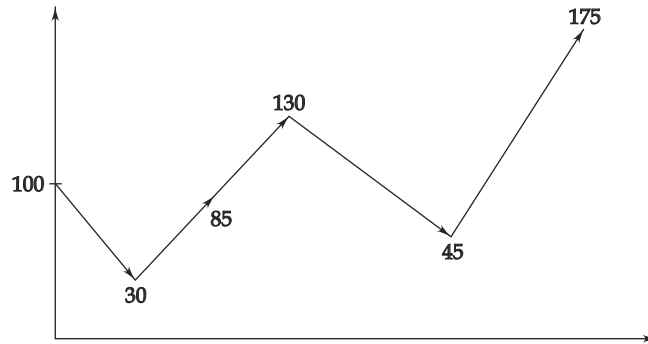


Fig. 10.5 FCFS disk scheduling

total seek time is estimated by head movement $|100 - 30| + |30 - 85| + |85 - 130| + |130 - 45| + |45 - 175|$
 $70 + 55 + 45 + 85 + 130 = 385$

10.5.2 SSTF (Shortest Seek Time First)

As the name suggests, it works for the shorter seek time among the disk queue. It compares the seek time of request and chooses which is closer in the work queue.

It is independent of the direction considering the above example:

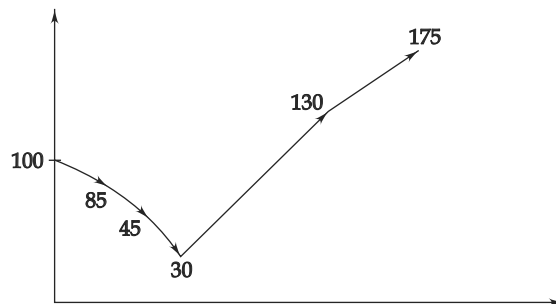


Fig. 10.6 SSTF disk scheduling

Total seek time:

$$\begin{aligned}
 &= |100 - 85| + |85 - 45| + |45 - 30| + |30 - 130| + |130 - 175| \\
 &= 15 + 40 + 15 + 100 + 45 \\
 &= 215
 \end{aligned}$$

10.5.3 Scan

It is also called *elevator algorithm* as it works on the basic principle of elevator.

It goes from the outside to the inside servicing requests and then back from the outside to the inside servicing requests.

This means, the disk arm behaves like elevator, first servicing all the requests going up (or down) and then reversing to service requests the other way.

Advantage

Reduces *variance* compared to SSTF.

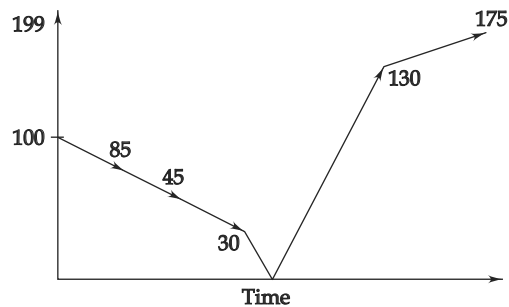


Fig. 10.7 Scan scheduling

Total seek time:

$$\begin{aligned}
 &= |10 - 85| + |85 - 45| + |45 - 30| + |30 - 0| + |0 - 130| + |130 - 175| \\
 &= 5 + 40 + 15 + 30 + 130 + 45 \\
 &= 75
 \end{aligned}$$

10.5.4 Look

It is similar to scan but stops moving inwards (or outwards) no more requests in that direction exist.

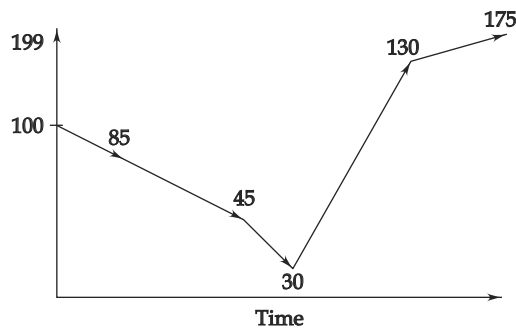


Fig. 10.8 SSTF scheduling

Total seek time

$$\begin{aligned}
 &= |100 - 85| + |85 - 45| + |45 - 30| + |30 - 130| + |130 - 175| \\
 &= 215
 \end{aligned}$$

10.5.5 C-Scan: (Circular Scan)

In this algorithm, disk head moves inwards servicing requests until it reaches the innermost cylinder, then jumps to the outside cylinder of the disk without servicing any requests.

This means, like in the above, e.g., after processing disk queue 30, there are no requests in that direction. Then it goes to outermost cylinder (199) in this case from innermost cylinder (0), without processing any request in that direction. After reaching the outermost cylinder it continues the algorithm again.

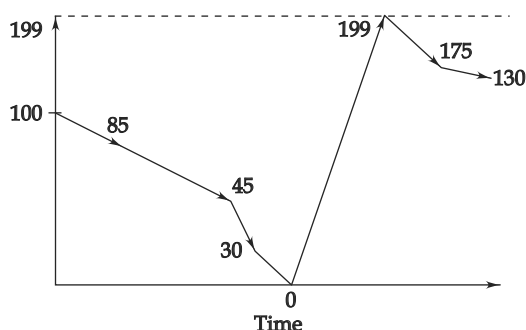


Fig. 10.9 C-Scan scheduling

Total seek time:

$$\begin{aligned}
 &|100 - 85| + |85 - 45| + |45 - 30| + |30 - 0| + |0 - 199| + |199 - 175| + |175 - 130| \\
 &= 368
 \end{aligned}$$

10.5.6 C-Look

Disk head moves towards servicing requests until there are no more requests in that direction. and then it jumps to the *outermost request*.

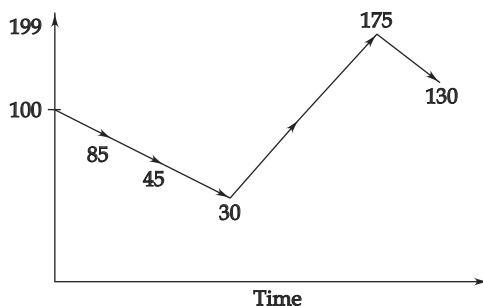


Fig. 10.10 C-Look scheduling

$$\text{Total seek time} = |100 - 85| + |85 - 45| + |45 - 30| + |30 - 175| + |175 - 130| = 360$$

Comparison of algorithms

	Algorithm description	
FCFS	Performs operations in the order of the request arrived	• Poor performance • Does not lead to starvation
SSTF	It looks for the shortest seek time among the disk queue. It compares the seek time of request and chooses which is closer in the work queue. Independent of direction.	• Leads to starvation • Too much switching slows down performance • Better than FCFS
Scan	Disk arm behaves like an elevator, first servicing all the requests going up (or down) and then reverse to service requests in other way.	• Better for systems that place a heavy load on disk
Look	It is similar to SCAN but stops moving inwards or outwards. When no more requests in that direction exist.	• Has low total seek time, thus improving performance.
Scan	Moves inwards servicing requests until it reaches the innermost cylinder then jumps to the outside cylinder of the disk without service any requests.	• Works for system with heavy loads
C Look	Moves towards servicing requests until there are no more requests in that direction and then it jumps to outermost request.	

10.6 RAID (REDUNDANT ARRAY OF INDEPENDENT DISKS)

RAID is a technology for managing how data is stored on multiple physical hard disks. RAID combines physical hard disks into a single virtual disk that is presented to OS as a single hard drive.

Basic functions

RAID array is a stand-alone unit with its own controller, cache and disks. It is attached to host via one or more standard ATA, SCSI or FC controllers. This system is used that have RAID software layers because of its simplicity and flexibility.

The three key concepts of RAID are:

1. **Mirroring:** This is copying of data to more than one disk. Redundancy is achieved by maintaining two copies of data on different disks. So that if one disk fails, the system can continue to operate by using the unaffected disk.
2. **Striping:** Splitting data across more than one disk. Disk is broken down into blocks of equal sized pieces called stripes. Each stripe created is written to a different disk across the physical disks in disk array, one row at a time. Each row is called a striped set.

The data in this striped set is written/read independently so it increases the performance.

3. **Parity/error correction:** Here redundancy data is created and stored to allow problems to be detected. The redundancy data is calculated from the sets of actual data values by performing XOR operations on binary numbers.

RAID Implementation

The distribution of data across multiple drives may be managed either by dedicated software or hardware or hybrid.

The implementations are given below.

1. Hardware RAID

- It requires a minimum special purpose RAID controller, e.g., may be PCI expansion card or capability built into motherboard.
- The drives may be ATA/IDE, SATA or combination.
- This type of implementation increases performance so no overhead to local CPU complex.

2. Software RAID

- This type of implementation is provided by most OS.
- System can have disks directly attached to buses.
- Software layer sits above the disk device drives and provides an abstraction layer between logical and physical drives.
- It requires very small amount of processing time.

3. Hybrid RAID are very popular with introduction of inexpensive RAID controllers, implemented using a standard disk controller and BIOS extensions to provide RAID functionality.

RAID Levels RAID can be classified into different levels based on its operation and level of redundancy provided. There is no “one size fits all” solutions as far as levels are concerned.

(1) RAID0/No RAID/Striping

In RAID0 system, data is split up in blocks/chunks that get written across all the drives in array. By using multiple disk (at least 2 at a time) RAID 0 offers better I/O performance as shown in Fig. 10.11.

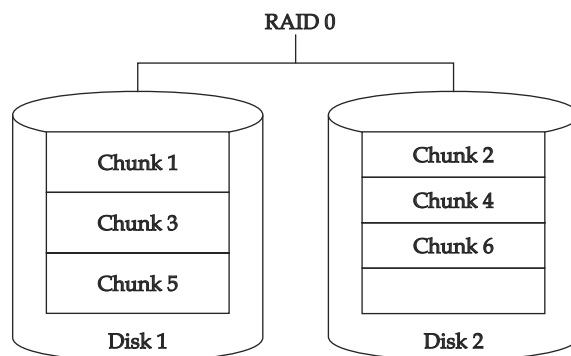


Fig. 10.11 RAID0

Advantages

1. It offers great performance both in read and write operations.
2. It is easy to implement.

Disadvantages

1. It does not provide any kind of redundancy.
2. It is not fault tolerant, i.e., if one disk fails all data in RAID0 array are lost.
3. It is not meant for critical systems.

(2) RAID 1 (Mirroring):

It is also known as mirroring. In this, data are stored twice by writing them to both the data disk and a mirror disk. If a disk fails, the controller either uses the data drive or mirror for data recovery and continues operation.

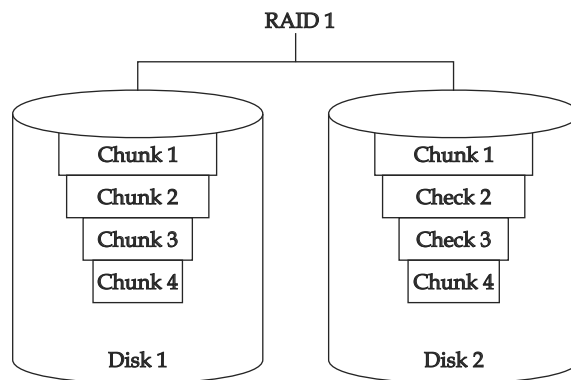


Fig. 10.12 RAID1

Advantages

1. RAID 1 offers an excellent read and write speed that is compatible to a single disk.
2. Maximum number of drives in RAID 1 are 32 starting from 2.
3. It can be used in fault tolerance situations.

Disadvantages

1. Hardware controller is required.
2. Effective storage capacity becomes half of total disk capacity.

(3) RAID 2 (Memory Style error correcting codes)

It is also known as memory style error correcting codes. Memory systems have long detected certain errors by using parity bits. Each byte in a memory system may have a parity bit associated with it that records whether number of bits are even (Parity 0) or odd (Parity 1).

Advantages

1. All single bit errors are detected by memory systems.

Disadvantages

If one of bits in bits in byte is damaged then parity of system changes.

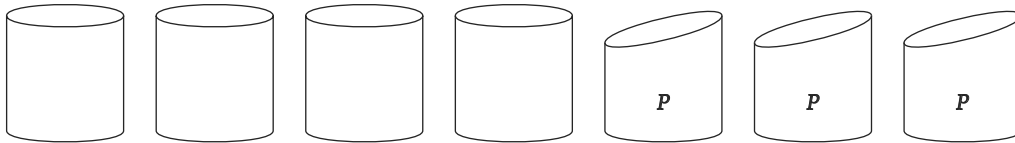


Fig. 10.13 RAID 2 Memory style error correcting codes

(4) RAID 3 (bit interleaved parity organization)

Data blocks are subdivided and written in parallel on two or more devices. An additional drive stores parity information.

Advantages

1. Raid 3 provides high throughput for large data transfer.
2. Disk failures do not significantly slow down throughput.

Disadvantages

1. This technology is fairly complex and too resource intensive to be done in software.
2. Performance is slower for random, small I/O operations.

RAID 5

RAID 5 is the most common secure RAID level. It is similar to RAID3 except that data are transferred to disks by independent read and write operations and not in parallel. Minimum disk required is 3 and maximum it goes to 32. It is the most cost-effective solution for both performance and redundancy.

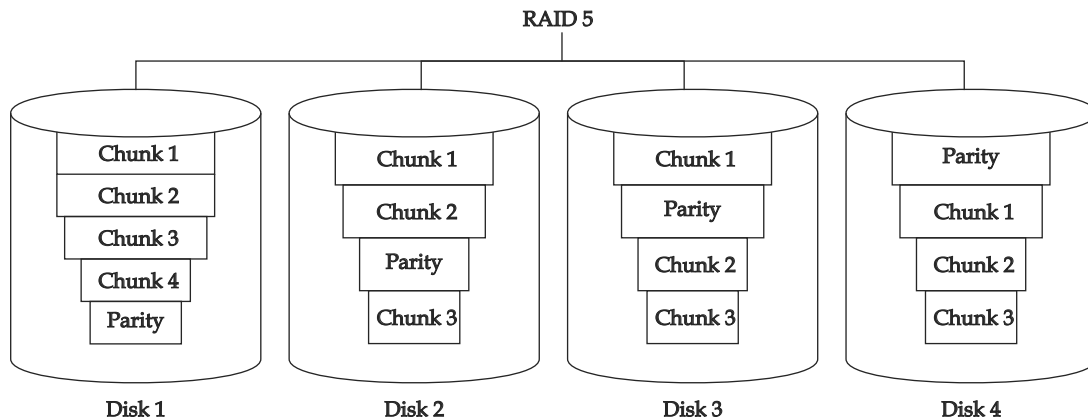


Fig. 10.14 RAID 5 (Parity across disk)

Advantage

Read data transactions are very fast.

Disadvantages

1. More complex
2. Maximize throughput.

RAID 10 (Mix of RAID 0 and RAID 1)

RAID 10 combines the benefits and limitations of RAID 0 and RAID 1 in a single system. It provides security by mirroring all data on a secondary disk while using stripping across each set of disks to speed up data transfer.

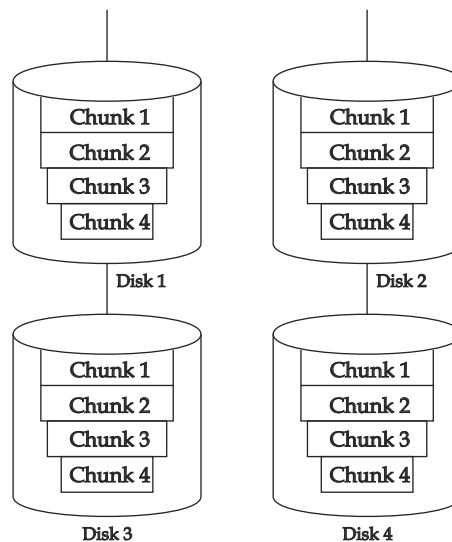


Fig. 10.15 RAID 10

Numerical Examples

10.1 Suppose that the disk drive has 5000 cylinders numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending request in FIFO order is:

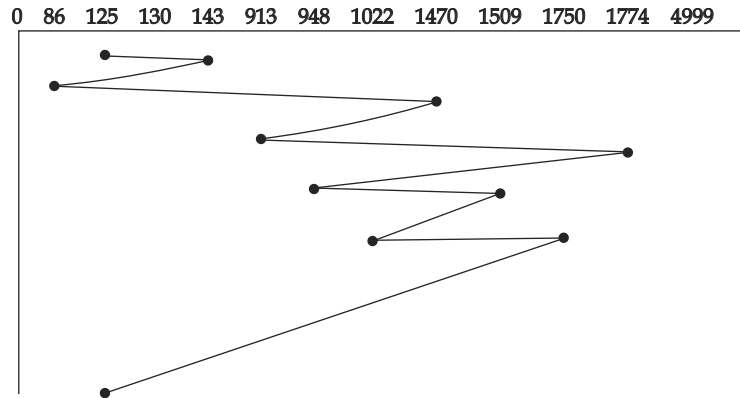
86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130

Starting from the current head position, what is the total distance (in cylinders) that the disk arms moves to satisfy the pending request for each of the following disk scheduling algorithms?

- | | |
|------------|----------|
| (a) FCFS | (b) SSTF |
| (c) SCAN | (d) LOOK |
| (e) C-Scan | |

Sol.

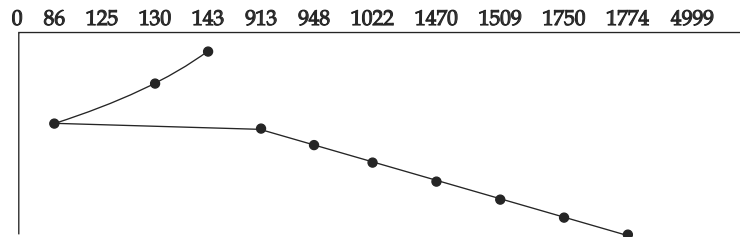
(a) FCFS



The *FCFS schedule* is 143, 86, 1470, 913, 1774, 948, 1509, 1022 1750, 130.

Total seek distance is (time) = $|143 - 86| + |86 - 1470| + |1470 - 913| + |1750 - 130| = 7081$

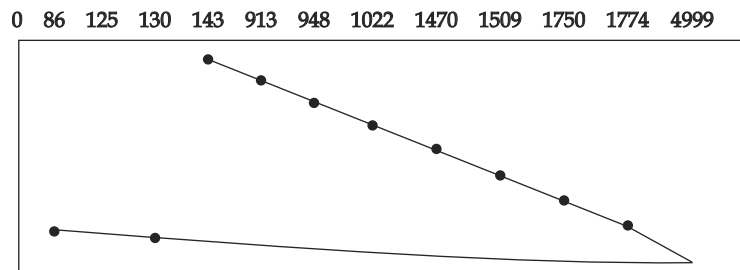
(b) SSTF



143, 130, 86, 913, 948, 1022, 1470, 1509, 1750, 1774

Total seek distance is = $|143 - 130| + |130 - 86| + |86 - 913| + |913 - 948| + |948 - 1022| + |1022 - 1470| + |1470 - 1509| + |1509 - 1750| + |1750 - 1774| = 1745$ Ans.

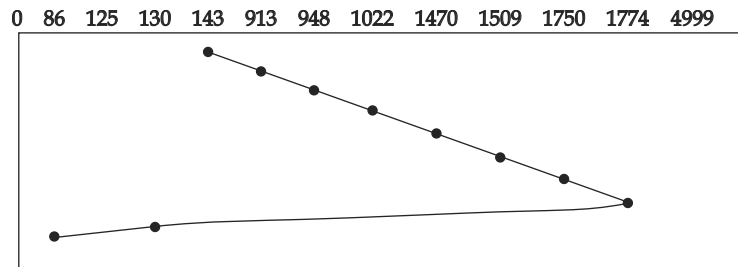
(c) Scan



Scan schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 130, 86

Total seek distance is = $|143 - 913| + |913 - 948| + |948 - 1022| + |1022 - 1470| + |1470 - 1509| + |1509 - 1750| + |1750 - 1774| + |1774 - 4999| + |4999 - 130| + |130 - 86| = 9769$

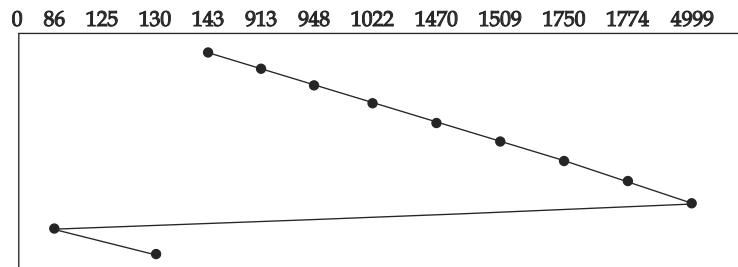
(d) Look



Look schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 130, 86

Total seek distance is = $|143 - 913| + |913 - 948| + |948 - 1022| + |1022 - 1470| + |1470 - 1509| + |1509 - 1750| + |1750 - 1774| + |1774 - 130| + |130 - 86| = 3319$

(e) C-Scan schedule



C-scan schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 86, 130

Total seek time is = $|143 - 913| + |913 - 948| + |948 - 1022| + |1022 - 1470| + |1470 - 1509| + |1509 - 1750| + |1750 - 1774| + |1774 - 4999| + |4999 - 86| + |86 - 130| = 9985$

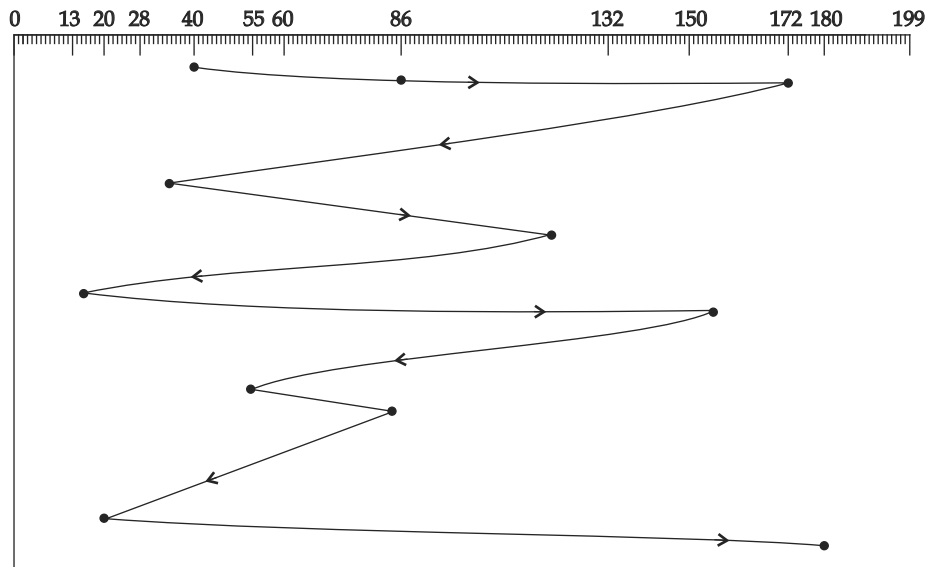
10.2 Suppose that the disk drive has 200 cylinders numbered from 0 to 199. The queue which is maintained is 86, 172, 28, 132, 13, 150, 55, 60, 20, 180 in order. Disk arm is initially at cylinder 40. Starting from the current head position what is the total distance (in cylinders) that disk arms moves to satisfy all the pending requests for

- | | | | | |
|-------------|---|------------|---|------------|
| (i) FCFS | ← | scheduling | ← | algorithms |
| (ii) SSTF | ← | scheduling | ← | algorithms |
| (iii) SCAN | ← | scheduling | ← | algorithms |
| (iv) C-SCAN | ← | scheduling | ← | algorithms |

- (v) Look ← scheduling ← algorithms
 (vi) C-Look ← scheduling ← algorithms

Sol.

(1) FCFS scheduling algorithm

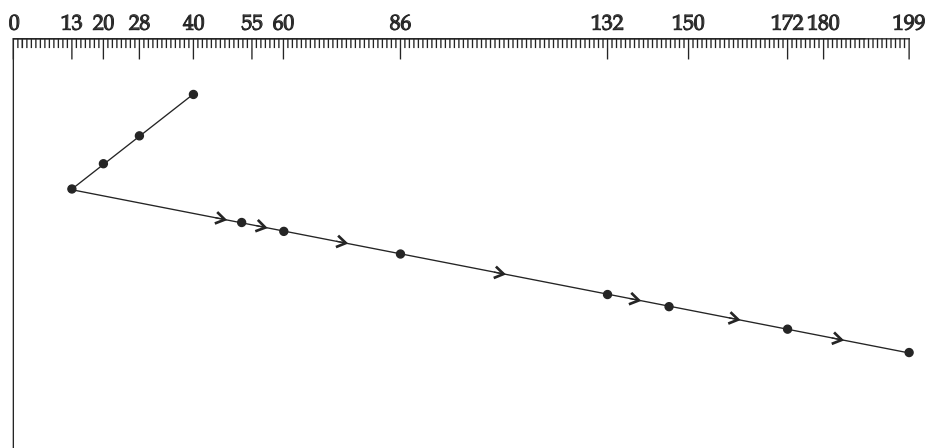


The FCFS sequence is

40 86 172 28 132 13 150 55 60 20 180

The seek distance is = $|40 - 86| + |86 - 172| + |172 - 28| + |28 - 132|$
 $+ |132 - 13| + |13 - 150| + |150 - 55| + |55 - 60|$
 $+ |60 - 20| + |20 - 180| = 936$

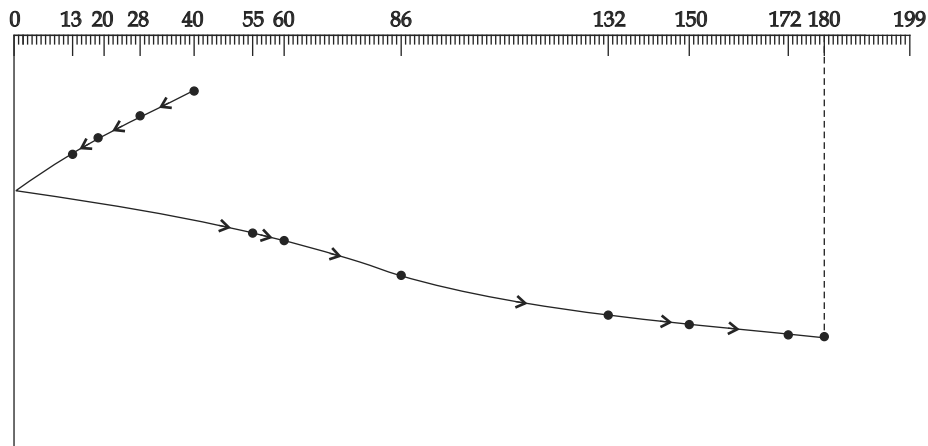
(2) SSTF scheduling algorithm



The seek distance is = $|40 - 28| + |28 - 20| + |20 - 13| + |13 - 55|$
 $+ |55 - 60| + |60 - 86| + |86 - 132| + |132 - 150|$
 $+ |150 - 172| + |172 - 180| = 194$ **Ans.**

(3) SCAN scheduling algorithm

Assumption: disk arm is moving towards 0.



SCAN sequence is

40 28 20 13 0 55 60 86 132 150 172 180

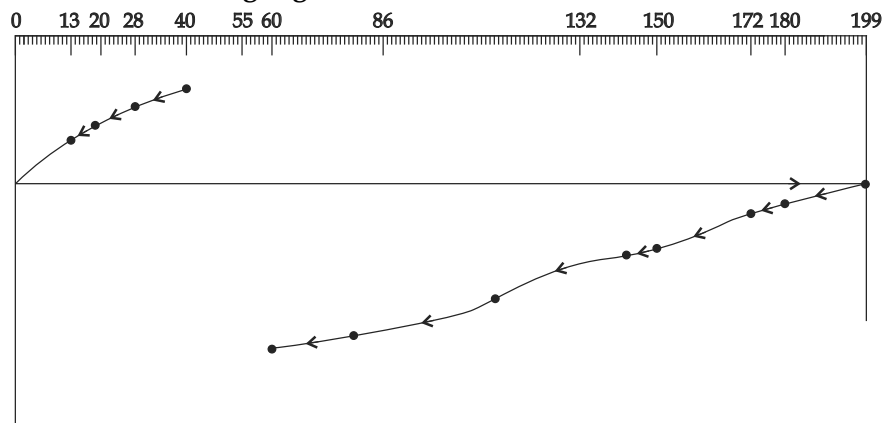
The seek distance is

$|40 - 28| + |28 - 20| + |20 - 13| + |13 - 0| + |0 - 55| + |55 - 60|$
 $+ |60 - 86| + |86 - 132| + |132 - 150| + |150 - 172| + |172 - 180|$

$\Rightarrow 12 + 8 + 7 + 13 + 55 + 5 + 26 + 46 + 18 + 22 + 8$

$\Rightarrow 220$

(4) C-Scan scheduling algorithm



The C-Scan sequence is: (assuming disk arm is moving towards 0)

40 28 20 13 0 199 180 172 150 132 86 60 55

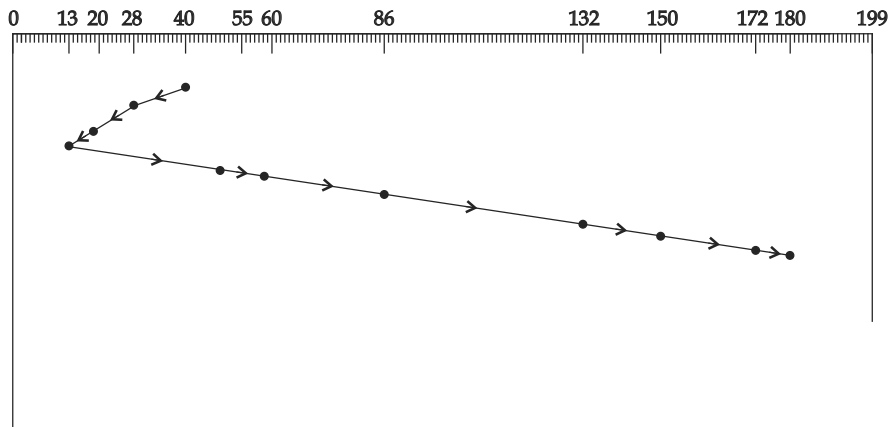
The seek distance is:

$$|40 - 28| + |28 - 20| + |20 - 13| + |13 - 0| + |0 - 199| + |199 - 180| \\ + |180 - 172| + |172 - 150| + |150 - 132| + |132 - 86| + |86 - 60| \\ + |60 - 55|$$

$$\Rightarrow 12 + 8 + 7 + 13 + 199 + 19 + 8 + 22 + 18 + 46 + 26 + 5$$

$$\Rightarrow 383$$

(5) Look scheduling algorithm



The look sequence is

40 28 13 55 60 80 132 150 172 180

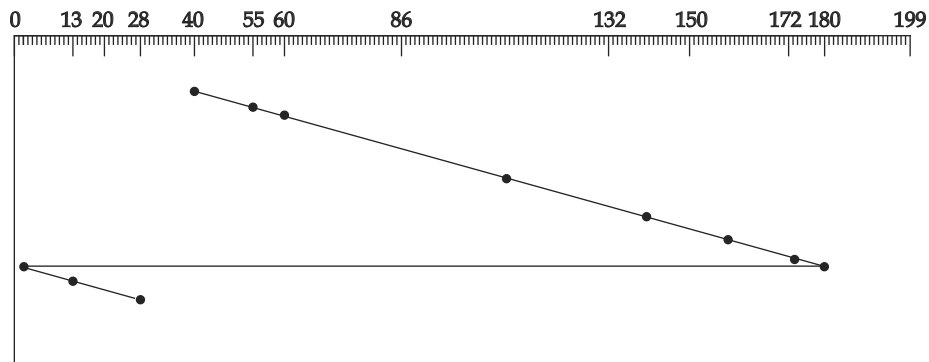
The seek distance is:

$$|40 - 28| + |28 - 20| + |20 - 13| + |13 - 35| + |55 - 60| + |60 - 86| \\ + |86 - 132| + |132 - 150| + |150 - 172| + |172 - 180|$$

$$\Rightarrow 12 + 8 + 7 + 42 + 57 + 26 + 42 + 18 + 22 + 8$$

$$\Rightarrow 190$$

(6) C-Look



40, 55, 60, 86, 132, 150, 172, 180, 0, 13, 20.

$$\begin{aligned} \text{Total head movements} &= |40 - 55| + |55 - 60| + |60 - 86| + |86 - 132| + \\ &|150 - 172| + |172 - 180| + |180 - 0| + |0 - 13| + |13 - 20| \\ &= 322 \text{ head movements} \end{aligned}$$

10.3 Explain the SSTF and SCAN disk scheduling policies. Obtain the total number of head movements needed to satisfy the following sequence of track requests for each of the two policies:

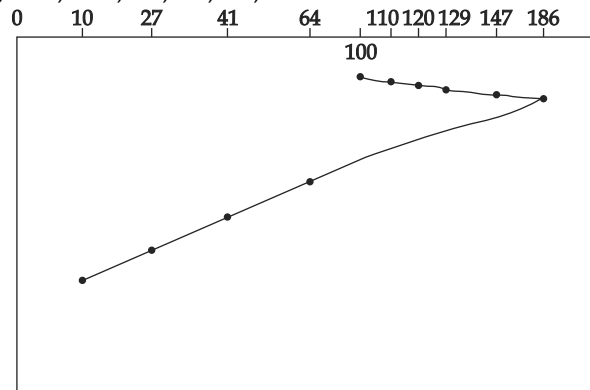
27, 129, 110, 186, 147, 41, 10, 64, 120

Assume that the disk head is initially positioned over track 100 and is moving in the direction of decreasing track number:

Sol. SSTF

The SSTF schedule is

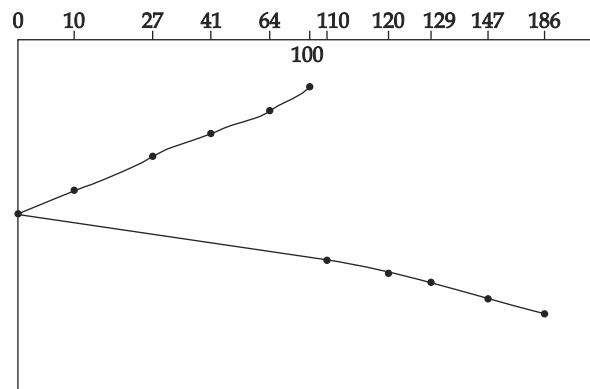
110, 120, 129, 147, 186, 64, 41, 27, 10



$$\begin{aligned} \text{Total seek distance is} &= |100 - 110| + |110 - 120| + |120 - 129| + |129 - 147| \\ &+ |147 - 186| + |186 - 64| + |64 - 41| + |41 - 27| + |27 - 10| \\ &= 262 \text{ Ans.} \end{aligned}$$

Scan: The SCAN schedule is

64 41 27 10 0 110 120 129 147



Total seek distance is $\rightarrow$

$$|100 - 64| + |64 - 41| + |41 - 27| + |27 - 10| + |10 - 0| + |0 - 110| + |110 - 120| + |120 - 129| + |129 - 147| + |147 - 186| = 386$$

10.4 On a disk with 1000 cylinders, numbers 0 to 999, compute the number of tracks the disk arm must move to satisfy all the requests in the disk queue. Assume the last request received was at track 345 and the head is moving towards track 0. The queue in FIFO order contains requests for the following tracks:

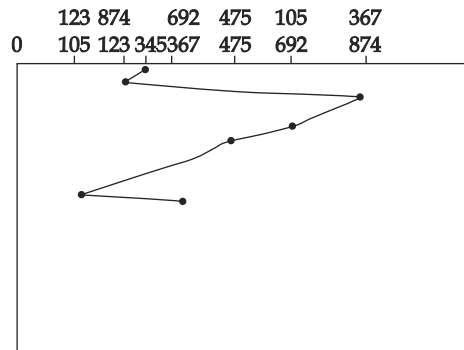
123, 874, 692, 475, 105, 367

Perform the computation for the following scheduling algorithm

(1) FIFO (2) SSTF (3) SCAN (4) LOOK (5) C-SCAN (6) C-LOOK

Sol.

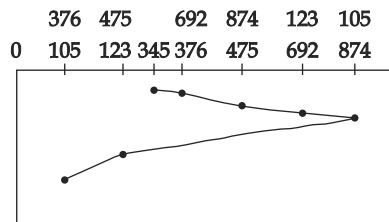
(i) FIFO $\rightarrow$ Schedule is



Total seek distance is

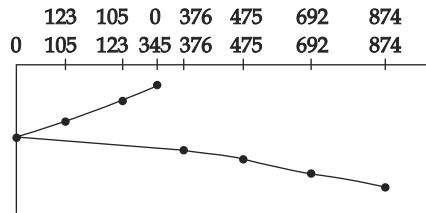
$$|123 - 345| + |874 - 123| + |692 - 874| + |475 - 692| + |105 - 457| + |367 - 105| = 2013$$

(ii) SSTF $\rightarrow$ Schedule is

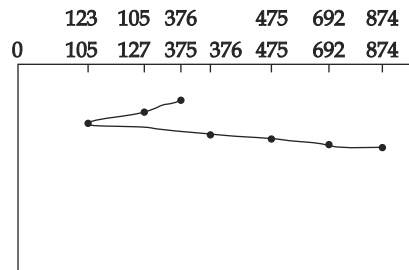


Total seek distance is

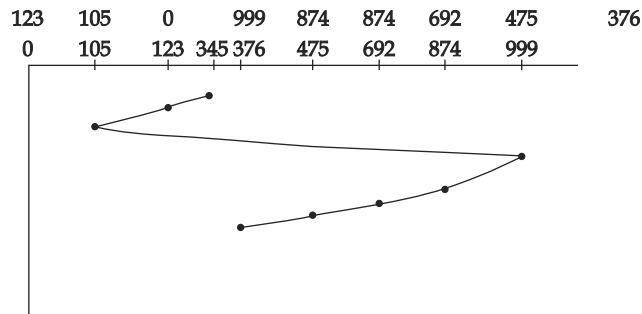
$$= |345 - 376| + |376 - 475| + |475 - 692| + |692 - 874| + |874 - 123| + |123 - 105| = 1298$$

(iii) SCAN → Schedule is

$$\text{Total seek distance} = |345 - 123| + |123 - 105| + |105 - 0| + |0 - 376| + |376 - 475| + |475 - 692| + |692 - 874| = 1219$$

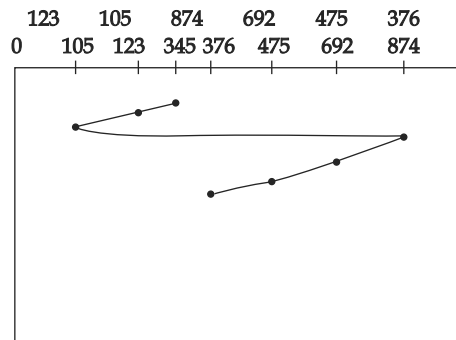
(iv) LOOK → Schedule is

$$\begin{aligned} \text{Total seek distance is} \\ &= |345 - 123| + |123 - 105| + |105 - 376| + |376 - 475| + |475 - 692| + |692 - 874| \\ &= 1009. \end{aligned}$$

(v) C-SCAN → Schedule is

$$\begin{aligned} \text{Total seek distance is} \\ &= |345 - 123| + |123 - 105| + |105 - 0| + |0 - 999| + |999 - 874| + |874 - 692| + |692 - 475| + |475 - 376| = 1967 \end{aligned}$$

(vi) C-LOOK → Schedule is



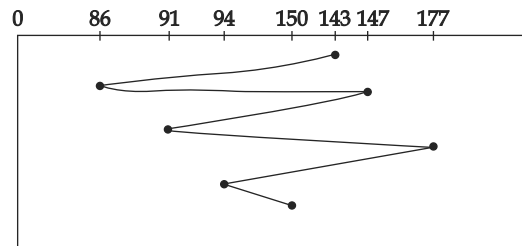
Total seek distance is

$$= |345 - 123| + |123 - 105| + |105 - 874| + |874 - 692| + |692 - 475| + |475 - 376| = 1507$$

10.5 Suppose the moving head disk with 200 tracks is currently serving a request for track 143 and has just finished a request for track 125. If the queue of request is kept in FIFO order 86, 147, 91, 177, 94, 150. What is the total head movement for the following scheduling:

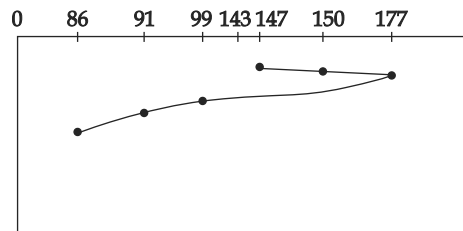
Sol.

(i) FCFS sequence → 86, 147, 91, 177, 94, 150



$$\text{Total seek distance is} = |143 - 86| + |86 - 147| + |147 - 91| + |91 - 177| + |177 - 94| + |94 - 150| = 499$$

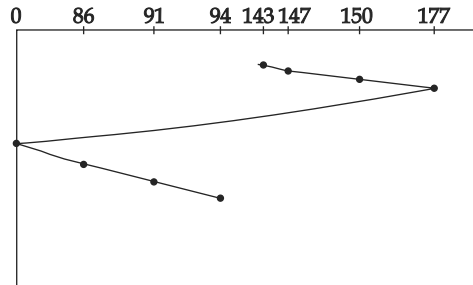
(ii) SSTF sequence is → 147, 150, 177, 94, 91, 86



$$\text{Total seek distance is} = |143 - 147| + |147 - 150| + |150 - 177| + |177 - 94| + |94 - 91| + |91 - 86| = 125$$

(iii) **C-SCAN Sequence** is $\rightarrow 86, 147, 91, 177, 94, 150$

147, 150, 177, 0, 86, 91, 94



Total seek distance is

$$= |143 - 147| + |147 - 150| + |150 - 177| + |177 - 0| + |0 - 86| + |86 - 91| + |91 - 94| = 305 \text{ Ans.}$$

Summary

- Disk drives are the major secondary storage I/O devices on most computers.
- Requests for disk I/O are generated by the file system and by virtual memory system.
- Each request specifies the address on the disk to be referenced in the form of logical block number.
- Disk scheduling algorithms can improve the effective bandwidth, average response time.
- Scheduling the I/O requests on hard disk is known as disk scheduling.
- Total time to fetch some data from disk is called access time.
- FCFS algorithm handles the first arrived request but it suffers from high seek time.
- SSTF algorithm has lowest seek time but it may suffer from starvation.
- The disk head starts from beginning in the SCAN algorithm.
- The C-Scan algorithm services the requests only in one direction.
- The C-Scan algorithm services the requests only in one direction.
- RAID is a data storage virtualization technology that combines multiple disk drives component into a logical unit for purpose of data redundancy and performance improvement.

Multiple-Choice Questions

1. In _____ information is recorded magnetically on platters.

(a) magnetic disks	(b) electrical disks
(c) assemblies	(d) cylinders

-
2. The heads of the magnetic disk are attached to a _____ that moves all the heads as a unit.
 - (a) spindle
 - (b) disk arm
 - (c) track
 - (d) None of these
 3. The set of tracks that are at one arm position make up
 - (a) magnetic disks
 - (b) electrical disks
 - (c) assemblies
 - (d) cylinders
 4. The time taken to move the disk arm to the desired cylinder is called the
 - (a) positioning time
 - (b) random access time
 - (c) seek time
 - (d) rotational latency
 5. The time taken for the desired sector to rotate to the disk head is called
 - (a) positioning time
 - (b) random access time
 - (c) seek time
 - (d) rotational latency
 6. When the head damages the magnetic surface, it is known as
 - (a) disk crash
 - (b) head crash
 - (c) magnetic damage
 - (d) All of these
 7. A floppy disk is designed to rotate _____ as compared to a hard disk drive.
 - (a) faster
 - (b) slower
 - (c) at the same speed
 - (d) None of these
 8. The host controller is
 - (a) controller built at the end of each disk
 - (b) controller at the computer end of the bus
 - (c) both (a) and (b)
 - (d) neither (a) nor (b)
 9. _____ controller sends the command placed into it via messages to the _____ controller.
 - (a) Host, host
 - (b) Disk, disk
 - (c) Host, disk
 - (d) Disk, host
 10. The disk bandwidth is
 - (a) the total number of bytes transferred
 - (b) total time between the first request for service and the completion on the last transfer
 - (c) the total number of bytes transferred divided by the total time between the first request for service and the completion on the last transfer
 - (d) None of these
 11. RAID level 3 supports a lower number of I/Os per second, because
 - (a) every disk has to participate in every I/O request
 - (b) only one disk participates per I/O request

- (c) I/O cycle consume a lot of CPU time
 (d) All of these
12. RAID level _____ is also known as block interleaved parity organisation and uses block level striping and keeps a parity block on a separate disk.
 (a) 1 (b) 2
 (c) 3 (d) 4
13. A performance problem with _____ is the expense of computing and writing parity.
 (a) non-parity based RAID levels (b) parity based RAID levels
 (c) all RAID levels (d) None of these
14. In RAID level 4, one block read, accesses
 (a) only one disk (b) all disks simultaneously
 (c) all disks sequentially (d) None of these
15. The overall I/O rate in RAID level 4 is
 (a) low (b) very low
 (c) high (d) None of these
16. Tertiary storage is built with
 (a) a lot of money (b) unremovable media
 (c) removable media (d) secondary storage
17. Floppy disk are examples of
 (a) primary storage (b) secondary storage
 (c) tertiary storage (d) None of these
18. A magneto-optic disk is
 (a) primary storage (b) secondary storage
 (c) tertiary storage (d) removable disk
19. The magneto-optic head files _____ the disk surface than a magnetic disk head does.
 (a) much farther from (b) much closer to
 (c) at the same distance as (d) None of these
20. Optical disks _____ magnetism.
 (a) use (b) do not use
 (c) may use (d) None of these

Answers

- | | | | | | |
|-----------------|---------|---------|---------|---------|---------|
| 1. (a) | 2. (b) | 3. (d) | 4. (c) | 5. (d) | 6. (b) |
| 7. (b) | 8. (b) | 9. (c) | 10. (c) | 11. (a) | 12. (d) |
| 13. (b) | 14. (a) | 15. (c) | 16. (c) | 17. (c) | |
| 18. (c) and (d) | 19. (a) | 20. (b) | | | |

11.1 INTRODUCTION TO PROTECTION

Protection and security measures prevent interference with the use of files or memory in a system. When these measures are applied to information they ensure that information is used by the authorised users only.

Protection refers to a mechanism for controlling the access of programs, processes or users to the resources defined by a computer system. This mechanism must provide a means for specification of the controls to be imposed together with some means of enforcement.

11.2 PROTECTION GOALS

An operating system consists of a collection of object, hardware or software. Each object has a unique name and can be accessed through a well-defined set of operations. Protection in operating system is needed to ensure that each object is accessed correctly and only by those processes that are allowed to do so, that is, to ensure fair and reliable resource usage. The main goal of protection is to prevent intentional or accidental destructive behaviour in multiprogramming systems. In these systems OSs enable users to safely share logical address space and the physical address space.

11.3 PROTECTION DOMAIN

The protection domain specifies a set of objects and types of operations that are performed on the objects. Protection domain is a collection of access rights of object name and right set, where right set is a subset of all valid operations.

Domains may overlap because a single object can participate in multiple domain with different access rights. The process may switch between different protection domain during execution.

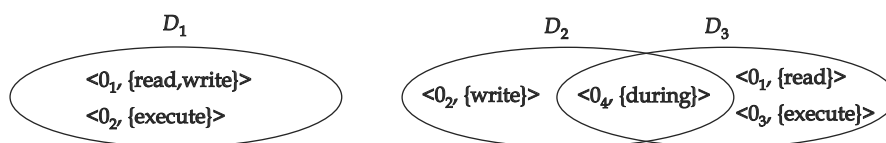


Fig. 11.1 System with three protection domains

11.4 ACCESS RIGHTS

The protection mechanism is different for primary and secondary memory. The authority to execute an operation on an object is known as access rights. Access rights specifies a object name and a right set, that is, all these valid operations that can be performed on each object. Access matrix represents all access rights of all the objects (hardware and software object) in a computer system. Access matrix can be viewed as a two-dimensional matrix.

Object		File 1	File 2	File 3	Printer
Domain	D ₁	Read write	Read		
	D ₂		Read	Execute	
	D ₃				Print
	D ₄		Read write		Print

Fig. 11.2 Access matrix

It is a useful matrix, but this matrix is not sufficient to store all the access rights. There are several access control mechanisms like access list, access hierarchies, list, etc.

11.5 ACCESS MATRIX IMPLEMENTATION

There are several methods of implementing the access matrix.

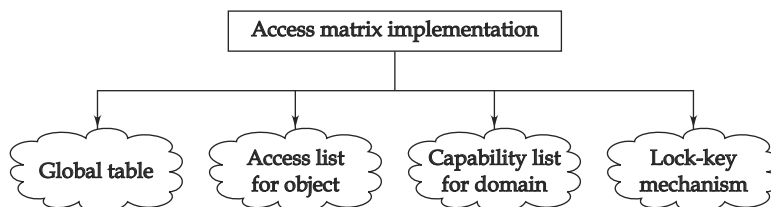


Fig. 11.3 Access matrix representation

1. Global Table

Global table is the simplest implementation of access matrix. It consist of $\langle \text{domain, object, right set} \rangle$, whenever an operation N is executed on an object D_i within domain

D_i , then the global table searches for $\langle D_i, D_i, R_k \rangle$, with NER_k , if it exists, then the operation is allowed to continue otherwise an error is raised.

2. Access List

Each non-empty entry in the column of the access matrix can be implemented as an access list for the specific object. Access list can be combined with other schemes to strengthen the protection. An access list can be extended easily to define a list plus a default set of access rights, whenever an operation N is executed on an object D_i with in domain D_i , then search the access list for object D_i , if the entry $\langle D_i, R_k \rangle$ with NER_k is found then the operation is allowed to continue, and if the entry is not found, then check the default set.

If N is in the default set, then the access is allowed otherwise an error is raised.

3. Capability List

A capability list for domain is a list that contains all the non-empty entry in the row of the access matrix, for the specific object. An object be represented by the physical name and address, called a capability. The capability list is associated with the domain, but it is never directly accessible to a process executing in that domain. Capabilities are usually distinguished from other data in two ways:

- Each object has a tag to denote whether it is accessible data or not.
- Address space is split into two parts, first is accessed by the program, which contains the data and instruction, and the second part is only accessed by the operating system and contain the capability list.

4. Lock-key Mechanism

The trade-off between the access list and capability list is referred as lock-key mechanism. The object has a list of unique bit patterns called lock, similarly the domain has a list of unique bit patterns called key. A process in a domain can access objects if and only if there is a key that matches the lock.

11.6 NEED OF SECURITY

Security refers to providing a protection system to computer system resources such as CPU, memory, disk, software programs and most importantly data/information stored in the computer system. If a computer program is run by unauthorized user then he/she may cause damage to computer or data stored in it. So a computer system must be protected against unauthorized access, malicious access to system memory, viruses, worms, etc.

11.7 AUTHENTICATION

Authentication refers to identifying each user of the system and associating the executing programs with those users. It is the responsibility of the operating system

to create a protection system which ensures that a user who is running a particular program is authentic. Operating system (OS) authentication is a method for identifying an individual user with credentials supplied by the operating system of the user's computer. These credentials can be the OS password or can include digital certificates in the user's computer. Operating systems generally identify/authenticate users using the following three ways:

- **User card/key:** User needs to punch card in card slot, or enter key generated by key generator in option provided by operating system to login into the system.
- **Username/Password:** User needs to enter a registered username and password with operating system to login into the system.
- **User attribute-fingerprint/eye retina pattern/signature:** User need to pass his/her attribute via designated input device used by operating system to login into the system.

11.8 PROGRAM THREATS

Operating system's processes and kernel perform the given task as instructed. If a user program makes these processes do malicious tasks then it is known as program threat. One of the common examples of program threat is a program installed in a computer which can store and send user credentials via network to some hacker. Following is the list of some well-known program threats.

Trojan Horse: Such program traps user login credentials and stores them to send to malicious user who can later on login to computer and can access system resources.

Trap Door: If a program which is designed to work as required, have a security hole in its code and perform illegal action without knowledge of user then it is called to have a trap door.

Logic Bomb: Logic bomb is a situation when a program misbehaves only when certain conditions met otherwise it works as a genuine program. It is harder to detect.

Virus: Virus as the name suggests can replicate them on computer system. They are highly dangerous and can modify/delete user files, crash systems. A virus is generally a small code embedded in a program. As user accesses the program, the virus starts getting embedded in other files/programs and can make system unusable for user.

Stack and Buffer Overflow: It exploits a bug in a program (overflow either the stack or memory buffers.)

11.9 SYSTEM THREATS

System threats refer to misuse of system services and network connections to put the user in trouble. System threats can be used to launch program threats on a complete network called program attack. System threats create such an environment that operating system resources or user files are misused. System and network threats involve the abuse of service and network connection some examples of system threat:

- **Worm:** Worm is a process which can choke down a system performance by using system resources to extreme level. A worm process generates its multiple copies where each copy uses system resources, prevent all other processes to get required resources. Worms process can even shut down an entire network.
- **Port scanning:** Port scanning is a mechanism or means by which a hacker can detect system vulnerabilities to make an attack on the system.
- **Denial of service:** Denial of service attack normally prevent the user to make legitimate use of the system. These attacks are generally network based.

11.10 THREAT MONITORING

Threat monitoring is a subscription based security service. This enables filtering of online attacks which protect data loss in business environment. This monitoring involves continuous analysis, assessment and review of security related stuff collected from all sources for detecting any type of theft or user during operation or in any system.

Various key points that one must keep in mind while threat monitoring are:

1. Understand the type of threat or attack that can be mounted against a system.
2. Understand the threat and its source while identifying data.
3. Understand the logic and cause behind that threat.

Thus, we may define threat as:

Potential possibility of considering unauthorized attempt to:

- (a) access information
- (b) manipulate information
- (c) make a system or information unreliable.

Therefore, threat monitoring is an essential part of OS and involves checking for suspicious patterns like incorrect pwd, etc. and scanning the system periodically for security holes; done when system is relatively unused or unreliable.

One may monitor threat in the following ways:

- (a) Update firmware and OS.
- (b) Perform monthly security scan of system.
- (c) Automatic monitoring with alerting.
- (d) Encrypt the data.

Summary

- Confidentiality is the requirement that information maintained by a computer system is accessible only by the authorised users.
- Integrity the requirement that a computer system resources can be modified only by authorised parties.
- Modification occurs when an authorised party not only gains access but changes a resource or the execution of running program.

- Authenticity is the requirement that a computer system can verify the identity of a user.
- Intruders and viruses are the two most publicised security threats.
- A trap door is a secret entry point into a program, often left by the program's developers or sometime via software update.
- A logic bomb is a code embedded in a program that "explodes" when certain conditions are met. Logic bombs often originate with the developers of the software.
- A Trojan horse is a program that contains hidden code to prevent some unwanted or harmful action.
- A virus is a program that can infect other programs by modification, as well as causing local damage. Such modification includes a copy of virus, which can then spread further to other programs.
- Passive attacks are hard to detect and takes the form of eavesdropping.
- Active attacks are hard to prevent so the emphasis is usually on detection and damage control.

Multiple-Choice Questions

1. The main goal of protection is to:
 - (a) Prevent accidental and intentional behaviour
 - (b) Implement multiprogramming
 - (c) Protect deadlocks
 - (d) None of the above
2. The process of conversion of plain text to cipher text is known as:
 - (a) Encryption
 - (b) Decryption
 - (c) Hash function
 - (d) None
3. The art of devising ciphers is called
 - (a) Message digest
 - (b) Hash functions
 - (c) Cryptography
 - (d) None
4. DES stands for
 - (a) Digital Equipment Standard
 - (b) Data Encryption
 - (c) Domain Encryption Standard
 - (d) None
5. Hash functions are used to produce the message digests which are then encrypted with a private key to get
 - (a) Cipher text
 - (b) Digital signature
 - (c) Public key
 - (d) None

Answers

- | | | | | |
|--------|--------|--------|--------|--------|
| 1. (a) | 2. (a) | 3. (c) | 4. (b) | 5. (b) |
|--------|--------|--------|--------|--------|

Appendices

Case Study of Linux & Window XP OS

WHAT IS LINUX?

Linux is, in simplest terms, an operating system. It is the software on a computer that enables applications and the computer operator to access the devices on the computer to perform desired functions. The operating system (OS) relays instructions from an application to, for instance, the computer's processor. The processor performs the instructed task, then sends the results back to the application via the operating system.

Explained in these terms, Linux is very similar to other operating systems, such as Windows and OS X.

But something sets Linux apart from these operating systems. The Linux operating system represented a \$25 billion ecosystem in 2008. Since its inception in 1991, Linux has grown to become a force in computing, powering everything from the New York Stock Exchange to mobile phones to supercomputers to consumer devices.

As an open operating system, Linux is developed collaboratively, meaning no one company is solely responsible for its development or ongoing support. Companies participating in the Linux economy share research and development costs with their partners and competitors. This spreading of development burden amongst individuals and companies has resulted in a large and efficient ecosystem and unheralded software innovation.

Over 1,000 developers, from at least 100 different companies, contribute to every kernel release. In the past two years alone, over 3,200 developers from 200 companies have contributed to the kernel – which is just one small piece of a Linux distribution.

All operating systems have kernels, built around the architectural metaphor that there must be a central set of instructions of direct device hardware, surrounded by various modular layers of functionality. The Linux kernel is unique and flexible because it is also modular in nature.

Modularity is desirable because it allows developers to shed parts of the kernel they don't need to use. Typically a smaller kernel is a faster kernel, because it isn't running processes it does not need.

If a device developer wants a version of Linux to run on a cell phone, she does not need the kernel functionality that deals with disk drives, Ethernet devices, or big monitor screens. She can pull out those pieces (and others), leaving just the optimized kernel to use for the phone.

The kernel of the Window operating system (which few people outside of Microsoft are allowed to look at without paying for the privilege) is a solidly connected piece of code, unable to be easily broken up into pieces. It is difficult (if not impossible) to pare down the Windows kernel to fit on a phone.

This modularity is significant to the success of Linux. The ability to scale down (or up) to meet the needs of a specific platform is a big advantage over other operating systems constrained to just a few possible platforms.

Modularity also effects stability and security as well. If one piece of the kernel code happens to fail, the rest of the kernel will not crash. Similarly, an illicit attack on one part of the kernel (or the rest of the operating system) might hamper that part of the code, but should not compromise the security of the whole device.

Linux is one of popular version of UNIX operating system. It is open source as its source code is feely available. It is free to use. Linux was designed considering UNIX compatibility. Its functionality list is quite similar to that of UNIX.

COMPONENTS OF LINUX SYSTEM

Linux Operating System has primarily three components

- **Kernel:** Kernel is the core part of Linux. It is responsible for all major activities of this operating system. It is consists of various modules and it interacts directly with the underlying hardware. Kernel provides the required abstraction to hide low level hardware details to system or application programs.
- **System Library:** System libraries are special function or programs using which application programs or system utilities accesses Kernel's features. These libraries implements most of the functionalities of the operating system and do not requires kernel module's code access rights.
- **System Utility:** System Utility programs are responsible to do specialized, individual level tasks.

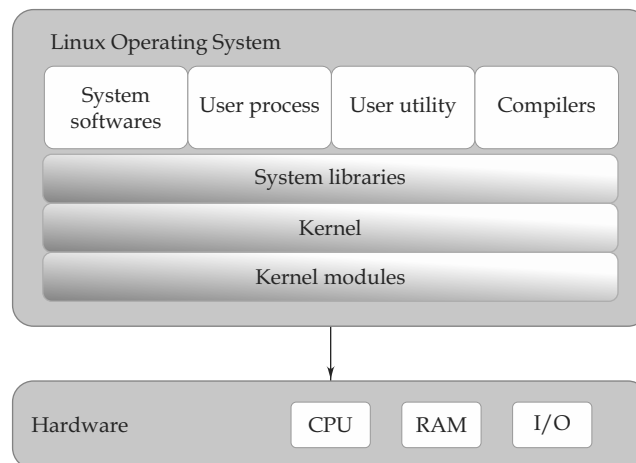


Fig. AI.1 Components of linux

KERNEL MODE VS USER MODE

Kernel component code executes in a special privileged mode called **kernel mode** with full access to all resources of the computer. This code represents a single process, executes in single address space and do not require any context switch and hence is very efficient and fast. Kernel runs each processes and provides system services to processes, provides protected access to hardwares to processes.

Support code which is not required to run in kernel mode is in System Library. User programs and other system programs works in **User Mode** which has no access to system hardwares and kernel code. User programs/utilities use System libraries to access Kernel functions to get system's low level tasks.

Basic Features

Following are some of the important features of Linux Operating System.

- **Portable:** Portability means softwares can works on different types of hardwares in same way. Linux kernel and application programs supports their installation on any kind of hardware platform.
- **Open Source:** Linux source code is freely available and it is community based development project. Multiple teams works in collaboration to enhance the capability of Linux operating system and it is continuously evolving.
- **Multi-User:** Linux is a multi user system means multiple users can access system resources like memory/ram/application programs at same time.
- **Multiprogramming:** Linux is a multiprogramming system means multiple applications can run at same time.
- **Hierarchical File System:** Linux provides a standard file structure in which system files/user files are arranged.
- **Shell:** Linux provides a special interpreter program which can be used to execute commands of the operating system. It can be used to do various types of operations, call application programs, etc.
- **Security:** Linux provides user security using authentication features like password protection/controlled access to specific files/encryption of data.

Architecture

Linux System Architecture is consist of following layers:

- **Hardware layer:** Hardware consists of all peripheral devices (RAM/HDD/CPU etc).
- **Kernel:** Core components of Operating System, interacts directly with hardware, provides low level services to upper layer components.
- **Shell:** An interface to kernel, hiding complexity of kernel's functions from users. Takes commands from user and executes kernel's functions.
- **Utilities:** Utility programs giving user most of the functionalities of an operating systems.

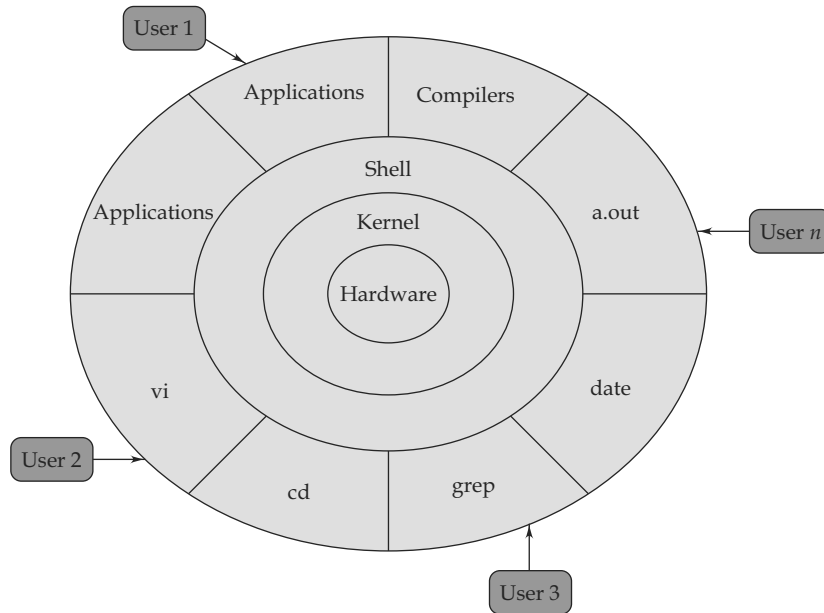


Fig. AI.2 Architecture of linux

Microsoft Windows Operating System series are one of the most popular operating systems. The first Windows was released in **1985 (Windows 1.0)** and since then till today the Microsoft OS has dominated the personal computer market. Windows has been written in **Assembly**, **C** and **C++** languages and it is simply a graphical user interface (**GUI**) that manages and maintains the users' computer resources and components. Computer software is categorized in **application software** and **system software**. Application software can perform a specific task, a text or image editor such as Microsoft Word or Adobe Photoshop are some examples of application software. But system software refers to the software that enables a computer to function efficiently. These special sorts of software consist of programs, interpreters, and languages to control the operation of a computer system. Here, we will talk about the **Windows Operating System evolution** and its **architecture** as system software. So before we start, let see what an operating system is exactly.

WINDOWS 3.X FAMILY

Windows 3.1

Microsoft **Windows 3.1** was the first member of windows family. It had a graphical user interface (**GUI**) and was based to run on the **DOS** (Disk Operating System). Windows 3.1x is a **16-bit** operating system was designed for personal computers.

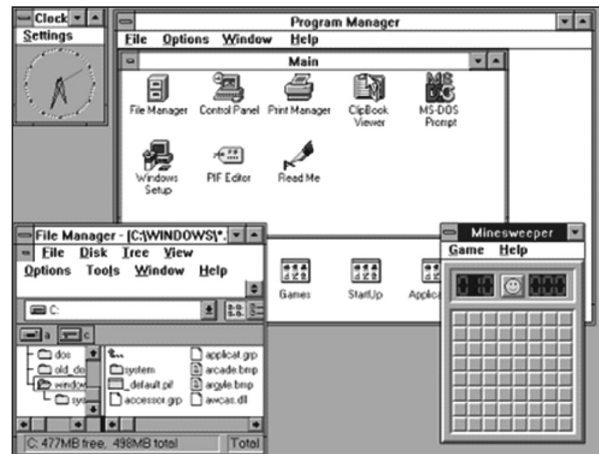


Fig. AI.3 GUI of Windows 3.1

Windows for Workgroups 3.11

The main feature of Windows for **Workgroup 3.11** was the **network** support. It supported **32-bit** access, full 32-bit network re-directors and ran on the **DOS**.

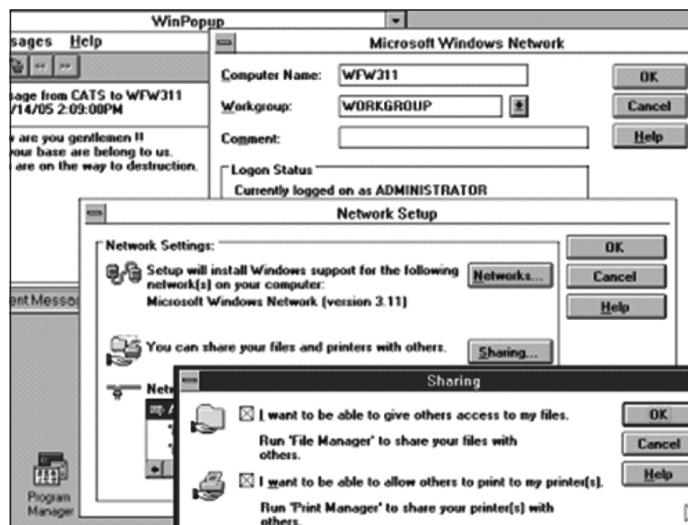


Fig. AI.4 GUI of workgroups 3.11

Windows 95

The **Windows 95** was a significant improvement from the previous Windows versions. It was designed for end-user desktop with better user interface, network support and **plug-and-play** feature (connect devices to the system without the need of configuration).

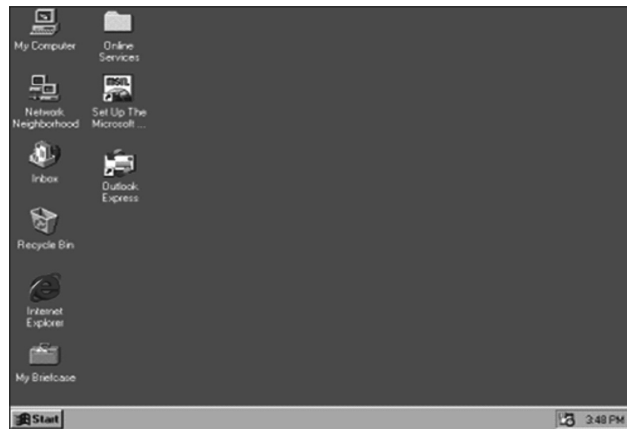


Fig. AI.5 GUI of Window 95

Windows 98

The second major release in the Windows 9x family was the **Windows 98**. It included new features such as **Wizards, Utilities, Tools** and resources for better performance. The Windows 98 is a **16-bit/32-bit** OS with an **MS-DOS** based boot stage. This operating system was faster than the Windows 95 and contained enhanced television, video playback, and new hardware support.

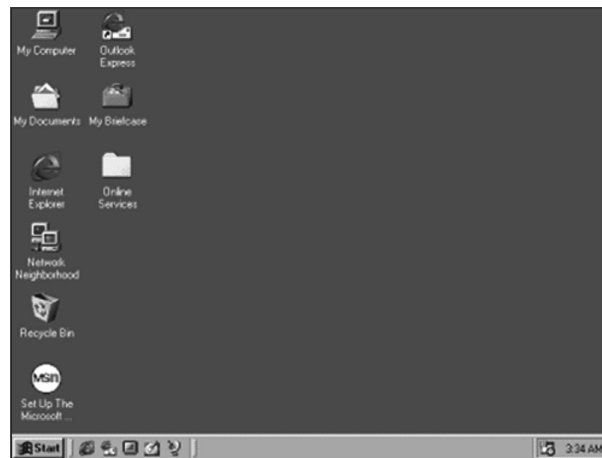


Fig. AI.6 GUI of Windows 98

Windows ME

The last operating system released in the Windows 9X series was the **Windows Millennium** or **ME** and it was designed by Microsoft for home users. It had the ability to connect users' computers at home to create a **local area network** with better Internet connectivity, and work with rich multimedia content such as photos, videos, and music.

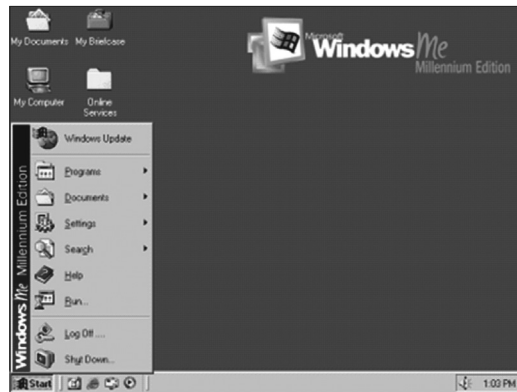


Fig. AI.7 GUI of Windows ME

WINDOWS NT FAMILY

Windows NT

The first version of **Windows NT (New Technology)** family was released in **1993**. Its design was based on Windows 95 user-interface and was a **multiprocessor** and **multiuser** operating system. It was a true **32-bit** operating system, which was optimized to work with 32-bit architecture hardware devices. At this stage, Microsoft started to remove the dependencies on DOS and began to fully rely on the **NT Kernel**.

Windows 2000

Windows 2000 design was built on Windows 98 user interface and made business user more productive. Windows 2000 introduced many of the new features of Windows 98 and Windows 98 SE into the NT family. Windows 2000 made **business user** more productive. Its integrated Web capabilities and support for mobile computers and hardware devices made Windows 2000 the easy way for business user to connect to the Internet anywhere and anytime.

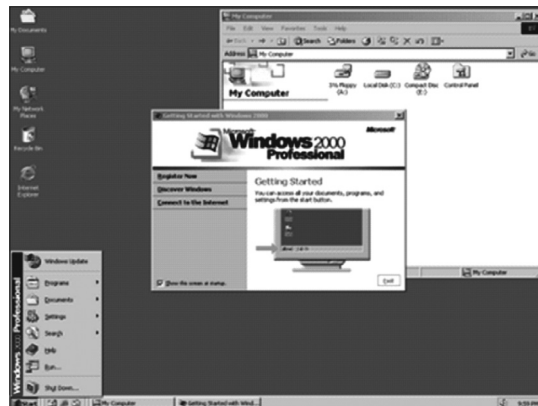


Fig. AI.8 GUI of Windows 2000

Windows XP

Microsoft used power of Windows 2000 and business features of Windows 98 and Me and made **Windows XP**. Windows XP was a smarter OS and introduced several new features to the Windows line, such as GDI+ graphics subsystem and improved image management, Start Menu and Taskbar improvements, Integrated Networking and Multimedia support and many more.



Fig. AI.9 GUI of Windows XP

The operating systems that evolved after Windows 2000 were based on the **Windows 2000 architecture** only. The other Windows operating systems after XP include **Windows Vista**, **7**, and **Windows 8**.

Windows Architecture

Windows 2000 architecture is the latest architecture of Windows operating system and the newer versions of the Windows OS have the same structure as Windows 2000. This structure is a **modular** structure, composed of several simple modules. These modules are:

- Hardware abstraction layer
- Kernel
- Executive services
- Protected subsystems
- Environment

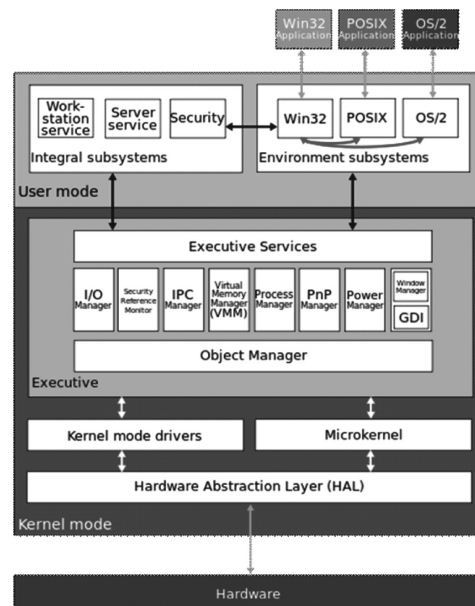


Fig. AI.10 Architecture of Windows

Hardware Abstraction Layer

What we mean by **hardware abstractions** is a set of routines that give a program direct access to the hardware resources. The **Hardware Abstraction Layer (HAL)** makes the **hardware dependencies** transparent to the rest of the operating system. This allows Windows 2000 to be **portable** from one hardware platform to another. It is a thin layer of software or programming that hides (**abstraction**) the hardware differences from the operating system. The HAL consists of a series of function that hides or abstracts differences between hardware platforms. For example, the processor used in your system can be from any vendor, such as Intel, Advanced Micro Devices, and Motorola. In the Windows NT OS, the HAL is placed in the **kernel** and in the space between hardware and the Win NT services.

Kernel

The **kernel** works very closely with the **HAL** and is the heart of the operating system. It schedules the activities to be performed by the CPU. On a computer that has multiple processors, the kernel synchronizes activity among processors to optimize performance. For example, if you open more than one application, such as, MSWord, MS Excel, and PowerPoint, the kernel schedules the processor's time within all applications.

Kernel-mode device drivers enable the Windows NT to interact with hardware components. These drivers have some system routines that represent all the system devices seen as a **file object** in the input/output manager for the user and the I/O manger can view them as **device objects**. These drivers exist in three levels: **high level, intermediate** and **low level drivers**. The high level relies on intermediate level which is contained of function drivers. The intermediate drivers rely on the lower level; this level also includes the Windows Driver Model (**WMD**). The lowest level directly controls the hardware and do not rely on the intermediate or high level drivers.

Another term that we are seeing in the Windows architecture is the **microkernel**. The microkernel is a collection of programs that can provide tasks such as **address space management, thread management** and **inter-process communication (IPC)**. The microkernel along with the Windows kernel are in charge to make the operating system work efficiently.

Executive Services

The **Executive Services**, which includes the kernel and the HAL, provides a set of common services that the user can use. This section interacts with **input/output devices, object management, process management** and the **system security**. Each group of services is managed by one of the components of the executive services, which are as follows:

- Object Manger
- Power Manger
- Process Manager

- I/O Manager
- Virtual Memory Manager
- Local Procedure Call Facility
- Cache Manager
- Security Reference Monitor
- Plug and Play Monitor
- Device Drive Manager

Object Manager

The **Object Manager** provides rules for **retention**, **naming** and **security** of objects. Objects can be, for example, files and folders saved in the file system. It also removes the duplicate object resources. The Object Manager considers each resource as an object whether it is a physical resource or a logical one like a file.

Creation and **insertion** of objects can be done in this section. The Object Manager first allocates an empty object and then reserves the required resources. After that **Ob** inserts the object and makes it accessible through its **name** or a **handle (cookie)**. A handle is an **identifier** that points to a certain kernel resource. The **lifetime** of an object also managed by the Ob and it will keep the object till the subsystem needs it and then it will be deleted by the Object Manager.

Power Manager

The **Power Manager** deals with power events like **power-off**, **stand-by**, and **hibernate**. Windows 2000 supports all of the latest standards in Power Management including the **Advanced Power Management (AMP)** and **Advanced Configuration and Power Interface (ACPI)**. Consequently, network devices can be powered off when not in use and dynamically reactivated when network access is required. Windows 2000 also supports **Wake-on-LAN** technology, allowing an entire machine to be powered down and then reactivated via incoming network requests. With **APM** and **ACPI** support, Windows 2000 has the potential to run for longer periods of time on battery powered systems due to its ability to conserve power when not in use and then be dynamically reactivated via incoming network requests.

Process Manager

The **Process Manager** manages the creation and deletion of processes. It provides a standard set of services for creating and using processes. The windows Process Manager works in conjunction with the **Security Model** and the **Virtual Memory Manager** to provide **inter-processes protection**.

I/O Manager

The **I/O Manager** manages all the **input** and **output** for the operating system. It supports all file system drivers, hardware device drivers and network drivers, and provides a

heterogeneous environment for them. The I/O Manager provides a common interface that all drivers, such as **FAT** file system driver and **NTFS** driver can call. This allows the I/O Manager to communicate with all drivers in the same way, without any knowledge of how the devices they control actually work. The I/O Manager provides a **cache manager** for improving the **disk performance** by caching the read request and then writes them on the disk in the background.

Virtual Memory Manager

The memory architecture for Windows 2000 is a **demand-paged virtual memory system**, accessed via **32-bit address**. **Virtual memory** is the term used to describe the technique where the operating system can allocate more memory than what is physically available. Each process is allocated its own unique virtual address space, which appears to be **4 gigabytes(GB)** in size. This address space is divided into block of equal size called **pages**.

Demand Paging is a method by which data is moved in pages from **physical memory** to a **temporary paging file** on the disk. As the process needs data, it is paged back into physical memory.

The Virtual Memory Manager maps virtual addresses in the process's address space to physical pages in the computer's memory. It hides the physical organization of memory from the processes. This ensures that processes do not access the memory of other processes.

Local Procedure Call Facility

Windows 2000 provides a communication mechanism between the applications and the environment subsystems. The executive system implements a message passing facility called a **Local Procedure Call (LPC)**. Applications communicate with the environment subsystems by passing messages via the LPC facility. The LPC contains **inter-process communication ports** that can be used by **user-mode subsystem** for communication.

Cache Manager

The **Cache Manager** is a part of the **I/O** architecture. It handles caching for the entire I/O system. Caching is used to improve the performance of the I/O systems. Instead of reading and writing directly to disk, frequently used files are temporarily stored in a cache in memory, and read and write operations are performed to these files in the memory. This improves the performance since reading and writing to memory is much **faster** than reading and writing to disk. The Cache Manager provides caching services for all file systems and network components under the control of the I/O Manager.

The Cache Manager dynamically changes the size of the cache as the amount of available RAM varies. When a process opens a file that is already present in the cache, the Cache Manager copies data from the cache to the process's memory, and vice versa, as read and write operations are performed.

Security Reference Monitor

The **Security Reference Monitor (SRM)** is responsible for enforcing the **access validation** and **audit-generation policy** defined by the local security subsystem. It is also responsible for controlling which objects have permissions to which resource. Each object has an **Access Control List (ACL)** that is queried when the object makes a service request. Access to resources is allowed or disallowed according to the right the module has in the ACL. For example, you have created a shared directory having read permission. If other users try to write into the directory, the Security Reference Monitor will check for the permission and will deny writing into the directory.

Plug and Play Manager

Plug and Play, which made its first appearance with Microsoft **Windows 95**, is now a feature of **Windows 2000**. Changes have been made within the system architecture of Windows 2000 to accommodate this facility. Compatible devices can now be introduced and Windows will configure the device. **PnP** supports **device detection** and **installation** at boot time which may require the reconfiguration of other devices. Plug-and-Play automatically tells the software (device drivers) where to find various pieces of hardware devices such as modems, network and sound cards, etc. Its task is to match the physical devices with the software that operates them and to establish channels of communication between each physical device and its driver.

Device Manager

Device Manager allows you to check the status of your **hardware devices** and to update **device drivers** for the hardware installed on your computer. It also checks how the hardware interacts with your computer programs. You can also use Device Manager to check the status of your hardware and update device drivers for the hardware installed on your computer.

Environment Subsystems

Windows 2000 allows many different types of applications to run on the same graphical desktop. It runs applications for operating systems such as MS-DOS, OS/2, Windows and POSIX. Windows 2000 support a variety of applications through the use of **Environment Subsystems**, which are Windows 2000 processes that emulate **different operating system environments**. For example, through command prompt of Windows 2000, you can get the CUI environment of DOS.

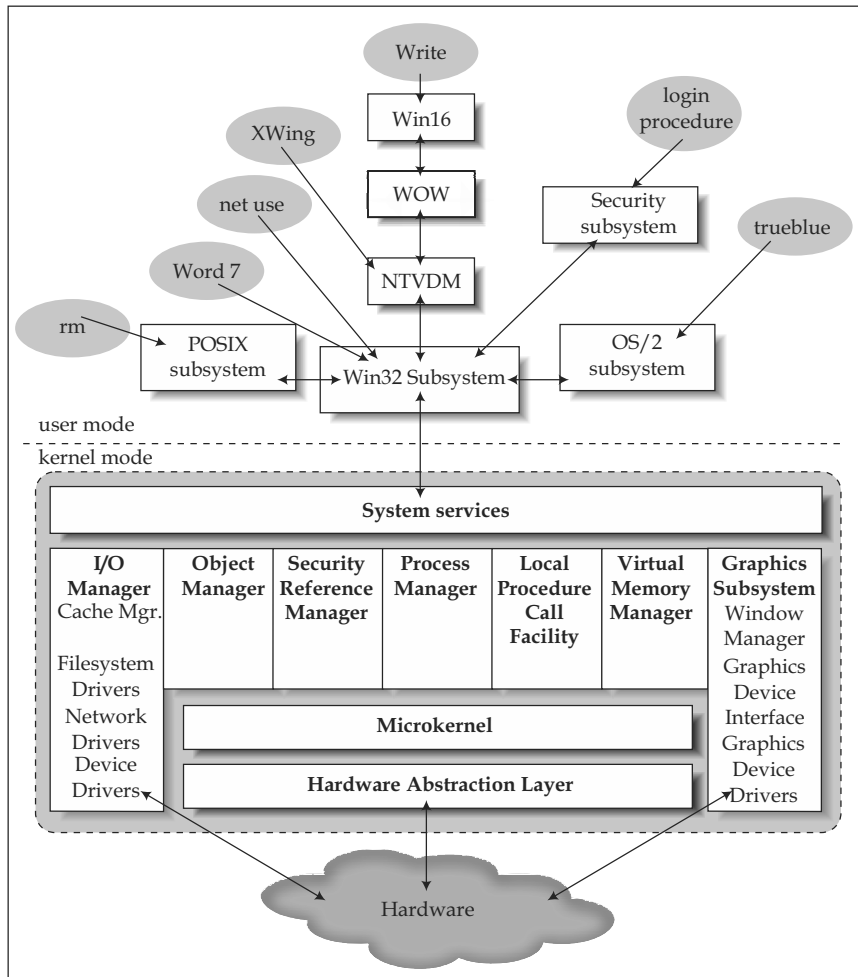


Fig. A1.11 Enviroment system of windows



Case Study of Mobile OS

Symbian OS is an operating system that runs on mobile “smartphone” platforms from several different manufacturers. Smartphones are so named because they run fully-featured operating systems and utilize the features of desktop computers. Symbian OS is designed so that it can be the basis of a wide variety of smartphones from several different manufacturers. It was carefully designed specifically to run on smartphone platforms: general-purpose computers with limited CPU, memory and storage capacity, focused on communication. Symbian OS has roots in systems that were developed in the 1990’s and its debut was in 2001. This should not be surprising, since the smartphone platform upon which Symbian OS runs has evolved only recently as well. Symbian OS has its roots in hand-held devices and has seen rapid development through several versions.

Symbian OS is unique among operating systems in that it was designed with smartphones as the target platform. It is not a generic operating system shoe-horned into a smartphone, nor is it an adaptation of a larger operating system for a smaller platform. It does, however, have many of the features of other larger operating systems, from multitasking to memory management to security issues.

The predecessors to Symbian OS have given their best features. Symbian OS is object-oriented, inherited from EPOC. It uses a microkernel design, which minimizes kernel overhead and pushes nonessential functionality to user-level processes, as introduced in version 6. It uses a client/server architecture, which mimics the engine model built into EPOC. It supports many desktop features, including multitasking and multithreading, and an extensible storage system. It also inherited a multimedia and communication emphasis from EPOC and the move to Symbian OS.

The Symbian OS Nanokernel

Symbian OS designers have addressed microkernel issues by implementing a **nanokernel** structure at the core of the operating system’s design. Just as certain system functions are pushed into user-space servers in microkernels, the design of Symbian OS separates functions that require complicated implementation into the Symbian OS kernel and keeps only the most basic functions in the nanokernel, the operating system’s core.

The nanokernel provides some of most basic functions in Symbian OS. In the nanokernel, simple threads operating in privileged mode implement services that are very primitive. Included among the implementations at this level are scheduling

and synchronization operations, interrupt handling, and synchronization objects, such as mutexes and semaphores. Most of the functions implemented at this level are preemptible. Functions at this level are very primitive (so that they can be fast); for example, dynamic memory allocation is a function too complicated for a nanokernel operation.

This nanokernel design requires a second level to implement more complicated kernel functions. The **Symbian OS kernel layer** provides the more complicated kernel functions that are needed by the rest of the operating system. Each operation at the Symbian OS kernel level is a privileged operation and combines with the primitive operations of the nanokernel to implement more complex kernel tasks. Complex object services, user-mode threads, process scheduling and context switching, dynamic memory, dynamically loaded libraries, complex syn-chronization objects and interprocess communication are just some of the operations implemented by this layer. This layer is fully preemptible and interrupts can cause this layer to reschedule any part of its execution even in the middle of con-text switching!

Figure below shows a diagram of the complete Symbian OS kernel structure.

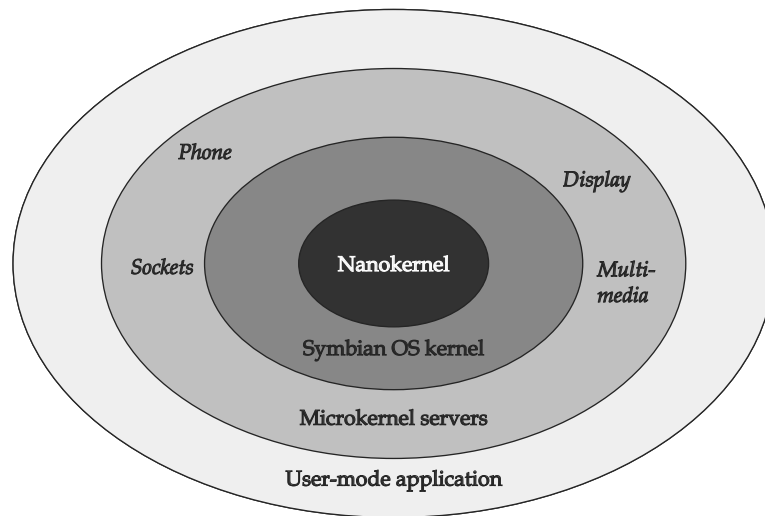


Fig. AII.1 The Symbian OS kernel structure has many layers

Symbian OS has some unique features that come from its focus on the smart-phone platform. It has a pluggable messaging architecture—one where new message types can be invented and implemented by developing modules that are dynamically loaded by the messaging server. The messaging system has been designed in layers, with specific types of object implementing the various layers. For example, message transport objects are separate from message type objects. A form of message transport, say, cellular wireless transport (like CDMA), could transport several different types of messages (standard text message types, SMS types, or system commands like BIO messages). New transport methods can be introduced by implementing a new object and loading it into the kernel.

Symbian OS has been designed at its core with APIs specialized for multimedia. Multimedia devices and content are handled by special servers and by a framework that lets the user implement modules that describe new and existing content and what to do with it. In much the same way messaging is implemented, multimedia is supported by various forms of objects, designed to interact with each other. The way sound is played is designed as an object that interacts with the way each sound format is implemented.

Processes and Threads in Symbian OS

Symbian OS is a multitasking operating system that uses the concepts of processes and threads much like other operating systems do. However, the structure of the Symbian OS kernel and the way it approaches the possible scarcity of resources influences the way that it views these multitasking objects.

Threads and Nanothreads

Instead of processes as the basis for multitasking, Symbian OS favours threads and is built around the thread concept. Threads form the central unit of multitasking. A process is simply seen by the operating system as a collection of threads with a process control block and some memory space.

Thread support in Symbian OS is based in the nanokernel with **nanothreads**. The nanokernel provides only simple thread support; each thread is supported by a nanokernel-based nanothread. The nanokernel provides for nanothread scheduling, synchronization (interthread communication) and timing services. Nanothreads run in privileged mode and need a stack to store their runtime environment data. Nanothreads cannot run in user mode. This fact means that the operating system can keep close, tight control over each one. Each nanothread needs a very minimal set of data to run: basically, the location of its stack and how big that stack is. The operating system keeps control of everything else, e.g., the code each thread uses, and stores a thread's context on its runtime stack.

Nanothreads have thread states like processes have states. The model used by the Symbian OS nanokernel adds a few states to the basic model. In addition to the basic states, nanothreads can be in the following states:

Suspended: This is when a thread suspends another thread and is meant to be different from the waiting state, where a thread is blocked by some upper layer object (e.g., a Symbian OS thread).

Fast Semaphore Wait: A thread in this state is waiting for a fast semaphore—a type of sentinel variable—to be signalled. Fast semaphores are nanokernel level semaphores.

DFC Wait: A thread in this state is waiting for a delayed function call or DFC to be added to the DFC queue. DFCs are used in device driver implementation. They represent calls to the kernel that can be queued and scheduled for execution by the Symbian OS kernel layer.

Sleep: Sleeping threads are waiting for a specific amount of time to elapse.

Other: There is a generic state that is used when developers implement extra states for nanothreads. Developers do this when they extend the nanokernel functional for new phone platforms (called personality layers).

Developers that do this must also implement how states are transitioned to and from their extended implementations.

Compare the nanothread idea with the conventional idea of a process. A nanothread is essentially an ultra light-weight process. It has a mini-context that gets switched as nanothreads get moved onto and out of the processor. Each nanothread has a state, as do processes. The keys to nanothreads are the tight control that the nanokernel has over them and the minimal data that make up the context of each one.

Symbian OS threads build upon nanothreads; the kernel adds support beyond what the nanokernel provides. User mode threads that are used for standard applications are implemented by Symbian OS threads. Each Symbian OS thread contains a nanothread and adds its own runtime stack to the stack the nanothread uses. Symbian OS threads can operate in kernel mode via system calls. Symbian OS also add exception handling and exit signaling to the implementation.

Symbian OS threads implement their own set of states on top of the nanothread implementation. Because Symbian OS threads add some functionality to the minimal nanothread implementation, the new states reflect the new ideas built into Symbian OS threads. Symbian OS adds seven new states that Symbian OS threads can be in, focussed on special blocking conditions that can happen to a Symbian OS thread. These special states include waiting and suspending on (normal) semaphores, mutex variables and condition variables. Remember that, because of the implementation of Symbian OS threads on top of nanothreads, these states are implemented in terms of nanothread states, mostly by using the suspended nanothread state in various ways.

Processes

Processes in Symbian OS, then, are Symbian OS threads grouped together under a single process control block structure with a single memory space. There may be only a single thread of execution or there may be many threads under one process control block. Concepts of process state and process scheduling have already been defined by Symbian OS threads and nanothreads. Scheduling a process, then, is really implemented by scheduling a thread and initializing the right process control block to use for its data needs.

Symbian OS threads, organized under a single process, work together in several ways. First, there is a single main thread that is marked as the starting point for the process. Second, threads share scheduling parameters. Changing parameters, that is, the method of scheduling, for the process changes the parameters for all threads. Third, threads share memory space objects, including device and other object descriptors. Finally, when a process is terminated, the kernel terminates all threads in the process.

How Symbian OS Addresses Memory

Since Symbian OS is a 32-bit operating system, addresses can range up to 4 GB. It employs the same abstractions as larger systems: programs must use virtual addresses,

which get mapped by the operating system to physical addresses. As with most systems, Symbian OS divides memory into virtual pages and physical frames. Frame size is usually 4 kB, but can be variable.

Since there can be up to 4 GB of memory, a frame size of 4 kB means a page table with over a million entries. With limited sizes of memory, Symbian OS cannot dedicate 1 MB to the page table. In addition, the search and access times for such a large table would be a burden to the system. To solve this, Symbian OS adopts a two-level page table strategy. The first level, called the **page directory**, provides a link to the second level and is indexed by a portion of the virtual address (first 12 bits). This directory is kept in memory and is pointed to by the **TTBR (translation table base register)**. A page directory entry points into the second level, which is a collection of page tables. These tables provide a link to a specific page in memory and are indexed by a portion of the virtual address (middle 8 bits). Finally, the word in the page referenced is indexed by the low-order 12 bits of the virtual address. Hardware assists in this virtual-to-physical address mapping calculation. While Symbian OS cannot assume the existence of any kind of hardware assistance, most of the architectures it is implemented for have MMUs. The ARM processor, for example, has an extensive MMU, with a translation lookaside buffer to assist in address computation.

When a page is not in memory, an error condition occurs because all application memory pages should be loaded when the application is started (no demand paging). Dynamically loaded libraries are pulled into memory explicitly by small stubs of code linked into the application executable, not by page faults.

Applications

Despite the lack of swapping, memory is very dynamic in Symbian OS. Applications are context switched through memory and, as stated above, have their memory requirements loaded into memory when they start execution. The memory pages each application requires can be statically requested from the operating system upon loading into memory. Dynamic space—that is, for the heap—is bounded, so static requests can be made for dynamic space as well. Memory frames are allocated to pages from a list of free frames; if no free frames are available, then an error condition is raised. Memory frames that are used cannot be replaced with pages from an incoming application, even if the frames are for an application that is not executing currently. This is because there is no swapping in Symbian OS and there is no place to copy the evicted pages to as the (very limited) flash memory is only for user files.

Set of Implemented Program in C/C++

EXPERIMENT 1

Program Name: WAP to implement FCFS CPU Scheduling Algorithm.

Theory Concept:

First-Come-First-Served algorithm is the simplest scheduling algorithm. Processes are dispatched according to their arrival time on the ready queue. Being a nonpreemptive discipline, once a process has a CPU, it runs to completion. FCFS scheme is not useful in scheduling interactive users because it cannot guarantee good response time. One of the major drawbacks of this scheme is that the average time is often quite long.

Implementation:

```
#include<stdio.h>
void main()
{
    int b[10],temp,k1,k2,j,tat[10],n,wt[10],s1=0,s2=0,i,at[10];
    float avg_tat,avg_wt;
    int ct[10];
    printf("Enter the no of processes ");
    scanf("%d",&n);
    printf("Enter the burst time of all processes\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&b[i]);
    }
    printf("Enter the arrival time of all processes\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&at[i]);
    }
    //sorting arrival time
    for(i=1;i<n;i++)
    {
        j=i-1;
        k1=at[i];
```

```
        k2=b[i];
while (at[j]>k1&& j>=0)
{
    at[j+1]=at[j];
    b[j+1]=b[j];
    j--;
}
at[j+1]=k1;
b[j+1]=k2;
}
ct[0]=b[0];
for(i=1;i<n;i++)
ct[i]=ct[i-1]+ b[i];
for(i=0;i<n;i++)
{
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-b[i];
    s1=s1+tat[i];
    s2=s2+wt[i];
}

avg_tat=(float)s1/n;
avg_wt=(float)s2/n;
printf("Average turn around time is %f\n",avg_tat);
printf("Average waiting time is %f\n",avg_wt);
}
```

EXPERIMENT 2

Program Name: WAP to implement Non-preemptive Shortest Job First Scheduling Algorithm.

Theory Concept:

Key concept of this algorithm is: "CPU is allocated to the process with least CPU-burst time." Amongst the processes in the ready queue, CPU is always assigned to the process with least CPU burst requirement. If the two processes having the same length, next CPU burst, FCFS scheduling is used i.e. one which arrives first, will be taken up first by the CPU.

Implementation:

```
#include<stdio.h>
int process[3][10]={0},num;
int getMin(int t)
{
    int i=0,mi=10000,x=-10;
    for(i=0; i<num; i++)
```

```

{
    if(process[2][i]!=-1 && process[1][i]<mi && process[0][i]<=t)
    {
        mi=process[1][i];
        x=i;
    }
}
return x;
}

```

```

void main()
{
    int i,m,t[2][10]={0},sum=0;
    float turnavg=0.0,waitavg=0.0;
    printf("\n Enter the number of process max 10 = ");
    scanf("%d",&num);
    for(i=0; i<num; i++)
    {
        printf("\n Enter the arrival time for P%d = ",i);
        scanf("%d",&process[0][i]);
        printf("\n Enter the burst time for P%d = ",i);
        scanf("%d",&process[1][i]);
    }
    for(i=0; i<num; i++)
    {
        X: m=getMin(sum);
        if(m==--10)
        {
            sum+=1;
            goto X;
        }
        process[2][m]=-1;
        sum+=process[1][m];
        t[0][m]=sum-process[0][m];
        turnavg+=t[0][m];
        t[1][m]=t[0][m]-process[1][m];
        waitavg+=t[1][m];
    }
    for(i=0; i<num; i++)
    {
        printf("\n Turn around time for P%d = %d",i,t[0][i]);
        printf("\n Waiting time for P%d = %d",i,t[1][i]);
    }
    printf("\n Average turn around time = %f ",turnavg/num);
    printf("\n Average waiting time = %f ",waitavg/num);
}

```

EXPERIMENT 3

Program Name: WAP to implement Preemptive Shortest Job First Scheduling Algorithm

Theory Concept:

The SJF algorithm can be either preemptive or non-preemptive. The choice arises when a new process arrives at the ready queue while a previous process is still executing. The next CPU burst of the newly arrived process may be shorter than what is left of the currently executing process. A preemptive SJF algorithm will preempt the currently executing process, whereas a nonpreemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called shortest-remaining-time-first scheduling.

Implementation:

```
#include<conio.h>
#include<stdio.h>
main()
{
    int i,j,n,process[5][50],k=0,temp,t=0,m;
    float tatime=0.0,wait=0.0;
    printf("Enter the number of processes=");
    scanf("%d",&n);
    printf("Enter the burst time of each process:-\n");
    for(i=0;i<n;i++)
    {
        process[0][i]=i+1;
        printf("Enter the arrival time for process %d=",i+1);
        scanf("%d",&process[1][i]);
        printf("Enter the burst time for process %d=",i+1);
        scanf("%d",&process[2][i]);
        process[3][i]=0;
        process[4][i]=0;
        k=k+process[2][i];
    }
    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if (process[2][j]>process[2][j+1])
            {
                temp=process[1][j];
                process[1][j]=process[1][j+1];
```

```

        process[1][j+1]=temp;

temp=process[0][j];
    process[0][j]=process[0][j+1];
    process[0][j+1]=temp;
temp=process[2][j];
    process[2][j]=process[2][j+1];
    process[2][j+1]=temp;
}
}
}
for(i=0;i<n-1;i++)
{
    for(j=0;j<n-i-1;j++)
    {
        if(process[1][j]>process[1][j+1])
        {
            temp=process[1][j];
            process[1][j]=process[1][j+1];
            process[1][j+1]=temp;

            temp=process[0][j];
            process[0][j]=process[0][j+1];
            process[0][j+1]=temp;

            temp=process[2][j];
            process[2][j]=process[2][j+1];
            process[2][j+1]=temp;
        }
    }
}

printf("\n\n process\tArrival time\tBurst time");
printf("\n-----");
for(i=0;i<n;i++)
printf("\n\n process  %d\t\t%d\t\t%d",process[0][i],process[1][i],process[2]
[i]);

Printf("\n-----\n\n");
for(i=0;i<k;i++) //My premotive sif code block
{
    printf("\nProcess  %d is running,,,,,",process[0][i]);

```

```
    process[2][t]=process[2][t]-1;
    process[4][t]=i+1;

    for(m=0;m<n;m++)
    {
        if(process[2][m] !=0 && m!=t)
            process[3][m]=i+1; //code not match process[3][m]=i+2 in other
    }
    for(j=0;j<n;j++)
    {
        if (process[2][t]>process[2][j] && process[1][j]<=i+1 && process[2][j]!=0)
        {
            t=j;
        }
        if (process[2][t]==0)
        {
            t=j;
        }
    }
}
printf(«\n-----\n\n»);

for(i=0;i<n;i++) //cacculating turn Around Time
{
    process[4][i]=process[4][i]-process[1][i];
    printf("\nTurn around time for process %d is= %d",process[0][i],process[4][i]);
    tatime=tatime+process[4][i];
}

tatime=tatime/n;

printf("\n\nAverage turn around time=%.3f\n",tatime);

printf("\n-----\n\n");

for(i=0;i<n;i++) //calculating turn around time
{
    process[3][i]=process[3][i]-process[1][i]; //code change process[3][i]=process[3][i]
    printf("\nWaiting time for process %d is= %i",process[0][i],process[3][i]);
```

```

wait=wait+process[3][i];
}
wait=wait/n;
printf("\nAverage waiting time for process is=%.3i",wait);

getch();
}

```

EXPERIMENT 4

Program Name: WAP to implement Round Robin Scheduling Algorithm.

Theory Concept:

In the round robin scheduling, processes are dispatched in a FIFO manner but are given a limited amount of CPU time called a time-slice or a quantum. If a process does not complete before its CPU-time expires, the CPU is preempted and given to the next process waiting in a queue. The preempted process is then placed at the back of the ready list. Round robin scheduling is preemptive (at the end of time-slice) therefore it is effective in time-sharing environments in which the system needs to guarantee reasonable response times for interactive users.

Implementation:

```

#include<stdio.h>
#include<conio.h>
int a[2][10],t=0,ct[2][20],n,tat[5],wt[5],q;
int search(int s)
{
int pos,i;
for(i=0;i<t;i++)
{
if(ct[0][i]==s)
pos=i;
}
return pos;
}
void sort()
{
int i,j,k1,k2;
for(i=1;i<n;i++)
{
k1=a[0][i];
k2=a[1][i];

```

```
j=i-1;
while(k1<a[0][j]&&j>=0)
{
a[0][j+1]=a[0][j];
a[1][j+1]=a[1][j];
j--;
}
a[0][j+1]=k1;
a[1][j+1]=k2;
}
}
void main()
{
int sum=0,s=0,i=0,r=0,b[2][10];
float stat=0.0,swt=0.0;
printf("enter the no of processes ");
scanf("%d",&n);
printf("Enter the quantum ");
scanf("%d",&q);
for(i=0;i<n;i++)
{
printf("enter the burst time of process p%d :",i);
scanf("%d",&a[1][i]);
printf("enter the arrival time of process p%d :",i);
scanf("%d",&a[0][i]);
b[0][i]=a[0][i];
b[1][i]=a[1][i];
}
sort();
for(i=0;i<n;i++)
sum=sum+a[1][i];
ct[1][t-1]=0;
while(s!=sum)
{
if(a[1][r]>0&&a[0][r]<=s)
{
ct[0][t]=r;
if(a[1][r]>=q)
{ct[1][t]=ct[1][t-1]+q;
a[1][r]-=q;
s=s+q;
}
```



```
else
{
    ct[1][t]=ct[1][t-1]+a[1][r];
        s=s+a[1][r];
        a[1][r]=0;
    }
    t++;
}
r=(r+1)%n;
}
for(i=0;i<n;i++)
{
    tat[i]=ct[1][search(i)]-b[0][i];
    wt[i]=tat[i]-b[1][i];
    stat+=tat[i];
    swt+=wt[i];
}
float avg_tat=stat/n;
float avg_wt=swt/n;
printf("\nAverage Turn Around Time is %f ",avg_tat);
printf("\nAverage Waiting Time is %f ",avg_wt);
getch();
}
```

EXPERIMENT 5

Program Name: WAP to implement Priority Based Scheduling Algorithm.

Theory Concept:

Allow CPU to be given preferentially process.

In priority scheduling, processes are allocated to the CPU on the basis of an externally assigned priority. The key to the performance of priority scheduling is in choosing priority for the processes.

Implementation:

```
#include<stdio.h>
int a[5][10]={0},num;
int getMin(int t)
{
    int i=0,mi=10000,x=-10;
    for(i=0;i<num;i++)
    {
        if(a[3][i]!=-1&&a[4][i]<mi&&a[0][i]<=t)
        {mi=a[4][i];

```

```
        x=i;
    }
}
return x;
}
void main()
{   int t[2][10]={0};
    int i=0,sum=0,n=0,m=0,j=0;
    float avg_tat=0.0,avg_wt=0.0;
printf("\nEnter the no of processes ");
scanf("%d",&num);
for(i=0;i<num;i++)
{   printf("\nEnter the arrival time ");
    scanf("%d",&a[0][i]);
    printf("\nEnter the Burst Time ");
    scanf("%d",&a[1][i]);
    printf("Enter the priority ");
    scanf("%d",&a[4][i]);
    a[2][i]=a[1][i];
}

while(n!=num)
{m=getMin(sum);
sum+=1;
if(m!=-10)
{
    a[2][m]=a[2][m]-1;
    if(a[2][m]==0)
        {n++;
        a[3][m]=-1;
        t[0][m]=sum-a[0][m];
        avg_tat+=t[0][m];
        t[1][m]=t[0][m]-a[1][m];
        avg_wt+=t[1][m];
        }
    }
}
for(i=0;i<num;i++)
{printf("\n Turn Around Time for p%d=%d",i,t[0][i]);
printf("\n Waiting Time for p%d=%d",i,t[1][i]);
}
printf("\n Turn Around Time is %f",avg_tat/num);
```

```
printf("\n Waiting Time is %f",avg_wt/num);
}
```

EXPERIMENT 6

Program Name: WAP to implement Banker's Deadlock Avoidance and Detection Algorithm

Theory Concept:

Banker's Algorithm

- multiple instances of resource types IMPLIES cannot use resource-allocation graph
- banks do not allocate cash unless they can satisfy customer needs when a new process enters the system
- declare in advance maximum need for each resource type
- cannot exceed the total resources of that type
- later, processes make actual request for some resources
- if the allocation leaves system in safe state grant the resources
- otherwise, suspend process until other processes release enough resources

Banker: Data Structures

```
#define MAXN 10 /* maximum number of processes */
#define MAXM 10 /* maximum number of resource types */
int Available[MAXM]; /* Available[j] = current # of unused resource j */
int Max[MAXN][MAXM]; /* Max[i][j] = max demand of i for resource j */
int Allocation[MAXN][MAXM]; /* Allocation[i][j] = i's current allocation of j */
int Need[MAXN][MAXM]; /* Need[i][j] = i's potential for more j */
/* Need[i][j] = Max[i][j] - Allocation[i][j] */
```

Notation:

$X \leq Y$ iff $X[i] \leq Y[i]$ for all i

(0, 3, 2, 1) is less than (1, 7, 3, 2)

(1, 7, 3, 2) is NOT less than (0, 8, 2, 1)

Each row of *Allocation* and *Need* are vectors: *Allocation_i* and *Need_i*

Banker: Example

Initially:

Available

A B C

10 5 7

Later Snapshot:

$\text{Max} - \text{Allocation} = \text{Need Available}$

	A B C	A B C	A B C	A B C
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

Banker: Safety Algorithm

- consider some sequence of processes
- if the first process has Need less than Available
- it can run until done
- then release all of its allocated resources
- allocation is increased for next process
- if the second process has Need less than Available
- then all of the processes will be able to run eventually
- IMPLIES system is in a **safe state**

Banker: Safety Algorithm

STEP 1: initialize

```
Work := Available;
for i = 1,2,...,n
  Finish[i] = false
```

STEP 2: find i such that both

- Finish[i] is false
 - Need_i <= Work
- if no such i, goto STEP 4

STEP 3:

```
Work := Work + Allocation_i
Finish[i] = true
goto STEP 2
```

STEP 4:

if Finish[i] = true for all i, system is in safe state

Banker: Safety Example

Using the previous example, P1,P3,P4,P2,P0 satisfies criteria.

Max - Allocation = Need <= Work Available

	A B C	A B C	A B C	A B C	A B C
P1	3 2 2	2 0 0	1 2 2	3 3 2	3 3 2
P3	2 2 2	2 1 1	0 1 1	5 3 2	
P4	4 3 3	0 0 2	4 3 1	7 4 3	

P2	9 0 2	3 0 2	6 0 0	7 4 5
P0	7 5 3	0 1 0	7 4 3	10 4 7
			10 5 7<<<	initial system

Banker: Resource-Request Algorithm

STEP 0: P_i makes Request_i for resources, say (1,0,2)

STEP 1: if Request_i ≤ Need_i

goto STEP 2

else ERROR

STEP 2: if Request_i ≤ Available

goto STEP 3

else suspend P_i

STEP 3: pretend to allocate requested resources

Available := Available - Request_i

Allocation_i := Allocation_i + Request_i;

Need_i := Need_i - Request_i

STEP 4: if pretend state is SAFE

then do a real allocation and P_i proceeds

else

restore the original state and suspend P_i

Banker: Resource-Request Algorithm

Say P1 requests (1,0,2)

Compare to Need₁: (1,0,2) ≤ (1,2,2)

Compare to Available: (1,0,2) ≤ (3 3 2)

Pretend to allocate resources:

Max - Allocation = Need Available

	A B C	A B C	A B C	A B C
P0	7 5 3	0 1 0	7 4 3	2 3 0<<<
P1	3 2 2	3 0 2<<<	0 2 0<<<	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

Is this safe? Yes: P1, P3, P4, P0, P2

Can P4 get (3,3,0)? No, (3,3,0) > (2,3,0) Available

Can P0 get (0,2,0)? (0,2,0) < (2,3,0) Available

Pretend: Available goes to (2,1,0)

But ALL Needs are greater than Available IMPLIES NOT SAFE

Implementation:

```
#include<stdio.h>
void main()
{
    int n,i,alloc[10][10],max[10][10],avail[10][10],a[10],need[10]
[10],sum[4]={0,0,0,0},p[5];
    printf("Enter the no. of Processes\n");
    scanf("%d",&n);
    printf("Now enter the Total no. of resources of type A,B,C,D\n");
    for(i=0;i<4;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("Now enter no. of allocated resources of A\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&alloc[0][i]);
        sum[0]=sum[0]+alloc[0][i];
    }
    printf("Now enter no. of allocated resources of B\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&alloc[1][i]);
        sum[1]=sum[1]+alloc[1][i];
    }
    printf("Now enter no. of allocated resources of C\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&alloc[2][i]);
        sum[2]=sum[2]+alloc[2][i];
    }
    printf("Now enter no. of allocated resources of D\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&alloc[3][i]);
        sum[3]=sum[3]+alloc[3][i];
    }
    printf("Now enter the no. of maximum resources of A\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&max[0][i]);
```

```
}
printf("Now enter the no. of maximum resources of B\n");
for(i=0;i<n;i++)
{
scanf("%d",&max[1][i]);
}
printf("Now enter the no. of maximum resources of C\n");
for(i=0;i<n;i++)
{
scanf("%d",&max[2][i]);
}
printf("Now enter the no. of maximum resources of D\n");
for(i=0;i<n;i++)
{
scanf("%d",&max[3][i]);
}
for(i=0;i<4;i++)
{
avail[i][0]=a[i]-sum[i];
}
printf("Now resource available for A=%d\t\n",avail[0][0]);
printf("Now resource available for B=%d\t\n",avail[1][0]);
printf("Now resource available for C=%d\t\n",avail[2][0]);
printf("Now resource available for D=%d\t\n",avail[3][0]);
for(i=0;i<n;i++)
{
need[0][i]=max[0][i]-alloc[0][i];
}
for(i=0;i<n;i++)
{
need[1][i]=max[1][i]-alloc[1][i];
}
for(i=0;i<n;i++)
{
need[2][i]=max[2][i]-alloc[2][i];
}
for(i=0;i<n;i++)
{
need[3][i]=max[3][i]-alloc[3][i];
}
printf("The need is...\n");
```

```
printf("A B C D\n");
for(i=0;i<n;i++)
{

printf("%d\t",need[0][i]);
printf("%d\t",need[1][i]);
printf("%d\t",need[2][i]);
printf("%d\t",need[3][i]);
printf("\n");
}
for(i=0;i<n;i++)
{
printf("Alloc=%d %d %d %d\n",alloc[0][i],alloc[1][i],alloc[2][i],alloc[3][i]);
}
for(i=0;i<n;i++)
{
if(need[0][i]<=avail[0][i]&&need[1][i]<=avail[1][i]&&need[2][i]<=avail[2][i]&&need[3][i]<=avail[3][i])
{
printf("Executing P[%d]\n",i);
avail[0][i+1]=avail[0][i]+alloc[0][i];
avail[1][i+1]=avail[1][i]+alloc[1][i];
avail[2][i+1]=avail[2][i]+alloc[2][i];
avail[3][i+1]=avail[3][i]+alloc[3][i];
printf("«Avail= %d %d %d %d\n»,avail[0][i+1],avail[1][i+1],avail[2][i+1],avail[3][i+1]);
}
else
{
p[i]=i;
avail[0][i+1]=avail[0][i];
avail[1][i+1]=avail[1][i];
avail[2][i+1]=avail[2][i];
avail[3][i+1]=avail[3][i];
}
}
printf("Thus Finally executing...");
for(i=0;i<n;i++)
{
if(p[i]==i)
{
```

```
printf("P[%d]\n",i);
}
}
}
```

EXPERIMENT 7

Program Name: WAP to implement FIFO Page Replacement Algorithm

Theory Concept:

Select the page that has been in the main memory for the longest time.

Use a queue (data structure)

Problem: although a page has been present for a long time, it may be really useful

Create a pool of the pages that have been remarked for removal.

Manage the pool in the same way as the rest of the pages

If a page is needed, take a page from the pool.

If a page in the pool is referenced again before being replaced in memory, it is simply reactivated. This is very efficient.

ALGORITHM

1. Start the process
2. Declare the size with respect to page length
3. Check the need of replacement from the page to memory
4. Check the need of replacement from old page to new page in memory
5. Form a queue to hold all pages
6. Insert the page require memory into the queue
7. Check for bad replacement and page fault
8. Get the number of processes to be inserted
9. Display the values
10. Stop the process

Implementation:

```
#include<stdio.h>
int main()
{
int
i,j,n,a[50],frame[10],no,k,avail,count=0;
printf("\n ENTER THE NUMBER OF PAGES:\n");
scanf("%d",&n);
printf("\n ENTER THE PAGE NUMBER :\n");
for(i=1;i<=n;i++)
scanf("%d",&a[i]);
```

```
printf("\n ENTER THE NUMBER OF FRAMES :");
scanf("%d",&no);
for(i=0;i<no;i++)
    frame[i]= -1;
j=0;
printf(«\tref string\t page frames\n»);
for(i=1;i<=n;i++)
{
    printf("%d\t\t",a[i]);
    avail=0;
    for(k=0;k<no;k++)
        if(frame[k]==a[i])
            avail=1;
    if (avail==0)
    {
        frame[j]=a[i];
        j=(j+1)%no;
        count++;
        for(k=0;k<no;k++)
            printf("%d\t",frame[k]);

        printf("\n");
    }
    printf("Page Fault Is %d",count);
    return 0;
}
```

EXPERIMENT 8

Program Name: WAP to implement LRU Page Replacement Algorithm

Theory Concept:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least recently used page by counter value
7. Stack them according the selection.
8. Display the values

9. Stop the process

Implementation:

```
#include<stdio.h>
main()
{
    int q[20],p[50],c=0,c1,d,f,i,j,k=0,n,r,t,b[20],c2[20];
    printf("Enter no of pages:");
    scanf("%d",&n);
    printf("Enter the reference string:");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter no of frames:");
    scanf("%d",&f);
    q[k]=p[k];
    printf(«\n\t%d\n»,q[k]);
    c++;
    k++;
    for(i=1;i<n;i++)
    {
        c1=0;
        for(j=0;j<f;j++)
        {
            if(p[i]!=q[j])
                c1++;
        }
        if(c1==f)
        {
            c++;
            if(k<f)
            {
                q[k]=p[i];
                k++;
                for(j=0;j<k;j++)
                    printf(«\t%d»,q[j]);
                printf("\n");
            }
            else
            {
                for(r=0;r<f;r++)
                {
                    c2[r]=0;
                    for(j=i-1;j<n;j--)
                    {
                        if(q[r]!=p[j])
                            c2[r]++;
                    }
                }
            }
        }
    }
}
```

```
                else
                break;
            }
        }
        for (r=0;r<f;r++)
        b[r]=c2[r];
        for (r=0;r<f;r++)
        {
            for (j=r;j<f;j++)
            {
                if (b[r]<b[j])
                {
                    t=b[r];
                    b[r]=b[j];
                    b[j]=t;
                }
            }
        }
        for (r=0;r<f;r++)
        {
            if (c2[r]==b[0])
            q[r]=p[i];
            printf("\t%d",q[r]);
        }
        printf("\n");
    }
}
printf("\nThe no of page faults is %d",c);
}
```

EXPERIMENT 9

Program Name: WAP to Simulate Paging technique of Memory Management

Theory Concept:

Step 1: Start the program.

Step 2: Enter the logical memory address.

Step 3: Enter the page table which has offset and page frame.

Step 4: The corresponding physical address can be calculated by,

$$PA = [\text{pageframe} * \text{No. of page size}] + \text{Page offset.}$$

Step 5: Print the physical address for the corresponding logical address.

Step 6: Terminate the program.

Implementation:

```

#include<stdio.h>
int main()
{
int lpage[10], pagetab[10], pframe[2][2], I,j;
int size, msize, cnt, t,k,pos,logadd,m=0;
int frameno;
long int l;
int i;
printf("\nEnter the memory size:");
scanf("%d", &msize);
printf("Enter the pase size:");
scanf("%d", &size);
cnt=(msize/size);
printf("\nEnter the logical page: ");
for(j=0;j
{
scanf("%d", &lpage[j]);
}
t=0;
k=0;
for(j=0;j
{
printf("\n");
if(t
{
printf("%d value is %d\n", t,lpage[j]);
t++;
}
else
{
t=0;
printf(«%d value is %d\n», t,lpage[j]);
t++;
}
}
printf("\nEnter the page table value: ");
for(i=0;i
{
scanf("%d", &pagetab[i]);

```

```
}
printf("\nEnter the logical address: ");
scanf("%d", &logadd);
for(k=0;k
{
if(logadd==lpage[k])
{
pos=(k/size);
m=k%size;
break;
}
else
continue;
}
printf("\nPage number : %d", pos);
frameno=pagetab[pos];
printf("\nThe frame number is %d\n", frameno);
printf("\nThe corresponding physical address is %d", (frameno*size)+m);
}
```

EXPERIMENT 10

Program Name: WAP to implement FIFO Disk Scheduling Algorithm.

Theory Concept:

The process running on a machine may have multiple outstanding requests for data from the disk. In what order should request be processed is done with the help of this.

Implementation:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
#include<graphics.h>
main()
{
    int gd=DETECT,gm,x,y,a1,x1,y1,a,b;
    int req[30],head,i,k,move,n,ch,t1,t2,max,min,g[30],c,h;
    initgraph(&gd,&gm,"z:\\tcpp\\bgi");
    /*int req[30],head,i,k,move,n,ch,t1,t2,max,min,g[30],c,h;*/
    clearviewport();
    printf("(1) LOOK(2) CLOOK(3) FCFS(0) EXIT Enter your choice: ");
    scanf("%d",&ch)
    while(ch)
    {
```

```

printf("Enter the current head position: ");
scanf("%d",&head); h=head;
printf("Enter the number of requests: ");
scanf("%d",&n); k=0; move=0;
printf("Total number of cylinders: 200.");
printf("Enter the requests (1 to 200):");
for(i=0;i<n;i++)
{
    scanf("%d",&req[i]);
    if(req[i]>200||req[i]<1)
    { printf("Enter requests only between 1-200."); goto
req; }
}
switch(ch)
{
case 1:
outtextxy(270,10,"SEEK PATTERN FOR LOOK");
max=req[0]; min=req[0]; t2=1; c=0;
for(i=1;i<n;i++)
{
    max=max>req[i]?max:req[i];
    min=min<req[i]?min:req[i];
}
if(max<=head) t2=0;
while(k<n)
{
    t1=0;
    for(i=0;i<n;i++)
    {
        if(t2)
        {
            if(req[i]&&req[i]>=head)
            {
                if(req[i]<req[t1]) t1=i;
                if(req[i]==max)
                { t2=0; g[c]=1; c++; }
            }
        }
        else
        {
            if(req[i]&&req[i]<=head)
            {
                if(req[i]>req[t1]) t1=i;
                if(req[i]==min)
                { t2=1; g[c]=200; c++; }
            }
        }
    }
}

```

```
        }/*for*/
        printf(«%d »,req[t1]);
        g[c]=req[t1]; c++;
        move+=abs(head-req[t1]);
        req[t1]=0; k++;
        head=req[t1];
        if(head==max) t2=0;
    }/*while*/
    break;
    case 2:
    outtextxy(270,10,"SEEK PATTERN FOR CLOOK");
    max=req[0]; min=req[0]; t2=1; c=0;
    for(i=1;i<n;i++)
    {
        max=max>req[i]?max:req[i];
        min=min<req[i]?min:req[i];
    }
    while(k<n)
    {
        t1=0;
        if(!t2) head=min;
        for(i=0;i<n;i++)
        {
            if(req[i]&&req[i]>=head)
            {
                if(req[i]<req[t1]) t1=i;
                else if(req[i]==max)
                { head=1; g[c]=1; c++; }
            }
        }
        }/*for*/
        printf("%d ",req[t1]);
        move+=abs(head-req[t1]);
        head=req[t1];
        g[c]=req[t1]; c++;
        req[t1]=0; k++;
        if(head==max) t2=0;
    }/*while*/
    break;
    case 3:
    outtextxy(270,10,"SEEK PATTERN FOR FCFS");
    for(i=0,c=0;i<n;i++,c++)
    {
        printf("%d ",req[i]);
        move+=abs(head-req[i]);
        head=req[i];
        g[c]=req[i];
    }
    break;
```

```

    }/*switch*/
    printf("Order of servicing:");
    printf("%d: ",h);
    for(i=0;i<n;i++)
        printf("%d ",g[i]);
    printf("Total Head Movements: %d",move);
    x=getmaxx();
    y=getmaxy();
    rectangle(0,20,x-5,y-5);
    a1=(x-30)/10;
    b=110; a=h+(3*a1); y1=125;
    fillellipse(a,b,2,2);
    for(i=0;i<n;i++)
    {
        int x1=g[i]+(3*a1);
        fillellipse(x1,y1,2,2);
        line(a,b,x1,y1);
        a=x1; b=y1;
        y1+=15;
    }
    getch();
    clrscr(); clearviewport();
    printf("(1) LOOK    (2) CLOOK    (3) FCFS    (0) EXIT
           Enter your choice:");
    scanf("%d",&ch);
}/*while*/
getch();
}/*main*/

```



Previous University Question Papers

B. Tech.

(Sem. IV) (Even Sem.) Theory

Examination, 2014-15

Operating System

(U.P.T.U., Lucknow)

Time: 3 Hours

Total Marks: 100

Note: Attempt all questions. All questions carry equal marks.

1. Attempt any **four** questions: (4*5 = 20)
 - (a) Describe the differences between symmetric and asymmetric multiprocessing?
 - (b) Discuss various operating system components.
 - (c) What are the difference between cell and kernel?
 - (d) Explain layered structure of an operating system. Also explain advantages and disadvantages of the layered approach to system design.
 - (e) What is spooling?
 - (f) Explain the principle of concurrency?
2. Attempt any **two** questions: (2*10 = 20)
 - (a) What do you understand by critical section? Discuss Bakery algorithm. Also show how it satisfies the requirement of a mechanism to control access to critical section.
 - (b) Explain the following terms briefly:
 - (i) Dekker's solution
 - (ii) Busy Waiting
 - (c) A barber chair consists of waiting room with n chairs and a barber room with 1 chair. If there are no customers to be served, the barber goes to sleep. If a customer enters the barbershop and all chairs are occupied then customer leaves the shop. If the barber is busy and chairs are available then customer sits in one of the free chairs. If the barber is asleep then customer wakes up the barber. Write an algorithm for the above synchronization problem using semaphores.

3. Attempt any **two** questions. (2*10 = 20)

- (a) Explain the following scheduling algorithm.
- Multilevel feedback queue-scheduling
 - Multiprocessor scheduling
- (b) Describe Banker's algorithm for deadlock avoidance. Consider a system with three processes and three resources. the snapshot of a system at a time is given below:

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P1	2	2	3	3	6	8	7	7	10
P2	2	0	3	4	3	3			
P3	1	2	4	3	4	4			

Determine the following:

- Is the current allocation in safe state?
 - Would the following requests be granted in the current state?
 - Process P2 requests (1,0,0)
 - Process P1 requests (1,0,0)
 - Find the need matrix
- (c) What is a thread? How thread is different from a process? What resources are used when a thread is created?
4. Attempt any **Two** of the following: (2*10=20)

- (a) On a system using paging and segmentation, the virtual address space consists of up to 16 segments where each segment can be up to 2^{16} bytes long. The hardware pages each segment into 512 bytes pages. How many bits in the virtual address specify the following?
- Segment number
 - Page number
 - Offset within page
 - Entire virtual address
- (b) Explain segmentation with diagram?
- (c) How many page faults would occur for the following reference string for four page frames using LRU and FIFO algorithms:
- 1,2,3,4,5,5,3,4,1,6,7,8,7,8,9,7,8,9,5,4,5,4,2

5. Attempt any **two** questions. (2*10 = 20)

- (a) Discuss the following terms.
- Access matrix
 - Boot Blocks
- (b) Describe schemes for defining logical structure of directory.
- (c) Discuss disk scheduling algorithms with example.

B. Tech.
(Sem. V) (Odd Sem.) Theory
Examination, 2014-15
Operating System
(U.P.T.U., Lucknow)

Time: 3 Hours

Total Marks: 100

- Note:** (i) Attempt **all** Questions
(ii) All questions carry equal marks
(iii) Be precise in your answer

1. Attempt any **four**: **4 × 5 = 20**
 - (a) Enumerate various OS components with their functions in brief.
 - (b) Why operating system is needed? Justify “operating system is the control program of the system”.
 - (c) Differentiate between (with one suitable example):
 - (i) Interactive and batch processing system.
 - (ii) Multiprogramming and time sharing system.
 - (d) Explain the difference between monolithic and microkernel systems with their advantages and disadvantages. What are the benefits of having kernel as re-entrant in the systems?
 - (e) Enumerate the advantages of the layered approach to system design? Also, explain virtual machine architecture in detail.
 - (f) What are semaphores? What is the usage of semaphores? Explain with a suitable example.
2. Attempt any **four**: **4 × 5 = 20**
 - (a) Draw and explain Process Control Block (PCB) with all its components. Also brief why context switching is an overhead in the system?
 - (b) Explain short-term, medium-term and long-term scheduling. Describe the differences among them.

- (c) Give the constraints given by Dijkstra that are imposed on any solution for critical section problem. Also, give Dekker's solution for two processes and check whether it satisfies the above constraints.
- (d) Give reader/writer problem and its solution using semaphores.
- (e) Draw the labelled process state transition diagram describing the various states. Also, give the different reasons for process termination.
- (f) Explain the need of process synchronization. How can the interprocess communication be achieved?
3. Attempt any two: $2 \times 10 = 20$
- (a) Consider the following set of processes:

Process	Arrival time	Burst time	Priority
P1	0	6	3
P2	1	4	1
P3	2	5	2
P4	3	8	4

- (i) Draw Gantt charts and find average waiting time and average turnaround time using shortest remaining time first (SRTF), round robin (time quantum = 3) and preemptive priority scheduling.
- (ii) If the scheduler takes 0.2 unit of CPU time in context switching, calculating the percentage of CPU time wasted in each case.
- (b) (i) What are deadlocks? Explain the different methods for dealing with the deadlocks.
- (ii) Consider the following snapshot of a system:

	Allocation			Max			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	2	3	3	6	8	4	3	10
P2	2	0	3	4	3	3			
P3	1	2	4	3	4	4			

Answer the following questions using the Banker's algorithm:

- (a) What is the content of the matrix Need?
- (b) Is the system in a safe state?
- (c) If a request from process P2 arrives for (2, 1, 0), can the request be granted immediately?
- (c) What do you mean by a thread? How it is different from a process? Explain various thread models with relative advantages and disadvantages.

4. Attempt any two: $2 \times 10 = 20$
- (a) (i) Explain the difference between internal and external fragmentation by taking suitable example. Which one occurs in pure paging, pure segmentation, demand paging and paged segmentation?
 - (ii) What is thrashing? When it does occur? Describe the actions taken by the operating system when a page fault occurs.
 - (b) (i) What do you mean by Belady's anomaly? Which algorithm suffers from Belady's anomaly?
 - (ii) Consider the following page reference string:
1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 5, 6
How many page faults would occur for the following replacement algorithms, assuming three frames.
 - (1) LRU page replacement
 - (2) FIFO page replacement
 - (3) Optimal page replacement
 - (c) Explain paged segmentation with its advantages and disadvantages.
In a paged segmented system, a virtual address consists of 32 bits of which 12 bits are for displacement, 11 bits are segment number and 9 bits are page number. Calculate the following:
 - (a) Page size
 - (b) Max segment size
 - (c) Max number of pages
 - (d) Max number of segments.
5. Attempt any two: $2 \times 10 = 20$
- (a) Discuss various file allocation strategies for disk space management. What criteria should be used in deciding which strategy is best utilized for a particular file?
 - (b) Suppose that a disk has 500 cylinders. The drive is currently serving a request at cylinder 125 and the previous request was at cylinder 100. The queue of pending request in FIFO order is
80, 147, 13, 177, 48 409, 22, 175, 301
What is the total distance that the disk arm moves for the following disk scheduling algorithm:
 - (1) SSTF
 - (2) LOOK
 - (3) C-SCAN
 - (c) Different between:
 - (1) Blocking and non-blocking I/O.
 - (2) Block and character devices.
 - (3) Linked list and bit map approach for free space memory management.

B. Tech.
(Sem. V) (Odd Semester) Theory
Examination, 2013-14
Operating System
(U.P.T.U., Lucknow)

Time: 3 Hours

Total Marks: 100

Note: (i) Attempt all questions. All questions carry equal marks
(ii) Use suitable diagram wherever necessary.

1. Attempt any **four** of the following: (4 × 5 = 20)
 - (a) What is an operating system? Define the components of an operating system.
 - (b) What is real-time operating system? What is the difference between Hard real time and soft real-time operating systems?
 - (c) Define the Services provided by the operating system.
 - (d) Write down the difference between multiprocessing and multiprogramming operating systems.
 - (e) Explain the need of system protection with the help of examples.
 - (f) What do you understand by system call? Enumerate five system calls used in process management.
2. Attempt any **four** of the following: (4 × 5 = 20)
 - (a) Define the different states of process with diagram. Explain the need of process suspension.
 - (b) State and describe the producer-consumer problem with its suitable solution.
 - (c) List the essential requirements of critical section implementation.
 - (d) Define message passing and shared memory inter-process communication.
 - (e) Differentiate between user thread and kernel thread. What is thread cancellation?
 - (f) Define Semaphore with suitable examples.
3. Attempt any **two** of the following: (2 × 10 = 20)
 - (a) Consider the set of the processes given in the table and the following scheduling algorithms:
 - (i) Round robin (Quantum = 1)
 - (ii) Round robin (Quantum = 2)
 - (iii) Shortest remaining job first

Process Id	Arrival time	Execution time
A	0	4
B	2	7
C	3	3
D	3.5	3
E	4	5

If there is tie within the processes, the tie is broken in the favour of the oldest process. Draw the Gantt chart and find the average waiting time, response time and find the average waiting time, response time and turn-around time for the algorithms. Comment on your result. Which one is better and why?

- (b) Describe the Banker's algorithm for safe allocation. Consider a system with five processes and three resource types and time T_0 the following snapshot of the system has been taken:

Process Id	Allocation			Maximum			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	1	1	2	4	3	3	3	1	0
P2	2	1	2	3	2	2			
P3	4	0	1	9	0	2			
P4	0	2	0	7	5	3			
P5	1	1	2	11	2	3			

- (i) Determine the total amount of resources of each type.
 - (ii) Compute the need of matrix.
 - (iii) Determine if the state is safe or not using Banker's algorithm.
 - (iv) Would the following request be granted in the current state?
 - (a) P1 $\langle 3, 3, 1 \rangle$
 - (b) P2 $\langle 2, 1, 0 \rangle$
 - (c) (i) What are the necessary conditions to hold a deadlock in a system?
 (ii) What are the approaches that can be used for Prevention of deadlock?
4. Attempt any two of the following: (2 × 10 = 20)
- (a) (i) Consider a logical address space of eight pages of 1024 words, each mapped onto a physical memory of 32 frames then:
 - (a) How many bits are in logical address?
 - (b) How many bits are in physical address?
 - (ii) Explain the difference between internal and external fragmentations.
 - (b) Consider the following reference string 12 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3 6.
 How many page faults will occur for:

- (i) First-In-First-Out
 - (ii) LRU page replacement algorithm?
- Assuming three and four frames (initially empty) in each case.
- (c) (i) What is thrashing? State the cause of thrashing and discuss its solution.
 - (ii) Write the difference between paging and segmentation.
5. Attempt any **two** of the following: **(2 × 10 = 20)**
- (a) Suppose the moving head disk with 200 tracks is currently serving a request for track 143 and has just finished a request for track 125. If the queue of request is kept in FIFO Order 86 147 91 177 94 150. What is total head movement for the following scheduling:
 - (i) FCFS
 - (ii) SSTF
 - (iii) C-Scan?
 - (b) What is a Directory? Define any two ways to implement the directory.
 - (c) Write short notes on:
 - (i) I/O buffering.
 - (ii) Sequential file
 - (iii) Indexed file

B. Tech.
(Semester-V) Theory Examination, 2012-13
Operating System
(U.P.T.U., Lucknow)

Time: 3 Hours

Total Marks: 100

Section-A

1. Attempt **all** question parts. **2 × 10 = 20**
- (a) List the various services provided by operating system.
 - (b) What is the difference between multiprogramming, multitasking and multiuser operating system?
 - (c) List the difference between a time sharing system and real time system.
 - (d) Define monolithic kernel and microkernel.
 - (e) List the various conditions for deadlock occurrences.
 - (f) What do you understand by critical section?
 - (g) What is paging and segmentation?
 - (h) What is Access Matrix and how it is used for protection?
 - (i) Differentiate between 'ready to run process in memory management' and 'ready to run process swapped'.
 - (j) Explain system protection in context of operating system.

Section-B

2. Attempt any **three** question parts. **3 × 10 = 30**
- (a) What is Thrashing? Explain its advantages and disadvantages. Consider the following pages of a reference string:
1, 2, 0, 3, 5, 1, 5, 7, 2, 0, 3, 5, 4, 1, 2, 5, 3, 7
Implement FCFS, LRU and optimal page replacement algorithm and calculate the number of page fault and hit ratio by considering three frames in a block.
 - (b) What do you mean by parallel (multiprocessor) system? What are its main types? Explain.
 - (c) What is bounded buffer problem? Discuss briefly with the help of producer and consumer process.
 - (d) Consider the following process:

Process	Arrival time	Burst time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

Consider non-preemptive and preemptive SJF algorithm, find out average waiting time in both cases.

- (e) On a simple paged system, associative register holds the most active page entries and the full page table is stored in the main memory. If references satisfied by the associative registers take 90 ns and reference through the main memory page table take 20 ns. What is the effective access time if 60% of all memory references find their entries in the associative registers?

Section-C

- Attempt all questions. $5 \times 10 = 50$
3. Attempt any **two** parts. $(2 \times 5 = 10)$
- What are the different techniques to remove fragmentation in case of multiprogramming with fixed partitions and variable partitions?
 - How is system call made? How is a system call handled by the system? Choose suitable example for explanation.
 - What is the reason behind dual-mode operation of processors?
4. Attempt any **two** parts. $(2 \times 5 = 10)$
- What do you mean by concurrent process? Discuss the inter-process communication in details.
 - What is Banker's Algorithm of deadlock avoidance? Explain.
 - Write an algorithm to explain the producer/consumer using semaphores.
5. Attempt any **two** parts. $(2 \times 5 = 10)$
- Discuss the performance criteria for CPU scheduling.
 - Draw and explain the process state transition diagram.
 - Give the solution to critical section problem. Explain its necessary conditions.
6. Attempt any **one** part. $(1 \times 10 = 10)$
- Suppose we have a processor with a hardware cache memory. Suppose a program is run by the CPU and this produces 500,000 memory references, out of which 475,000 hits the cache memory. Suppose average memory access time is 70 ns and cache access time 20 ns, of which 1 ns are for lookup. Suppose the system following the write-through policy. (a) What is the hit ratio? (b) What is the average memory reference time as seen by the CPU if, (i) all the

memory references are read, (ii) 80% if the memory references are read, and (iii) 90% of the memory references are read?

- (b) In a segmentation-based system, the main memory has the following holes in this order: 21K, 5K, 90K, 54K, 10K, 25K and 56K; there are three new requests for memory of sizes 10K, 7K and 22K. The system does first come first serve service. Explain what holes will be taken for each of First Fit and Best Fit memory allocation schemes.

7. Attempt any **two** parts. (2 × 5 = 10)

- (a) What are the various file access methods? Explain sequential access method. How it can be simulated on direct access file?
- (b) What are the different file organizations? Discuss access mechanisms.
- (c) Describe the file system implementation and directory system implementation in detail.

B. Tech.
(Semester-V) Theory Examination, 2011-12
Operating System
(U.P.T.U., Lucknow)

Time: 3 Hours

Total Marks: 100

- Note:** (i) Attempt **all** questions.
(ii) All questions carry equal marks.
(iii) Notations/Symbols/Abbreviations used have usual meaning.
(iv) Make suitable assumptions, wherever required.

1. Attempt any **two** parts of the following:
 - (a) Explain the following terms clearly:
 - (i) Process
 - (ii) Busy waiting
 - (iii) Multiprogramming
 - (iv) Multithreading
 - (b) Differentiate between the following:
 - (i) Shell and kernel
 - (ii) Deadlock and starvation
 - (iii) Monolithic kernel and microkernel
 - (iv) Concurrent execution and parallel execution.
 - (c) (i) What do you understand by system call? How is a system call made? How is a system call handled by the system? Choose suitable examples for explanation.
(ii) What is the reason behind dual-mode operation of processors?
2. Attempt any **two** parts of the following:
 - (a) What do you understand by critical section? What are the requirements of a solution? What are the requirements of a solution to the critical section problem.
Discuss mutual exclusion implementation with the help of test-and-set machine instruction.
 - (b) State the finite buffer producer-consumer problem. Give solution of the problem using semaphores.
 - (c) Write a short note on interprocess communication.

3. Attempt any **two** parts of the following:
- (a) (i) Draw and explain the process state transition diagram.
 - (ii) In a system, n processes share m resource units that can be reserved and released only one at a time. The maximum need of each process does not exceed m and sum of all maximum need is less than $m + n$. Show that a deadlock cannot occur.
 - (b) List various performance criteria for scheduling algorithms. Five processes A, B, C, D, require CPU burst of 3, 5, 2, 5 and 5 units respectively. Their arrival times in the system are 0, 1, 3, 9 and 12 respectively. Draw Gantt chart and compute the average turnaround time and average waiting time of these processes for the Shortest Job First (SJF) and Shortest Remaining Time First (SRTF) scheduling algorithms.
 - (c) Write and explain Banker's algorithm for avoidance of deadlock.
4. Attempt any **two** parts of the following:
- (a) What is paging? Describe how logical address is translated to physical address in a paged system. Further give reasons as to why page sizes are always kept in power of 2.
 - (b) Consider a demand paged system. Page tables are stored in main memory which has an access time of 100 nanoseconds. The TLB can hold 8 page table entries and has an access time of 10 nanoseconds. During the execution of a process, it is found that 80% of the time, a required page table entry exists in TLB and only 2% of the references cause page faults. The average time to service page fault is 2 milliseconds. Compute the effective memory access time.
 - (c) Discuss the LRU page replacement policy. Prove that this policy do not suffer from Belady's anomaly. How many page faults would occur with LRU policy for the following reference string:
1, 2, 3, 4, 1, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 8, 7, 8, 9, 5, 4, 4, 2
Assume that there are four frames which are initially empty.
5. Attempt any **two** parts of the following:
- (a) Explain the SSTF and SCAN disk scheduling policies. Obtain the total number of head movements needed to satisfy the following sequence of track requests for each of the two policies:
27, 129, 110, 186, 147, 41, 10, 64, 120
Assume that the disk head is initially positioned over track 100 and is moving in the direction of decreasing track number.
 - (b) Describe the various file allocation methods with their relative advantages and disadvantages.
 - (c) Write a short note on File System Protection.

End Term Examination
Fifth Semester (B. Tech.) December 2011
(GGSIPU, Delhi)

Paper Code: ETCS-305

Time: 3 Hours

Subject: Operating System

Maximum Marks: 75

Note: Attempt any five questions

Q.1. Attempt **all parts** of the following: **(3 × 5 = 15)**

- (a) Explain the batch processing system with its advantages.
- (b) Define a process. What are the cooperating processes?
- (c) What is role played by buffering in the device management?
- (d) How the swap-space management is carried out?
- (e) How the directory is implemented? List the steps required for recovery of a file system.

Q.2. Attempt **any one** part (a or b) of the following: **(12)**

- (a) Consider a set of 5 processes whose arrival time, CPU time needed and priority are given below. Assuming 1 to be the highest and 5 the lowest priority. Calculate the average waiting time, average turnaround time for the following scheduling policies.

Processes	Arrival time (ms)	CPU time (ms)	Priority
P1	0	10	5
P2	0	5	2
P3	2	3	1
P4	5	20	4
P5	10	2	3

- (A) SJF with preemption
- (B) Round robin with time quantum 3 ms.
- (C) Priority scheduling with preemption.

(i) How are the Inter-process communication is achieved? **(3)**

(b) Discuss the following: **(3 + 3 + 4 + 2 + 3)**

- (i) Various stages of a process.
- (ii) Monitors and their uses.
- (iii) Atomic transactions
- (iv) Synchronization in solaris 2.
- (v) Real-time scheduling.

Q.3. Attempt **any one** part (a or b) of the following: (15)

(a) Consider the following page references string:

5, 6, 7, 1, 2, 3, 5, 6, 7, 1, 2, 3, 5, 6, 7, 1, 8, 9, 9, 1, 2, 3, 4

How many page faults would occur for the following page replacement algorithms, assuming four frames? All frames are initially empty.

(i) LRU replacement (ii) FIFO replacement

(iii) Optimal replacement

(b) (i) Discuss the differences between paged-segmentation and segmented paging. (5)

(ii) Define and discuss the effects of thrashing. How can a system recover from the same? (5)

(iii) Define and discuss the distributed system and real-time systems with their advantages. (5)

Q.4. Attempt **any one** part (a or b) of the following: (3 × 5)

(a) Discuss the following with examples:

(i) Shared devices (ii) Channels and control units

(iii) Disk reliability (iv) Block multiplexing

(v) Deadlock handling

(b)

(i) What are the necessary and sufficient conditions for a deadlock to occur? (4)

(ii) Discuss the steps for recovery from a deadlock. (6)

(iii) Discuss any four disk scheduling algorithms with examples. (5)

Q.5. Attempt **any one** part (a or b) of the following:

(a)

(i) Discuss the general model of a file system. Explain the differences between a physical files system and logical file system? How the access verification is carried out in a file system? Explain with suitable examples. (12)

(ii) Discuss the techniques for consistency in a file system. (3)

(b) Explain the following: (5 × 3 = 15)

(i) Different allocation methods with their merits and demerits.

(ii) Free space management in the file system.

(iii) Directory Implementation and its performance evaluation.

B. Tech.
(Semester-V) ODD Semester Theory Examination, 2010-11
Operating System

[Time: 3 Hours]

[Total Marks: 100]

Note: Attempt **all** questions.

1. Attempt any **four** parts of the following: (5 × 4 = 20)
 - (a) What is an operating system? Discuss the main services of the operating system.
 - (b) Discuss the differences between a time sharing system and real time system.
 - (c) Discuss the objectives of the multiprocessor systems.
 - (d) What do you mean by kernel? Explain monolithic kernel and microkernel.
 - (e) Describe the steps involved in booting.
 - (f) Explain the following:
 - (i) Multitasking
 - (ii) Multithreading
2. Attempt any **two** parts of the following: (10 × 2 = 20)
 - (a) Write an algorithm to explain the producer/consumer using semaphores.
 - (b) (i) Compare and contrast the use of monitors and semaphores operations.
(ii) What is critical section? Discuss.
 - (c) Discuss one classical problem related to the process synchronisation.
3. Attempt any **two** parts of the following: (10 × 2 = 20)
 - (a) (i) Explain the need for Process Control Block (PCB).
(ii) Discuss the performance criteria for CPU scheduling.
 - (b) (i) Describe the necessary condition for deadlock to occur.
(ii) In the respect of Banker's Algorithm discuss whether system is safe or unsafe. If a system is safe, show how it is possible for all user to complete:

	Current loan	Maximum need
user (1)	2	6
user (2)	4	7
user (3)	5	6
user (4)	0	2
available	1	

- (c) Explain the following scheduling algorithm:
 - (i) Multilevel feedback queues scheduling
 - (ii) First-In-First-Out (FIFO) scheduling
- 4. Attempt any **two** parts of the following: (10 × 2 = 20)
 - (a) What do you understand by fragmentation? What are the different techniques to remove fragmentation in case of multiprogramming with fixed partitions and variable partitions? Discuss.
 - (b) Define virtual memory concepts and also discuss page replacement algorithms in brief.
 - (c) Write short notes on:
 - (i) Thrashing
 - (ii) Cache memory organisation
- 5. Attempt any **two** parts of the following: (10 × 2 = 20)
 - (a) What are the different file organizations? Discuss access mechanisms.
 - (b) Explain the following:
 - (i) Directory system
 - (ii) File protection
 - (c) Discuss the disk scheduling algorithms.

B. Tech.
(Semester-V) Even Semester Theory Examination, 2009-10
Operating System

Time: 3 Hours

Total Marks: 100

Note: (i) Attempt **all** the questions.
(ii) All questions carry equal marks

1. Attempt any **four** of the following: **(4 × 5 = 20)**
 - (a) Write down the advantages of batch processing system.
 - (b) What are the major functions of operating system?
 - (c) Explain the main features of an real time operating system.
 - (d) Draw the layered structure of an operating system.
 - (e) Draw the evolution of operating system.
 - (f) Write down about the following in brief:
 - (i) System protection
 - (ii) System components
2. Attempt any **four** parts of the following: **(4 × 5 = 20)**
 - (a) Draw the process state diagram and explain the state transaction.
 - (b) Provide the solution of critical section problem.
 - (c) Explain the binary semaphores with an example.
 - (d) What do you understand by concurrent process? Explain with an example.
 - (e) Write a brief note on inter-process communication.
 - (f) How concurrency problems are solved with producer and consumer problems?
3. Attempt any **four** parts of the following: **(4 × 5 = 20)**
 - (a) What do you understand by CPU scheduling? Which one is best and why?
 - (b) Calculate turnaround time and waiting time for following processes, if these processes are using:
 - (i) SJF
 - (ii) FCFS

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- (c) Explain the different conditions of deadlock.
 - (d) Write down the methods for deadlock prevention.
 - (e) Discuss about Multiprocessor scheduling in brief.
 - (f) How the recovery from deadlock is done using combined approach?
4. Attempt any **two** parts of the following: (2 × 10 = 20)
- (a) Discuss the paging system for memory management in details. Also give it, advantages and disadvantages.
 - (b) Discuss about following in details:
 - (i) Demand paging
 - (ii) Thrashing
 - (c) (i) What do you understand by Page replacement? Name the algorithm available for Page replacement.
 - (ii) How many Page faults occur for optimal Page replacement algorithm with following reference string for four page frames:
1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4, 5, 4, 2
5. Attempt any **two** parts of the following: (2 × 10 = 20)
- (a) Explain the file system and its function and how system calls are related with file system.
 - (b) Write short notes on following:
 - (i) Memory mapped I/O.
 - (ii) Direct Access Method for file.
 - (c) (i) Write down the criterion for selection of disk-scheduling algorithm.
 - (ii) Suppose, a disk have 5000 cylinders, numbered 0 to 4999. The drive is currently sending a request a cylinder 143 and the previous was a cylinder 125. The queue of pending request in FIFO order is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130.

End Term Examination

Fourth Semester (B. Tech.) May 2010

Paper Code: ETCS-212

Subject: Operating System & System Programming

Paper ID: 32212

Time: 3 Hours

Maximum Marks: 75

Note: Attempt any five questions

Q.1. Answer the following in brief: (2.5 × 10 = 25)

- (a) What are the main advantages of multiprogramming?
- (b) Define the essential properties of batch operating system.
- (c) What are the differences between a trap and an interrupt?
- (d) Define the differences between preemptive and non-preemptive scheduling.
- (e) What are the benefits and the detriments of each of the following? Consider both the system and the programmer's level.
 - (i) Symmetric and asymmetric communication.
 - (ii) Fixed-sized and variable-sized messages.
- (f) Discuss three major complications that concurrent processing adds to an operating system.
- (g) Can "busy waiting" be avoided altogether? Explain your answer.
- (h) What is the cause of thrashing? How does the system detect thrashing?
- (i) In what situations would using memory as a RAM disk be more useful than using it as a disk cache?

Unit-I

Q.2.

- (a) Describe the following allocation algorithm: (4)
 - (i) First Fit (ii) Best Fit (iii) Worst Fit
- (b) Why are sometimes segmentation and paging combined into one scheme? (4)
- (c) Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs. (4.5)

OR

Q.3.

- (a) Why are page sizes always powers of '2'? Explain. (3)
- (b) Consider the following segment table: (5)

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	13327	580
4	1952	96

What are physical addresses for the following logical addresses?

(i) 0430 (ii) 110 (iii) 2500 (iv) 3400 (v) 4112

- (c) Under what circumstances would a user be better off using a time-sharing system, rather than a RC or single-user workstation? **(4.5)**

Unit-II

Q.3.

- What are the two differences between user-level threads and Kernel-level threads? Under what circumstances is one type better than the other? **(4)**
- Show that, if the 'wait' and 'signal' operations are not executed atomically, then mutual exclusion may be avoided (violated). **(3.5)**
- Suppose that a scheduling algorithm (at level of short term scheduling) favours those processes that have used the least processor time in the recent past. Why will this algorithm favour I/O bound programs and yet riot permanently starve CPU-bound programs? **(5)**

OR

Q.3.

- Explain the differences in degrees to which the following scheduling algorithms discriminate in favour of short processes: **(6.5)**
- Write a bounded buffer monitor in which the buffers (portions) are embedded within the monitor itself. **(6)**

Unit-III

Q.4.

- Consider a system consisting of six tape drives, with n processes competing for them. Each process may need two drives. For which values of n , is the system deadlock free. **(4.5)**
- Why is rotational latency usually not considered in disk scheduling? How would you modify SSTF, SCAN and C-SCAN to include latency optimization? **(8)**

OR

Q.4.

- (a) Describe three circumstances separately under which blocking I/O and non-blocking I/O should be used. Why not just implement non-blocking I/O and have processes busy-wait until their devices are ready? Explain. (4)
- (b) Is disk scheduling other than FCFS scheduling, useful in a single user environment? Explain your answer. (4.5)
- (c) Is it possible to have a deadlock involving only one process? Explain your answer. (4)

Unit-IV

Q.5.

- (a) Systems that support sequential files always have an operation to rewind files. Do systems that support random access files need this too? Why or why not, explain. (4.5)
- (b) Some systems provide file sharing by maintaining a single copy of a file, other systems maintain several copies, one for each of the users sharing the file. Discuss the relative merits of each approach. (4)
- (c) Why must the bit map for file allocation be kept on mass storage, rather than in main memory? Explain. (4)

OR

Q.5.

- (a) An operating system only supports a single directory but allows that directory to have arbitrarily many files with arbitrarily long file names. Can something approximating a hierarchical file system be simulated? How? (5)
- (b) Why is it advantageous to the user for an operating system to dynamically allocate its internal tables? What are the penalties to the operating system for doing so? (4)
- (c) Some systems support many types of structures for a file's data, while others simply support a stream of bytes. What are the advantages and disadvantages? (3.5)

B. Tech.
(Semester-VI) Examination, 2008-09
Operating System

[Time: 3 Hours]

[Total Marks: 100]

1. Attempt any **four**:
 - (a) Compare multitasking and multiuser operating system.
 - (b) What are the desirable and essential characteristics of an operating system?
 - (c) Explain in brief real-time operating system. Illustrate some areas where they are used.
 - (d) What are the different services provided by an operating system?
 - (e) Draw the layered structure of an operating system
 - (f) What do you mean by system protection? How it is achieved?
2. Attempt any **four**:
 - (a) What is PCB (Process Control Block)?
 - (b) Explain the principle of concurrency.
 - (c) Demonstrate process synchronization using procedure consumer problem.
 - (d) What is critical section? Design algorithm to solve this problem.
 - (e) How can the interprocess communication be achieved?
 - (f) Define the following:
 - (i) Dispatch
 - (ii) Context switching.
3. Attempt any **four**:
 - (a) Define the following terms:
 - (i) Average waiting time.
 - (ii) Time slice or quantum.
 - (iii) Response time.
 - (iv) Turnaround time.
 - (v) CPU utilization.
 - (b) What should be the selection criteria for scheduling algorithm?
 - (c) Calculate turnaround time and average waiting time for the following set of processes, if these processes are scheduled using:
 - (i) SJF
 - (ii) Priority (both preemptive)

Process	Burst time	Priority	Arrival time
P_1	7	1	0
P_2	3	2	4
P_3	9	3	7

- (d) What is deadlock and its conditions?
 - (e) How deadlock can be avoided?
 - (f) Explain the differences between busy waiting and blocking.
4. Attempt any **two**:
- (a) Explain the difference between internal fragmentation and external fragmentation? Which one occurs in paging system? Which one occurs in systems using pure segmentation? Discuss various ways of removing fragmentation.
 - (b) Explain the concept of virtual memory and how it is obtained by demand paging and segmentation?
 - (c) Write short notes on the following:
 - (i) Thrashing
 - (ii) Cache memory.
 - (iii) Allocation of frame.
 - (iv) Dining-philosopher-problem.
5. Attempt any **four**:
- (a) Define the following terms.
 - (i) Seek time.
 - (ii) Rotational latency
 - (iii) File sharing.
 - (b) Explain indexed allocation method of disk allocation.
 - (c) What is DMA?
 - (d) What are the functions of a file system?
 - (e) Draw the file structure for UNIX operating system or disk operating system (DOS).
 - (f) List five system calls related to file system.

First Term Examination
(Regular)
Fourth Semester (B. Tech.)
(GGSIPU, Delhi)

Paper Code: ETCS-212

Subject: Operating System & System Programming-I

Paper ID: 32212

Time: 11/2 Hours

Maximum Marks: 30

Note: Q.1 is Compulsory, attempt any two from the rest.

Q.1.

- (a) Name two system calls each for process creation and process termination. (2 × 5)
- (b) Explain Belady's anomaly.
- (c) List the differences between hard real time system & soft real time systems.
- (d) What is the role of medium term scheduler. Draw its diagram also.
- (e) Sketch segmentation with paging.

Q.2.

- (a) Explain the use of TLB in paging H/W along with diagram. (4)
- (b) Following is the reference string 1, 2, 3, 4, 5, 6, 2, 1, 2, 3, 7, 6, 3, 1, 6, 3, 2.
Apply (i) LRU, (ii) FIFO (iii) OPTIMAL
Replacement algorithms on the given string to calculate the number of page faults. (6)

Q.3.

- (a) Consider the following set of processes with the CPU burst & arrival time in milliseconds. (6)

Process	Arrival time	Burst time
P1	0.0	8
P2	0.4	4
P3	0.9	3
P4	1.2	5

Assuming the context switch time to be 10% of CPU burst time. Draw the Gantt Chart and find: (6)

- (i) Average turnaround time for these processes with the FCFS scheduling algorithm.
- (ii) Average turnaround time for the these processes with the SJF scheduling algorithm.
- (b) What is the problem with priority scheduling algorithms, give the solution to the above problem with examples. (4)

Q.4. Write short notes on any **four**. (2.5 × 4)

- (a) Inverted page tables
- (b) Thrashing
- (c) O.S. as resource manager
- (d) Types of queues in queuing diagram
- (e) Produce consumer problem

Second Term Examination
(Regular) April - 2009
Fourth Semester (B. Tech.)
(GGSIPU, Delhi)

Paper Code: ETCS-212

Time: 1½ Hours

Subject: Operating System

Maximum Marks: 30

Note: Q.1 is compulsory, attempt any two from the rest.

Q.1.

- (a) Write the solution (code) to reader-writer problem using semaphores. **(10)**
- (b) What is a critical section and what is its problem?
- (c) Briefly explain rotational latency and seek time.
- (d) RAID stand for –
 - (i) Retrieval Array of Independent Disks
 - (ii) Round Array of Independent Disks
 - (iii) Round Allocation of Independent disksAnd how many levels does it supports?
- (e) Which disk scheduling algorithm is known as elevator algorithm and why?

Q.2.

- (a) Suppose a disk has 2000 cylinders and currently it is serving a request at 143 cylinders and previous request was at 130. The queue of pending request in FIFO order is 90, 1400, 973, 948, 1765, 1420, 1035, 1765, 1420, 1035, 1710, 140. Calculate the total distance that the disk arm covers using the following algorithm: **(6)**
 - (i) FCFs (ii) SSTF (iii) SCAN (iv) CLOOK
- (b) Draw a wait for graph from resource allocation graph using four processors and five resources, where resource type 2, 4 and 5 have multiple instances, such that (i) the graph has a cycle but processes are not deadlocked. (ii) The graph has a cycle and processes are deadlock. **(4)**

Q.3.

- (a) Consider the following snapshot system

Process	Allocation	Max.	Available
P_0	0012	0012	1520
P_1	1000	1750	
P_2	1354	2356	
P_3	0632	0652	
P_4	0014	0656	

Assuming the following using the “Banker’s algorithm”.

- (i) What is the context of the array “Need”?
 - (ii) Is the system in a “Safe state”?
 - (iii) If a request from process P_1 arrives for (0, 4, 2, 0) can the request be immediately granted?
 - (b) Explain dining philosophers problem and give its solution. **(4)**
- Q.4.** Write short notes on any **four** **(2.5 × 4 = 10)**
- (a) Deadlock recovery
 - (b) Monitors
 - (c) Buffering
 - (d) Dedicated and shared devices
 - (e) BAD blocks

B. Tech.
(Semester-VI) Examination, 2007-08
Operating System

[Time: 3 Hours]

[Total Marks: 100]

- Note:** (i) Attempt **all** questions.
(ii) All questions carry equal marks.
(iii) Use suitable diagram wherever necessary

1. Attempt any **four** of the following: **5 × 4 = 20**
 - (a) Enumerate various OS components and give their functions in brief.
 - (b) Differentiate between Smart Card OS and Personal Computer OS.
 - (c) Differentiate between (with one suitable example):
 - (i) Interactive and batch processing systems
 - (ii) Multiprogramming and multitasking.
 - (d) List various operating system structures and explain virtual machine architecture in detail.
 - (e) One of the major functions of OS is to act as a resource manager. Is it true or false? Give reasons in support of your answer.
 - (f) Differentiate between
 - (i) System software and application software
 - (ii) General purpose OS and real-time OS.
2. Attempt any **two** of the following: **10 × 2 = 20**
 - (a) Consider a system consisting of processes $P_1, P_2, \dots, P_n$, each of which has a unique priority number. Write a monitor that allocates three identical line printers to these processes, using the priority numbers for deciding the order of allocation.
 - (b) Define mutual exclusion and its need. One solution to the critical section problem or mutual exclusion implementation is with test and set instruction (test and lock). Explain this approach in detail and give its relative advantages and disadvantages.
 - (c) It is said Inter process communication is best provided by a message passing system. Explain the implementation issue in massive passing system for inter process communication.

3. Attempt any **two** of the following: **10 × 2 = 20**

- (a) Explain priority scheduling and its major drawbacks with their solutions. Draw the Gantt chart and find average waiting time and response time of the process set given in the following table.

Process Id	Arrival time	Execution time	Priority
A	0	10	3
B	0	2	1
C	1	3	3
D	2	1	5
E	2	4	2

- (b) Discuss in brief any two evaluation methods which can be for scheduling algorithms
- (c) Write short notes on:
- (i) Resource allocation graph and resource allocation graph algorithm.
 - (ii) Recovery from deadlock

4. Attempt any **two** of the following: **10 × 2 = 20**

- (a) What do you understand by Belady's anomaly. Which popular page replacement algorithm suffers from the Belady's anomaly? Also give the name of the class of algorithms, which can never suffer from Belady's and why?

A system using demand-paged memory, takes 250 ns to satisfy a memory request if the page is in memory. If the page is not in memory, the request takes on an average 5 ms if a free frame is available or the page to the swapped out has not been modified or 12 ms if the page to be swapped out has been modified. What is the effective access time if the page fault rate is 2%, and 40% of the time the page to be replaced has been modified? Assume the system is running only a single process and the CPU is idle during page swaps.

- (b) (i) Describe the First Fit, Best Fit and Worst Fit memory allocation algorithms.
- (ii) On a system using a disk cache, the mean access time is 41.2 ms, the mean disk access time is 2 ms, the mean disk access time is 100 ms, and the system has 8 MB of cache memory. For each doubling of the amount of memory, the miss rate is halved. How much memory must be added to reduce the mean access time to 20 ms? Assuming the amount of memory may only increase by doubling.
- (c) Differentiate between paging and segmentation. On a system using paging and segmentation, the virtual address space consists of up to 8 segments where each segment can be up to 2^{29} bytes long. The hardware pages each segment into 256-byte pages. Determine the bits needed in the virtual address to specify the:

- (i) Segment number
 - (ii) Page number
 - (iii) Offset within page
 - (iv) Entire virtual address
5. Attempt any **two** of the following: **10 × 2 = 20**
- (a) Differentiate between:
 - (i) Block and character devices.
 - (ii) Block and non-blocking I/Os.
 - (iii) Link list and bit map approach for free space memory management.
 - (iv) Double buffering and single buffering.
 - (b) Discuss the following with respect to file system:
 - (i) Consistency checking
 - (ii) Common file attributes and file operations.
 - (c) None of the disk scheduling discipline, except FCFS, are truly fair (starvation may occur)
 - (i) Explain why this assertion is true.
 - (ii) Describe a way to modify algorithms such as SCAN to ensure fairness.
 - (iii) Explain why fairness is an important goal in a time-sharing system.
 - (iv) Give two examples of circumstances in which it is important that the operating system be unfair in serving I/O requests.

End Term Examination
Fourth Semester (B. Tech.) May 2008
(GGSIPU, Delhi)

Paper Code: ETCS212

Subject: Operating System and System Programming-I

Paper ID: 27212

(Batch: 2001-2003)

Time: 3 Hours

Maximum Marks: 75

Note: Attempt any five questions. All questions carry equal marks.

Q.1.

- (a) Explain in brief the services provided by an operating system.
- (b) What is the main advantage for an operating system designer of using a virtual machine architecture? What is the main advantage for a user?

Q.2.

- (a) Define the essential properties of following type of operating system:
 - (i) Time Sharing
 - (ii) Real Time
 - (iii) Distributed
 - (iv) Network
- (b) What is critical section problem? What are the major requirements that a solution to critical section problem must satisfy?

Q.3.

- (a) Consider the following set of processes:

Process	Burst time (msec)	Arrival time (msec)
P0	75	0
P1	40	10
P2	25	10
P3	20	80
P4	45	85

Suppose a system uses round-robin scheduling with a quantum of 15 msec.

- (i) Create a Gantt Chart illustrating the execution of these processes.
- (ii) What is the turnaround time for process P3?
- (iii) What are the average waiting and average turnaround times for these processes?
- (b) Explain the static and dynamic memory allocation techniques, with examples.

Q.4.

- (a) Explain the techniques for preventing and avoiding deadlocks.

- (b) Given memory partitions of 100 kB, 500 kB, 200 kB, 300 kB and 600 kB (in order), how would algorithms place process of 212 kB, 417 kB, 112 kB and 426 kB (in order)? Which algorithm makes the most efficient use of memory?

Q.5.

- (a) Consider the following page-reference string:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

How many page faults would occur for the following page replacement algorithms, assuming 3 frames:

- (i) FIFO (ii) Optimal
(ii) LRU
- (b) Explain the various types of directing structures.

Q.6.

- (a) Explain the different protection methods for file system in UNIX Operating System.
- (b) Suppose that a disk drive has 5000 cylinders, numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests in FIFO order is 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130 starting from the current head position. What is the total head distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk scheduling algorithms:
- (i) FCFS (ii) SSTF
(iii) SCAN (iv) C-LOOK

Q.7.

- (a) Explain following:
- (i) spooling (ii) online and offline processing
(iii) single buffering and double buffering
- (b) What is the difference between deadlock and starvation?
- (c) What is the difference between virtual address space and physical address space?
- (d) Why are page sizes always powers of 2?

End-Term Examination
Fourth Semester (B. Tech.) – May 2008
(GGSIPU, Delhi)

Paper Code: ETCS-212

Paper ID: 28212

Time: 3 Hours

Subject: Operating System

Batch: 2004-2006

Maximum Marks: 75

Note: Attempt any five questions.

Q.1.

- (a) Explain multiprogramming, time-sharing and distributed operating system. (6)
- (b) Distinguish between kernel and microkernel of an operating system. Describe the essential modules of kernel and explain the purpose of system calls. (9)

Q.2.

- (a) Explain the following terms with the help of examples: (6)
 - (i) Race-condition
 - (ii) Critical section
 - (iii) Mutual exclusion with busy waiting
- (b) Explain the reader's and writer's classical IPC problem and its solution. (9)

Q.3.

- (a) What is CPU scheduling? Explain the long-term, medium-term and short-term scheduling. (6)
- (b) Given the following information: (9)

Process	Arrival time	Burst time	Priority
A	0	3	3
B	1	6	5
C	2	1	2
D	3	4	1
E	4	2	4

Compute average waiting time and average turnaround time for the following scheduling algorithms:

- (i) Round robin (Quantum = 1)
- (ii) Priority preemptive
- (iii) Preemptive SJF

Q.4. (a) Describe Banker's algorithm to avoid a deadlock. (7)

(b) Consider a system with five processes (P_0, P_1, P_2, P_3, P_4) and three resources types (A, B, C) and having the following snapshot: (8)

	Allocation			Max.			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

Find

(i) Is system in a safe state?

(ii) Suppose process P_1 requests one additional instance of resource type A and two instances of resource type C. Can request be immediately granted?

Q.5.

(a) Explain in details: (9)

(i) Paged segmentation

(ii) Segmented paging

(b) In a paged segmented system, a virtual address consists of 32 bits of which 12 bits are used for displacement, 11 bits are a segment number and 9 bits are a page number. Calculate:

(i) Page size

(ii) Maximum number of pages

(iii) Maximum segment size

(iv) Maximum number of segments

Q.6.

(a) Describe various disk space allocation strategies with their merits and demerits. And discuss also the criteria that have to decide which strategy is to be adopted for a particular file. (8)

(b) A disk drive has 50 cylinders, numbered 0 to 49. The drive is currently serving a request at cylinder 15, and the previous request was at cylinder 9. The queue of pending requests, in FIFO order is 4, 40, 11, 35, 7, 14.

(i) SSTF

(ii) SCAN

(iii) LOOK

(iv) C-SCAN

Q.7. Write short notes on (any three): (3 × 5)

(a) Direct Memory Access (DMA)

(b) File protection and security

(c) Semaphores

(d) Virtual memory

B. Tech.
(Semester-VI) Examination, 2006-07
Operating System

Time: 3 Hours

Total Marks: 100

Note: (i) Attempt **all** questions. All questions carry equal marks.
(ii) Use suitable diagram wherever necessary

1. Attempt any **four** of the following: **4 × 5 = 20**
 - (a) Enumerate the basic functions of an operating system and explain each in brief.
 - (b) Write short notes on the following:
 - (i) Time sharing system
 - (ii) Real-time system
 - (c) Explain the microkernel approach to operating system design. Also enumerate its advantages and disadvantages.
 - (d) List five services provided by an operating system. Explain how each of them provides convenience to the users. Explain in which cases it would be impossible for user-level programs to provide these services.
 - (e) What is the purpose of the system calls or application programmer. Enumerate five system calls used in process management or file management.
 - (f) Define and explain the need of system protection. Give two examples.
2. Attempt any **four** of the following: **4 × 5 = 20**
 - (a) Draw the state diagram of a process and label various transitions. Explain the need of process suspension.
 - (b) Define the following terms:
 - (i) Dispatcher
 - (ii) Dispatch latency
 - (iii) Scheduling
 - (iv) Swapping
 - (v) Context switching.
 - (c) Differentiate between user thread and kernel thread. What is thread cancellation? Explain its type.
 - (d) Explain various thread models with their relative advantages and disadvantages.

- (e) Define a critical section problem and its solution by using semaphore. Use this approach to solve producer/consumer problem.
- (f) Explain with respect to IPC.
- Synchronous and asynchronous communication.
 - Need of buffering and its implementation.
3. Attempt any **two** of the following: **2 × 10 = 20**
- What are necessary conditions to hold a deadlock in a system? Explain the resource allocation graph algorithm to deal with deadlock problem. What are the limitations of this approach?
 - Consider the set of the processes given in Table 1 and the following scheduling algorithms:
 - First-Come First-Served
 - Round robin (Quantum = 2)
 - Round robin (Quantum = 1)

Table 1

Process Id	Arrival time	Execution time
A	0	4
B	2	7
C	3	3
D	3.5	3
E	4	5

If there is tie within the process, there is broken in the favour of the oldest process.

- Draw the Gantt chart and find the average waiting time and response time for the algorithms. Comment on your result which one is better and why?
 - If the scheduler takes 0.2 unit of CPU time in context switching for the completed job and 0.1 unit of additional CPU time for incomplete jobs for saving their context, calculate the percentage of CPU time wasted in each case.
- (c) Differentiate between: (two differences)
- Multiprocessor scheduling and uniprocessor scheduling
 - Deadlock prevention and avoidance
 - I/O bound and CPU bound job
 - Preemptive and non-preemptive scheduling
 - Multilevel queue and multilevel feedback queue scheduling.

4. Attempt any **two** of the following: **2 × 10 = 20**

- (a) A process has four page frames allocated to it. The time of the last loading of a page into each page frame, the time of last access to the page, the virtual page number in each frame and the referenced (R) and modified (M) bits for each page frame are shown in the table below. Times are in clock ticks from the process start-time at time 0.

Virtual page #	Frame #	Time loaded	Time referenced	M	R
2	0	60	161	0	1
1	1	130	160	0	0
0	2	26	162	1	0
3	3	20	163	1	1

A page fault to virtual page 4 has occurred. Which page frame will have its contents replaced under each of the following replacement algorithms?

- (i) FIFO (ii) LRU

Give reason in support of your answer. Also briefly explain the working of these algorithms.

- (b) Differentiate between prepaging and demand paging. On a system using demand-paged memory, it takes 120 ns to satisfy a memory request if the page is in memory. If the page is not in memory, the request takes on an average 5 ms. What would the page fault rate need to be achieved an effective access time of 1 ms? Assume the system is running only a single process and the CPU is idle during page swaps.

- (c) Write short notes on the following:

- (i) Temporal and spatial locality of reference
(ii) Memory protection mechanism
(iii) Memory allocation algorithms

5. Attempt any **two** of the following **2 × 10 = 20**

- (a) A disk with 1000 cylinders, numbers 0 to 999, compute the number of tracks the disk arm must move to satisfy all the requests in the disk queue. Assume the last request serviced was at track 345 and the head is moving towards track 0. The queue in FIFO order contains request for the following tracks: 123, 874, 692, 475, 105, 376.

Perform the computation for the following scheduling algorithms:

- (i) FIFO, (ii) SSTF, (iii) SCAN**

- (b) Discuss the multilevel indexing file allocation schemes. Which allocation scheme will minimize the amount of space required in directory structure and why?

- (c) Discuss the following with respect to I/O system:

- (i) Basic functions (ii) Block and character devices (iii) Spooling

End-Term Examination
Fourth Semester (B. Tech.) – May, June 2006
GGSIPU, Delhi
B. Tech CSE/IT

Paper Code: ETCS-212

Subject: Operating System

Paper ID: 27212

Time: 3 Hours

Maximum Marks: 75

Note: Attempt all questions. Q. No. 1 is compulsory. First question is of 25 marks, each subpart among of 2.5 marks. Each of the rest four questions carries 12.5 marks having internal choice among part a and b one part is to be attempted out of Q. No. 2 to 5.

Q.1. Attempt all parts:

- (a) What is the main advantage for an operating system designer of using a virtual machine architecture? What is the advantage for a user?
- (b) Describe the actions taken by a kernel to context switch among processes.
- (c) What advantage is there in having different quantum sizes on different levels of a multilevel queuing system?
- (d) Show that, if the wait and signal operations are not executed automatically, then mutual exclusion may be violated?
- (e) Can a system detect that some of its processes are starving? Justify your answer.
- (f) Why are the pages sizes always a power of 2?
- (g) When do page faults occur? Describe the actions taken by the operating system when a page fault occurs.
- (h) Why do some systems keep track of the types of a file, while others leave it to the user or simply to do not implement multiple file types? Which system is better?
- (i) How caches help improve the performance in a file system? Why do you not use more or larger caches if they are so useful?
- (j) Is disk scheduling, other than FCFS, useful in a single-user environment? Explain your answer.

Q.2. Attempt any one of the following:

- (a) Explain the working of paging and segmentation. Differentiate between the segmented paging and paged segmentation with a suitable example.

- (b) (I) Define the terms:
- (i) Distributed systems
 - (ii) Real-time systems
 - (iii) Multiprogramming
- (II) Given memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 660 KB (in order), how would each of the first-fit, best-fit, and worst-fit algorithms place processes of 212 KB, 417 KB, 112 KB and 426 KB (in order)? Which algorithm makes the most efficient use of memory?

Q3. Attempt any one of the following:

- (a) Consider a system consisting of processes $P_1, P_2, P_3 \dots P_n$, each of which has a unique priority number. Write a monitor that allocates three identical line printers to these processes using the priority numbers for deciding the order of allocation.
- (b) Consider the inter-process communication scheme where mailboxes are used.
- (i) Suppose a process P wants to wait for two messages, one from mailbox A and other from mailbox B . What sequence of send and receive should it execute?
 - (ii) What sequence of send and receive should P execute if P wants to wait for one message either from mailbox A or from mailbox B from both.
 - (iii) A receive operation makes a process wait until the mailbox is nonempty. Either devise a scheme that allows a process to wait until a mailbox is empty or explain why such a situation can't exist.

Q4. Attempt any one part of the following:

- (a) (I) Consider the following snapshot of a system:

	Allocation	Max	Available
	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	0 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 2	

Answer the following using banker's algorithm:

- (i) What is the content of the matrix need?
 - (ii) If the request in a safe state?
 - (iii) If a request from P_1 arrives for (0, 4, 2, 9), can the request be granted immediately?
- (II) If disk scheduling, other than FCFS scheduling useful in a single-user environment? Explain your answer.

- (b) Is disk scheduling, other than FCFS scheduling, useful in a single-user environment? Explain your answer.

86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130.

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requested for each of the following disk scheduling algorithms?

- | | |
|--------------|-------------|
| (i) SSTF | (ii) LOOK |
| (iii) C-SCAN | (iv) C-LOOK |
- (III) Discuss the following operating/terms with suitable examples:
- | | |
|---------------------------|---|
| (i) Buffering | (ii) Block multiplexing |
| (iii) Virtual devices and | (iv) Devices allocation considerations. |

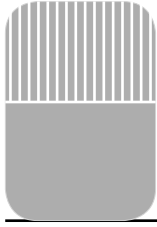
Q.5. Attempt any one part of the following:

- (a) (i) What problem could arise if a system allow a file system to be mounted simultaneously at more than one location?
- (ii) How do caches help improve the performance of file system? Why do systems not use more or larger caches if they are so useful?
- (iii) Explain why logging metadata updates ensure recovery of a file system after a file system crash.
- (b) (i) Explain the working of a virtual file system.
- (ii) Discuss the linked allocation and indented allocation schemes for a file allocation. Compare the index allocation scheme with the contiguous allocation scheme.



Bibliography

1. Galvin S., "Operating System Concepts", Addison Wesley, 5th Edition, 2001.
2. W. Stallings, Operating System, Pearson Education.
3. John Donovan, "System Programming", Tata McGraw-Hill, 23rd Reprint, 2000.
4. Andrew S. Tanenebaum, "Operating System Design", PHI Edition, 2000.
5. William Stallings, "Operating System Internals and Design Principles", PHI, 6th Edition 2002.
6. D.M. Dhamdhere, "Operating Systems: A Concept Based Approach, 2nd Edition, Tata McGraw-Hill, 2007.
7. J. Donovan & E. Madnick, "Operating System Concept and Design", Tata McGraw-Hill, 2001.
8. Dr. Rajiv Chopra, "Operating Systems (A Practical Approach), S. Chand, 2009.
9. I.A. Dhotre, "Operating System", Technical Publications, 2015.



Index

A

- Access list 225
- Access matrix implementation 224
- Access rights 224
- Address binding 121
- Algorithm description 206
- Authentication 225

B

- Bakery algorithm 74
- Banker's algorithm 98
- Bare machine 127
- Basic page replacement 161
- Batch operating systems 4
- Belady's anomaly 164
- Bit vector 192
- Block device 181

C

- Cache manager 241
- Cache memory organization 168
- Capability list 225
- Circular buffering 183
- Circular scan 205
- Client-server architecture 14
- C-Look 205
- Command interpreter system 12
- Contiguous allocation 190
- Contiguous memory allocation 126
- CPU scheduler 37
- CPU scheduling algorithms 39
- CPU-I/O burst cycle 36
- Critical section problem 70, 71
- C-scan 205

D

- Deadlock 91
 - avoidance of 97
 - cause of 92
 - detection of 100
 - handling of 95
 - necessary conditions of 92
 - prevention of 95
 - recovery from 102
- Deadlock state 103
- Demand paging 157, 158, 159
 - performance of 160
- Device manager 242
- Dining philosophers problem 82
- Direct access method 189
- Directory implementation 194, 195
- Disk bandwidth 200
- Disk performance parameters 200
- Disk scheduling 202
- Disk scheduling algorithms 202
- Disk storage 200
- Dispatcher 37
- Distributed systems 7
- Double buffering 183
- Dynamic linking 123
- Dynamic loading 122
- Dynamic relocation 125

E

- Environment subsystems 242
- Executive services 239
- External fragmentation 131, 146

F

FCFS (First Come-First Serve) 202
File 187
 access methods of 188
 attributes of 187
 operations of 188
 types of 187
File allocation methods 190
File directory 193
File management 9
File sharing 195
First-come, first-served 40
First-in-First-out (FIFO) algorithm 163
Fragmentation 131
Free space management 192

G

Global table 224

H

Hardware abstraction layer 239
Hardware RAID 207
Hashed page tables 139
Hybrid RAID 207

I

I/O buffering 182
I/O devices 181
I/O manager 240
I/O software 181
I/O subsystem 181
I/O system management 10
Indexed allocation 192
Indexed sequential access 190
Interactive computer systems 4
Internal fragmentation 131, 146
Interprocess communication (IPC) 29

K

Kernel mode 233
Kernels 16, 239

L

Layered approach 12
Least Recently Used (LRU) algorithm 165
Linked allocation 191

Linux system 231, 232
Local procedure call facility 241
Lock-key mechanism 225
Logical address space 124
Long-term scheduler 26, 27

M

Magnetic disks 199
Main memory 147
Main memory management 9
Mass storage structure 199
Medium-term scheduler 27
Memory allocation policies 146
Memory management 119
 basic hardware for 120
 requirements for 119
Memory management techniques 146
Message passing system 30
Microlithic kernel 16
MMS 125
Mobile OS 244
Monolithic kernel 16
Multilevel feedback queue scheduling 47
Multilevel paging 137
Multilevel queue scheduling 46
Multiprogramming operating systems 4
Multitasking operating systems 5

N

Network operating systems 8
Networking 10
No buffering 183
Non-contiguous memory allocation 132
Non-preemptive scheduling 38

O

Object manager 240
Operating system
 classification of 4
 components of an 8
 functions of 2, 3
 objectives of 1
 services 15
 structure of 11
 viewpoints on 2

Optimal algorithm 164

Overlays 123

P

Page fault 160

Page replacement algorithms 163, 166

Page table 135

Paging 145

Paging hardware 133

Parallel system 6

Peterson's solution 73

Physical address space 124

Plug-and-play manager 242

Power manager 240

Preemptive scheduling 38

Priority scheduling algorithm 43

Process 21, 25

Process control block 23

Process creation 27

Process management 9

Process manager 240

Process states 22

Process termination 28

Processes operations 27

Producer-consumer problem 69

Program threats 226

Protection 223

 domain of 223

 goals of 223

Protection system 11

P-threads 25

R

Race condition 68

Rag with a cycle but no deadlock 93

RAID (Redundant Array of Independent Disks) 206–210

RAID Levels 207

Readers-writers problem 81

Real-time operating systems 6

Re-entrant kernel 17

Resource allocation graph (RAG) 93

Resource allocation graph algorithm 97

Round robin scheduling 45

S

Safe state 97

Schedulers 26

Scheduling

 basic concepts of 36

 criteria 39

Secondary storage management 10

Security, need of 225

Security reference monitor 242

Segmentation 141, 145

 with paging 144

Semaphores 77

Sequential access method 189

Shared file, implementation of 195

Shared memory system 30

Shared pages 141

Shortest-job first scheduling 42

Short-term scheduler 26, 27

Single buffering 182

Sleeping barber problem 85

Software RAID 207

Solaris threads in 24

SSTF (Shortest Seek Time First) 203

Starvation 103

Swapping 125

Symbian OS

 application of 248

Symbian OS addresses memory 247

Symbian OS kernel layer 245

Symbian OS nanokernel 244

Symbian OS processes and threads 246

Synchronization 68

Synchronization hardware 75

Synchronization

 classic problems of 80

System model 91

System threats 226

 monitoring of 227

T

Thrashing 166

Threads 24, 25

Time sharing operating systems 6

U

User mode 233

V

Valid-invalid bit 159

Variable partition 130

Virtual address space 157

Virtual machine 14

Virtual memory manager 241

Virtual memory, concept of 156

W

Windows (2000) 237

Windows (95) 235

Windows (98) 236

Windows 3.1 234

Windows 3.X family 234

Windows ME 236

Windows NT 237

Windows NT family 237

Windows XP 238

Windows, enviroment system of 243

OPERATING SYSTEMS

A Concept-based Approach

The book describes the development of modern operating systems, explaining the evolution starting from the mainframe systems. This book deals with the subject matter in a novel, all-round manner. Since operating system is considered to be a concept-rich subject, this book discusses each concept in depth and explains the same in a lucid manner. The book also covers the practical aspects of the subject, emphasising shell programming. It also covers all the major programs discussed in lab sessions for students. The different features of various operating systems are also explained. Linux, Windows XP and Symbian Mobile operating systems are emphasised in the case studies.

This book has been designed after reviewing the syllabi of various Indian universities and, therefore, provides a wide coverage of the subject. Each chapter begins with learning objectives and ends with a summary containing a quick review of important concepts discussed in the chapter. Every chapter also provides a plenty of solved examples along with the conceptual material in the text for a hands-on understanding of the methods involved. Multiple-choice questions, review questions and brain teasers provided at the end of each chapter will assist the faculty in preparing students for the examinations.

With its coverage and approach, this book will be extremely useful for undergraduate and postgraduate students of computer science, IT engineering and MCA.

Salient Features:

- Discusses different types of OSs and OS architectures, and also explains how the modern operating systems have been developed.
- Highlights the hardware issues necessary to understand operating systems concepts.
- Covers every concept in depth and provides numerous solved examples interspersed within the text.
- Provides specially designed brain teasers at the end of each chapter for students to develop an analytical approach to problem solving.
- Includes case studies of OSs, namely Linux, Windows and Symbian Mobile OS.
- Contains hands-on session for programs that will be helpful for operating system laboratories.

Inderjeet Kaur is Associate Professor in the Department of Computer Science & Engineering at Ajay Kumar Garg Engineering College, Ghaziabad. She is B.Tech., M.Tech. and Ph.D and has over 11 years of experience in teaching undergraduate and postgraduate students. She has contributed over 20 papers in various national and international journals/conferences. She is a lifetime member of ACM, IETE. Her main interests lie in the designing and development of smart devices for various applications, and study of various operating system concepts.

Sonam Gupta is Assistant Professor in the Department of Computer Science & Engineering at Ajay Kumar Garg Engineering College, Ghaziabad. She is B.E., M. Tech. and is pursuing her Ph.D and has over 6 years of experience in teaching. She has published over 10 papers in various national/international journals. She is a reviewer of many international journals. Her research interests include operating systems and their applications.



978-93-89795-53-0



9 789389 795530